图论

Morning Flower and Evening Oath

XCPC vector 模板分册

建设稿：验证状态与适用边界见各条目及仓库 verification/。

目录

使用约定 vi

第一章 DFS（图论） 1
1.1 顶点划分：各诱导子图全为奇度 . . . 1

第二章 树上问题 4
2.1 树的直径 . . . 4
2.1.1 边权修改动态树直径 . . . 4
2.1.2 树上点集直径摘要 . . . 9
2.2 最近公共祖先 . . . 12
2.2.1 离线 Tarjan LCA . . . 12
2.2.2 欧拉序与 ST 表 LCA . . . 15
2.2.3 倍增 LCA、祖先与最大边权 . . . 18
2.2.4 两条树路径的交集 . . . 23
2.3 树链剖分 . . . 25
2.3.1 长链剖分与常数时间 K 级祖先 . . . 25
2.3.2 重链剖分与 LCA . . . 30
2.4 树上启发式合并 . . . 42
2.4.1 树上启发式合并：子树众数颜色和 . . . 42
2.5 虚树 . . . 45
2.5.1 虚树：关键点压缩与路径信息 . . . 45
2.6 树分治 . . . 49
2.6.1 点分治：树上距离点对计数 . . . 49
2.6.2 树上新增市场的支配范围 . . . 53
2.7 动态树分治 . . . 57
2.7.1 动态点分树：点权赋值与距离球求和 . . . 57
2.7.2 动态点分树：最近活动点与点号方案 . . . 62
2.7.3 动态点分树：有符号边权活动点直径与端点 . . . 66
2.8 树哈希 . . . 72
2.8.1 确定性有根与无根树同构 . . . 72

第三章 拓扑排序 75
3.1 DAG 有符号权最长路与路径方案 . . . 75

第四章 最短路问题 81
4.1 最短路 . . . 81
4.1.1 Johnson：负边全源最短路 . . . 81
4.1.2 非负权最短路 . . . 84
4.1.3 顶点开放时间的单位边最短路 . . . 86
4.1.4 Floyd 全源最短路与原边方案 . . . 88
4.1.5 最小总边权最短路树（含零权边） . . . 95
4.1.6 稠密图最短路：矩阵 Dijkstra . . . 101
4.1.7 SPFA：队列松弛、负环判定与有限路径 . . . 105
4.1.8 Bellman–Ford：负环影响与路径方案 . . . 112

4.2 k 短路 . . . 119
4.2.1 非负权严格次短行走 . . . 119
4.2.2 正权前 K 短行走 . . . 122

第五章 生成树问题 125
5.1 最小生成树 . . . 125
5.1.1 Kruskal 重构森林与阈值连通块 . . . 125
5.1.2 曼哈顿最小生成树 . . . 130
5.1.3 严格与非严格次小生成树、换边方案 . . . 133
5.1.4 稠密图 Prim 最小生成森林 . . . 138
5.1.5 Kruskal 最小生成树与生成森林 . . . 142
5.1.6 曼哈顿最小生成树：闭锥扫描与平局割证明 . . . 146
5.2 最小树形图 . . . 147
5.2.1 有向最小树形图 . . . 147

第六章 连通性相关 150
6.1 强连通分量 . . . 150
6.1.1 Tarjan 强连通分量与缩点 . . . 150
6.1.2 函数图分解、跳转与到达 . . . 155
6.1.3 强连通分量 . . . 160
6.2 双连通分量 . . . 163
6.2.1 点双与边双：共享 lowlink 核心 . . . 163
6.2.2 边双缩点森林（保留桥编号） . . . 167
6.2.3 属于简单奇环的全部顶点 . . . 169
6.2.4 边双缩点的大小与内部边数 . . . 172
6.2.5 无向边定向与强连通定向 . . . 175
6.3 割点和桥 . . . 176
6.3.1 由点双分解计算删点连通块数 . . . 176
6.3.2 连通无向图最少加边方案 . . . 179
6.3.3 割点与桥 . . . 180
6.4 圆方树 . . . 183
6.4.1 静态仙人掌森林最短路 . . . 183
6.4.2 由点双构造圆方森林 . . . 188

第七章 环计数问题 190
7.1 简单无向图四元环计数 . . . 190
7.2 无向简单图三元环枚举 . . . 192

第八章 最小环 194
8.1 有向图最小平均权环：线性空间 Karp . . . 194
8.2 非负权无向图最小环与原边方案 . . . 198

第九章 2-SAT 203
9.1 2-SAT 与任意可行方案 . . . 203
9.2 递归染色与回滚求最小赋值 . . . 206

第十章 欧拉图 209
10.1 有向图字典序最小欧拉路 . . . 209
10.2 字典序最小单词链 . . . 212
10.3 无向图欧拉路与边方向 . . . 214

10.4 欧拉回路与指定端点定向 . . . 217
10.5 任意起终点的混合欧拉路径 . . . 220

第十一章 网络流 221
11.1 最大流 . . . 221
11.1.1 HLPP 最高标号预流推进与流方案 . . . 221
11.1.2 最大流与方案 . . . 227
11.1.3 最大流中单位容量边的可行性与必选性 . . . 233
11.1.4 递归 ISAP 最大流 . . . 237
11.1.5 最大流方案唯一性 . . . 241
11.2 最小割 . . . 243
11.2.1 正容量最小割的可行边与必选边 . . . 243
11.2.2 最大权闭合子图与选点方案 . . . 245
11.2.3 Gomory–Hu 最小割树 . . . 247
11.2.4 最小割树全点对查询 . . . 249
11.3 费用流 . . . 250
11.3.1 势能最小费用最大流 . . . 250
11.3.2 SPFA 最小费用最大流 . . . 254
11.3.3 允许负环的最小费用最大流 . . . 258
11.3.4 zkw 最小费用最大流 . . . 261
11.4 上下界网络流 . . . 265
11.4.1 上下界可行循环流 . . . 265
11.4.2 有源汇上下界最值流与边流量 . . . 267
11.5 Stoer–Wagner 算法 . . . 271
11.5.1 无向全局最小割 . . . 271

第十二章 图的匹配 274
12.1 二分图最大匹配 . . . 274
12.1.1 Hopcroft–Karp 最大匹配与最小点覆盖 . . . 274
12.1.2 二分图最大匹配的可行边与必选边 . . . 278
12.1.3 二分图最大独立集方案 . . . 279
12.2 二分图最大权匹配 . . . 281
12.2.1 矩形二分图最小权匹配 . . . 281
12.2.2 二分图最大权匹配（缺边与可选匹配） . . . 283
12.2.3 按匹配基数求最大权与顶标证书 . . . 286
12.3 一般图最大匹配 . . . 292
12.3.1 一般图最大基数匹配：带花树 . . . 292

第十三章 Prüfer 序列 295
13.1 Prüfer 双射与有标号树计数 . . . 295
13.1.1 双射为何成立 . . . 295
13.1.2 Cayley、指定度数与叶子数 . . . 295
13.1.3 加权 Cayley 与连接已有连通块 . . . 296

第十四章 矩阵树定理 297
14.1 带权矩阵树：无向树与内外向树形图 . . . 297
14.2 任意正模数带权矩阵树 . . . 299
14.3 精确整数带权生成树计数 . . . 302
14.4 带权矩阵树定理与方向约定 . . . 305

14.4.1 为什么这个余子式恰好计数 . . . 305
14.4.2 既有接口与验证记录 . . . 305
14.5 任意正模数的矩阵树计数 . . . 306

第十五章 LGV 引理 307
15.1 LGV：DAG 路径权值行列式 . . . 307

第十六章 支配树 309
16.1 支配树：Lengauer–Tarjan . . . 309
16.2 DAG 支配树：拓扑序与在线倍增 LCA . . . 313

第十七章 配套知识与验证范围 318
17.1 树路径交：投影与单点交 . . . 318
17.2 非负权树的点集直径摘要 . . . 319
17.3 函数图的入环链与到达查询 . . . 320
17.4 杭州 2023 H：随机事件顺序与函数图 . . . 321
17.5 杭州 2023 G：开放时间的单位边最短路 . . . 322
17.6 SCC 模板题与速度榜口径 . . . 322
17.7 SPFA 费用流与残量方案 . . . 322
17.8 2-SAT 编号方向与双连通定义 . . . 323
17.9 排列约束与最大流单位边分类 . . . 324
17.10 点分治、启发式合并与虚树用法 . . . 324
17.11 严格次小生成树与换边 . . . 325
17.12 闭合集、树形图与全局割的完整用法 . . . 325

第一部分 跨类依赖 326

附录 A 数论 327
A.1 模算术简介 . . . 327
A.1.1 静态模整数 . . . 327
A.2 卢卡斯定理 . . . 329
A.2.1 Lucas：小素数模的大下标组合数 . . . 329

附录 B 组合数学 331
B.1 排列组合 . . . 331
B.1.1 阶乘组合数 . . . 331

附录 C 线性代数 333
C.1 行列式 . . . 333
C.1.1 素数模行列式 . . . 333
C.1.2 任意正模数行列式：欧几里得消元 . . . 335
C.1.3 精确整数行列式（Bareiss） . . . 337

附录 D 并查集 339
D.1 并查集 . . . 339
D.1.1 并查集 . . . 339

附录 E 树状数组 341
E.1 树状数组第 k 小与组合依赖 . . . 341

附录 F 线段树 **343**
F.1 线段树基础 343
F.1.1 通用懒标记线段树与边界搜索 343
F.1.2 通用线段树与左右边界搜索 348
F.1.3 线段树：区间乘、加、赋值与模意义求和 353

使用约定

C++20，默认包含 bits/stdc++.h 与 cassert，并使用 std 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 vector 代码。

第一章　DFS（图论）

1.1　顶点划分：各诱导子图全为奇度

代码依赖：dsu（第 339 页，D.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

odd_induced_partition(n,edges) 接收 0..n-1 的无向简单图，禁止自环、重边。返回 optional<vector<int»：无解为 nullopt，否则每点一个正组号，同组顶点的诱导子图中每个点度数均为奇数。不删选组内边，不要求组数最少。充要条件是每个连通分量点数为偶数；空图返回空划分。先递归 DFS 取得原深度和后序，再合并未完成分组；内部点已为奇度、根为偶度，奇数个子组可封口，偶数时留一个可上挂的子组给祖先。level 只记录活动根的挂接深度，原 depth 不修改。祖先邻点按原深度排序，以二分寻找最近的严格祖先，避免反复扫描高出度根的全部邻居。时间 O((n+m) log(n+1))，空间 O(n+m)，递归栈 O(n)；节点数、边数须可由 int 表示。依赖 dsu 的按调用方向连接根语义。

```
optional<vector<int>> odd_induced_partition(int n, const vector<pair<int, int>> &edges)
{
    vector<vector<int>> g(n), up(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> depth(n, -1), order;
    auto dfs = [&](auto &&self, int u, int h) -> int
    {
        depth[u] = h;
        int size = 1;
        for (int v : g[u])
            if (depth[v] == -1) size += self(self, v, h + 1);
        order.push_back(u);
        return size;
    };
    for (int u = 0; u < n; u++)
        if (depth[u] == -1 && dfs(dfs, u, 0) % 2) return nullopt;
    for (auto [u, v] : edges)
    {
        if (depth[u] > depth[v]) swap(u, v);
        up[v].push_back(u);
    }
    for (auto &a : up)
        sort(a.begin(), a.end(), [&](int u, int v) { return depth[u] < depth[v]; });
```

合并未完成分组（接上页同一函数）：

```
    dsu d(n);
    vector<int> level = depth, color(n);
    int count = 0;
    for (int u : order)
    {
        vector<int> children;
        for (int v : g[u])
            if (level[v] == level[u] + 1 && !color[d.find(v)]) children.push_back(v);
        if (children.empty()) continue;
        int skip = -1, best = -1;
        if (children.size() % 2 == 0)
            for (int v : children)
            {
                auto it = lower_bound(up[v].begin(),
                                      up[v].end(),
                                      depth[u],
                                      [&](int w, int h) { return depth[w] < h; });
                if (it != up[v].begin() && depth[*prev(it)] > best)
                {
                    skip = v;
                    best = depth[*prev(it)];
                }
            }
        if (skip != -1) level[skip] = best + 1;
        if (children.size() % 2 || skip != -1) color[d.find(u)] = ++count;
        for (int v : children)
            if (v != skip) d.merge(u, v);
    }
    for (int u = 0; u < n; u++) color[u] = color[d.find(u)];
    return color;
}
```

应用题补充：奇度诱导子图划分（本地接口示例）

题意：输入无向简单图，点号减 1 后调用；输出同组顶点的诱导子图全部为奇度的顶点分组，无解输出 -1。组号本身无唯一答案，验证时按原图所有同组边计算度数。本例是本地接口演示，链接为存在性定理来源，不是 OJ 记录。

所需模板：dsu（第 339 页，D.1.1 节）、odd_induced_partition（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://people.maths.ox.ac.uk/scott/Papers/odds2.pdf

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> edges(m);
    for (auto &[u, v] : edges)
    {
        cin >> u >> v;
        u--;
        v--;
    }
    auto color = odd_induced_partition(n, edges);
    if (!color)
        cout << -1 << '\n';
    else
    {
        for (int x : *color) cout << x << ' ';
        cout << '\n';
    }
}
```

第二章　树上问题

2.1　树的直径

相关应用：固定拓扑下修改非负边权，递归欧拉序与区间加维护动态直径。

2.1.1　边权修改动态树直径

DynamicDiameter(n)，n≥1，点号 1..n。add(u,v,w) 返回从 0 开始的原边编号；输入须恰为一棵非负权无向树。build(root=1) 预处理；update(id,w) 设置原边权值；diameter() 返回最长路径长度，单点树为 0。仅返回长度，不支持 link/cut 和负权。可换根重建、复制；所有内部成员仅供只读。全部边权之和始终 ≤LLONG_MAX/4，保证中间值安全。

递归 DFS 记录进入顶点及每次孩子返回后的根距离，得到 2n-1 项欧拉序。修改父边等价于对子树首次至最后出现的闭区间加 Δ。维护 max(d[i]-2d[j]+d[k])，i≤j≤k；非负权下区间最小根距离等于 LCA 根距离，故最大值即直径。线段树五项 high、low、left、right、best 分别表示 d、-2d、前两项、后两项、完整三项最大值。统一加 Δ 后依次加 Δ、-2Δ、-Δ、-Δ、0。建树 O(n)，修改 O(log n)，查询 O(1)，空间 O(n)。DFS 保持递归，深链须给足原生栈。线上验证尚待提交。

字段、加边与五项合并：

```
struct DynamicDiameter
{
    using ll = long long;
    struct Node
    {
        ll high, low, left, right, best, lazy;
    };
    struct Edge
    {
        int u, v;
        ll w;
    };
    int n, size = 0;
    ll total = 0;
    vector<Edge> edges;
    vector<vector<pair<int, int>>> g;
    vector<int> first, last, child;
    vector<Node> t;

    explicit DynamicDiameter(int n) : n(n), g(n + 1), first(n + 1), last(n + 1)
    {
        assert(n >= 1);
    }
    int add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(w >= 0 && w <= LLONG_MAX / 4 - total);
        int id = edges.size();
        edges.push_back({u, v, w});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        total += w;
        size = 0;
        return id;
    }
    static Node merge(const Node &a, const Node &b)
    {
        Node c;
        c.high = max(a.high, b.high);
        c.low = max(a.low, b.low);
        c.left = max({a.left, b.left, a.high + b.low});
        c.right = max({a.right, b.right, a.low + b.high});
        c.best = max({a.best, b.best, a.left + b.high, a.high + b.right});
        c.lazy = 0;
        return c;
    }
```

递归欧拉遍历与建树（接上页结构体）：

```
    void build(int root = 1)
    {
        assert(1 <= root && root <= n && (int)edges.size() == n - 1);
        vector<ll> d;
        child.resize(edges.size());
        auto dfs = [&](auto &&self, int u, int parent, ll depth) -> void
        {
            first[u] = d.size();
            d.push_back(depth);
            for (auto [v, id] : g[u])
            {
                if (v == parent) continue;
                child[id] = v;
                self(self, v, u, depth + edges[id].w);
                d.push_back(depth);
            }
            last[u] = d.size() - 1;
        };
        dfs(dfs, root, 0, 0);
        size = d.size();
        t.resize(4 * size);
        auto init = [&](auto &&self, int x, int l, int r) -> void
        {
            if (l == r)
            {
                t[x] = {d[l], -2 * d[l], -d[l], -d[l], 0, 0};
                return;
            }
            int mid = (l + r) / 2;
            self(self, x * 2, l, mid);
            self(self, x * 2 + 1, mid + 1, r);
            t[x] = merge(t[x * 2], t[x * 2 + 1]);
        };
        init(init, 1, 0, size - 1);
    }
```

区间加与原边修改（接上页结构体）：

```
    void apply(int x, ll delta)
    {
        t[x].high += delta;
        t[x].low -= 2 * delta;
        t[x].left -= delta;
        t[x].right -= delta;
        t[x].lazy += delta;
    }
    void change(int x, int l, int r, int ql, int qr, ll delta)
    {
        if (ql <= l && r <= qr)
        {
            apply(x, delta);
            return;
        }
        apply(x * 2, t[x].lazy);
        apply(x * 2 + 1, t[x].lazy);
        t[x].lazy = 0;
        int mid = (l + r) / 2;
        if (ql <= mid) change(x * 2, l, mid, ql, qr, delta);
        if (qr > mid) change(x * 2 + 1, mid + 1, r, ql, qr, delta);
        t[x] = merge(t[x * 2], t[x * 2 + 1]);
    }
    void update(int id, ll w)
    {
        assert(size && 0 <= id && id < (int)edges.size());
        assert(w >= 0 && w <= LLONG_MAX / 4 - (total - edges[id].w));
        ll delta = w - edges[id].w;
        total += delta;
        edges[id].w = w;
        int u = child[id];
        change(1, 0, size - 1, first[u], last[u], delta);
    }
    ll diameter() const
    {
        assert(size);
        return t[1].best;
    }
};
```

模板题使用：CEOI2019 Dynamic Diameter（LOJ3163 待复核提交）

题意：每次修改一条边后输出直径；用上次答案解码边号和新权，边编号 0-based。

所需模板：DynamicDiameter（第 4 页，2.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://loj.ac/p/3163

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    long long w;
    cin >> n >> q >> w;
    DynamicDiameter tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        tree.add(u, v, c);
    }
    tree.build();
    long long last = 0;
    while (q--)
    {
        long long d, e;
        cin >> d >> e;
        int id = (d + last) % (n - 1);
        tree.update(id, (e + last) % w);
        last = tree.diameter();
        cout << last << '\n';
    }
}
```

2.1.2 树上点集直径摘要

默认空点集，a=b=-1、length=0。insert(v,distance) 插入顶点，重复插入合法；merge(other,distance) 合并两个点集摘要，允许自合并；farthest(v,distance) 返回 optional<pair<int,long long»（最远顶点、距离），空集返回 nullopt，距离并列时可返回任意最远点。distance(u,v) 回调必须精确返回同一棵固定非负权树的距离，点号为非负整数；所有距离 ≤LLONG_MAX。每次插入或最远查询最多两次距离调用，合并最多四次，摘要只占 O(1) 空间。保存 a、b 为点集中某对直径端点，不保留全部点；字段只读，不能用一般图度量或负权树代替。复制摘要即可建立前缀、后缀或线段树聚合，重置用默认构造。

```
struct TreeDiameter
{
    using ll = long long;
    int a = -1, b = -1;
    ll length = 0;

    // distance is the metric of one fixed tree with nonnegative edge weights.
    template <class Distance> void insert(int v, const Distance &distance)
    {
        assert(v >= 0);
        if (a == -1)
        {
            a = v;
            b = v;
            return;
        }
        ll x = distance(a, v), y = distance(b, v);
        if (x >= y && x > length)
        {
            b = v;
            length = x;
        }
        else if (y > length)
        {
            a = v;
            length = y;
        }
    }
```

合并与最远点查询（接上页同一结构体）：

```
    template <class Distance> void merge(TreeDiameter other, const Distance &distance)
    {
        if (other.a == -1) return;
        insert(other.a, distance);
        insert(other.b, distance);
    }

    template <class Distance>
    optional<pair<int, ll>> farthest(int v, const Distance &distance) const
    {
        if (a == -1) return nullopt;
        ll x = distance(a, v), y = distance(b, v);
        return x >= y ? make_pair(a, x) : make_pair(b, y);
    }
};
```

应用题补充：Codeforces 379F New Year Tree

题意：初始 1 连接 2、3、4；每次给指定叶子添两个新叶子，输出直径边数。q≤500000，最终点数 4+2q。先离线建最终树并做 HLD，再按编号顺序 insert；添叶不会改变旧点间距离，满足固定非负权树前提。只维护已出现顶点。O(q log q) 时间、O(q) 空间，DFS 递归；只调用 insert 及 length，merge/farthest 由接口测试验证。

所需模板：HLD（第 30 页，2.3.2 节）、TreeDiameter（第 9 页，2.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/379/F

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    int n = 4 + 2 * q;
    HLD tree(n);
    for (int v = 2; v <= 4; v++) tree.add(1, v);
    for (int i = 0; i < q; i++)
    {
        int v;
        cin >> v;
        tree.add(v, 5 + 2 * i);
        tree.add(v, 6 + 2 * i);
    }
    tree.build();
    auto dis = [&](int u, int v)
    {
        return tree.dep[u] + tree.dep[v] - 2 * tree.dep[tree.lca(u, v)];
    };
    TreeDiameter cur;
    for (int v = 1; v <= 4; v++) cur.insert(v, dis);
    for (int i = 0; i < q; i++)
    {
        cur.insert(5 + 2 * i, dis);
        cur.insert(6 + 2 * i, dis);
        cout << cur.length << '\n';
    }
    return 0;
}
```

2.2 最近公共祖先

2.2.1 离线 Tarjan LCA

构造 OfflineLCA(n)，add(u,v) 加无向树边，add_query(u,v) 登记查询并返回从 0 开始的编号，run(root=1) 后读取 answer[id]。输入必须是非空树，点号 1..n；允许同点、重复查询及零个查询。run 可重复调用或换根，不清空查询；追加查询后需重新 run，旧答案在重算前不可按新根使用。DFS 与并查集查找均递归。并查集按大小合并并压缩路径，ancestor[find(u)] 保存已合并部分对应的当前树祖先；不能直接拿并查集代表当 LCA。每个子树完成后合并到父点并更新 ancestor；查询另一端已完成时，从其集合读取祖先。总时间 O((n+q) α(n))，空间 O(n+q)。全部点对和根的父链对拍、50 万点链及 P3379、POJ1470 完整驱动本地验证，在线待补。

```
struct OfflineLCA
{
    int n;
    vector<vector<int>> g;
    vector<vector<pair<int, int>>> queries;
    vector<int> parent, size, ancestor, done, answer;

    OfflineLCA(int n)
        : n(n),
          g(n + 1),
          queries(n + 1),
          parent(n + 1),
          size(n + 1),
          ancestor(n + 1),
          done(n + 1)
    {
    }

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int add_query(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = answer.size();
        answer.push_back(0);
        queries[u].push_back({v, id});
        queries[v].push_back({u, id});
        return id;
    }
```

```
    int find(int u)
    {
        if (parent[u] != u) parent[u] = find(parent[u]);
        return parent[u];
    }

    void merge(int u, int v)
    {
        u = find(u);
        v = find(v);
        if (size[u] < size[v]) swap(u, v);
        parent[v] = u;
        size[u] += size[v];
    }

    void dfs(int u, int p)
    {
        for (int v : g[u])
            if (v != p)
            {
                dfs(v, u);
                merge(u, v);
                ancestor[find(u)] = u;
            }
        done[u] = 1;
        for (auto [v, id] : queries[u])
            if (done[v]) answer[id] = ancestor[find(v)];
    }

    // Input must be a nonempty tree. Query IDs are 0-based.
    void run(int root = 1)
    {
        assert(1 <= root && root <= n);
        iota(parent.begin(), parent.end(), 0);
        iota(ancestor.begin(), ancestor.end(), 0);
        fill(size.begin(), size.end(), 1);
        fill(done.begin(), done.end(), 0);
        fill(answer.begin(), answer.end(), 0);
        dfs(root, 0);
    }
};
```

模板题使用：Luogu P3379

题意：给定以 root 为根的树，回答两点的最近公共祖先。先登记全部查询，再 run(root)，answer 按查询加入顺序输出；点号为 1..n。

所需模板：OfflineLCA（第 12 页，2.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3379

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, root;
    cin >> n >> m >> root;
    OfflineLCA graph(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add_query(u, v);
    }
    graph.run(root);
    for (int u : graph.answer) cout << u << '\n';
    return 0;
}
```

2.2.2 欧拉序与 ST 表 LCA

构造 EulerLCA(n)，add(u,v,w=1) 加非负权无向树边，build(root=1) 预处理；lca(u,v) 返回最近公共祖先，distance(u,v) 仍返回无权路径边数；weighted_distance(u,v) 返回 long long 带权距离。边权非负，所有树上路径的权和须 ≤LLONG_MAX。输入必须为非空树，点号 1..n；允许同点查询和重复 build 换根。只可在 build 后查询，加边会清除 ready 标志。递归 DFS 在进入顶点及从每个儿子返回时记录顶点，euler 长度为 2n-1，first[u] 是从 0 开始的首次出现位置；depth[root]=0。两点首次出现位置间深度最小的顶点就是 LCA，用 ST 表保存该顶点。预处理 O(n log n) 时间和空间，每次查询 O(1)。ST 内部下标从 0 开始；first、depth、euler、root_dist 可直接读取；root_dist 为当前根的带权距离。50 万点链、全部小树根与点对以及 P3379、POJ1330 双风格完整程序本地验证；上述无权题在线待补；QOJ8235 的带权应用已双 AC。

```
struct EulerLCA
{
    using ll = long long;
    int n;
    bool ready = false;
    vector<vector<int>> g, st;
    vector<vector<ll>> cost;
    vector<ll> root_dist;
    vector<int> first, depth, euler, lg;

    EulerLCA(int n)
        : n(n), g(n + 1), cost(n + 1), root_dist(n + 1), first(n + 1), depth(n + 1)
    {
    }

    void add(int u, int v, ll w = 1)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(w >= 0);
        g[u].push_back(v);
        g[v].push_back(u);
        cost[u].push_back(w);
        cost[v].push_back(w);
        ready = false;
    }

    int shallower(int u, int v) const { return depth[u] <= depth[v] ? u : v; }
```

递归欧拉序与 RMQ（接上页同一结构体）：

```
    void dfs(int u, int parent)
    {
        first[u] = euler.size();
        euler.push_back(u);
        for (int i = 0; i < int(g[u].size()); i++)
        {
            int v = g[u][i];
            if (v != parent)
            {
                depth[v] = depth[u] + 1;
                root_dist[v] = root_dist[u] + cost[u][i];
                dfs(v, u);
                euler.push_back(u);
            }
        }
    }

    // Input must be a nonempty tree. Rebuilding may change the root.
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        euler.clear();
        depth[root] = 0;
        root_dist[root] = 0;
        dfs(root, 0);
        int m = euler.size();
        lg.assign(m + 1, 0);
        for (int i = 2; i <= m; i++) lg[i] = lg[i / 2] + 1;
        st.assign(lg[m] + 1, {});
        st[0] = euler;
        for (int k = 1; k <= lg[m]; k++)
        {
            st[k].resize(m - (1 << k) + 1);
            for (int i = 0; i + (1 << k) <= m; i++)
                st[k][i] = shallower(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
        }
        ready = true;
    }
```

LCA、边数距离与带权距离（接上页同一结构体）：

```
    int lca(int u, int v) const
    {
        assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
        int l = first[u], r = first[v];
        if (l > r) swap(l, r);
        int k = lg[r - l + 1];
        return shallower(st[k][l], st[k][r - (1 << k) + 1]);
    }

    int distance(int u, int v) const
    {
        int z = lca(u, v);
        return depth[u] + depth[v] - 2 * depth[z];
    }

    ll weighted_distance(int u, int v) const
    {
        int z = lca(u, v);
        return (root_dist[u] - root_dist[z]) + (root_dist[v] - root_dist[z]);
    }
};
```

模板题使用：Luogu P3379

题意：给定以 root 为根的树，回答两点的最近公共祖先。加完所有边后 build(root)，再逐条调用 lca；点号为 1..n。此题只使用无权 LCA 接口。

所需模板：EulerLCA（第 15 页，2.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3379

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q, root;
    cin >> n >> q >> root;
    EulerLCA graph(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    graph.build(root);
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        cout << graph.lca(u, v) << '\n';
    }
    return 0;
}
```

2.2.3 倍增 LCA、祖先与最大边权

构造 LiftingLCA(n)，add(u,v,w=1) 加边，build(root=1) 预处理非空无向树，点号 1..n。jump(u,k) 返回 k 级祖先，k 非负，超过根返回 0；lca(u,v) 求最近公共祖先；distance 返回路径边数，不是权值和；max_edge 返回路径最大边权，同点返回 nullopt，其他情况返回有值的 optional<long long>。支持负权和 LLONG_MIN，LOW 只是最大值合并的单位元，不表示空路径。DFS 递归，根深度为 0；up[k][u] 与 mx[k][u] 是 2 的 k 次方长度向上段的祖先与最大边权，0 号节点为哨兵。可重复 build 换根；修改图后必须重建。预处理 O(n log n) 时间与空间，各查询 O(log n)。不需要边权最值时，可删去 mx 和 max_edge 相关部分。全小树点对/根、极值权重及 50 万点负权链本地验证；P3379、POJ1330 驱动只验证 LCA，最大边权仍是本地证据。

```
struct LiftingLCA
{
    using ll = long long;
    static constexpr ll LOW = numeric_limits<ll>::lowest();
    int n, levels;
    bool ready = false;
    vector<vector<pair<int, ll>>> g;
    vector<vector<int>> up;
    vector<vector<ll>> mx;
    vector<int> depth;

    LiftingLCA(int n) : n(n), g(n + 1), depth(n + 1)
    {
        assert(n > 0);
        levels = 1;
        while ((1LL << levels) <= n) levels++;
        up.assign(levels, vector<int>(n + 1));
        mx.assign(levels, vector<ll>(n + 1, LOW));
    }

    void add(int u, int v, ll w = 1)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
        ready = false;
    }
```

```
    void dfs(int u, int parent, ll weight)
    {
        up[0][u] = parent;
        mx[0][u] = weight;
        for (int k = 1; k < levels; k++)
        {
            int p = up[k - 1][u];
            up[k][u] = up[k - 1][p];
            mx[k][u] = max(mx[k - 1][u], mx[k - 1][p]);
        }
        for (auto [v, w] : g[u])
            if (v != parent)
            {
                depth[v] = depth[u] + 1;
                dfs(v, u, w);
            }
    }

    // Nonempty tree only. Node 0 means no ancestor.
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        depth[root] = 0;
        dfs(root, 0, LOW);
        ready = true;
    }

    int jump(int u, long long steps) const
    {
        assert(ready && 1 <= u && u <= n && steps >= 0);
        if (steps > depth[u]) return 0;
        for (int k = 0; k < levels; k++)
            if (steps >> k & 1) u = up[k][u];
        return u;
    }
```

```
    int lca(int u, int v) const
    {
        assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
        if (depth[u] < depth[v]) swap(u, v);
        u = jump(u, depth[u] - depth[v]);
        if (u == v) return u;
        for (int k = levels - 1; k >= 0; k--)
            if (up[k][u] != up[k][v])
            {
                u = up[k][u];
                v = up[k][v];
            }
        return up[0][u];
    }

    int distance(int u, int v) const
    {
        return depth[u] + depth[v] - 2 * depth[lca(u, v)];
    }

    optional<ll> max_edge(int u, int v) const
    {
        assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
        if (u == v) return nullopt;
        if (depth[u] < depth[v]) swap(u, v);
        ll answer = LOW;
        int delta = depth[u] - depth[v];
        for (int k = 0; k < levels; k++)
            if (delta >> k & 1)
            {
                answer = max(answer, mx[k][u]);
                u = up[k][u];
            }
        if (u == v) return answer;
        for (int k = levels - 1; k >= 0; k--)
            if (up[k][u] != up[k][v])
            {
                answer = max(answer, max(mx[k][u], mx[k][v]));
                u = up[k][u];
                v = up[k][v];
            }
        return max(answer, max(mx[0][u], mx[0][v]));
    }
};
```

模板题使用：Luogu P3379

题意：给定以 root 为根的树，回答两点的最近公共祖先。加完所有边后 build(root)，再逐条调用 lca；点号为 1..n。此题不涉及跳祖先或路径最大边权。

所需模板：LiftingLCA（第 18 页，2.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3379

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q, root;
    cin >> n >> q >> root;
    LiftingLCA graph(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    graph.build(root);
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        cout << graph.lca(u, v) << '\n';
    }
    return 0;
}
```

应用题补充：SPOJ QTREE2 / 路径距离与第 k 个点

题意：多测 t≤25、n≤10000、边长 c≤100000，DIST u v 查距离，KTH u v k 查从 u 到 v 的第 k 个点（1-based 且合法），DONE 结束，每组后空行。根距离 d 额外用递归 DFS 求出；切勿把 h.distance 的边数当权值和。令 a、b 为两端到 LCA 的边数，k≤a+1 从 u 跳 k−1，否则从 v 跳 a+b+1−k。预处理 O(n log n)、查询 O(log n)、空间 O(n log n)；本用法所有根距离、差和答案须在 long long 范围。

所需模板：LiftingLCA（第 18 页，2.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.spoj.com/problems/QTREE2/

SPOJ QTREE2 / 路径距离与第 k 个点：使用代码

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        LiftingLCA h(n);
        for (int i = 1; i < n; i++)
        {
            int u, v;
            long long w;
            cin >> u >> v >> w;
            h.add(u, v, w);
        }
        h.build();
        vector<long long> d(n + 1);
        auto dfs = [&](auto &&self, int u, int p) -> void
        {
            for (auto [v, w] : h.g[u])
            {
                if (v == p) continue;
                d[v] = d[u] + w;
                self(self, v, u);
            }
        };
        dfs(dfs, 1, 0);
        string cmd;
        while (cin >> cmd && cmd != "DONE")
        {
            int u, v;
            cin >> u >> v;
            int p = h.lca(u, v);
            if (cmd == "DIST") cout << (d[u] - d[p]) + (d[v] - d[p]) << '\n';
            else
            {
                int k;
                cin >> k;
                int a = h.depth[u] - h.depth[p];
                int b = h.depth[v] - h.depth[p];
                int ans = k <= a + 1 ? h.jump(u, k - 1) : h.jump(v, a + b + 1 - k);
                cout << ans << '\n';
            }
        }
        cout << '\n';
    }
}
```

2.2.4 两条树路径的交集

path_intersection(a,b,c,d,depth,lca) 返回路径 a-b 与 c-d 的公共路径，字段 u、v 是交集端点，vertices 是公共顶点数；空交返回 {0,0,0}，单点交的 u=v。点号从 1 开始，depth 数组与 lca 回调必须来自同一棵非空树、同一个根，深度按边数计算，不能传带权根距离。支持数组或 vector 深度表，回调接收两个点并返回 LCA。将 c、d 分别投影到 a-b 得 p、q；投影是 lca(a,b)、lca(a,x)、lca(b,x) 中最深者。若 p≠q，c-d 必经过 p-q；若 p=q，检验 p 是否在 c-d 上即可区分单点交与空交。不依赖顶点编号大小。常数次 LCA 调用、O(1) 额外空间；使用 EulerLCA 时为 O(1)，HLD/倍增时为 O(log n)。公共边数为 max(0,vertices-1)。全小树根与四端点、随机换号树用路径点集独立验证；P3398 完整程序只检查是否相交，端点及数量由本地验证覆盖。

```
struct PathIntersection
{
    int u, v, vertices;
};

// Same nonempty tree and root for depth[] and lca(). Empty result is {0,0,0}.
template <class Depth, class Lca>
PathIntersection
path_intersection(int a, int b, int c, int d, const Depth &depth, Lca lca)
{
    int z = lca(a, b);
    auto project = [&](int x)
    {
        int p = z;
        for (int v : {lca(a, x), lca(b, x)})
            if (depth[v] > depth[p]) p = v;
        return p;
    };
    auto distance = [&](int u, int v)
    {
        return 1LL * depth[u] + depth[v] - 2LL * depth[lca(u, v)];
    };
    int p = project(c), q = project(d);
    if (p == q && distance(c, p) + distance(p, d) != distance(c, d)) return {0, 0, 0};
    return {p, q, int(distance(p, q) + 1)};
}
```

应用题补充：Luogu P3398

题意：树上两条路径是否有公共顶点，n,q≤100000。点号 1..n，EulerLCA 的 depth 与 lca 必须来自同一根，depth 是边数而非带权距离。vertices 非零输出 Y，否则 N；此题只使用是否相交，交集端点和顶点数另由本地路径点集验证。EulerLCA 预处理 O(n log n)，每次查询 O(1)，DFS 递归。

所需模板：EulerLCA（第 15 页，2.2.2 节）、path_intersection（第 23 页，2.2.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3398

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    EulerLCA tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        tree.add(u, v);
    }
    tree.build();
    auto lca = [&](int u, int v)
    {
        return tree.lca(u, v);
    };
    while (q--)
    {
        int a, b, c, d;
        cin >> a >> b >> c >> d;
        auto result = path_intersection(a, b, c, d, tree.depth, lca);
        cout << (result.vertices ? 'Y' : 'N') << '\n';
    }
    return 0;
}
```

2.3 树链剖分

2.3.1 长链剖分与常数时间 K 级祖先

LongChain(n)，1≤n<INT_MAX，点号 1..n；add(u,v) 加入无向树边，调用方保证全部边恰好构成一棵树，不支持环、重边、自环或森林。build(root=1) 以指定根重建，dep[root]=1；kth(u,k) 返回 k 级祖先，k≥0，k=0 返回自身，越过根返回 0。查询前必须 build，加边后必须重建；支持重复 build、换根与值复制。son 按子树最大高度选长儿子，不能换成子树大小；len 含自身。链顶 t 的 down[t] 从 t 向下，up[t][0]=t 且向上越根补 0；成员只读，dfs1/dfs2 仅内部使用。预处理 O(n log(n+1))，空间 O(n log(n+1))，两次 DFS 均递归、栈深 O(n)。查询先跳最高 2 的幂，再在链顶上下表中查一次，32 位字长模型下 O(1)；k=0 必须先处理，不能求 0 的最高位。

```
struct LongChain
{
    int n;
    vector<int> dep, len, son, top;
    vector<vector<int>> g, fa, up, down;

    LongChain(int n) : n(n), g(n + 1)
    {
        assert(1 <= n && n < INT_MAX);
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs1(int u, int p)
    {
        fa[0][u] = p;
        dep[u] = dep[p] + 1;
        for (int j = 1; j < (int)fa.size(); j++)
            fa[j][u] = fa[j - 1][fa[j - 1][u]];
        for (int v : g[u])
        {
            if (v == p) continue;
            dfs1(v, u);
            if (len[v] + 1 > len[u])
            {
                len[u] = len[v] + 1;
                son[u] = v;
            }
        }
    }
```

递归划分长链并填表（接上页同一结构体）：

```
    void dfs2(int u, int t)
    {
        top[u] = t;
        if (u == t)
        {
            down[t].reserve(len[u]);
            up[t].resize(len[u]);
            int x = u;
            for (int &v : up[t])
            {
                v = x;
                x = fa[0][x];
            }
        }
        down[t].push_back(u);
        if (son[u]) dfs2(son[u], t);
        for (int v : g[u])
        {
            if (v != fa[0][u] && v != son[u])
                dfs2(v, v);
        }
    }
```

重建与常数时间查询（接上页同一结构体）：

```
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        dep.assign(n + 1, 0);
        len.assign(n + 1, 1);
        son.assign(n + 1, 0);
        top.assign(n + 1, 0);
        fa.assign(bit_width((unsigned)n), vector<int>(n + 1));
        up.assign(n + 1, {});
        down.assign(n + 1, {});
        dfs1(root, 0);
        dfs2(root, root);
    }

    int kth(int u, int k) const
    {
        assert(1 <= u && u <= n && k >= 0 && !fa.empty());
        if (k >= dep[u]) return 0;
        if (k == 0) return u;
        int j = bit_width((unsigned)k) - 1;
        u = fa[j][u];
        k -= 1 << j;
        int t = top[u];
        int d = dep[u] - dep[t];
        if (k <= d) return down[t][d - k];
        return up[t][k - d];
    }
};
```

为什么一次跳跃就够 先向上跳 p（不超过 k 的最大 2 的幂），到达 u，剩余 r 小于 p。原查询点是 u 下方距离 p 的后代，所以 u 的子树高度至少 p+1。u 所在长链顶 t 的链长至少 dep[u]-dep[t]+p+1，足以覆盖 r；若 r 不超过 u 到 t 的距离就查 down，否则查 up。选择长儿子必须按高度，不能按子树大小。

边界与空间 up[t][0] 和 down[t][0] 都等于 t；up 越过根填 0。先处理 k=0 和越根，避免对 0 求最高位。各长链长度之和为 n，上下表各存 n 个 int；倍增表为 O(n log n)。预处理两次 DFS 都保持递归，栈深可达 n。

模板题使用：P5903 【模板】树上 K 级祖先

题意：按题意用 32 位无符号 xorshift 顺序生成询问；根深度为 1，先生成 x 再生成 k，按 64 位累计 i* 答案的异或；n≤500000，q≤5000000。

所需模板：LongChain（第 25 页，2.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5903

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    uint32_t s;
    cin >> n >> q >> s;
    LongChain tree(n);
    int root = 0;
    for (int u = 1; u <= n; u++)
    {
        int p;
        cin >> p;
        if (p) tree.add(u, p);
        else root = u;
    }
    tree.build(root);
    auto get = [&]()
    {
        s ^= s << 13;
        s ^= s >> 17;
        s ^= s << 5;
        return s;
    };
    int last = 0;
    unsigned long long ans = 0;
    for (int i = 1; i <= q; i++)
    {
        int x = (get() ^ last) % n + 1;
        int k = (get() ^ last) % tree.dep[x];
        last = tree.kth(x, k);
        ans ^= 1ULL * i * last;
    }
    cout << ans << '\n';
}
```

接口用法演示：API 任意根与越根祖先

演示约定：输入 n root q、n-1 条 1-based 树边和 q 组 u k；逐行输出 k 级祖先，k=0 为自身，越根为 0。

所需模板：LongChain（第 25 页，2.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/hld/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, root, q;
    cin >> n >> root >> q;
    LongChain tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        tree.add(u, v);
    }
    tree.build(root);
    while (q--)
    {
        int u, k;
        cin >> u >> k;
        cout << tree.kth(u, k) << '\n';
    }
}
```

2.3.2 重链剖分与 LCA

add(u,v) 建无向树；build(root) 后读取 fa、dep、siz、son、top、dfn、rk。子树对应 [dfn[u],dfn[u]+siz[u]-1]。lca(u,v) 为 O(log n)，path(u,v,work,edge) 分解路径；work(l,r) 仅适用于可交换聚合，边权模式排除 LCA 位置。输入必须是连通无环树，建树 O(n)，两遍递归 DFS；支持改变根后重复 build，dfn 会重新编号。path_ordered(u, v, work, edge) 的 edge 默认为 false；按真实 u 到 v 顺序回调 work(l, r, rev)：1-based 闭区间 l≤r，rev=true 按 r 到 l 访问，否则按 l 到 r。支持非交换聚合，O(log n) 段与额外空间；结合线段树通常每次查询 $O(\log^2 n)$。edge=true 仅删除 LCA 对应位置，仍须自行处理边算子的方向；不自动求逆。u=v 时点模式一次单点回调，边模式无回调；改变根后须重建依赖 dfn 的结构。

```
struct HLD
{
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> fa, dep, siz, son, top, dfn, rk;

    HLD(int n)
        : n(n),
          g(n + 1),
          fa(n + 1),
          dep(n + 1),
          siz(n + 1),
          son(n + 1),
          top(n + 1),
          dfn(n + 1),
          rk(n + 1)
    {
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs1(int u, int p)
    {
        fa[u] = p;
        siz[u] = 1;
        son[u] = 0;
        for (int v : g[u])
        {
            if (v == p) continue;
            dep[v] = dep[u] + 1;
            dfs1(v, u);
            siz[u] += siz[v];
            if (!son[u] || siz[v] > siz[son[u]]) son[u] = v;
        }
    }
```

重编号、LCA 与可交换路径（接上页同一结构体）：

```
    void dfs2(int u, int t)
    {
        top[u] = t;
        dfn[u] = ++timer;
        rk[timer] = u;
        if (son[u]) dfs2(son[u], t);
        for (int v : g[u])
        {
            if (v != fa[u] && v != son[u]) dfs2(v, v);
        }
    }

    // Connected tree only. Recursive DFS; subtree interval is [dfn, dfn+siz-1].
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        timer = 0;
        dep[root] = 0;
        dfs1(root, root);
        dfs2(root, root);
        assert(timer == n);
    }

    int lca(int u, int v) const
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            u = fa[top[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }

    // Commutative operations only; edge=true excludes the LCA's position.
    template <class F> void path(int u, int v, F work, bool edge = false) const
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            work(dfn[top[u]], dfn[u]);
            u = fa[top[u]];
        }
        if (dep[u] > dep[v]) swap(u, v);
        if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v]);
    }
```

按真实方向分解路径（接上页同一结构体）：

```
    // Emit u-to-v order; reverse=true traverses the closed interval from r to l.
    template <class F> void path_ordered(int u, int v, F work, bool edge = false) const
    {
        vector<pair<int, int>> down;
        while (top[u] != top[v])
        {
            if (dep[top[u]] >= dep[top[v]])
            {
                work(dfn[top[u]], dfn[u], true);
                u = fa[top[u]];
            }
            else
            {
                down.emplace_back(dfn[top[v]], dfn[v]);
                v = fa[top[v]];
            }
        }
        if (dep[u] >= dep[v])
        {
            if (dfn[v] + edge <= dfn[u]) work(dfn[v] + edge, dfn[u], true);
        }
        else if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v], false);
        for (auto it = down.rbegin(); it != down.rend(); ++it)
            work(it->first, it->second, false);
    }
};
```

模板题使用：Luogu P3384

题意：给定有根点权树，支持路径加、路径和、子树加、子树和，答案模 mod。先按 dfn 重排初值再构造线段树；path 回调给出 1-based 闭区间，子树区间为 [dfn[u],dfn[u]+siz[u]-1]。

所需模板：HLD（第 30 页，2.3.2 节）、AffineSegTree（第 353 页，F.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3384

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, root, mod;
    cin >> n >> m >> root >> mod;
    vector<long long> values(n + 1);
    for (int u = 1; u <= n; u++) cin >> values[u];
    HLD h(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        h.add(u, v);
    }
    h.build(root);
    vector<long long> ordered(n);
    for (int u = 1; u <= n; u++) ordered[h.dfn[u] - 1] = values[u];
    AffineSegTree seg(ordered, mod);
    while (m--)
    {
        int op, u;
        cin >> op >> u;
        if (op == 1)
        {
            int v;
            long long x;
            cin >> v >> x;
            h.path(u, v, [&](int l, int r) { seg.update(l, r, 1, x); });
        }
        else if (op == 2)
        {
            int v;
            cin >> v;
            long long ans = 0;
            h.path(u, v, [&](int l, int r) { ans = (ans + seg.query(l, r)) % mod; });
            cout << ans << '\n';
        }
        else if (op == 3)
        {
            long long x;
            cin >> x;
            seg.update(h.dfn[u], h.dfn[u] + h.siz[u] - 1, 1, x);
        }
        else
            cout << seg.query(h.dfn[u], h.dfn[u] + h.siz[u] - 1) << '\n';
    }
}
```

模板题使用：Library Checker Vertex Set Path Composite

题意：静态无向树，1≤N,Q≤200000。点函数 f(x)=a*x+b，对 998244353 取模；题面 1≤a,c<mod，0≤b,d,x<mod。0 p c d 改单点函数，1 u v x 按 u 到 v（含两端点）依次代入。题目点号 0-based，传 HLD 时加 1；HLD 闭区间 [l,r] 对应 segtree 半开 [l−1,r)。S 存正向与反向复合，不能把组合顺序反过来或用逆函数代替反向序列。点改 O(log N)，路径 O($\log^2$ N)，空间 O(N)；保留递归 DFS，最大链须足够大栈。

所需模板：HLD（第 30 页，2.3.2 节）、segtree（第 348 页，F.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/vertex_set_path_composite

```
using S = array<long long, 4>;
const int mod = 998244353;

S op(S a, S b)
{
    return {b[0] * a[0] % mod,
            (b[0] * a[1] + b[1]) % mod,
            a[2] * b[2] % mod,
            (a[2] * b[3] + a[3]) % mod};
}

S e()
{
    return {1, 0, 1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n), val(n);
    for (auto &x : a)
    {
        cin >> x[0] >> x[1];
        x[2] = x[0];
        x[3] = x[1];
    }
    HLD h(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        h.add(u + 1, v + 1);
    }
    h.build();
    for (int u = 1; u <= n; u++) val[h.dfn[u] - 1] = a[u - 1];
    segtree<S, op, e> s(val);
```

继续处理操作（接上页同一 main）：

```
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(h.dfn[p + 1] - 1, {c, d, c, d});
        }
        else
        {
            int u, v;
            long long x;
            cin >> u >> v >> x;
            h.path_ordered(u + 1, v + 1, [&](int l, int r, bool rev)
            {
                S z = s.prod(l - 1, r);
                int k = rev ? 2 : 0;
                x = (z[k] * x + z[k + 1]) % mod;
            });
            cout << x << '\n';
        }
    }
}
```

接口用法演示：路径点权增减与点查（kuangbin 3.3.1 源码模型）

演示约定：读到 EOF：n,m,q、n 个初值、m 条树边，要求 m=n−1。I u v x/D u v x 给路径所有点加/减 x，Q u 查点值。1-based，初值保留在 a，Fenwick 仅记录增量差分；HLD 闭区间 [l,r] 在 l 加 x、r+1 减 x，包含 LCA 与两端。演示 n,q≤50000、| 初值 |≤10^12、0≤x≤10^6，全部中间值用 long long。每次更新 $O(\log^2 n)$、点查 O(log n)、空间 O(n)。原 HDU3966 题面未核验。

所需模板：HLD（第 30 页，2.3.2 节）、Fenwick（第 341 页，E.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/kuangbin/ACM-ICPC

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    while (cin >> n >> m >> q)
    {
        vector<long long> a(n + 1);
        for (int u = 1; u <= n; u++) cin >> a[u];
        HLD h(n);
        for (int i = 0; i < m; i++)
        {
            int u, v;
            cin >> u >> v;
            h.add(u, v);
        }
        h.build();
        auto bit = Fenwick<long long>(n);
        while (q--)
        {
            char op;
            int u;
            cin >> op >> u;
            if (op == 'Q') cout << a[u] + bit.sum(h.dfn[u]) << '\n';
            else
            {
                int v;
                long long x;
                cin >> v >> x;
                if (op == 'D') x = -x;
                h.path(u, v, [&](int l, int r)
                {
                    bit.add(l, x);
                    bit.add(r + 1, -x);
                });
            }
        }
    }
}
```

应用题补充：SPOJ QTREE / 单边修改与路径最大边权

题意：多测 t≤20、n≤10000。输入边按 1..n−1 编号；CHANGE i x 修改第 i 条边，QUERY u v 查路径最大边权，DONE 结束本组。把边存到较深端点的 dfn−1；path 的 edge=true 排除 LCA，闭 [l,r] 转为 segtree 的半开 [l−1,r)。空路径约定输出 0，非空路径支持 signed64 负权及最小值；题面仅写 c≤10^6，未写修改值下界。预处理 O(n)、修改 O(log n)、查询 O($\log^2$ n)、空间 O(n)，递归 DFS。

所需模板：HLD（第 30 页，2.3.2 节）、segtree（第 348 页，F.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.spoj.com/problems/QTREE/

```
auto op(long long x, long long y)
{
    return max(x, y);
}

auto e()
{
    return numeric_limits<long long>::lowest();
}
```

使用入口（接上页配置函数）：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        HLD h(n);
        vector<int> a(n), b(n), child(n);
        vector<long long> w(n), base(n, e());
        for (int i = 1; i < n; i++)
        {
            cin >> a[i] >> b[i] >> w[i];
            h.add(a[i], b[i]);
        }
        h.build();
        for (int i = 1; i < n; i++)
        {
            child[i] = h.dep[a[i]] > h.dep[b[i]] ? a[i] : b[i];
            base[h.dfn[child[i]] - 1] = w[i];
        }
        auto seg = segtree<long long, op, e>(base);
        string cmd;
        while (cin >> cmd && cmd != "DONE")
        {
            int u;
            cin >> u;
            if (cmd == "CHANGE")
            {
                long long x;
                cin >> x;
                seg.set(h.dfn[child[u]] - 1, x);
            }
            else
            {
                int v;
                cin >> v;
                long long ans = e();
                h.path(u, v, [&](int l, int r)
                {
                    ans = max(ans, seg.prod(l - 1, r));
                }, true);
                cout << (u == v ? 0 : ans) << '\n';
            }
        }
    }
}
```

接口用法演示：树边翻转与奇路径计数（kuangbin HDU5039 模型）

演示约定：来源协议：T 组，每组 n、n 个不同名字、n-1 条 (名字, 名字,0/1) 边、q 条操作；Q 开头查询，其余操作接 1-based 输入边号并翻转该边。输出 Case #k: 及每次有向端点对数量，故答案 2*zero*one；无向点对则不乘 2。固定根 1，根路径异或值不同的端点间为奇路径；翻边即翻较深端点整棵子树，HLD 闭区间转 lazy_segtree 半开 [dfn-1,dfn-1+siz)。树上预处理 O(n)，名字 map 映射另计字符串比较和字典查询；改边 O(log n)，查询 O(1)，空间 O(n)，DFS 递归。官方 HDU 题面和限制未核验，只作来源模型/API 用法。

所需模板：HLD（第 30 页，2.3.2 节）、lazy_segtree（第 343 页，F.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://acm.hdu.edu.cn/showproblem.php?pid=5039

```
using S = pair<int, int>;

S op(S a, S b)
{
    return {a.first + b.first, a.second + b.second};
}

S e()
{
    return {0, 0};
}

S mapping(bool flip, S a)
{
    if (flip) swap(a.first, a.second);
    return a;
}

bool composition(bool f, bool g)
{
    return f ^ g;
}

bool id()
{
    return false;
}
```

使用入口（接上页配置函数）：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    for (int tc = 1; tc <= tests; tc++)
    {
        int n;
        cin >> n;
        HLD h(n);
        map<string, int> names;
        for (int u = 1; u <= n; u++)
        {
            string name;
            cin >> name;
            names[name] = u;
        }
        vector<int> u(n), v(n), bit(n), child(n), parity(n + 1);
        for (int i = 1; i < n; i++)
        {
            string a, b;
            cin >> a >> b >> bit[i];
            u[i] = names.at(a);
            v[i] = names.at(b);
            h.add(u[i], v[i]);
        }
        h.build();
        for (int i = 1; i < n; i++)
        {
            child[i] = h.fa[u[i]] == v[i] ? u[i] : v[i];
            parity[child[i]] = bit[i];
        }
        vector<S> values(n);
        for (int i = 1; i <= n; i++)
        {
            int x = h.rk[i];
            if (x != 1) parity[x] ^= parity[h.fa[x]];
            values[i - 1] = {parity[x] == 0, parity[x] == 1};
        }
        lazy_segtree<S, bool, op, e, mapping, composition, id> seg(values);
```

继续处理操作（接上页同一 main）：

```
        int q;
        cin >> q;
        cout << "Case #" << tc << ":\n";
        while (q--)
        {
            string command;
            cin >> command;
            if (command[0] == 'Q')
            {
                auto [zero, one] = seg.all();
                cout << 2LL * zero * one << '\n';
            }
            else
            {
                int edge;
                cin >> edge;
                int x = child[edge];
                int l = h.dfn[x] - 1;
                seg.apply(l, l + h.siz[x], true);
            }
        }
    }
}
```

2.4 树上启发式合并

2.4.1 树上启发式合并：子树众数颜色和

SubtreeColors(n)，点号 1..n，add(u,v) 建无向树；要求图连通且无环，n=0 可空运行。run(colors,root=1) 传入长度 n+1 的 int 颜色数组，colors[0] 忽略。answer[u] 为子树内所有最高频颜色的原始编号之和，max_freq[u] 为该频次；支持负颜色与稀疏编号，内部离散化，和用 long long。重复 run 可更换颜色或根。重儿子信息保留，轻儿子通过 DFS 序区间加入；清空整棵临时子树后再重置最大值，不尝试逐次删除时维护众数。O(n log n) 时间、O(n) 空间，全部 DFS 递归。insert 是内部增量统计，可按题目修改统计量，但相应清空规则也必须调整。

```
struct SubtreeColors
{
    int n, best = 0;
    long long sum = 0;
    vector<vector<int>> g;
    vector<int> siz, heavy, tin, tout, order, color, value, freq;
    vector<int> max_freq;
    vector<long long> answer;

    SubtreeColors(int n) : n(n), g(n + 1) {}

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void prepare(int u, int p)
    {
        siz[u] = 1;
        tin[u] = order.size();
        order.push_back(u);
        for (int v : g[u])
        {
            if (v == p) continue;
            prepare(v, u);
            siz[u] += siz[v];
            if (siz[v] > siz[heavy[u]]) heavy[u] = v;
        }
        tout[u] = (int)order.size() - 1;
    }

    void insert(int u)
    {
        int c = color[u];
        freq[c]++;
        if (freq[c] > best)
        {
            best = freq[c];
            sum = value[c];
        }
        else if (freq[c] == best)
            sum += value[c];
    }
```

启发式合并与运行入口（接上页同一结构体）：

```
    void solve(int u, int p, bool keep)
    {
        for (int v : g[u])
        {
            if (v != p && v != heavy[u]) solve(v, u, false);
        }
        if (heavy[u]) solve(heavy[u], u, true);
        for (int v : g[u])
        {
            if (v == p || v == heavy[u]) continue;
            for (int i = tin[v]; i <= tout[v]; i++) insert(order[i]);
        }
        insert(u);
        max_freq[u] = best;
        answer[u] = sum;
        if (!keep)
        {
            for (int i = tin[u]; i <= tout[u]; i++) freq[color[order[i]]]--;
            best = 0;
            sum = 0;
        }
    }

    // A connected tree, colors[1..n]. Sum all colors tied for maximum frequency.
    void run(const vector<int> &colors, int root = 1)
    {
        assert((int)colors.size() == n + 1);
        siz.assign(n + 1, 0);
        heavy = siz;
        tin = siz;
        tout = siz;
        color = siz;
        max_freq = siz;
        answer.assign(n + 1, 0);
        order.clear();
        value.assign(colors.begin() + 1, colors.end());
        sort(value.begin(), value.end());
        value.erase(unique(value.begin(), value.end()), value.end());
        freq.assign(value.size(), 0);
        best = 0;
        sum = 0;
        if (!n) return;
        assert(1 <= root && root <= n);
        for (int u = 1; u <= n; u++)
            color[u] =
                lower_bound(value.begin(), value.end(), colors[u]) - value.begin();
        prepare(root, 0);
        solve(root, 0, false);
    }
};
```

应用题补充：Codeforces 600E

题意：以 1 为根，对每个子树输出出现次数最多的所有颜色编号之和；并列颜色都计入。颜色数组 1-based，run 后直接读 answer，和用 long long。n 至 10 万、颜色在 1..n；比赛应用，不替代正式模板题。

所需模板：SubtreeColors（第 42 页，2.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/600/E

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    std::cin >> n;
    std::vector<int> colors(n + 1);
    for (int u = 1; u <= n; u++) std::cin >> colors[u];
    SubtreeColors tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        std::cin >> u >> v;
        tree.add(u, v);
    }
    tree.run(colors);
    for (int u = 1; u <= n; u++) std::cout << tree.answer[u] << (u == n ? '\n' : ' ');
}
```

2.5 虚树

2.5.1 虚树：关键点压缩与路径信息

VirtualTree(n)，n≥1，点号 1..n；add(u,v,w=1) 建非负权连通树，无环和重边。prepare(root=1) 递归预处理，可换根重建；根到任一点距离须在 long long 范围内。build(keys) 自动去重并补齐必要 LCA，返回 vertices 和 edges；全部编号沿用原树，vertices 按 DFS 序排列，非空时首项即虚树根。edges 为 (from,to,distance,minimum)，分别是祖先到后代的虚边、压缩路径长度和路径最小边权。空关键点返回空结果，单点返回单节点；原树根不会强制加入，需要时由调用者加入 keys。边按后代 DFS 序排列，可逆序做自底向上的 DP。不分配或清空 n 个查询邻接表；每次仅返回 O(k) 结果。预处理 O(n log n) 时间空间，单次 O(k log k+k log n)。lca、ancestor 也可直接使用，要求有效点号且已经 prepare。构造用的祖先链栈属于虚树算法，原树 DFS 保持递归。

```
struct VirtualTree
{
    using ll = long long;

    struct Edge
    {
        int from, to;
        ll distance, minimum;
    };

    struct Result
    {
        vector<int> vertices;
        vector<Edge> edges;
    };

    int n, log = 1, timer = 0;
    vector<vector<pair<int, ll>>> g;
    vector<vector<int>> up;
    vector<vector<ll>> mn;
    vector<int> depth, tin, tout;
    vector<ll> dist;

    VirtualTree(int n) : n(n), g(n + 1) {}

    void add(int u, int v, ll w = 1)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
```

预处理与最近公共祖先（接上页同一结构体）：

```
    void dfs(int u, int p)
    {
        tin[u] = ++timer;
        up[0][u] = p;
        for (int j = 1; j < log; j++)
        {
            up[j][u] = up[j - 1][up[j - 1][u]];
            mn[j][u] = min(mn[j - 1][u], mn[j - 1][up[j - 1][u]]);
        }
        for (auto [v, w] : g[u])
        {
            if (v == p) continue;
            assert(dist[u] <= numeric_limits<ll>::max() - w);
            depth[v] = depth[u] + 1;
            dist[v] = dist[u] + w;
            mn[0][v] = w;
            dfs(v, u);
        }
        tout[u] = timer;
    }

    void prepare(int root = 1)
    {
        assert(1 <= root && root <= n);
        log = 1;
        while ((1LL << log) <= n) log++;
        up.assign(log, vector<int>(n + 1));
        mn.assign(log, vector<ll>(n + 1, numeric_limits<ll>::max()));
        depth.assign(n + 1, 0);
        tin = depth;
        tout = depth;
        dist.assign(n + 1, 0);
        timer = 0;
        dfs(root, root);
    }

    bool ancestor(int u, int v) const { return tin[u] <= tin[v] && tout[v] <= tout[u]; }

    int lca(int u, int v) const
    {
        if (ancestor(u, v)) return u;
        if (ancestor(v, u)) return v;
        for (int j = log - 1; j >= 0; j--)
        {
            if (!ancestor(up[j][u], v)) u = up[j][u];
        }
        return up[0][u];
    }
```

压缩路径与虚树构造（接上页同一结构体）：

```
    Edge path(int u, int v) const
    {
        assert(u != v && ancestor(u, v));
        ll minimum = numeric_limits<ll>::max();
        int x = v, steps = depth[v] - depth[u];
        for (int j = 0; j < log; j++)
        {
            if (steps >> j & 1)
            {
                minimum = min(minimum, mn[j][x]);
                x = up[j][x];
            }
        }
        return {u, v, dist[v] - dist[u], minimum};
    }

    // Uses original vertex IDs. Empty input gives an empty result.
    Result build(vector<int> keys) const
    {
        if (keys.empty()) return {};
        for (int u : keys) assert(1 <= u && u <= n);
        auto cmp = [&](int u, int v)
        {
            return tin[u] < tin[v];
        };
        sort(keys.begin(), keys.end(), cmp);
        keys.erase(unique(keys.begin(), keys.end()), keys.end());
        int k = keys.size();
        for (int i = 1; i < k; i++) keys.push_back(lca(keys[i - 1], keys[i]));
        sort(keys.begin(), keys.end(), cmp);
        keys.erase(unique(keys.begin(), keys.end()), keys.end());
        Result result{keys, {}};
        vector<int> st;
        for (int u : keys)
        {
            while (!st.empty() && !ancestor(st.back(), u)) st.pop_back();
            if (!st.empty()) result.edges.push_back(path(st.back(), u));
            st.push_back(u);
        }
        return result;
    }
};
```

应用题补充：Luogu P2495 [SDOI2011] 消耗战

题意：每次给若干非根关键点，求删边使它们都与根 1 断开的最小代价。显式加入根构造虚树，逆序遍历虚边，使用路径最小边权与子树代价取小；只清空本次顶点和标记。n 至 25 万、询问及关键点总数至 50 万。省选应用，正式非比赛模板题映射仍待补。

所需模板：VirtualTree（第 45 页，2.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2495

```cpp
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    std::cin >> n;
    VirtualTree tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long w;
        std::cin >> u >> v >> w;
        tree.add(u, v, w);
    }
    tree.prepare();
    std::vector<int> marked(n + 1);
    std::vector<long long> dp(n + 1);
    int q;
    std::cin >> q;
    while (q--)
    {
        int k;
        std::cin >> k;
        std::vector<int> keys(k);
        for (int &u : keys)
        {
            std::cin >> u;
            marked[u] = 1;
        }
        keys.push_back(1);
        auto result = tree.build(keys);
        for (int u : result.vertices) dp[u] = 0;
        for (int i = (int)result.edges.size() - 1; i >= 0; i--)
        {
            auto e = result.edges[i];
            long long cost = marked[e.to] ? e.minimum : std::min(e.minimum, dp[e.to]);
            dp[e.from] += cost;
        }
        std::cout << dp[1] << '\n';
        for (int u : keys) marked[u] = 0;
    }
}
```

2.6 树分治

2.6.1 点分治：树上距离点对计数

CentroidPairs(n)，点号 1..n；add(u,v,w) 建树，允许森林但禁止环、重边、自环，边权非负；所有简单路径长度须能用 long long 表示。build() 预处理，可重复调用；改变边后须重建。count_leq(k) 统计距离 ≤k 的无序不同顶点对，count_exact(k) 统计距离恰为 k 的点对；负 k 返回 0，零距离只统计不同顶点，跨连通块不配对。parent[c] 是点分树父节点，根为 0。all[c] 为该层分治块到重心的距离，含重心距离 0；part[c] 为去掉重心后各子块的距离。预处理 $O(n \log^2 n)$，空间 $O(n \log n)$，每次计数 $O(n \log n)$；固定树上多次查询可以复用排序结果。DFS 递归，距离和比较采用减法避免溢出。不是动态点分树更新模板。

```
struct CentroidPairs
{
    using ll = long long;
    int n;
    vector<vector<pair<int, ll>>> g;
    vector<int> siz, removed, parent;
    vector<vector<ll>> all;
    vector<vector<vector<ll>>> part;

    CentroidPairs(int n)
        : n(n),
          g(n + 1),
          siz(n + 1),
          removed(n + 1),
          parent(n + 1),
          all(n + 1),
          part(n + 1)
    {
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }

    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }
```

重心与分解（接上页同一结构体）：

```
    int centroid(int u, int p, int total)
    {
        for (auto [v, w] : g[u])
        {
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        }
        return u;
    }

    void collect(int u, int p, ll d, vector<ll> &out)
    {
        out.push_back(d);
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            assert(d <= numeric_limits<ll>::max() - w);
            collect(v, u, d + w, out);
        }
    }

    void decompose(int entry, int p)
    {
        size_dfs(entry, 0);
        int c = centroid(entry, 0, siz[entry]);
        removed[c] = 1;
        parent[c] = p;
        all[c] = {0};
        for (auto [v, w] : g[c])
        {
            if (removed[v]) continue;
            part[c].push_back({});
            auto &d = part[c].back();
            collect(v, c, w, d);
            sort(d.begin(), d.end());
            all[c].insert(all[c].end(), d.begin(), d.end());
        }
        sort(all[c].begin(), all[c].end());
        for (auto [v, w] : g[c])
        {
            if (!removed[v]) decompose(v, c);
        }
    }
```

重建与距离计数（接上页同一结构体）：

```
    // Input must be a forest. Rebuild after changing edges.
    void build()
    {
        fill(removed.begin(), removed.end(), 0);
        for (int u = 1; u <= n; u++)
        {
            all[u].clear();
            part[u].clear();
        }
        for (int u = 1; u <= n; u++)
        {
            if (!removed[u]) decompose(u, 0);
        }
    }

    ll pairs(const vector<ll> &d, ll k) const
    {
        int l = 0, r = (int)d.size() - 1;
        ll answer = 0;
        while (l < r)
        {
            if (d[l] <= k - d[r])
            {
                answer += r - l;
                l++;
            }
            else
                r--;
        }
        return answer;
    }

    // Unordered pairs of distinct connected vertices. Nonnegative edge weights.
    ll count_leq(ll k) const
    {
        if (k < 0) return 0;
        ll answer = 0;
        for (int c = 1; c <= n; c++)
        {
            answer += pairs(all[c], k);
            for (const auto &d : part[c]) answer -= pairs(d, k);
        }
        return answer;
    }

    ll count_exact(ll k) const
    {
        if (k < 0) return 0;
        return count_leq(k) - (k ? count_leq(k - 1) : 0);
    }
};
```

模板题使用：Luogu P3806

题意：正权树上查询距离为 k 的不同顶点对是否存在。先 build 一次，再用 count_exact 的结果是否非零输出 AYE/NAY。n 至 1 万、询问至 100、边权至 1 万；该程序查询存在性，底层计数另支持零权和森林。

所需模板：CentroidPairs（第 49 页，2.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3806

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    CentroidPairs tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long w;
        std::cin >> u >> v >> w;
        tree.add(u, v, w);
    }
    tree.build();
    while (m--)
    {
        long long k;
        std::cin >> k;
        std::cout << (tree.count_exact(k) ? "AYE" : "NAY") << '\n';
    }
}
```

2.6.2 树上新增市场的支配范围

固定非负边权树，n≥1，点号 0..n-1；add(u,v,w)，w 为非负 long long。solve(existing) 返回各点新建市场后支配的顶点数，existing 长度 n、0/1 表示原有市场；原有市场点返回 0，不允许重复建。归属按 (距离, 市场点号) 字典序，距离相同时编号小者胜；零边合法。无原有市场时各点答案 n；全是市场时全 0。可在同一树上换 existing 重复 solve，每次重算。int128 保存距离和比较阈值；最近市场多源 Dijkstra，递归点分治总集合加、同分支减。O(n $\log^2$ n) 时间、O(n) 辅助空间，递归深度最坏 O(n)。返回完整方案计数，取 max 即最佳新增市场；内部字段和辅助方法不外改。不支持拓扑动态更新。

```
struct TreeMarket
{
    using I = __int128_t;
    using Key = pair<I, int>;
    int n;
    vector<vector<pair<int, long long>>> g;
    vector<int> size, removed, market, answer;
    vector<Key> best;

    TreeMarket(int n) : n(n), g(n), size(n), removed(n), answer(n)
    {
    }

    void add(int u, int v, long long w)
    {
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }

    void size_dfs(int u, int p)
    {
        size[u] = 1;
        for (auto [v, w] : g[u])
            if (v != p && !removed[v])
            {
                size_dfs(v, u);
                size[u] += size[v];
            }
    }

    int centroid(int u, int p, int total)
    {
        for (auto [v, w] : g[u])
            if (v != p && !removed[v] && size[v] > total / 2)
                return centroid(v, u, total);
        return u;
    }
```

接上页同一结构体：

```
    void collect(int u, int p, I d, vector<pair<int, I>> &nodes)
    {
        nodes.push_back({u, d});
        for (auto [v, w] : g[u])
            if (v != p && !removed[v]) collect(v, u, d + w, nodes);
    }

    void count(const vector<pair<int, I>> &nodes, int sign)
    {
        vector<Key> keys;
        for (auto [u, d] : nodes) keys.push_back({best[u].first - d, best[u].second});
        sort(keys.begin(), keys.end());
        for (auto [u, d] : nodes)
            if (!market[u])
            {
                int pos = upper_bound(keys.begin(), keys.end(), Key{d, u}) - keys.begin();
                answer[u] += sign * ((int)keys.size() - pos);
            }
    }

    void decompose(int entry)
    {
        size_dfs(entry, -1);
        int c = centroid(entry, -1, size[entry]);
        vector<pair<int, I>> nodes;
        collect(c, -1, 0, nodes);
        count(nodes, 1);
        removed[c] = 1;
        for (auto [v, w] : g[c])
            if (!removed[v])
            {
                nodes.clear();
                collect(v, c, w, nodes);
                count(nodes, -1);
                decompose(v);
            }
    }
```

接上页同一结构体：

```
    // Existing markets get answer 0. No existing market: every site wins n.
    vector<int> solve(const vector<int> &existing)
    {
        market = existing;
        fill(answer.begin(), answer.end(), 0);
        fill(removed.begin(), removed.end(), 0);
        best.assign(n, {I(1) << 120, n});
        using State = tuple<I, int, int>;
        priority_queue<State, vector<State>, greater<State>> q;
        for (int u = 0; u < n; u++)
            if (market[u])
            {
                best[u] = {0, u};
                q.push({0, u, u});
            }
        if (q.empty())
        {
            fill(answer.begin(), answer.end(), n);
            return answer;
        }
        while (!q.empty())
        {
            auto [d, id, u] = q.top();
            q.pop();
            if (best[u] != Key{d, id}) continue;
            for (auto [v, w] : g[u])
                if (Key{d + w, id} < best[v])
                {
                    best[v] = {d + w, id};
                    q.push({d + w, id, v});
                }
        }
        decompose(0);
        return answer;
    }
};
```

接口用法演示：树上新增市场（来源模型接口演示）

演示约定：按 kuangbin 来源模型：每组输入 n、n-1 条 1-based 非负边权树边和 n 个 0/1 市场标记，读到 EOF。新增市场只能选原来没有市场的点，按距离、市场编号字典序分配顶点，输出最多可支配的顶点数；无候选点输出 0，无原有市场输出 n。调用时点号减一，编号相对顺序保持。官方 HDU5016 题面和资源限制尚未核验，此处只是来源模型/API 演示，不登记正式题目 AC。

所需模板：TreeMarket（第 53 页，2.6.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://acm.hdu.edu.cn/showproblem.php?pid=5016

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    while (cin >> n)
    {
        TreeMarket tree(n);
        for (int i = 1; i < n; i++)
        {
            int u, v;
            long long w;
            cin >> u >> v >> w;
            tree.add(u - 1, v - 1, w);
        }
        vector<int> existing(n);
        for (int &x : existing) cin >> x;
        auto answer = tree.solve(existing);
        cout << *max_element(answer.begin(), answer.end()) << '\n';
    }
}
```

2.7 动态树分治

2.7.1 动态点分树：点权赋值与距离球求和

代码依赖：Fenwick（第 341 页，E.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

构造 CentroidSum(n)，T 默认为 long long；n≥1，输入是固定连通单位边树。点号 0..n-1；先 add(u,v)，再 build(values)。set(u,x) 是点权赋值；query(u,k) 返回到 u 距离 ≤k 的全部点权和，含 u 自身；负半径返回 0，半径用 long long。build 可在同一拓扑上重建并替换全部权值，建树后不再 add 边，不提供 link/cut。每点保存重心祖先、距离和分支，使用总距离桶减同支桶容斥；距离 d 对应 Fenwick 下标 d+1。线性建每层 Fenwick，构造/空间 O(n log n)，修改/查询 O($\log^2$ n)。T 支持零值及加减，所有桶和、赋值差与容斥中间值均须可表示；可用负权，不使用 kth。内部图、路径、桶和辅助函数不能外部改写。原树 DFS 递归深度最坏 O(n)，不等于重心层数；最大链测试提供 256MiB 栈，仍需核对目标评测栈限制。

```
template <class T = long long> struct CentroidSum
{
    struct Record
    {
        int centroid, distance, branch;
    };
    int n;
    vector<vector<int>> g;
    vector<int> siz, removed;
    vector<T> value;
    vector<vector<Record>> path;
    vector<Fenwick<T>> all;
    vector<vector<Fenwick<T>>> part;

    CentroidSum(int n)
        : n(n), g(n), siz(n), removed(n), value(n), path(n), all(n), part(n)
    {
        assert(n > 0);
    }

    void add(int u, int v)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }
```

子树大小、重心与距离记录（接上页同一结构体）：

```
    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (int v : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }

    int centroid(int u, int p, int total)
    {
        for (int v : g[u])
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        return u;
    }

    void collect(int u, int p, int d, int c, int b, vector<T> &bucket)
    {
        path[u].push_back({c, d, b});
        if ((int)bucket.size() <= d + 1) bucket.resize(d + 2);
        bucket[d + 1] += value[u];
        for (int v : g[u])
            if (v != p && !removed[v]) collect(v, u, d + 1, c, b, bucket);
    }
```

线性构建距离桶和递归分解（接上页同一结构体）：

```
    // Raw distance d is stored at 1-based Fenwick index d+1.
    void make_bit(Fenwick<T> &bit, vector<T> bucket)
    {
        bit.n = (int)bucket.size() - 1;
        bit.a = move(bucket);
        for (int i = 1; i <= bit.n; i++)
            if (i + (i & -i) <= bit.n) bit.a[i + (i & -i)] += bit.a[i];
    }

    void decompose(int entry)
    {
        size_dfs(entry, -1);
        int c = centroid(entry, -1, siz[entry]);
        removed[c] = 1;
        path[c].push_back({c, 0, -1});
        vector<T> total{T{}, value[c]};
        for (int v : g[c])
        {
            if (removed[v]) continue;
            vector<T> bucket(1);
            collect(v, c, 1, c, (int)part[c].size(), bucket);
            if (total.size() < bucket.size()) total.resize(bucket.size());
            for (int d = 1; d < (int)bucket.size(); d++) total[d] += bucket[d];
            part[c].emplace_back();
            make_bit(part[c].back(), move(bucket));
        }
        make_bit(all[c], move(total));
        for (int v : g[c])
            if (!removed[v]) decompose(v);
    }
```

重建、点权赋值与距离球求和（接上页同一结构体）：

```
    // Rebuild resets all values and indices; do not change the tree after building.
    void build(const vector<T> &values)
    {
        assert((int)values.size() == n);
        value = values;
        fill(removed.begin(), removed.end(), 0);
        for (int u = 0; u < n; u++)
        {
            path[u].clear();
            part[u].clear();
        }
        decompose(0);
    }

    void set(int u, T x)
    {
        assert(0 <= u && u < n);
        T delta = x - value[u];
        value[u] = x;
        for (auto [c, d, b] : path[u])
        {
            all[c].add(d + 1, delta);
            if (b != -1) part[c][b].add(d + 1, delta);
        }
    }

    T prefix(const Fenwick<T> &bit, long long radius) const
    {
        if (radius < 0) return T{};
        return bit.sum(radius >= bit.n ? bit.n : (int)radius + 1);
    }

    T query(int u, long long radius) const
    {
        assert(0 <= u && u < n);
        T answer{};
        if (radius < 0) return answer;
        for (auto [c, d, b] : path[u])
        {
            answer += prefix(all[c], radius - d);
            if (b != -1) answer -= prefix(part[c][b], radius - d);
        }
        return answer;
    }
};
```

模板题使用：Luogu P6329 【模板】点分树 / 震波

题意：固定单位边树上修改点权，查询到中心距离不超过 k 的权值和。操作 0 查询，操作 1 赋值；每次先将 x 及 y/k 与最近一次查询答案异或，再把 1-based 顶点 x 减一。更新不改变 last，操作类型不异或。n,m≤100000，点权/新值在 1..10000，解码后 k 在 0..n-1。构造后先 build；原树递归 DFS 需要足够栈空间。

所需模板：Fenwick（第 341 页，E.1 节）、CentroidSum（第 57 页，2.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6329

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> value(n);
    for (auto &x : value) std::cin >> x;
    CentroidSum tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        std::cin >> u >> v;
        tree.add(u - 1, v - 1);
    }
    tree.build(value);
    long long last = 0;
    while (m--)
    {
        int op;
        long long x, y;
        std::cin >> op >> x >> y;
        x ^= last;
        y ^= last;
        if (op == 0)
        {
            last = tree.query((int)x - 1, y);
            std::cout << last << '\n';
        }
        else
            tree.set((int)x - 1, y);
    }
}
```

2.7.2 动态点分树：最近活动点与点号方案

CentroidNearest(n)，n≥1，点号 0..n-1；先 add(u,v,w=1) 建固定连通树，禁止环、重边、自环。边权非负，所有原树简单路径长度须能用 long long 表示。build() 清空活动集，可在同一拓扑上重建，建树后不再 add。set(u,on) 启用或停用点，重复设置无影响；query(u) 返回 optional<pair<long long,int»，即最近距离和同距离中最小点号，空集返回 nullopt，距离 LLONG_MAX 合法。active[u] 只读，可用 set(u,!active[u]) 翻转；其他成员均为内部数据，不直接修改。每个重心用 set 维护 (距离, 原点号)，删除时精确到点号。查询跳过溢出的重心绕路距离；最优点与查询点的某个公共重心在原路径上，因此仍能取得精确答案。构建 O(n log n)，单次更新 $O(\log^2 n)$，查询 O(log n)，空间 O(n log n)。所有 DFS 递归，原树最大递归深度 O(n)。不支持负边权或动态拓扑。

```
struct CentroidNearest
{
    using ll = long long;
    int n;
    bool built = false;
    vector<vector<pair<int, ll>>> g, path;
    vector<int> siz, removed, active;
    vector<std::set<pair<ll, int>>> bag;

    CentroidNearest(int n)
        : n(n), g(n), path(n), siz(n), removed(n), active(n), bag(n)
    {
        assert(n > 0);
    }

    void add(int u, int v, ll w = 1)
    {
        assert(!built && 0 <= u && u < n && 0 <= v && v < n);
        assert(u != v && w >= 0);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }

    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }

    int centroid(int u, int p, int total)
    {
        for (auto [v, w] : g[u])
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        return u;
    }
```

重心距离与递归分解（接上页同一结构体）：

```
    void collect(int u, int p, ll d, int c)
    {
        path[u].push_back({c, d});
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            assert(d <= LLONG_MAX - w);
            collect(v, u, d + w, c);
        }
    }

    void decompose(int entry)
    {
        size_dfs(entry, -1);
        int c = centroid(entry, -1, siz[entry]);
        removed[c] = 1;
        collect(c, -1, 0, c);
        for (auto [v, w] : g[c])
            if (!removed[v]) decompose(v);
    }
```

清空重建、活动点增删与最近点查询（接上页同一结构体）：

```
    // Fixed connected nonnegative-weight tree; rebuild clears all active points.
    void build()
    {
        fill(removed.begin(), removed.end(), 0);
        fill(active.begin(), active.end(), 0);
        for (int u = 0; u < n; u++)
        {
            path[u].clear();
            bag[u].clear();
        }
        decompose(0);
        built = true;
    }

    void set(int u, bool on)
    {
        assert(built && 0 <= u && u < n);
        if (active[u] == on) return;
        active[u] = on;
        for (auto [c, d] : path[u])
        {
            if (on) bag[c].insert({d, u});
            else bag[c].erase({d, u});
        }
    }

    // Minimum (distance, vertex ID); nullopt means no active point.
    optional<pair<ll, int>> query(int u) const
    {
        assert(built && 0 <= u && u < n);
        optional<pair<ll, int>> ans;
        for (auto [c, d] : path[u])
        {
            if (bag[c].empty()) continue;
            auto [x, v] = *bag[c].begin();
            // A detour may overflow even when every simple path fits.
            if (x > LLONG_MAX - d) continue;
            pair<ll, int> now{d + x, v};
            if (!ans || now < *ans) ans = now;
        }
        return ans;
    }
};
```

应用题补充：SPOJ QTREE5 / Query on a tree V（Luogu SP2939）

题意：N,Q≤100000，单位边树点号 1..N，初始全黑；先 N 和 N-1 条边，再 Q 条操作。0 u 翻转黑白，1 u 输出最近白点距离（可为自身），没有白点输出-1。内部转换为 0-based，白点对应活动点。原站注明 modified from ZJOI07，按应用用法收录；非独立非比赛模板题。

所需模板：CentroidNearest（第 62 页，2.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.spoj.com/problems/QTREE5/

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    std::cin >> n;
    CentroidNearest tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        std::cin >> u >> v;
        tree.add(u - 1, v - 1);
    }
    tree.build();
    int q;
    std::cin >> q;
    while (q--)
    {
        int op, u;
        std::cin >> op >> u;
        --u;
        if (op == 0)
            tree.set(u, !tree.active[u]);
        else
        {
            auto ans = tree.query(u);
            std::cout << (ans ? ans->first : -1) << '\n';
        }
    }
}
```

2.7.3 动态点分树：有符号边权活动点直径与端点

CentroidDiameter(n)，固定连通无向树、0-based 点号，n>0 且 2n 不溢出 int。add(u,v,w) 仅在首次 build 前；允许负边权，所有简单路径权值和必须能用 long long 表示。build() 递归分解并清空活动集，可反复重建；set(u,on) 幂等启停点。query() 返回 optional<tuple<long long,int,int»，内容为最大距离与任意一对达到最优值的活动端点，允许两端相同，所以非空集答案至少 0；空集 nullopt。各重心按不同分支取最大距离，只合并跨分支真实路径，不能套用非负权直径端点摘要；并列不承诺最小编号。空集预处理 O(n log n)、set O($\log^2$ n)、query O(1)，空间 O(n log n)；逐点激活全树 O(n $\log^2$ n)。支持值拷贝；active 只读用于反色，其余字段与 sum/size_dfs/centroid/collect/decompose/candidate 为内部状态。DFS 保持递归，树输入合法性与足够栈空间由调用方保证。

```
struct CentroidDiameter
{
    using ll = long long;
    using Item = tuple<ll, int, int>;
    using Best = tuple<ll, int, int, int>;
    struct Entry
    {
        int c, b;
        ll d;
    };
    int n;
    bool built = false;
    vector<vector<pair<int, ll>>> g;
    vector<vector<Entry>> path;
    vector<int> siz, removed, active;
    vector<std::set<pair<ll, int>>> bag;
    vector<std::set<Item>> top;
    std::set<Best> best;
    std::set<int> live;

    CentroidDiameter(int n)
        : n(n), g(n), path(n), siz(n), removed(n), active(n), top(n)
    {
        assert(0 < n && n <= INT_MAX / 2);
    }

    ll sum(ll a, ll b) const
    {
        assert(b >= 0 ? a <= LLONG_MAX - b : a >= LLONG_MIN - b);
        return a + b;
    }

    void add(int u, int v, ll w = 1)
    {
        assert(!built && 0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
```

递归分解与不同分支的距离（接上页同一结构体）：

```
    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }

    int centroid(int u, int p, int total)
    {
        for (auto [v, w] : g[u])
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        return u;
    }

    void collect(int u, int p, ll d, int c, int b)
    {
        path[u].push_back({c, b, d});
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            collect(v, u, sum(d, w), c, b);
        }
    }

    void decompose(int entry)
    {
        size_dfs(entry, -1);
        int c = centroid(entry, -1, siz[entry]);
        removed[c] = 1;
        path[c].push_back({c, (int)bag.size(), 0});
        bag.emplace_back();
        for (auto [v, w] : g[c])
        {
            if (removed[v]) continue;
            int b = bag.size();
            bag.emplace_back();
            collect(v, c, w, c, b);
        }
        for (auto [v, w] : g[c])
            if (!removed[v]) decompose(v);
    }
```

清空重建与重心候选（接上页同一结构体）：

```
    // Fixed tree, signed weights. Rebuild clears all active points.
    void build()
    {
        fill(removed.begin(), removed.end(), 0);
        fill(active.begin(), active.end(), 0);
        for (int u = 0; u < n; u++)
        {
            path[u].clear();
            top[u].clear();
        }
        bag.clear();
        best.clear();
        live.clear();
        decompose(0);
        built = true;
    }

    optional<Best> candidate(int c) const
    {
        if (top[c].size() < 2) return nullopt;
        auto it = top[c].rbegin();
        auto [x, u, a] = *it++;
        auto [y, v, b] = *it;
        if (u > v) swap(u, v);
        return Best{sum(x, y), u, v, c};
    }
```

活动点设置与端点查询（接上页同一结构体）：

```
    void set(int u, bool on)
    {
        assert(built && 0 <= u && u < n);
        if (active[u] == on) return;
        active[u] = on;
        if (on) live.insert(u);
        else live.erase(u);
        for (auto [c, b, d] : path[u])
        {
            if (auto old = candidate(c)) best.erase(*old);
            if (!bag[b].empty())
            {
                auto [x, v] = *bag[b].rbegin();
                top[c].erase({x, v, b});
            }
            if (on) bag[b].insert({d, u});
            else bag[b].erase({d, u});
            if (!bag[b].empty())
            {
                auto [x, v] = *bag[b].rbegin();
                top[c].insert({x, v, b});
            }
            if (auto now = candidate(c)) best.insert(*now);
        }
    }

    // Maximum distance and any attaining pair; equal endpoints are allowed.
    optional<Item> query() const
    {
        assert(built);
        if (live.empty()) return nullopt;
        if (!best.empty())
        {
            auto [d, u, v, c] = *best.rbegin();
            if (d > 0) return Item{d, u, v};
        }
        int u = *live.begin();
        return Item{0, u, u};
    }
};
```

应用题补充：SPOJ QTREE4 / 有符号边权动态白点直径

题意：n,q≤100000，边权 −1000..1000，初始全白；C u 反色，A 查白点对的最大距离，两端允许相同。无白点输出 They have disappeared.。题目 1-based，传入模板时减 1；query() 非空时取 get<0>。固定树且允许负边，不能改用只支持非负权的 TreeDiameter 摘要。

所需模板：CentroidDiameter（第 66 页，2.7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.spoj.com/problems/QTREE4/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    CentroidDiameter t(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u - 1, v - 1, w);
    }
    t.build();
    for (int u = 0; u < n; u++) t.set(u, true);
    int q;
    cin >> q;
    while (q--)
    {
        char op;
        cin >> op;
        if (op == 'C')
        {
            int u;
            cin >> u;
            --u;
            t.set(u, !t.active[u]);
        }
        else
        {
            auto ans = t.query();
            if (ans) cout << get<0>(*ans) << '\n';
            else cout << "They have disappeared.\n";
        }
    }
}
```

接口用法演示：活动点最大距离与端点方案（接口演示）

演示约定：输入 n,q、n−1 条 0-based 带权树边，初始全不活动。0 u on 设置状态（重复设置不变）；1 查询，空集输出 −1，否则输出最大距离 d 及任意最优活动点 u,v。允许负边和 u=v，并列不指定点号；所有简单路径和须在 signed64 内。

所需模板：CentroidDiameter（第 66 页，2.7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/tree-divide/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    CentroidDiameter t(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    t.build();
    while (q--)
    {
        int op;
        cin >> op;
        if (op == 0)
        {
            int u, on;
            cin >> u >> on;
            t.set(u, on);
        }
        else
        {
            auto ans = t.query();
            if (!ans) cout << -1 << '\n';
            else
            {
                auto [d, u, v] = *ans;
                cout << d << ' ' << u << ' ' << v << '\n';
            }
        }
    }
}
```

2.8 树哈希

相关应用：Deterministic sorted-child interning and centroid rooting; exact alternative to randomized tree hashing.

2.8.1 确定性有根与无根树同构

TreeIsomorphism iso；rooted(g,root) 返回有根树编号，unrooted(g) 返回无根树 pair 编号。g 是 0-based 非空简单无向连通树，邻接表双向各存一次；root 在 0..n-1。递归 DFS，不验证非法图。所有待比较的树必须共用同一个 iso；编号依赖插入次序，不同对象或清空 codes 前后的编号不能直接比较。对子节点编号排序并用 map 精确驻留，无随机哈希碰撞。无根树取重心：单重心返回 (code,0)，双重心分别以两点为根编码并排序。支持单点；不考虑点/边标签，也不返回顶点映射。codes 只读，编号数须 <INT_MAX，n<=INT_MAX。S 为本次结束时驻留签名数，一次 n 点调用 O(n log(n+1)+n log(S+1))，临时 O(n)，跨调用累计存储至多 O(T)，T 为累计编码遍历的顶点数。

```
struct TreeIsomorphism
{
    map<vector<int>, int> codes;

    int rooted(const vector<vector<int>> &g, int root)
    {
        assert(0 <= root && root < int(g.size()));
        auto dfs = [&](auto &&self, int u, int p) -> int
        {
            vector<int> children;
            for (int v : g[u])
            {
                if (v == p) continue;
                children.push_back(self(self, v, u));
            }
            sort(children.begin(), children.end());
            int next = int(codes.size()) + 1;
            return codes.emplace(children, next).first->second;
        };
        return dfs(dfs, root, -1);
    }
```

继续求解（接上页同一结构体）：

```
    pair<int, int> unrooted(const vector<vector<int>> &g)
    {
        int n = g.size();
        assert(n > 0);
        vector<int> centers;
        auto dfs = [&](auto &&self, int u, int p) -> int
        {
            int size = 1, largest = 0;
            for (int v : g[u])
            {
                if (v == p) continue;
                int child = self(self, v, u);
                size += child;
                largest = max(largest, child);
            }
            largest = max(largest, n - size);
            if (largest <= n / 2) centers.push_back(u);
            return size;
        };
        dfs(dfs, 0, -1);
        int a = rooted(g, centers[0]);
        if (centers.size() == 1) return {a, 0};
        int b = rooted(g, centers[1]);
        if (a > b) swap(a, b);
        return {a, b};
    }
};
```

模板题使用：P5043 【模板】树同构 / [BJOI2015] 树的同构

题意：M 棵无根树，每棵 N 个点，1<=N,M<=50。输入 N 个父亲编号，0 表示根；转换无向树后忽略输入根。输出与本树同构的最早树编号；所有树共用同一驻留表。

所需模板：TreeIsomorphism（第 72 页，2.8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5043

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> m;
    TreeIsomorphism iso;
    map<pair<int, int>, int> first;
    for (int id = 1; id <= m; id++)
    {
        int n;
        cin >> n;
        vector<vector<int>> g(n);
        for (int u = 0; u < n; u++)
        {
            int p;
            cin >> p;
            if (p == 0) continue;
            p--;
            g[u].push_back(p);
            g[p].push_back(u);
        }
        auto code = iso.unrooted(g);
        auto result = first.emplace(code, id);
        cout << result.first->second << '\n';
    }
}
```

第三章　拓扑排序

3.1　DAG 有符号权最长路与路径方案

DagLongest(n)，点号 1..n，0<n<INT_MAX；add(u,v,w) 返回 0-based 原边号，支持重边和完整 signed64 权值，边数 ≤INT_MAX。run(s) 求指定起点的最长路；run(0) 允许任意起点及零边路径。每次运行重建入度，返回 false 表示整张图存在环（包括与 s 不连通的环），此时全部 dis/pre/ord 结果无效；它不是一般图最长路求解器。

成功时 valid=true，ord 为全图拓扑序，dis[v]==−inf 为不可达，其余为 int128 最长距离；path(v) 返回正向原边号，不可达和零边路径都为空，结合 dis 区分。并列任选一条，不承诺字典序。add 使旧结果失效；支持重复改源点和独立对象复制。

run 时间和空间 O(n+m)，path 为 O(路径长)。拓扑处理必须包含不可达点，并正常消减其出边入度，但禁止用不可达哨兵参与加法。有限路径至多 n−1 边，绝对值 <(n · 2^63)<2^94，int128 和 inf=2^126 足够，不能无条件转 long long。e/g/dis/pre/ord/valid 只读。拓扑序用 vector 作 FIFO 队列，与 DFS 改写为显式栈无关。

```
struct DagLongest
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 126;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    bool valid = false;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<I> dis;
    vector<int> pre, ord;

    DagLongest(int n) : n(n), g(n + 1)
    {
        assert(0 < n && n < INT_MAX);
    }

    int add(int u, int v, long long w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(e.size() < INT_MAX);
        int id = e.size();
        e.push_back({u, v, w});
        g[u].push_back(id);
        valid = false;
        return id;
    }
```

全图拓扑序与最长路（接上页同一结构体）：

```
    // s=0 allows any starting vertex, including a zero-edge path.
    // A cycle anywhere returns false and invalidates all results.
    bool run(int s)
    {
        assert(0 <= s && s <= n);
        valid = false;
        dis.assign(n + 1, -inf);
        pre.assign(n + 1, -1);
        ord.clear();
        vector<int> deg(n + 1);
        for (auto [u, v, w] : e) deg[v]++;
        for (int u = 1; u <= n; u++)
        {
            if (!deg[u]) ord.push_back(u);
            if (!s || u == s) dis[u] = 0;
        }
        for (int i = 0; i < (int)ord.size(); i++)
        {
            int u = ord[i];
            for (int id : g[u])
            {
                auto [from, v, w] = e[id];
                if (dis[u] != -inf && dis[v] < dis[u] + w)
                {
                    dis[v] = dis[u] + w;
                    pre[v] = id;
                }
                if (!--deg[v]) ord.push_back(v);
            }
        }
        valid = (int)ord.size() == n;
        return valid;
    }
```

成功运行后的原边路径（接上页同一结构体）：

```
    // Forward original-edge IDs; inspect dis to distinguish unreachable.
    vector<int> path(int t) const
    {
        assert(valid && 1 <= t && t <= n);
        vector<int> ids;
        if (dis[t] == -inf) return ids;
        while (pre[t] != -1)
        {
            ids.push_back(pre[t]);
            t = e[pre[t]].u;
        }
        reverse(ids.begin(), ids.end());
        return ids;
    }
};
```

应用题补充：Luogu P1807 / 最长路

题意：有符号权 DAG 中求 1 到 n 的最长路，不可达输出 −1。n≤1500，m≤50000，|w|≤100000，有限结果可转 long long；负数也可能是合法答案，不能以 dis<0 判不可达。

所需模板：DagLongest（第 75 页，3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1807

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    DagLongest t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    t.run(1);
    if (t.dis[n] == -DagLongest::inf) cout << -1 << '\n';
    else cout << (long long)t.dis[n] << '\n';
}
```

应用题补充：CSES 1680 / Longest Flight Route

题意：DAG 中求从 1 到 n 经过城市数最多的路线；每条航线加权 1，path 返回原边号，依次输出边终点。不可达输出 IMPOSSIBLE；城市数为边数 +1。n≤100000，m≤200000。

所需模板：DagLongest（第 75 页，3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1680/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    DagLongest t(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        t.add(u, v, 1);
    }
    t.run(1);
    if (t.dis[n] == -DagLongest::inf)
    {
        cout << "IMPOSSIBLE\n";
        return 0;
    }
    auto p = t.path(n);
    cout << p.size() + 1 << '\n';
    cout << 1;
    for (int id : p) cout << ' ' << t.e[id].v;
    cout << '\n';
}
```

接口用法演示：DAG 最长距离、全图判环与原边路径（接口演示）

演示约定：输入 n,m,s 及 signed64 有向边；s=0 允许任意起点。整图有环输出 CYCLIC，否则逐点输出 INF 或 “int128 最长距离边数正向原边号”。INF 表示不可达，负数距离有效，边号从 0 计。

所需模板：DagLongest（第 75 页，3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/topo/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    DagLongest t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    if (!t.run(s))
    {
        cout << "CYCLIC\n";
        return 0;
    }
    auto print = [](__int128 x)
    {
        if (x < 0)
        {
            cout << '-';
            x = -x;
        }
        string a;
        do
        {
            a += char('0' + x % 10);
            x /= 10;
        } while (x);
        reverse(a.begin(), a.end());
        cout << a;
    };
```

继续处理操作（接上页同一 main）：

```
    for (int u = 1; u <= n; u++)
    {
        if (t.dis[u] == -DagLongest::inf) cout << "INF\n";
        else
        {
            auto path = t.path(u);
            print(t.dis[u]);
            cout << ' ' << path.size();
            for (int id : path) cout << ' ' << id;
            cout << '\n';
        }
    }
}
```

第四章　最短路问题

4.1　最短路

4.1.1　Johnson：负边全源最短路

Johnson(n)，点号 1..n，1≤n<INT_MAX，总边数不超过 INT_MAX；add(u,v,w) 加有向边，允许重边、自环和完整 long long 边权。build() 用初值全零的 Bellman–Ford 求势能，等价于超级源点连向所有点；全图任意位置有负环时返回 false，否则返回 true。仅成功后可 distances(s)，返回 vector<int128> 的原图距离，d[s]=0，不可达为 Johnson::inf；0 号不用。可按需逐源调用，不强制存 n^2 矩阵。加边会使预处理失效，再 build 后查询；可重复 build。重赋权为 w+h[u]-h[v]，非负；查询后加 h[v]-h[s] 还原。势能与距离全程 int128，不能无条件转为 long long。预处理 O(nm+n) 时间，单源用惰性优先队列，O(n+m log(m+1)) 时间、O(n+m) 辅助空间；整个结构 O(n+m)。g、edges、h 仅供读取，勿手动修改。

```
struct Johnson
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 120;
    int n;
    vector<tuple<int, int, long long>> edges;
    vector<vector<pair<int, long long>>> g;
    vector<I> h;
    bool ready = false;

    Johnson(int n) : n(n), g(n + 1), h(n + 1) { assert(n > 0); }

    void add(int u, int v, long long w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        edges.push_back({u, v, w});
        g[u].push_back({v, w});
        ready = false;
    }
```

势能预处理、单源查询与距离还原（接上页同一结构体）：

```
    bool build()
    {
        ready = false;
        fill(h.begin(), h.end(), 0);
        for (int step = 0; step < n; step++)
        {
            bool changed = false;
            for (auto [u, v, w] : edges)
                if (h[v] > h[u] + w)
                {
                    h[v] = h[u] + w;
                    changed = true;
                }
            if (!changed)
            {
                ready = true;
                return true;
            }
        }
        return false;
    }

    vector<I> distances(int s) const
    {
        assert(ready && 1 <= s && s <= n);
        vector<I> d(n + 1, inf);
        priority_queue<pair<I, int>, vector<pair<I, int>>, greater<pair<I, int>>> q;
        d[s] = 0;
        q.push({0, s});
        while (!q.empty())
        {
            auto [value, u] = q.top();
            q.pop();
            if (value != d[u]) continue;
            for (auto [v, w] : g[u])
            {
                I next = value + w + h[u] - h[v];
                if (next < d[v])
                {
                    d[v] = next;
                    q.push({next, v});
                }
            }
        }
        for (int v = 1; v <= n; v++)
            if (d[v] != inf) d[v] += h[v] - h[s];
        return d;
    }
};
```

模板题使用：Luogu P5905

题意：有向图可能有负边，若任意位置存在负环输出 -1。否则逐源求距离，按题意把不可达距离换为 10^9，再求编号加权和。题目规模和边权界保证最终和可转为 long long。

所需模板：Johnson（第 81 页，4.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5905

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Johnson graph(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        graph.add(u, v, w);
    }
    if (!graph.build())
    {
        cout << -1 << '\n';
        return 0;
    }
    for (int s = 1; s <= n; s++)
    {
        auto d = graph.distances(s);
        Johnson::I answer = 0;
        for (int v = 1; v <= n; v++)
            answer += v * (d[v] == Johnson::inf ? 1000000000 : d[v]);
        cout << (long long)answer << '\n';
    }
}
```

4.1.2 非负权最短路

add(u,v,w)、run(s)，dis 为距离，pre 为前驱，path(t) 返回点序列。不可达用 LLONG_MAX；要求所有有限最短距离严格小于该哨兵，w≥0。懒删除堆在多重边图中按 O(n+m log(m+1)) 计，空间 O(n+m)；简单图常写 O((n+m)log(n+1))。

```
struct Dijkstra
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX;
    int n;
    vector<vector<pair<int, ll>>> g;
    vector<ll> dis;
    vector<int> pre;

    Dijkstra(int n) : n(n), g(n + 1) {}

    void add(int u, int v, ll w)
    {
        assert(w >= 0);
        g[u].push_back({v, w});
    }

    void run(int s)
    {
        dis.assign(n + 1, inf);
        pre.assign(n + 1, -1);
        dis[s] = 0;
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (d != dis[u]) continue;
            for (auto [v, w] : g[u])
                if (w < inf - d && d + w < dis[v])
                {
                    dis[v] = d + w;
                    pre[v] = u;
                    q.push({dis[v], v});
                }
        }
    }

    vector<int> path(int t) const
    {
        if (dis[t] == inf) return {};
        vector<int> ans;
        for (; t != -1; t = pre[t]) ans.push_back(t);
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

模板题使用：Luogu P4779

题意：给非负权有向图和源点 s，输出 s 到各点的最短距离。加完边后 run(s)，读取 dis[1..n]；本题保证源点可达所有点，一般输入的不可达值须按目标题目格式另行转换。

所需模板：Dijkstra（第 84 页，4.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4779

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m, s;
    std::cin >> n >> m >> s;
    Dijkstra graph(n);
    while (m--)
    {
        int u, v;
        long long w;
        std::cin >> u >> v >> w;
        graph.add(u, v, w);
    }
    graph.run(s);
    for (int i = 1; i <= n; i++) std::cout << graph.dis[i] << (i == n ? '\n' : ' ');
}
```

4.1.3 顶点开放时间的单位边最短路

release_bfs(open,source,neighbors) 返回各点最早到达时间，不可达为 -1；点号 0..n-1，n>0，open[v] 为非负整数，源点初始时间为 open[source]。允许原地等待，每条边耗时恰为 1，neighbors(u,visit) 枚举所有出邻点并调用 visit(v)，可直接枚举网格而不建边。令 R=max(open)，要求 R+n<INT_MAX 且内存可承受；时间 O(n+m+R)，空间 O(n+R)。第一次发现即可定距，只适用于单位边和固定开放时间；不能用于一般带权边、关闭时间或周期可用边。

```
template <class Neighbors>
vector<int> release_bfs(const vector<int> &open, int source, Neighbors neighbors)
{
    int n = open.size();
    assert(0 <= source && source < n);
    int last = *max_element(open.begin(), open.end());
    assert(last >= 0 && last < INT_MAX - n);
    vector<vector<int>> pending(last + 1);
    vector<int> dist(n, -1), queue;
    queue.reserve(n);
    dist[source] = open[source];
    queue.push_back(source);
    int head = 0, time = 0, waiting = 0;
    while (head < int(queue.size()) || waiting)
    {
        int limit = head < int(queue.size()) ? dist[queue[head]] : INT_MAX;
        while (time <= last && time <= limit && pending[time].empty()) time++;
        int u;
        if (time <= last && time <= limit)
        {
            u = pending[time].back();
            pending[time].pop_back();
            waiting--;
        }
        else
            u = queue[head++];
        neighbors(u,
                  [&](int v)
                  {
                      if (dist[v] != -1) return;
                      dist[v] = max(dist[u] + 1, open[v]);
                      if (dist[v] == dist[u] + 1)
                          queue.push_back(v);
                      else
                      {
                          pending[open[v]].push_back(v);
                          waiting++;
                      }
                  });
    }
    return dist;
}
```

应用题补充：QOJ 8236 / ICPC 2023 Hangzhou G —Snake Move

题意：蛇可移动或缩尾，求蛇头到各格最少步数的平方和，模 2^64。初始身体按 0-based i（不含头）设 open=k-i，尾格为 1；源点时间 0，障碍不连边。n,m 至多 3000，k 至多 min(nm,100000)。不可达不计，使用 unsigned long long 自然回绕；这是比赛应用，不是独立模板题。

所需模板：release_bfs（第 86 页，4.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/gym/104976/attachments/download/24253/hangzhou(en).pdf

```
int main()
{
    int n, m, k;
    scanf("%d%d%d", &n, &m, &k);
    vector<int> open(n * m);
    int source = 0;
    for (int i = 0; i < k; i++)
    {
        int x, y;
        scanf("%d%d", &x, &y);
        int u = (x - 1) * m + y - 1;
        if (i == 0)
            source = u;
        else
            open[u] = k - i;
    }
    vector<string> grid(n);
    vector<char> row(m + 1);
    for (auto &s : grid)
    {
        scanf("%s", row.data());
        s = row.data();
    }
    auto dist =
        release_bfs(open,
                    source,
                    [&](int u, auto visit)
                    {
                        int x = u / m, y = u % m;
                        const int dx[] = {-1, 1, 0, 0};
                        const int dy[] = {0, 0, -1, 1};
                        for (int i = 0; i < 4; i++)
                        {
                            int a = x + dx[i], b = y + dy[i];
                            if (a >= 0 && a < n && b >= 0 && b < m && grid[a][b] == '.')
                                visit(a * m + b);
                        }
                    });
    unsigned long long answer = 0;
    for (int d : dist)
        if (d >= 0) answer += 1ULL * d * d;
    printf("%llu\n", answer);
}
```

4.1.4 Floyd 全源最短路与原边方案

Floyd(n)，点号 1..n，0<n<INT_MAX；add(u,v,w) 返回 0-based 原边号，有向边，允许重边、自环及完整 signed64 权值，边数 ≤INT_MAX。无向图需分别加入双向边。run() 重新建立矩阵；存在任何负环返回 false，valid=false，全部结果不可读，不能只取未受负环影响的点对。

成功时 dis[u][v]==inf 为不可达，其他为 int128 最短距离，dis[u][u]=0。path(u,v) 返回正向原边号，并列任选；不可达和 u=v 均为空，结合 dis 区分。nxt 为恢复路径的首条原边号矩阵，通常只需调用 path；e/dis/nxt/valid 只读。add 清除 valid，支持重复运行、加边后重跑与独立值拷贝。

run 时间 $O(n^3+m)$，空间 $O(n^2+m)$，path 时间与输出空间 O(路径长)。每次处理 k 之前检查 dis[k][k]<0，立即拒绝，不能延后到所有松弛结束，否则负环下中间量可指数增长。通过的枢轴不含负环；有限标签取简单路径或闭合路径，拼接候选绝对值 <2n · 2^63<2^95，int128 与 inf=2^126 足够。两段都有限才相加；不要无条件转 long long。矩阵占用约 $20(n+1)^2$ 字节，另计 vector 和原边表，n 应按时间/内存选择。

```
struct Floyd
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 126;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    bool valid = false;
    vector<Edge> e;
    vector<vector<I>> dis;
    vector<vector<int>> nxt;

    Floyd(int n) : n(n)
    {
        assert(0 < n && n < INT_MAX);
    }

    int add(int u, int v, long long w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(e.size() < INT_MAX);
        int id = e.size();
        e.push_back({u, v, w});
        valid = false;
        return id;
    }
```

矩阵初始化与 Floyd 松弛（接上页同一结构体）：

```
    // Any negative cycle invalidates all results, including finite pairs.
    bool run()
    {
        valid = false;
        dis.assign(n + 1, vector<I>(n + 1, inf));
        nxt.assign(n + 1, vector<int>(n + 1, -1));
        for (int u = 1; u <= n; u++) dis[u][u] = 0;
        for (int id = 0; id < (int)e.size(); id++)
        {
            auto [u, v, w] = e[id];
            if (w < dis[u][v])
            {
                dis[u][v] = w;
                nxt[u][v] = id;
            }
        }
        for (int k = 1; k <= n; k++)
        {
            // Stop before using a negative diagonal as a pivot.
            if (dis[k][k] < 0) return false;
            for (int u = 1; u <= n; u++)
            {
                if (dis[u][k] == inf) continue;
                for (int v = 1; v <= n; v++)
                    if (dis[k][v] != inf && dis[u][v] > dis[u][k] + dis[k][v])
                    {
                        dis[u][v] = dis[u][k] + dis[k][v];
                        nxt[u][v] = nxt[u][k];
                    }
            }
        }
        valid = true;
        return true;
    }
```

成功运行后的原边路径（接上页同一结构体）：

```
    // Forward original-edge IDs; empty for unreachable or s=t.
    vector<int> path(int s, int t) const
    {
        assert(valid && 1 <= s && s <= n && 1 <= t && t <= n);
        vector<int> ids;
        if (dis[s][t] == inf) return ids;
        while (s != t)
        {
            int id = nxt[s][t];
            ids.push_back(id);
            s = e[id].v;
        }
        return ids;
    }
};
```

模板题使用：AOJ GRL_1_C / All Pairs Shortest Path

题意：0-based 有向图输入转换为 1-based；整图有负环输出 NEGATIVE CYCLE，否则逐行输出距离矩阵，不可达为 INF。n≤100、m≤9900、|w|≤20000000，有限结果可转 long long。题面无自环重边，模板额外支持。

所需模板：Floyd（第 88 页，4.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=GRL_1_C&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Floyd t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u + 1, v + 1, w);
    }
    if (!t.run())
    {
        cout << "NEGATIVE CYCLE\n";
        return 0;
    }
    for (int u = 1; u <= n; u++)
    {
        for (int v = 1; v <= n; v++)
        {
            if (v > 1) cout << ' ';
            if (t.dis[u][v] == Floyd::inf) cout << "INF";
            else cout << (long long)t.dis[u][v];
        }
        cout << '\n';
    }
}
```

应用题补充：CSES 1672 / Shortest Routes II

题意：无向正权图多次查询最短距离；每条道路加入双向边，重边保留最小值，不可达输出 −1。n≤500、m≤n^2、q≤100000、w≤10^9，有限结果用 long long。

所需模板：Floyd（第 88 页，4.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1672/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    Floyd t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
        t.add(v, u, w);
    }
    t.run();
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        if (t.dis[u][v] == Floyd::inf) cout << -1 << '\n';
        else cout << (long long)t.dis[u][v] << '\n';
    }
}
```

接口用法演示：Floyd 全源距离与指定点对路径（接口演示）

演示约定：输入 n,m,q 及 signed64 有向边，再输入 q 个点对。整图有负环只输出 NEGATIVE CYCLE；否则每问输出 INF 或 “int128 距离边数正向原边号”，原边号从 0 计。同点零边路径与不可达分别处理。

所需模板：Floyd（第 88 页，4.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/shortest-path/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    Floyd t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    if (!t.run())
    {
        cout << "NEGATIVE CYCLE\n";
        return 0;
    }
    auto print = [](__int128 x)
    {
        if (x < 0)
        {
            cout << '-';
            x = -x;
        }
        string a;
        do
        {
            a += char('0' + x % 10);
            x /= 10;
        } while (x);
        reverse(a.begin(), a.end());
        cout << a;
    };
```

继续处理操作（接上页同一 main）：

```
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        if (t.dis[u][v] == Floyd::inf) cout << "INF\n";
        else
        {
            auto path = t.path(u, v);
            print(t.dis[u][v]);
            cout << ' ' << path.size();
            for (int id : path) cout << ' ' << id;
            cout << '\n';
        }
    }
}
```

4.1.5 最小总边权最短路树（含零权边）

ShortestPathTree(n)，无向非负权图，1-based 点号，0<n<INT_MAX；add(u,v,w) 自动加入双向邻接并返回 0-based 无向原边号，支持自环重边，0≤w≤LLONG_MAX，边数 ≤INT_MAX。run(s) 返回可达点数（含 s），构造所有可达点上的最短路树，并在这些树中最小化所选原边权值总和 weight。断开图只覆盖 s 所在连通块；不是最小生成树。

运行后 valid=true，dis 为 int128 最短距离，inf 为不可达；ids 为树边原编号，weight 为 int128 总边权。path(v) 返回从 s 到 v 的正向无向原边号，按当前点决定边方向；s=v 与不可达均为空，结合 dis 区分。pre[v] 为树中父边号，根与不可达点为 −1。并列任选，不保证字典序。add 使旧结果失效；支持反复换源点及独立复制。

Dijkstra 求距离，递归划分可达零权连通块；每块选零权生成树，非根块选择最轻的紧入边，最后递归定向整棵树。时间 O(n+m log(m+1))、空间 O(n+m)，path 为 O(路径长)。zero/dfs 与 bel/tree 为内部辅助，成员只读；递归深度最坏 O(n)，调用方保证栈空间。有限距离和总树权绝对值 <2^94，int128 和 inf=2^126 足够，不能无条件转 long long。

```
struct ShortestPathTree
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 126;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n, root = 0;
    bool valid = false;
    I weight = 0;
    vector<Edge> e;
    vector<vector<pair<int, int>>> g, tree;
    vector<I> dis;
    vector<int> bel, ids, pre;

    ShortestPathTree(int n) : n(n), g(n + 1)
    {
        assert(0 < n && n < INT_MAX);
    }

    int add(int u, int v, long long w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && w >= 0);
        assert(e.size() < INT_MAX);
        int id = e.size();
        e.push_back({u, v, w});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        valid = false;
        return id;
    }
```

递归辅助与 run 中的 Dijkstra（接上页同一结构体）：

```
    // Internal: span one reachable zero-weight component recursively.
    void zero(int u, int c)
    {
        bel[u] = c;
        for (auto [v, id] : g[u])
            if (!e[id].w && !bel[v])
            {
                ids.push_back(id);
                zero(v, c);
            }
    }

    // Internal: orient the selected undirected tree from the query root.
    void dfs(int u, int p)
    {
        for (auto [v, id] : tree[u])
            if (v != p)
            {
                pre[v] = id;
                dfs(v, u);
            }
    }

    // Minimum total edge weight among shortest-path trees of reachable vertices.
    // Returns the number of reachable vertices, including s.
    int run(int s)
    {
        assert(1 <= s && s <= n);
        valid = false;
        root = s;
        dis.assign(n + 1, inf);
        priority_queue<pair<I, int>, vector<pair<I, int>>, greater<pair<I, int>>> q;
        dis[s] = 0;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (d != dis[u]) continue;
            for (auto [v, id] : g[u])
                if (dis[v] > d + e[id].w)
                {
                    dis[v] = d + e[id].w;
                    q.push({dis[v], v});
                }
        }
```

继续 run：零权块、最轻入口与定向（接上页同一结构体）：

```
        bel.assign(n + 1, 0);
        ids.clear();
        int cnt = 0, reached = 0;
        for (int u = 1; u <= n; u++)
            if (dis[u] != inf)
            {
                reached++;
                if (!bel[u]) zero(u, ++cnt);
            }
        vector<int> best(cnt + 1, -1);
        for (int u = 1; u <= n; u++)
            if (dis[u] != inf)
                for (auto [v, id] : g[u])
                    if (bel[u] != bel[v] && dis[v] == dis[u] + e[id].w)
                    {
                        int &b = best[bel[v]];
                        if (b == -1 || e[id].w < e[b].w) b = id;
                    }
        for (int c = 1; c <= cnt; c++)
            if (c != bel[s])
            {
                assert(best[c] != -1);
                ids.push_back(best[c]);
            }
        tree.assign(n + 1, {});
        weight = 0;
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            tree[u].push_back({v, id});
            tree[v].push_back({u, id});
            weight += w;
        }
        pre.assign(n + 1, -1);
        dfs(s, 0);
        valid = true;
        return reached;
    }
```

读取最短路树中的路径（接上页同一结构体）：

```
    // Forward undirected edge IDs; follow each edge from the current vertex.
    vector<int> path(int t) const
    {
        assert(valid && 1 <= t && t <= n);
        vector<int> p;
        if (dis[t] == inf) return p;
        while (t != root)
        {
            int id = pre[t];
            p.push_back(id);
            auto [u, v, w] = e[id];
            t = u == t ? v : u;
        }
        reverse(p.begin(), p.end());
        return p;
    }
};
```

应用题补充：Codeforces 545E / Paths and Trees

题意：连通无向正权图，给定源点，输出总边权最小的最短路树：先输出 weight，再输出 ids 加 1。n,m≤300000，w≤10^9，总树权可转 long long。题面不含零权、自环和重边，额外接口范围另由 API 验证。

所需模板：ShortestPathTree（第 95 页，4.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/545/E

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    ShortestPathTree t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    int s;
    cin >> s;
    t.run(s);
    cout << (long long)t.weight << '\n';
    for (int id : t.ids) cout << id + 1 << ' ';
    cout << '\n';
}
```

接口用法演示：零权最短路树、断开图与原边路径（接口演示）

演示约定：输入 n,m,s 及无向非负 signed64 边，每条仅 add 一次。首行输出“可达点数最小总边权树边数原边号”；随后逐点输出 INF 或“最短距离路径边数正向无向原边号”。所有权和用 int128 打印，原边号从 0 计。

所需模板：ShortestPathTree（第 95 页，4.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/shortest-path/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    ShortestPathTree t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    int reached = t.run(s);
    auto print = [](__int128 x)
    {
        if (x < 0)
        {
            cout << '-';
            x = -x;
        }
        string a;
        do
        {
            a += char('0' + x % 10);
            x /= 10;
        } while (x);
        reverse(a.begin(), a.end());
        cout << a;
    };
```

继续处理操作（接上页同一 main）：

```
    cout << reached << ' ';
    print(t.weight);
    cout << ' ' << t.ids.size();
    for (int id : t.ids) cout << ' ' << id;
    cout << '\n';
    for (int u = 1; u <= n; u++)
    {
        if (t.dis[u] == ShortestPathTree::inf) cout << "INF\n";
        else
        {
            auto path = t.path(u);
            print(t.dis[u]);
            cout << ' ' << path.size();
            for (int id : path) cout << ' ' << id;
            cout << '\n';
        }
    }
}
```

4.1.6 稠密图最短路：矩阵 Dijkstra

DenseDijkstra(n)，1-based 点号，0<n<INT_MAX；add(u,v,w) 加入有向非负 signed64 边，矩阵合并重边取最小值，允许零权、自环。无向图分别加双向；不保存原边编号。cost[u][v]=−1 表示无边，LLONG_MAX 仍是合法边权。

run(s) 每次从头求指定源点的最短距离，成功后 valid=true；dis 为 int128，inf 表示不可达。path(t) 返回正向顶点列表，不可达为空，源点自身为 {s}；pre 为父点，根及不可达为 −1。相同距离任选方案，不承诺字典序。add 使旧结果失效，支持重复换源与独立值拷贝；cost/dis/pre/root/valid 只读。

构造时间和空间 $O(n^2)$，add 为 O(1)，run 为 $O(n^2)$，不使用堆；path 为 O(路径长)。矩阵约 $8(n+1)^2$ 字节，另加 O(n) 的距离和辅助数组，按内存选择 n；稀疏图优先堆 Dijkstra。有限最短路径至多 n−1 条边，距离 <2^94，int128 及 inf=2^126 足够。每步选择未定点中距离最小者，只松弛未定点并严格改善，零权环不会形成父点环。

```
struct DenseDijkstra
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 126;
    int n, root = 0;
    bool valid = false;
    vector<vector<long long>> cost;
    vector<I> dis;
    vector<int> pre;

    DenseDijkstra(int n) : n(n), cost(n + 1, vector<long long>(n + 1, -1))
    {
        assert(0 < n && n < INT_MAX);
    }

    void add(int u, int v, long long w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && w >= 0);
        auto &x = cost[u][v];
        if (x == -1 || w < x) x = w;
        valid = false;
    }
```

矩阵扫描、松弛与路径输出（接上页同一结构体）：

```
    void run(int s)
    {
        assert(1 <= s && s <= n);
        root = s;
        dis.assign(n + 1, inf);
        pre.assign(n + 1, -1);
        vector<bool> vis(n + 1);
        dis[s] = 0;
        for (int step = 0; step < n; step++)
        {
            int u = 0;
            for (int v = 1; v <= n; v++)
            {
                if (!vis[v] && (!u || dis[v] < dis[u])) u = v;
            }
            if (!u || dis[u] == inf) break;
            vis[u] = true;
            for (int v = 1; v <= n; v++)
                if (!vis[v] && cost[u][v] >= 0 && dis[v] > dis[u] + cost[u][v])
                {
                    dis[v] = dis[u] + cost[u][v];
                    pre[v] = u;
                }
        }
        valid = true;
    }

    // Forward vertex IDs; empty iff unreachable, {root} for the root.
    vector<int> path(int t) const
    {
        assert(valid && 1 <= t && t <= n);
        if (dis[t] == inf) return {};
        vector<int> p;
        for (; t != -1; t = pre[t]) p.push_back(t);
        reverse(p.begin(), p.end());
        return p;
    }
};
```

模板题使用：AOJ ALDS1_12_B / Single Source Shortest Path

题意：输入 n 和 n 行 “0-based 点号出度邻点权值…”，求 0 到各点距离，按原点号顺序逐行输出 “点号距离”。n≤100，m≤10000，0≤w≤100000，题面保证从 0 全部可达，距离可转 long long。

所需模板：DenseDijkstra（第 101 页，4.1.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=ALDS1_12_B&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    DenseDijkstra g(n);
    for (int i = 0; i < n; i++)
    {
        int u, k;
        cin >> u >> k;
        while (k--)
        {
            int v;
            long long w;
            cin >> v >> w;
            g.add(u + 1, v + 1, w);
        }
    }
    g.run(1);
    for (int u = 1; u <= n; u++)
    {
        cout << u - 1 << ' ' << (long long)g.dis[u] << '\n';
    }
}
```

接口用法演示：矩阵 Dijkstra 距离与顶点路径（接口演示）

演示约定：输入 n,m,s 及 1-based 有向非负 signed64 边，逐点输出 INF 或 “int128 距离路径顶点数正向顶点列表”。重边合并为最小权；源点路径有一个点，与不可达空路径不同。

所需模板：DenseDijkstra（第 101 页，4.1.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/shortest-path/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    DenseDijkstra g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u, v, w);
    }
    g.run(s);
    for (int v = 1; v <= n; v++)
        if (g.dis[v] == DenseDijkstra::inf)
            cout << "INF\n";
        else
        {
            __int128 x = g.dis[v];
            string a;
            do
            {
                a += char('0' + x % 10);
                x /= 10;
            } while (x);
            reverse(a.begin(), a.end());
            cout << a;
            auto p = g.path(v);
            cout << ' ' << p.size();
            for (int u : p) cout << ' ' << u;
            cout << '\n';
        }
}
```

4.1.7 SPFA：队列松弛、负环判定与有限路径

Spfa(n)，点号 1..n，0<n<INT_MAX。add(u,v,w) 返回 0-based 原边号，允许重边、自环、完整 signed64 权值，边数 ≤INT_MAX；加边令 valid=false。run(s) 查从 s 可达的负环，s=0 同时初始化所有点为 0 以查全图。返回 false 时 valid=false，全部 dis/pre 均为未完成状态，不能读取为最短路或恢复路径；需要负环方案或逐点负无穷分类用 BellmanFord。

run 返回 true 时 valid=true，dis[v]==inf 为不可达，其余为 int128 最短距离；run(0) 距离为任意起点出发的最小值，且本身就是全部约束 x_v−x_u≤w 的一个可行解。path(v) 返回正向原边号；不可达与零边路径均为空，结合 dis 区分。每次 run 重置临时队列/标记/路径边数，支持反复改起点与对象复制。

FIFO 队列，最坏 O(nm+n+m) 时间，O(n+m) 空间，不保证近线性；队列去重不等于永久访问标记。严格松弛后维护标签所代表游走的边数，达到 n 立即判负环。停止前标签最多 n 条边，绝对值 ≤n · 2^63<2^94，int128 与 inf=2^126 足够。不要无条件转 long long；e/g/dis/pre/valid 只读，其余按接口使用。

```
struct Spfa
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 126;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    bool valid = false;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<I> dis;
    vector<int> pre;

    Spfa(int n) : n(n), g(n + 1)
    {
        assert(0 < n && n < INT_MAX);
    }

    int add(int u, int v, long long w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(e.size() < INT_MAX);
        int id = e.size();
        e.push_back({u, v, w});
        g[u].push_back(id);
        valid = false;
        return id;
    }
```

FIFO 队列与负环判定（接上页同一结构体）：

```
    // s=0 checks the whole graph. A false result invalidates all distances.
    bool run(int s)
    {
        assert(0 <= s && s <= n);
        valid = false;
        dis.assign(n + 1, inf);
        pre.assign(n + 1, -1);
        vector<int> in(n + 1), len(n + 1);
        queue<int> q;
        for (int u = 1; u <= n; u++)
            if (!s || u == s)
            {
                dis[u] = 0;
                in[u] = 1;
                q.push(u);
            }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            in[u] = 0;
            for (int id : g[u])
            {
                auto [from, v, w] = e[id];
                if (dis[v] <= dis[u] + w) continue;
                dis[v] = dis[u] + w;
                pre[v] = id;
                len[v] = len[u] + 1;
                if (len[v] >= n) return false;
                if (!in[v])
                {
                    in[v] = 1;
                    q.push(v);
                }
            }
        }
        valid = true;
        return true;
    }
```

成功运行后的有限路径（接上页同一结构体）：

```
    // Forward original-edge IDs; empty for unreachable or a zero-edge path.
    vector<int> path(int t) const
    {
        assert(valid && 1 <= t && t <= n);
        vector<int> ids;
        if (dis[t] == inf) return ids;
        while (pre[t] != -1)
        {
            ids.push_back(pre[t]);
            t = e[pre[t]].u;
        }
        reverse(ids.begin(), ids.end());
        return ids;
    }
};
```

模板题使用：Luogu P3385 / 从 1 可达的负环

题意：T 组图，每条输入边权非负时加双向边，负数仅加 u 到 v；run(1) 只查从 1 可达的负环，有则 YES，否则 NO。m 是输入记录数，不是最终边数；不可达部分的负环不影响本题。

所需模板：Spfa（第 105 页，4.1.7 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3385

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T;
    cin >> T;
    while (T--)
    {
        int n, m;
        cin >> n >> m;
        Spfa t(n);
        while (m--)
        {
            int u, v, w;
            cin >> u >> v >> w;
            t.add(u, v, w);
            if (w >= 0) t.add(v, u, w);
        }
        cout << (t.run(1) ? "NO" : "YES") << '\n';
    }
}
```

模板题使用：Luogu P5960 / 差分约束

题意：输入 x_u−x_v≤w，加入 v 到 u 权 w。run(0) 覆盖全部约束；无解输出 NO，否则直接输出 dis[1..n] 为任意可行解。n,m≤5000、|w|≤10000，有限解在 int 范围内；不必另建超级源点。

所需模板：Spfa（第 105 页，4.1.7 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5960

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Spfa t(n);
    while (m--)
    {
        int u, v, w;
        cin >> u >> v >> w;
        t.add(v, u, w);
    }
    if (!t.run(0)) cout << "NO\n";
    else
        for (int u = 1; u <= n; u++)
            cout << (long long)t.dis[u] << " \n"[u == n];
}
```

应用题补充：Luogu P1993 / 小 K 的农场

题意：三类关系：1 a b c 表示 x_a−x_b≥c，加入 a 到 b 权 −c；2 表示 ≤c，加入 b 到 a 权 c；3 表示相等，加入双向 0。run(0) 成功输出 Yes，否则 No。解可整体平移到非负，不要额外加入上界。

所需模板：Spfa（第 105 页，4.1.7 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1993

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Spfa t(n);
    while (m--)
    {
        int op, a, b, c;
        cin >> op >> a >> b;
        if (op != 3) cin >> c;
        if (op == 1) t.add(a, b, -c);
        else if (op == 2) t.add(b, a, c);
        else
        {
            t.add(a, b, 0);
            t.add(b, a, 0);
        }
    }
    cout << (t.run(0) ? "Yes" : "No") << '\n';
}
```

接口用法演示：SPFA 有限最短距离与路径（接口演示）

演示约定：输入 n,m,s 和 1-based 有向边，权值 signed64；s=0 查全图。发现相关负环只输出 NEGATIVE CYCLE；否则逐点输出 INF 或 “int128 距离边数正向原边号”。全图负环与指定起点可达负环必须区分。

所需模板：Spfa（第 105 页，4.1.7 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/shortest-path/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    Spfa t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    if (!t.run(s))
    {
        cout << "NEGATIVE CYCLE\n";
        return 0;
    }
    auto print = [](__int128 x)
    {
        if (x < 0)
        {
            cout << '-';
            x = -x;
        }
        string a;
        do
        {
            a += char('0' + x % 10);
            x /= 10;
        } while (x);
        reverse(a.begin(), a.end());
        cout << a;
    };
```

继续处理操作（接上页同一 main）：

```
    for (int u = 1; u <= n; u++)
    {
        if (t.dis[u] == Spfa::inf) cout << "INF\n";
        else
        {
            auto path = t.path(u);
            print(t.dis[u]);
            cout << ' ' << path.size();
            for (int id : path) cout << ' ' << id;
            cout << '\n';
        }
    }
}
```

4.1.8 Bellman–Ford：负环影响与路径方案

BellmanFord(n)，点号 1..n，0<n<INT_MAX。add(u,v,w) 加有向边并返回 0-based 原边号，允许重边、自环和完整 signed64 边权，总边数 ≤INT_MAX。run(s) 返回是否不存在从 s 可达的负环；run(0) 把全体点初值设 0，等价于超级源点，用于查全图负环。每次 run 重置状态；add 后旧结果失效，须重新 run。

运行后 dis[v]==inf 表示不可达；neg[v] 为真表示从起点可达的负环能到达 v，距离为负无穷（这时不可读取 dis 作为答案）；其他点的 dis 为有限 int128 最短距离。path(v) 返回有限最短路的正向原边号；不可达、负无穷或空路径都返回空 vector，须结合 dis/neg 判定。cycle 为空表示无相关负环，否则是一个相关负环的正向原边号，可通过 e[id].u/v/w 还原顶点和权值。run(0) 的有限距离是从任意起点出发的最小值，允许零边路径，不是某个固定单源距离。

O(nm+n+m) 时间、O(n+m) 空间；path 输出 O(路径长)。inf=2^126，全过程 int128 累加，不能无条件转为 long long。在给定 n,m 的 int 边界内，n 轮最多 nm 次更新，标签对应游走至多 nm 条边，绝对值 <2^125，不碰 inf 也不溢出。e/g/dis/pre/neg/cycle 和 ready 只读，不手改。

```
struct BellmanFord
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 126;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    bool ready = false;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<I> dis;
    vector<int> pre, neg, cycle;

    BellmanFord(int n) : n(n)
    {
        assert(0 < n && n < INT_MAX);
        g.resize(n + 1);
    }

    int add(int u, int v, long long w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(e.size() < INT_MAX);
        int id = e.size();
        e.push_back({u, v, w});
        g[u].push_back(v);
        ready = false;
        return id;
    }
```

run：逐轮松弛与负环种子（接上页同一结构体）：

```
    // s=0 initializes every vertex to zero, for detecting any negative cycle.
    bool run(int s)
    {
        assert(0 <= s && s <= n);
        dis.assign(n + 1, s ? inf : 0);
        pre.assign(n + 1, -1);
        neg.assign(n + 1, 0);
        cycle.clear();
        dis[s] = 0;
        queue<int> q;
        int last = -1;
        for (int step = 0; step < n; step++)
        {
            last = -1;
            for (int id = 0; id < (int)e.size(); id++)
            {
                auto [u, v, w] = e[id];
                if (dis[u] == inf || dis[v] <= dis[u] + w) continue;
                dis[v] = dis[u] + w;
                pre[v] = id;
                last = v;
                if (step == n - 1 && !neg[v])
                {
                    neg[v] = 1;
                    q.push(v);
                }
            }
            if (last == -1) break;
        }
```

继续 run：恢复负环并传播；随后为 path（接上页同一结构体）：

```
        if (last != -1)
        {
            int x = last;
            for (int i = 0; i < n; i++) x = e[pre[x]].u;
            int start = x;
            do
            {
                cycle.push_back(pre[x]);
                x = e[pre[x]].u;
            } while (x != start);
            reverse(cycle.begin(), cycle.end());
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int v : g[u])
                if (!neg[v])
                {
                    neg[v] = 1;
                    q.push(v);
                }
        }
        ready = true;
        return cycle.empty();
    }

    // Finite path in forward original-edge IDs; empty also for unreachable/-inf.
    vector<int> path(int t) const
    {
        assert(ready && 1 <= t && t <= n);
        vector<int> ids;
        if (dis[t] == inf || neg[t]) return ids;
        while (pre[t] != -1)
        {
            ids.push_back(pre[t]);
            t = e[pre[t]].u;
        }
        reverse(ids.begin(), ids.end());
        return ids;
    }
};
```

模板题使用：AOJ GRL_1_B / 含负边的单源最短路

题意：0-based 有向图，n≤1000、m≤2000、边权绝对值 ≤10000。从 r 可达负环时输出 NEGATIVE CYCLE，否则逐点输出距离，不可达输出 INF。转成模板 1-based 编号；本题有限距离在 signed64 内，故可显式转 long long 输出。不可达位置的负环不影响此题判断。

所需模板：BellmanFord（第 112 页，4.1.8 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=GRL_1_B

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    BellmanFord t(n);
    while (m--)
    {
        int u, v, w;
        cin >> u >> v >> w;
        t.add(u + 1, v + 1, w);
    }
    if (!t.run(s + 1)) cout << "NEGATIVE CYCLE\n";
    else
        for (int u = 1; u <= n; u++)
        {
            if (t.dis[u] == BellmanFord::inf) cout << "INF\n";
            else cout << (long long)t.dis[u] << '\n';
        }
}
```

应用题补充：CSES1197 / 输出全图负环

题意：n≤2500、m≤5000，查任意位置负环并输出一条闭合顶点序列。调用 run(0) 而不是 run(1)；cycle 存正向边号，先输出第一条边的起点，再逐边输出终点。无负环 NO，有负环 YES。原题输出顶点，模板保留边号以区分平行边。

所需模板：BellmanFord（第 112 页，4.1.8 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1197/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BellmanFord t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    if (t.run(0)) cout << "NO\n";
    else
    {
        cout << "YES\n" << t.e[t.cycle[0]].u;
        for (int id : t.cycle) cout << ' ' << t.e[id].v;
        cout << '\n';
    }
}
```

接口用法演示：有限路径、不可达与负无穷（接口演示）

演示约定：输入 n,m,s 和 1-based 边，边权 signed64；s=0 代表全体点为起点。逐点输出 INF、−INF，或“int128 距离边数正向原边号列表”；末行 C 边数负环边号列表。空有限路径与不可达不同，负无穷点不用当前 dis 作答案。边号按输入顺序从 0 计。

所需模板：BellmanFord（第 112 页，4.1.8 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/shortest-path/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    BellmanFord t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u, v, w);
    }
    t.run(s);
    auto print = [](__int128 x)
    {
        if (x < 0)
        {
            cout << '-';
            x = -x;
        }
        string a;
        do
        {
            a += char('0' + x % 10);
            x /= 10;
        } while (x);
        reverse(a.begin(), a.end());
        cout << a;
    };
```

继续处理操作（接上页同一 main）：

```
    for (int u = 1; u <= n; u++)
    {
        if (t.dis[u] == BellmanFord::inf) cout << "INF\n";
        else if (t.neg[u]) cout << "-INF\n";
        else
        {
            auto path = t.path(u);
            print(t.dis[u]);
            cout << ' ' << path.size();
            for (int id : path) cout << ' ' << id;
            cout << '\n';
        }
    }
    cout << "C " << t.cycle.size();
    for (int id : t.cycle) cout << ' ' << id;
    cout << '\n';
}
```

4.2 k 短路

4.2.1 非负权严格次短行走

StrictSecondShortest(n)，点号 1..n，n<INT_MAX；add(u,v,w) 添加有向边，无向边添加两次。允许重边、自环及零权，0<=w<LLONG_MAX。run(s) 返回 dis[v][0/1]，分别为最短距离与严格大于它的最小距离；同长路线不重复计数，允许重复点边，s 到自身包含空路。不存在或长度 >=LLONG_MAX 返回 inf=LLONG_MAX；只保证此阈值以下的精确答案，超界扩展跳过。只读 g/n；可重复求解、加边后重算。O((n+m) log(n+m+1)) 时间，O(n+m) 空间，不返回行走方案。

```
struct StrictSecondShortest
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX;
    int n;
    vector<vector<pair<int, ll>>> g;

    StrictSecondShortest(int n) : n(n), g(n + 1)
    {
        assert(n > 0 && n < INT_MAX);
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(0 <= w && w < inf);
        g[u].push_back({v, w});
    }
```

继续求解（接上页同一结构体）：

```
    vector<array<ll, 2>> run(int s) const
    {
        assert(1 <= s && s <= n);
        vector<array<ll, 2>> dis(n + 1, {inf, inf});
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
        dis[s][0] = 0;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (d > dis[u][1]) continue;
            for (auto [v, w] : g[u])
            {
                if (w >= inf - d) continue;
                ll next = d + w;
                if (next < dis[v][0])
                {
                    swap(next, dis[v][0]);
                    q.push({dis[v][0], v});
                }
                if (dis[v][0] < next && next < dis[v][1])
                {
                    dis[v][1] = next;
                    q.push({next, v});
                }
            }
        }
        return dis;
    }
};
```

应用题补充：P2865 [USACO06NOV] Roadblocks G

题意：无向正权图，从 1 到 n 求严格次短距离，允许回溯；多条等长最短路不算次短。n<=5000，m<=100000，边权 1..5000，题目保证次短存在。

所需模板：StrictSecondShortest（第 119 页，4.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2865

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    StrictSecondShortest solver(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        solver.add(u, v, w);
        solver.add(v, u, w);
    }
    cout << solver.run(1)[n][1] << '\n';
}
```

4.2.2 正权前 K 短行走

KShortestWalks(n)，点号 1..n，n<INT_MAX；add(u,v,w) 添加有向边，要求 0<w<LLONG_MAX，零权边/环不在接口范围内。允许自环与重边，每次 add 视为不同边。run(s,t,k) 按非降序返回至多 k 个行走长度，k>=0；不同边序列分别计数，即使等长。允许重复点边，不是简单路径算法。s=t 时第一个答案为空路 0；若排除空路，调用 k+1 后删除首项，需 k<INT_MAX。终点仍可继续走再回来。长度 >=LLONG_MAX 不返回；只有阈值以下答案有保证。每个顶点至多扩展 k 次，O(n+km log(km+2)) 时间、O(n+km) 空间；k 大时应改用更高效算法。只读 g/n，可重复求解和加边后重算，不返回方案。

```
struct KShortestWalks
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX;
    int n;
    vector<vector<pair<int, ll>>> g;

    KShortestWalks(int n) : n(n), g(n + 1)
    {
        assert(n > 0 && n < INT_MAX);
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(0 < w && w < inf);
        g[u].push_back({v, w});
    }
```

继续求解（接上页同一结构体）：

```
    vector<ll> run(int s, int t, int k) const
    {
        assert(1 <= s && s <= n && 1 <= t && t <= n && k >= 0);
        vector<ll> answer;
        if (k == 0) return answer;
        vector<int> count(n + 1);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (count[u] == k) continue;
            count[u]++;
            if (u == t)
            {
                answer.push_back(d);
                if (int(answer.size()) == k) break;
            }
            for (auto [v, w] : g[u])
            {
                if (count[v] < k && w < inf - d)
                {
                    q.push({d + w, v});
                }
            }
        }
        return answer;
    }
};
```

应用题补充：P2901 [USACO08MAR] Cow Jogging G

题意：从 n 到 1 求前 k 条路线长度，等长按边序列分别计数，不足输出-1。n<=1000，m<=10000，k<=100，边权 1..10^6；边均从大编号指向小编号。此题为 DAG 应用，不验证有环接口域。

所需模板：KShortestWalks（第 122 页，4.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2901

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    KShortestWalks solver(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        solver.add(u, v, w);
    }
    auto answer = solver.run(n, 1, k);
    for (int i = 0; i < k; i++)
    {
        cout << (i < int(answer.size()) ? answer[i] : -1) << '\n';
    }
}
```

第五章　生成树问题

5.1　最小生成树

5.1.1　Kruskal 重构森林与阈值连通块

代码依赖：dsu（第 339 页，D.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

KruskalTree(n)，0≤n≤INT_MAX/2，原点 0..n-1；add(u,v,w) 加入无向边，返回插入编号，边数 ≤INT_MAX，支持完整 signed64 权、重边、自环、负权和断连。build(descending=false) 重建并返回连通块数；默认升序，true 降序。每次成功合并创建父点：ch 为左右儿子，val 为合并边权；叶点 ch={-1,-1} 且 val 无意义。总点数 k=2n-连通块数，父编号大于子编号；可按 n..k-1 聚合叶点信息，不需要模拟 DFS。up[0] 为父点，根为-1。

component(u,w,strict=false) 返回阈值子图中 u 所属分量的最高重构节点；默认升序保留边权 ≤w，降序保留 ≥w，strict 改成 < 或 >。其子树叶点恰为分量，可从聚合数组查大小、最小值等。同权边必须连续爬完所有合法祖先；不以叶点 val 充当无穷哨兵。构建 O(m log(m+1)+n log(n+1))，空间 O(m+n log(n+1))，查询 O(log(n+1))。依赖 dsu，合并前按大小交换根。支持重复 build、切换方向与值复制；加边后必须重建再查询，构建后成员只读。空图只允许 build、不允许查询。

```
struct KruskalTree
{
    struct Edge
    {
        int u, v;
        long long w;
    };

    int n;
    bool down = false;
    vector<Edge> e;
    vector<long long> val;
    vector<array<int, 2>> ch;
    vector<vector<int>> up;

    KruskalTree(int n) : n(n)
    {
        assert(0 <= n && n <= INT_MAX / 2);
    }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        e.push_back({u, v, w});
        return (int)e.size() - 1;
    }
```

重建森林与倍增表（接上页同一结构体）：

```
    int build(bool descending = false)
    {
        down = descending;
        ch.assign(n, {-1, -1});
        val.assign(n, 0);
        dsu d(n);
        vector<int> top(n), ids(e.size());
        iota(top.begin(), top.end(), 0);
        iota(ids.begin(), ids.end(), 0);
        sort(ids.begin(), ids.end(), [&](int a, int b)
        {
            if (e[a].w != e[b].w)
                return down ? e[a].w > e[b].w : e[a].w < e[b].w;
            return a < b;
        });
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            u = d.find(u);
            v = d.find(v);
            if (u == v) continue;
            if (d.sz[u] < d.sz[v]) swap(u, v);
            int x = ch.size();
            ch.push_back({top[u], top[v]});
            val.push_back(w);
            d.merge(u, v);
            top[u] = x;
        }
        int k = ch.size();
        int h = max(1, (int)bit_width((unsigned)k));
        up.assign(h, vector<int>(k, -1));
        for (int u = n; u < k; u++)
        {
            for (int v : ch[u]) up[0][v] = u;
        }
        for (int j = 1; j < h; j++)
        {
            for (int u = 0; u < k; u++)
            {
                int v = up[j - 1][u];
                if (v != -1) up[j][u] = up[j - 1][v];
            }
        }
        return 2 * n - k;
    }
```

阈值分量查询（接上页同一结构体）：

```
    int component(int u, long long w, bool strict = false) const
    {
        assert(0 <= u && u < n && !up.empty());
        for (int j = (int)up.size() - 1; j >= 0; j--)
        {
            int v = up[j][u];
            if (v == -1) continue;
            bool ok = down ? val[v] >= w : val[v] <= w;
            if (strict && val[v] == w) ok = false;
            if (ok) u = v;
        }
        return u;
    }
};
```

为何阈值查询正确 升序处理完所有权不超过 w 的边后，并查集恰好是该阈值子图的连通分量。每次成功合并的两棵子树叶点集合不交，新父点记录其并集；沿叶点到根的内部点权单调不减。因此最高合法祖先的叶点集合就是答案。降序情形反向比较；严格阈值排除等权合并。同权内部点可能连续出现，倍增必须爬到最高合法点。

如何聚合分量 叶点先存原数据，随后按编号 n 到 k-1 合并两个儿子的结果，可求大小、最小值、最大值等。不必为聚合另跑 DFS；这利用的是建点时已经确定的父子拓扑顺序。断连图的各棵树独立，无须添加超级根。

归程建模 先用长度求每个点到 1 号点的最短步行距离。汽车只能走海拔严格大于水位的边，故选降序构建和 strict=true；可在分量内任意点下车，答案为该重构子树的叶点距离最小值。普通最小生成树的总权不能代替这个分量查询。

应用题补充：P4768 [NOI2018] 归程

题意：汽车只能经过海拔严格大于水位的边；可在可达分量任意点下车，再步行到 1 号点。Dijkstra 预处理步行距离，降序重构树聚合最小值，支持依赖上次答案的在线解码。

所需模板：KruskalTree（第 125 页，5.1.1 节）、Dijkstra（第 84 页，4.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4768

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        int n, m;
        cin >> n >> m;
        KruskalTree tr(n);
        Dijkstra g(n);
        for (int i = 0; i < m; i++)
        {
            int u, v;
            long long l, a;
            cin >> u >> v >> l >> a;
            g.add(u, v, l);
            g.add(v, u, l);
            tr.add(u - 1, v - 1, a);
        }
        g.run(1);
        tr.build(true);
        vector<long long> best(tr.ch.size());
        for (int u = 0; u < n; u++) best[u] = g.dis[u + 1];
        for (int u = n; u < (int)tr.ch.size(); u++)
        {
            auto [l, r] = tr.ch[u];
            best[u] = min(best[l], best[r]);
        }
        int q, k;
        long long s, last = 0;
        cin >> q >> k >> s;
        while (q--)
        {
            long long v, p;
            cin >> v >> p;
            v = (v + k * last - 1) % n;
            p = (p + k * last) % (s + 1);
            last = best[tr.component(v, p, true)];
            cout << last << '\n';
        }
    }
}
```

接口用法演示：API 阈值连通块聚合

演示约定：输入 n m q down，点 0-based；m 条 u v w，无向可断连。先输出全图连通块数。每次 u w strict，输出对应阈值连通块的点数与最小原点号；down 和 strict 取 0/1。

所需模板：KruskalTree（第 125 页，5.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/mst/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q, down;
    cin >> n >> m >> q >> down;
    KruskalTree tr(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        tr.add(u, v, w);
    }
    cout << tr.build(down) << '\n';
    int k = tr.ch.size();
    vector<int> cnt(k, 1), low(k);
    iota(low.begin(), low.end(), 0);
    for (int u = n; u < k; u++)
    {
        auto [l, r] = tr.ch[u];
        cnt[u] = cnt[l] + cnt[r];
        low[u] = min(low[l], low[r]);
    }
    while (q--)
    {
        int u, strict;
        long long w;
        cin >> u >> w >> strict;
        int v = tr.component(u, w, strict);
        cout << cnt[v] << ' ' << low[v] << '\n';
    }
}
```

5.1.2 曼哈顿最小生成树

代码依赖：dsu（第 339 页，D.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

Point 为 pair<long long,long long>，坐标支持完整有符号 64 位，点数须 ≤INT_MAX 且内存可分配。candidates(points) 给出最多 4n 条 (weight,u,v) 候选，允许重复边；solve(points) 返回精确 128 位 weight 与原下标边对 edges。空/单点返回 0 且无边，重合点仍为不同顶点；同权最优树不唯一。先转 128 位再变换/加减，输入不变。四方向扫描与 Kruskal 为 O(n log(n+1))、O(n) 空间；调用处选较大 dsu 根在前，不改变普通 dsu 语义。候选保证只针对无约束完整 L1 图，不能直接删禁边后声称仍含受限 MST。

```
struct ManhattanMST
{
    using I = __int128;
    using Point = pair<long long, long long>;
    using Edge = tuple<I, int, int>;
    struct Result
    {
        I weight = 0;
        vector<pair<int, int>> edges;
    };

    static vector<Edge> candidates(const vector<Point> &points)
    {
        assert(points.size() <= INT_MAX);
        int n = int(points.size());
        vector<pair<I, I>> p(points.begin(), points.end());
        vector<int> id(n);
        iota(id.begin(), id.end(), 0);
        vector<Edge> e;
        e.reserve(size_t(n) * 4);
        for (int dir = 0; dir < 4; dir++)
        {
            sort(id.begin(), id.end(), [&](int i, int j)
            {
                return pair{p[i].first + p[i].second, i} <
                       pair{p[j].first + p[j].second, j};
            });
```

活动点扫描与 Kruskal（接上页同一结构体，仍在四轮循环中）：

```
            map<I, int> sweep;
            for (int i : id)
            {
                auto it = sweep.lower_bound(-p[i].second);
                while (it != sweep.end())
                {
                    int j = it->second;
                    I dx = p[i].first - p[j].first;
                    I dy = p[i].second - p[j].second;
                    if (dx < dy) break;
                    e.emplace_back(dx + dy, i, j);
                    it = sweep.erase(it);
                }
                sweep[-p[i].second] = i;
            }
            for (auto &[x, y] : p)
                if (dir & 1) x = -x;
                else swap(x, y);
        }
        return e;
    }

    static Result solve(const vector<Point> &points)
    {
        auto e = candidates(points);
        int n = int(points.size());
        Result r;
        if (n < 2) return r;
        sort(e.begin(), e.end());
        dsu uf(n);
        for (auto [w, u, v] : e)
        {
            int a = uf.find(u), b = uf.find(v);
            if (a == b) continue;
            if (uf.size(a) < uf.size(b)) swap(a, b);
            uf.merge(a, b);
            r.weight += w;
            r.edges.emplace_back(u, v);
            if (r.edges.size() == points.size() - 1) break;
        }
        assert(r.edges.size() == points.size() - 1);
        return r;
    }
};
```

模板题使用：Library Checker Manhattan MST

题意：1≤N≤200000，N 个点的坐标 0..10^9，可重合且每个输入位置仍是不同顶点。完整图边权为曼哈顿距离；输出最小总权及 N−1 条 0-based 原编号边，任意最优树均可。结果 weight 为 128 位；本题粗上界 2e9 乘以 N−1，严格小于 4e14，所以此驱动才可转 long long 打印。N=1 输出 0、不输出边；库的空输入和有符号 64 位坐标另属 API 扩展。

所需模板：dsu（第 339 页，D.1.1 节）、ManhattanMST（第 130 页，5.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/manhattanmst

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<ManhattanMST::Point> p(n);
    for (auto &[x, y] : p) cin >> x >> y;
    auto r = ManhattanMST::solve(p);
    cout << (long long)r.weight << '\n';
    for (auto [u, v] : r.edges) cout << u << ' ' << v << '\n';
}
```

5.1.3 严格与非严格次小生成树、换边方案

代码依赖：dsu（第 339 页，D.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

SecondMST(n)，n≥1，点号 0..n-1。add(u,v,w) 返回 0-based 边号；允许自环、重边、完整 signed64 位权。solve(strict=true) 返回 connected、128 位 MST 总权 weight、所选边号 tree，以及可选 128 位次小权 next 和换入/换出边号 in/out。strict=true 要求权值严格增大；false 要求边集与返回的 MST 不同，可同权。不连通返回默认空结果；无次小树时 next 为空且 in/out=-1。存在时将 tree 中的 out 删去并加入 in 即得方案。可加边后重算；每次 solve 重建。依赖 dsu，沿用路径减半及 y 根挂 x 根；不用按秩合并。时间 O((n+m)log(n+m))，空间 O(m+n log n)，DFS 递归深度 O(n)。merge/dfs/path 与工作数组为内部实现；原始 e 只读。

```
struct SecondMST
{
    using I = __int128_t;
    using Pair = array<int, 2>;

    struct Edge
    {
        int u, v;
        long long w;
    };

    struct Result
    {
        bool connected = false;
        I weight = 0;
        vector<int> tree;
        optional<I> next;
        int in = -1, out = -1;
    };

    int n, h;
    vector<Edge> e;
    vector<vector<pair<int, int>>> g;
    vector<int> dep;
    vector<vector<int>> up;
    vector<vector<Pair>> mx;

    SecondMST(int n) : n(n), h(bit_width((unsigned)n)) { assert(n > 0); }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        e.push_back({u, v, w});
        return (int)e.size() - 1;
    }
```

不同权摘要与递归预处理（接上页同一结构体）：

```
    // Keep edge IDs of the two largest DISTINCT weights; -1 means absent.
    Pair merge(Pair a, Pair b) const
    {
        for (int x : b)
        {
            if (x == -1) continue;
            if (a[0] == -1 || e[x].w > e[a[0]].w)
            {
                a[1] = a[0];
                a[0] = x;
            }
            else if (e[x].w != e[a[0]].w && (a[1] == -1 || e[x].w > e[a[1]].w))
                a[1] = x;
        }
        return a;
    }

    void dfs(int u, int p)
    {
        for (int j = 1; j < h; j++)
        {
            int v = up[j - 1][u];
            up[j][u] = up[j - 1][v];
            mx[j][u] = merge(mx[j - 1][u], mx[j - 1][v]);
        }
        for (auto [v, id] : g[u])
            if (v != p)
            {
                dep[v] = dep[u] + 1;
                up[0][v] = u;
                mx[0][v] = {id, -1};
                dfs(v, u);
            }
    }
```

路径查询（接上页同一结构体）：

```
    Pair path(int u, int v) const
    {
        Pair ans{-1, -1};
        if (dep[u] < dep[v]) swap(u, v);
        for (int j = h - 1; j >= 0; j--)
            if ((dep[u] - dep[v]) >> j & 1)
            {
                ans = merge(ans, mx[j][u]);
                u = up[j][u];
            }
        if (u == v) return ans;
        for (int j = h - 1; j >= 0; j--)
            if (up[j][u] != up[j][v])
            {
                ans = merge(ans, merge(mx[j][u], mx[j][v]));
                u = up[j][u];
                v = up[j][v];
            }
        return merge(ans, merge(mx[0][u], mx[0][v]));
    }
```

Kruskal 与逐边换入（接上页同一结构体）：

```
    Result solve(bool strict = true)
    {
        vector<int> ids(e.size());
        iota(ids.begin(), ids.end(), 0);
        sort(ids.begin(),
             ids.end(),
             [&](int a, int b) { return tie(e[a].w, a) < tie(e[b].w, b); });
        dsu ds(n);
        vector<bool> used(e.size());
        g.assign(n, {});
        Result ans;
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            if (!ds.merge(u, v)) continue;
            used[id] = true;
            ans.weight += (I)w;
            ans.tree.push_back(id);
            g[u].push_back({v, id});
            g[v].push_back({u, id});
        }
        if ((int)ans.tree.size() != n - 1) return {};
        ans.connected = true;
        dep.assign(n, 0);
        up.assign(h, vector<int>(n));
        mx.assign(h, vector<Pair>(n, Pair{-1, -1}));
        dfs(0, -1);
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            if (used[id] || u == v) continue;
            auto p = path(u, v);
            int out = p[0];
            if (strict && e[out].w == w) out = p[1];
            if (out == -1) continue;
            I cost = ans.weight + (I)w - (I)e[out].w;
            if (!ans.next || cost < *ans.next)
            {
                ans.next = cost;
                ans.in = id;
                ans.out = out;
            }
        }
        return ans;
    }
};
```

应用题补充：Luogu P4180 [BJWC2010] 严格次小生成树

题意：无向图求总权严格大于 MST 的最小生成树权值；n 至 10 万、m 至 30 万、边权 $0..10^9$，允许自环并保证答案存在。1-based 顶点读入后减一，solve 默认严格；题目答案可安全转 long long。比赛应用，不计非比赛模板覆盖。

所需模板：dsu（第 339 页，D.1.1 节）、SecondMST（第 133 页，5.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4180

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    SecondMST mst(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        mst.add(u - 1, v - 1, w);
    }
    auto ans = mst.solve();
    // The problem guarantees a strict second tree and a 64-bit answer.
    assert(ans.connected && ans.next);
    cout << (long long)*ans.next << '\n';
}
```

5.1.4 稠密图 Prim 最小生成森林

Prim(n)，0≤n≤INT_MAX，点号 0..n-1；add(u,v,w) 自动加无向边，允许完整 signed64 权、零权、负权、自环，重边只保留最小权。不保存原边编号；cost 为 int128 矩阵，inf=2^126 表示缺边，−1 和 LLONG_MAX 均为合法边权。

run() 每次重建最小生成森林，返回原图连通块数。n>0 时返回 1 表示 MST，空图返回 0。weight 为 int128 总权；pre[v] 为父点，根为 −1，非根选边权为 cost[pre[v]][v]。父点森林无环，支持断连和孤立点；同权任选，不承诺字典序。加边后须重新 run，支持重复运行与独立复制；成员只读。

构造和存储 $O(n^2)$，add 为 O(1)，run 为 $O(n^2)$，矩阵约 $16n^2$ 字节，按实际内存选择 n；稀疏图优先 Kruskal。每步选到已选点集的最轻边；所有剩余点均无跨边时开启新树，不把 inf 计入总权。只向未选点严格更新父点，负自环不入树。总权绝对值 <2^94，可为 −1 或超出 long long，不能用权值判断断连。

```
struct Prim
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 126;
    int n;
    vector<vector<I>> cost;
    vector<int> pre;
    I weight = 0;

    Prim(int n) : n(n), cost(n, vector<I>(n, inf))
    {
        assert(n >= 0);
    }

    void add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        cost[u][v] = cost[v][u] = min(cost[u][v], I(w));
    }
```

矩阵扫描、割边松弛与父点森林（接上页同一结构体）：

```
    // Rebuild a minimum spanning forest; return its component count.
    int run()
    {
        vector<I> d(n, inf);
        vector<bool> vis(n);
        pre.assign(n, -1);
        weight = 0;
        int cnt = 0;
        for (int step = 0; step < n; step++)
        {
            int u = -1;
            for (int v = 0; v < n; v++)
            {
                if (!vis[v] && (u == -1 || d[v] < d[u])) u = v;
            }
            if (d[u] == inf)
            {
                cnt++;
            }
            else
            {
                weight += d[u];
            }
            vis[u] = true;
            for (int v = 0; v < n; v++)
                if (!vis[v] && cost[u][v] < d[v])
                {
                    d[v] = cost[u][v];
                    pre[v] = u;
                }
        }
        return cnt;
    }
};
```

模板题使用：AOJ ALDS1_12_A / Minimum Spanning Tree

题意：输入 n 及 n×n 对称邻接矩阵，−1 表示缺边，其余为 0..2000；n≤100 且保证连通。只读取上三角的有效边，输出 MST 总权；本题可安全转 long long。注意输入 −1 缺边是题目约定，Prim.add 的 −1 是合法负权。

所需模板：Prim（第 138 页，5.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=ALDS1_12_A&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    Prim g(n);
    for (int u = 0; u < n; u++)
        for (int v = 0; v < n; v++)
        {
            int w;
            cin >> w;
            if (u < v && w != -1) g.add(u, v, w);
        }
    g.run();
    cout << (long long)g.weight << '\n';
}
```

接口用法演示：矩阵 Prim 总权与父点森林（接口演示）

演示约定：输入 n,m 及 0-based 无向 signed64 边，输出连通块数、int128 最小生成森林总权，以及 pre[0..n-1]；根为 −1，其余为父点。允许空图、断连、负权、自环和重边，重边合并为最小权；不输出原边编号。

所需模板：Prim（第 138 页，5.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/mst/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Prim g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u, v, w);
    }
    cout << g.run() << ' ';
    __int128 x = g.weight;
    if (x < 0)
    {
        cout << '-';
        x = -x;
    }
    string a;
    do
    {
        a += char('0' + x % 10);
        x /= 10;
    } while (x);
    reverse(a.begin(), a.end());
    cout << a;
    for (int p : g.pre) cout << ' ' << p;
    cout << '\n';
}
```

5.1.5 Kruskal 最小生成树与生成森林

代码依赖：dsu（第 339 页，D.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

Kruskal(n)，0≤n≤INT_MAX，点号 0..n-1；add(u,v,w) 加入无向边并返回 0-based 原边号，边权为完整 signed64，支持重边、自环和负权，边数 ≤INT_MAX。run() 每次从全部原边重建最小生成森林，返回连通块数；n>0 时返回 1 表示存在 MST，空图返回 0。weight 为 int128 总权，ids 为所选原边号，按 (权值, 原编号) 排序；e 保持插入顺序。加边后必须重新 run，旧结果不代表新图。支持重复运行和独立值复制。

依赖普通 dsu，调用方先把较大根放在前，保留 dsu 的 fa[y]=x 语义。时间 O(n+m log(m+1)+m α(n+1))，空间 O(n+m)；add 均摊 O(1)。总权绝对值 <2^94，不能无条件转 long long。自环不入树，断开图仍返回各块最小树；不使用 −1 总权判断失败，合法 MST 总权也可为 −1。成员只读，n 和 e 不直接修改。

```
struct Kruskal
{
    using I = __int128_t;
    struct Edge
    {
        int u, v;
        long long w;
    };

    int n;
    vector<Edge> e;
    vector<int> ids;
    I weight = 0;

    Kruskal(int n) : n(n)
    {
        assert(n >= 0);
    }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(e.size() < INT_MAX);
        e.push_back({u, v, w});
        return int(e.size()) - 1;
    }
```

边排序、合并与原边方案（接上页同一结构体）：

```
    // Rebuild a minimum spanning forest; return its component count.
    int run()
    {
        vector<int> order(e.size());
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int a, int b)
        {
            return tie(e[a].w, a) < tie(e[b].w, b);
        });
        dsu d(n);
        ids.clear();
        weight = 0;
        int cnt = n;
        for (int id : order)
        {
            auto [u, v, w] = e[id];
            u = d.find(u);
            v = d.find(v);
            if (u == v) continue;
            if (d.sz[u] < d.sz[v]) swap(u, v);
            d.merge(u, v);
            ids.push_back(id);
            weight += I(w);
            cnt--;
        }
        return cnt;
    }
};
```

模板题使用：Luogu P3366 / 最小生成树

题意：无向图求最小生成树总权，不连通输出 orz；n≤5000、m≤200000、边权 1..10000，允许自环重边。输入 1-based 点号减一，run()!=1 表示断连；本题总权可安全转 long long。

所需模板：dsu（第 339 页，D.1.1 节）、Kruskal（第 142 页，5.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3366

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Kruskal g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u - 1, v - 1, w);
    }
    if (g.run() != 1)
    {
        cout << "orz\n";
    }
    else
    {
        cout << (long long)g.weight << '\n';
    }
}
```

接口用法演示：最小生成森林与原边方案（接口演示）

演示约定：输入 n,m 及 0-based 无向 signed64 边，输出连通块数、int128 最小生成森林总权、选边数和 0-based 原边编号。支持空图、负权、自环重边与断连；方案按 (权值, 原编号) 排序。接口演示，不计在线模板题。

所需模板：dsu（第 339 页，D.1.1 节）、Kruskal（第 142 页，5.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/mst/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Kruskal g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u, v, w);
    }
    cout << g.run() << ' ';
    __int128 x = g.weight;
    if (x < 0)
    {
        cout << '-';
        x = -x;
    }
    string a;
    do
    {
        a += char('0' + x % 10);
        x /= 10;
    } while (x);
    reverse(a.begin(), a.end());
    cout << a << ' ' << g.ids.size();
    for (int id : g.ids) cout << ' ' << id;
    cout << '\n';
}
```

5.1.6 曼哈顿最小生成树：闭锥扫描与平局割证明

接口见第 5.1.2 节（第 130 页），正式题用法见第 5.1.2 节（第 132 页）。完整无向图的边权是 $d(p,q) = |p_x - q_x| + |p_y - q_y|$。不枚举二次边集：每个点只保留四个向右闭锥中的最近邻，再对候选边作 Kruskal。候选图含有至少一棵最小生成树，不保证含所有最优边；禁边、颜色等额外约束不能直接套用。

一个方向的扫描 变换后的锥取 $q_x - p_x \geq q_y - p_y \geq 0$。按 $(x+y, \text{原编号})$ 排序，活动点用 $-y$ 作 map 键。依次从 $y_p \leq y_q$ 的点开始检查，直到 $x_q - x_p < y_q - y_p$。活动点的 $x - y$ 随 y 递减而严格递增，因此首次失败后可停止。第一次满足条件的后继具有最小 $x+y$，其距离正是 $(x_q + y_q) - (x_p + y_p)$。删掉已找到邻居的活动点，每点每轮最多删除一次，故每轮至多 $n-1$ 条候选边。四轮坐标分别为 $(x,y), (y,x), (-y,x), (x,-y)$，覆盖右侧四个闭锥。任意无向边都能从左端点指向其中一个锥；竖直边任选一个方向。总时间 $O(n \log n)$，额外空间 $O(n)$。

重复坐标与闭边界 重复坐标是不同顶点。排序平局按原编号，连续代表之间生成零权链；同一 $x+y$ 的其他位置不能以非零距离删掉该代表，最后一个代表继续寻找非重合邻居。可先在证明中收缩这些零链。若一个割拆开重合点类，零链已有最小的过割边；否则割可直接下降到收缩图。闭锥内只能使用 $d(r,q) \leq \max(d(p,r), d(p,q))$，不能写严格小于：$p = (0,0), q = (2,0), r = (1,1)$ 的三条距离都是 2。

平局也成立的割证明 在非重合点图的任意割中，选距离最小为 δ 的过割点对 p, q，并在平局中令 $p_x + q_x$ 最大；令 p 为左端点。选择包含 q 的右向锥，若边恰在对角线上，选靠近水平轴的浅锥。该锥的最近邻 r 满足 $d(p,r) \leq \delta$。若 r 和 q 在割的同侧，则 pr 已是候选最小过割边。否则 rq 过割且 $d(r,q) \leq \delta$：严格小于违反最小性，等于也会违反上述平局选择。

后一个结论可在局部锥坐标 $X \geq Y \geq 0$ 中检查。满足 $d(p,r) \leq \delta$ 的点位于顶点 $(0,0), (\delta,0), (\delta/2, \delta/2)$ 的三角形。当 q 严格位于锥内时，距离达到 δ 只能取已排除的原点；q 在主轴时，非零等号点在 $X = Y$ 的射线上；q 在对角线时，非零等号点只能是 $(\delta, 0)$。后两种情况在所选右向锥中都使 $r_x > p_x$（对角线选浅锥正是为了保证这一点），于是过割点对 r, q 的横坐标和更大，矛盾。所以每个割都保留最小边；在候选图上应用 Prim 的割性质，得到的树也是完整图的 MST。

整数范围与抄写合同 输入为 signed 64 位整数对；先整体转换为 signed 128 位，再进行和、差、变号及距离求和。单边最大距离为 $2^{65} - 2$；在通常 32 位 int 编号上限内，总权远小于 signed 128 位上限。返回原始 0 基编号和 128 位总权；空集与单点返回 0 和空边集。正式题限制 $1 \leq n \leq 200000$、$0 \leq x, y \leq 10^9$，仅该驱动可安全转回 long long 输出。并查集只在本调用点按大小选择根的合并顺序，不改普通 dsu 的接口语义。

参考：Zhou–Shenoy–Nicholls (2002)，DOI `10.1016/S0020-0190(01)00232-0`；四轮 map 实现参考 KACTL `ManhattanMST.h`（CC0，固定版本与字节哈希见来源记录）。原论文半开锥的严格不等式不能直接代入这里的闭锥代码。

5.2 最小树形图

5.2.1 有向最小树形图

solve(n,root,edges)，点号 0..n-1，n≥1、0≤root<n；返回以 root 为根向外可达的最小树形图费用，无解 nullopt。允许负权、重边和自环（自环不选），n=1 的费用为 0；费用 -1 与无解可区分。输入、缩环后边权及累计费用绝对值须小于 2^120，类型为 int128。每个非根点选最小入边；若成环则缩点，跨环入边减去原终点已选入边费用，以计算替换代价。O(VE)，edges 按值传入，原图保持不变。仅返回费用，不恢复边方案。双版本已做所有根的入边枚举与 800 点连续缩环核验，暂无专门 OJ AC。

```
struct Arborescence
{
    using I = __int128_t;

    struct Edge
    {
        int u, v;
        I w;
    };

    // Directed minimum rooted OUT-arborescence cost, 0-based, or nullopt.
    static optional<I> solve(int n, int root, vector<Edge> e)
    {
        const I inf = I(1) << 120;
        I ans = 0;
        while (true)
        {
            vector<I> in(n, inf);
            vector<int> pre(n, -1), id(n, -1), vis(n, -1);
            for (auto a : e)
                if (a.u != a.v && a.w < in[a.v])
                {
                    in[a.v] = a.w;
                    pre[a.v] = a.u;
                }
            in[root] = 0;
            for (I x : in)
            {
                if (x == inf) return nullopt;
                ans += x;
            }
```

判环与缩点（接上页同一函数）：

```
            int cnt = 0;
            for (int i = 0; i < n; i++)
            {
                int u = i;
                while (vis[u] != i && id[u] == -1 && u != root)
                {
                    vis[u] = i;
                    u = pre[u];
                }
                if (u != root && id[u] == -1)
                {
                    id[u] = cnt;
                    for (int v = pre[u]; v != u; v = pre[v]) id[v] = cnt;
                    ++cnt;
                }
            }
            if (!cnt) return ans;
            for (int i = 0; i < n; i++)
                if (id[i] == -1) id[i] = cnt++;
            for (auto &a : e)
            {
                int v = a.v;
                a.u = id[a.u];
                a.v = id[a.v];
                if (a.u != a.v) a.w -= in[v];
            }
            root = id[root];
            n = cnt;
        }
    }
};
```

模板题使用：Luogu P4716

题意：给定根的有向最小树形图费用，无解输出-1。n 至 100、m 至 10^4、正权至 10^6；顶点和根从 1-based 转 0-based。optional 有值时才解引用，本题总费用可转 long long；核心只返回费用，不恢复选边。

所需模板：Arborescence（第 147 页，5.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4716

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, r;
    cin >> n >> m >> r;
    vector<Arborescence::Edge> e;
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        e.push_back({u - 1, v - 1, w});
    }
    auto ans = Arborescence::solve(n, r - 1, e);
    // This problem's answer fits in long long.
    cout << (ans ? (long long)*ans : -1) << '\n';
}
```

第六章　连通性相关

6.1　强连通分量

6.1.1　Tarjan 强连通分量与缩点

TarjanSCC(n)，点号 1..n；add(u,v) 加有向边，run() 从头计算。bel[u] 为 1..cnt 的强连通分量编号，跨分量边总从大编号指向小编号，可按 cnt..1 做拓扑 DP。dag() 返回缩点邻接表，保留重边；若 DP 需要对不同邻点去重，调用者应排序去重。支持自环、重边、不连通图和重复 run。时间及图存储 O(n+m)，递归 DFS 另需 O(n) 调用栈；本地验证应配置足够栈内存。注意已有 SCC 模板的编号方向与本模板相反。

```
struct TarjanSCC
{
    int n, timer = 0, cnt = 0;
    vector<vector<int>> g;
    vector<int> dfn, low, bel, st;

    TarjanSCC(int n) : n(n), g(n + 1) {}

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back(v);
    }

    void dfs(int u)
    {
        dfn[u] = low[u] = ++timer;
        st.push_back(u);
        for (int v : g[u])
        {
            if (!dfn[v])
            {
                dfs(v);
                low[u] = min(low[u], low[v]);
            }
            else if (!bel[v])
                low[u] = min(low[u], dfn[v]);
        }
        if (low[u] == dfn[u])
        {
            ++cnt;
            while (true)
            {
                int v = st.back();
                st.pop_back();
                bel[v] = cnt;
                if (v == u) break;
            }
        }
    }

```

```
    // IDs 1..cnt are in reverse topological order. Recursive DFS needs O(n) stack.
    void run()
    {
        timer = cnt = 0;
        dfn.assign(n + 1, 0);
        low = dfn;
        bel = dfn;
        st.clear();
        for (int u = 1; u <= n; u++)
        {
            if (!dfn[u]) dfs(u);
        }
    }

    // Retains parallel edges. Use after run().
    vector<vector<int>> dag() const
    {
        vector<vector<int>> result(cnt + 1);
        for (int u = 1; u <= n; u++)
        {
            for (int v : g[u])
            {
                if (bel[u] != bel[v]) result[bel[u]].push_back(bel[v]);
            }
        }
        return result;
    }
};
```

模板题使用：Library Checker scc

题意：把有向图分成强连通分量，并按缩点图的拓扑顺序输出各分量。题目点号为 0..n-1，调用时加 1，输出时减 1。

所需模板：TarjanSCC（第 150 页，6.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/scc

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    TarjanSCC g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        g.add(u + 1, v + 1);
    }
    g.run();
    vector<vector<int>> c(g.cnt + 1);
    for (int u = 1; u <= n; u++) c[g.bel[u]].push_back(u - 1);
    cout << g.cnt << '\n';
    for (int i = g.cnt; i >= 1; i--)
    {
        cout << c[i].size();
        for (int u : c[i]) cout << ' ' << u;
        cout << '\n';
    }
}
```

应用题补充：Luogu P3275 / 糖果

题意：每人至少 1 颗，五类关系转成 x_v≥x_u+w，w 只取 0/1。强连通分量内出现权 1 边即无解 −1；其余按 Tarjan 分量编号从大到小递推最长路，每分量初值 1，按原顶点累加最小糖果总量。n,k≤100000，O(n+k)，总量用 long long；不能把每分量只计一次。

所需模板：TarjanSCC（第 150 页，6.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3275

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    TarjanSCC t(n);
    vector<array<int, 3>> e;
    auto add = [&](int u, int v, int w)
    {
        t.add(u, v);
        e.push_back({u, v, w});
    };
    while (m--)
    {
        int op, a, b;
        cin >> op >> a >> b;
        if (op == 1)
        {
            add(a, b, 0);
            add(b, a, 0);
        }
        else if (op == 2) add(a, b, 1);
        else if (op == 3) add(b, a, 0);
        else if (op == 4) add(b, a, 1);
        else add(a, b, 0);
    }
```

继续处理操作（接上页同一 main）：

```
    t.run();
    vector<vector<pair<int, int>>> g(t.cnt + 1);
    for (auto [u, v, w] : e)
    {
        int a = t.bel[u], b = t.bel[v];
        if (a == b)
        {
            if (w)
            {
                cout << -1 << '\n';
                return 0;
            }
        }
        else g[a].push_back({b, w});
    }
    vector<int> dis(t.cnt + 1, 1);
    for (int u = t.cnt; u >= 1; u--)
        for (auto [v, w] : g[u])
            dis[v] = max(dis[v], dis[u] + w);
    long long sum = 0;
    for (int u = 1; u <= n; u++) sum += dis[t.bel[u]];
    cout << sum << '\n';
    return 0;
}
```

相关应用：函数图包含尾链与环分解，导航无独立页；需正文定位，不应等同一般 SCC 算法。

6.1.2 函数图分解、跳转与到达

FunctionalGraph(to) 或 build(to) 重建，每个点恰有一条出边，to[u] 为后继，点号 0..n-1；空图允许，但不能查询。advance(u,k) 返回第 k 步到达的点，k 为完整 unsigned long long；steps(u,v) 返回首次到达 v 的最少步数，无法到达返回 -1，自身为 0。component[u] 为所在环的编号；cycles[id] 按后继方向保存环；entry[u] 为首次入环点；depth[u] 为入环步数；position 仅在环点上有效，非环点为 -1。order 为入度剥离顺序，逆序可用于从后继向入环树传播状态；up 是倍增表。所有字段只读。O(n log n) 预处理及空间，入环前查询 O(log n)，入环后取模 O(1)，不为 64 位步数分配 64 层表。默认构造为空图，支持自环、多个分量及重建。n≤INT_MAX，实际规模须满足内存。

```
struct FunctionalGraph
{
    using U = unsigned long long;
    int n;
    vector<int> component, position, entry, depth, order;
    vector<vector<int>> cycles, up;

    explicit FunctionalGraph(vector<int> to = {}) { build(move(to)); }

    void build(vector<int> to)
    {
        assert(to.size() <= INT_MAX);
        n = to.size();
        vector<int> degree(n);
        for (int v : to)
        {
            assert(0 <= v && v < n);
            degree[v]++;
        }
        order.clear();
        for (int u = 0; u < n; u++)
            if (degree[u] == 0) order.push_back(u);
        for (int i = 0; i < int(order.size()); i++)
        {
            int v = to[order[i]];
            if (--degree[v] == 0) order.push_back(v);
        }
```

环、入环树与倍增表（接上页同一结构体与函数）：

```
        component.assign(n, -1);
        position.assign(n, -1);
        entry.resize(n);
        depth.assign(n, 0);
        cycles.clear();
        for (int u = 0; u < n; u++)
            if (degree[u] && component[u] == -1)
            {
                vector<int> cycle;
                int v = u, id = cycles.size();
                do
                {
                    component[v] = id;
                    position[v] = cycle.size();
                    entry[v] = v;
                    cycle.push_back(v);
                    v = to[v];
                } while (v != u);
                cycles.push_back(move(cycle));
            }
        for (int i = int(order.size()) - 1; i >= 0; i--)
        {
            int u = order[i], v = to[u];
            component[u] = component[v];
            entry[u] = entry[v];
            depth[u] = depth[v] + 1;
        }
        up.assign(1, to);
        for (int k = 1; (1ULL << k) <= U(n); k++)
        {
            vector<int> row(n);
            for (int u = 0; u < n; u++) row[u] = up[k - 1][up[k - 1][u]];
            up.push_back(move(row));
        }
    }
```

跳转与最少步数接口（接上页同一结构体）：

```
    int advance(int u, U k) const
    {
        assert(0 <= u && u < n);
        if (k >= U(depth[u]))
        {
            k -= depth[u];
            u = entry[u];
            const auto &cycle = cycles[component[u]];
            return cycle[(U(position[u]) + k % cycle.size()) % cycle.size()];
        }
        for (int bit = 0; k; bit++, k >>= 1)
            if (k & 1) u = up[bit][u];
        return u;
    }

    int steps(int u, int v) const
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        if (component[u] != component[v] || depth[u] < depth[v]) return -1;
        if (depth[v] > 0)
        {
            int d = depth[u] - depth[v];
            return advance(u, d) == v ? d : -1;
        }
        int offset = position[v] - position[entry[u]];
        if (offset < 0) offset += int(cycles[component[u]].size());
        return depth[u] + offset;
    }
};
```

模板题使用：CSES 1750 Planets Queries I

题意：给每点唯一后继，查询从 x 走 k 步后的点，n,q≤200000，0≤k≤10^9。题目 1-based 点号读入减 1、输出加 1；k 用 unsigned long long，模板也支持完整 64 位步数。advance 先处理入环链再对环长取模，不能只建 30 层就宣称支持任意 64 位。预处理 O(n log n)，查询 O(log n)，入环后 O(1)。

所需模板：FunctionalGraph（第 155 页，6.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1750

```
int main()
{
    int n, q;
    scanf("%d%d", &n, &q);
    vector<int> to(n);
    for (auto &v : to)
    {
        scanf("%d", &v);
        v--;
    }
    FunctionalGraph graph(to);
    while (q--)
    {
        int u;
        unsigned long long k;
        scanf("%d%llu", &u, &k);
        printf("%d\n", graph.advance(u - 1, k) + 1);
    }
}
```

模板题使用：CSES 1160 Planets Queries II

题意：同样每点一个后继，查询 u 到 v 的最少步数，不可达输出-1；n,q≤200000。1-based 读入转 0-based，steps 返回步数，不能加 1；同一点答案 0。处于同一弱连通分量也可能不可达，例如不同入环分支或从环走回链。预处理 O(n log n)，单次 O(log n)。

所需模板：FunctionalGraph（第 155 页，6.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1160

```
int main()
{
    int n, q;
    scanf("%d%d", &n, &q);
    vector<int> to(n);
    for (auto &v : to)
    {
        scanf("%d", &v);
        v--;
    }
    FunctionalGraph graph(to);
    while (q--)
    {
        int u, v;
        scanf("%d%d", &u, &v);
        printf("%d\n", graph.steps(u - 1, v - 1));
    }
}
```

应用题补充：Luogu P2921

题意：每点一个后继，输出从各点出发直到首次重复前访问的不同点数，包含起点，n≤100000。结果为入环链长 depth[u] 加所在环的长度；component 是内部 0-based 环编号，不能当成环顶点。自环答案 1。这份用法只读取分解字段，不调用 advance/steps。当前模板含倍增表，建图 O(n log n)，输出 O(n)。

所需模板：FunctionalGraph（第 155 页，6.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2921

```
int main()
{
    int n;
    scanf("%d", &n);
    vector<int> to(n);
    for (auto &v : to)
    {
        scanf("%d", &v);
        v--;
    }
    FunctionalGraph graph(to);
    for (int u = 0; u < n; u++)
        printf("%d\n", graph.depth[u] + int(graph.cycles[graph.component[u]].size()));
}
```

6.1.3 强连通分量

add(u,v)、run()，bel[1..n] 为分量编号，cnt 为数量；分量编号按缩点图拓扑序递增。递归 Kosaraju，按用户要求保留 DFS 写法；O(V+E)，同时存正反图。

```
struct SCC
{
    int n, cnt = 0;
    vector<vector<int>> g, rg;
    vector<int> bel;

    SCC(int n) : n(n), g(n + 1), rg(n + 1) {}

    void add(int u, int v)
    {
        g[u].push_back(v);
        rg[v].push_back(u);
    }

    void dfs1(int u, vector<int> &vis, vector<int> &order)
    {
        vis[u] = 1;
        for (int v : g[u])
        {
            if (!vis[v]) dfs1(v, vis, order);
        }
        order.push_back(u);
    }

    void dfs2(int u)
    {
        bel[u] = cnt;
        for (int v : rg[u])
        {
            if (!bel[v]) dfs2(v);
        }
    }
```

重算入口与拓扑编号（接上页同一结构体）：

```
    // Recursive Kosaraju; component IDs are in topological order, 1..cnt.
    void run()
    {
        vector<int> vis(n + 1), order;
        cnt = 0;
        bel.assign(n + 1, 0);
        for (int s = 1; s <= n; s++)
        {
            if (!vis[s]) dfs1(s, vis, order);
        }
        reverse(order.begin(), order.end());
        for (int s : order)
        {
            if (!bel[s])
            {
                ++cnt;
                dfs2(s);
            }
        }
    }
};
```

模板题使用：Library Checker scc (Kosaraju)

题意：给有向图，输出强连通分量并按缩点拓扑序排列；N、M≤500000，可有重边和自环。题面零基点号先加 1；SCC.bel 为 1..cnt 且沿跨分量边递增，按 1..cnt 输出并将点号减 1。不要照搬 TarjanSCC 的逆序循环。时间、空间 O(N+M)，DFS 递归。

所需模板：SCC（第 160 页，6.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/scc

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    SCC g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        g.add(u + 1, v + 1);
    }
    g.run();
    vector<vector<int>> c(g.cnt + 1);
    for (int u = 1; u <= n; u++) c[g.bel[u]].push_back(u - 1);
    cout << g.cnt << '\n';
    for (int i = 1; i <= g.cnt; i++)
    {
        cout << c[i].size();
        for (int u : c[i]) cout << ' ' << u;
        cout << '\n';
    }
}
```

6.2 双连通分量

6.2.1 点双与边双：共享 lowlink 核心

BiconnectedCore(n)，点号 1..n；add(u,v) 返回原边的 0-based 编号，run() 从头计算，允许重复运行。

点双结果：blocks 为点双的顶点列表，桥两端成块，孤立点单独成块，割点可出现在多个块；cut[u] 标记割点。

边双结果：bridge[id] 标记桥，bel[u] 是 1..cnt 的边双编号；cnt 是全图边双个数。重边按边号处理；自环保留边号但不进 DFS，不是桥，不额外产生点双。一次共享 DFS 后递归染色，O(n+m) 时间和空间，DFS 保持递归。n=0 合法。run 后才可读取结果或构造森林；g、edges 为原图，不在 DFS 期间修改。

```
struct BiconnectedCore
{
    int n, timer = 0, cnt = 0;
    vector<vector<pair<int, int>>> g;
    vector<pair<int, int>> edges;
    vector<vector<int>> blocks;
    vector<int> dfn, low, cut, bridge, bel, st;

    BiconnectedCore(int n) : n(n), g(n + 1) {}

    int add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = edges.size();
        edges.push_back({u, v});
        if (u != v)
        {
            g[u].push_back({v, id});
            g[v].push_back({u, id});
        }
        return id;
    }
```

```
    void dfs(int u, int parent_edge)
    {
        dfn[u] = low[u] = ++timer;
        st.push_back(u);
        int children = 0;
        for (auto [v, id] : g[u])
        {
            if (id == parent_edge) continue;
            if (!dfn[v])
            {
                children++;
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) bridge[id] = 1;
                if (low[v] >= dfn[u])
                {
                    if (parent_edge != -1 || children > 1) cut[u] = 1;
                    blocks.push_back({u});
                    while (true)
                    {
                        int x = st.back();
                        st.pop_back();
                        blocks.back().push_back(x);
                        if (x == v) break;
                    }
                }
            }
            else
                low[u] = min(low[u], dfn[v]);
        }
        if (parent_edge == -1 && children == 0) blocks.push_back({u});
    }
```

```
    void paint(int u)
    {
        bel[u] = cnt;
        for (auto [v, id] : g[u])
        {
            if (!bridge[id] && !bel[v]) paint(v);
        }
    }
```

```
    void run()
    {
        timer = cnt = 0;
        dfn.assign(n + 1, 0);
        low = dfn;
        cut = dfn;
        bel = dfn;
        bridge.assign(edges.size(), 0);
        blocks.clear();
        st.clear();
        for (int u = 1; u <= n; u++)
        {
            if (!dfn[u])
            {
                dfs(u, -1);
                st.pop_back();
            }
        }
        for (int u = 1; u <= n; u++)
        {
            if (bel[u]) continue;
            ++cnt;
            paint(u);
        }
    }
};
```

模板题使用：Luogu P8435

题意：给定无向图，输出所有点双连通分量的顶点集合；割点可出现在多个集合中，孤立点单独成块。点号 1..n。

所需模板：BiconnectedCore（第 163 页，6.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P8435

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    BiconnectedCore graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        std::cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    const auto &blocks = graph.blocks;
    std::cout << blocks.size() << '\n';
    for (const auto &block : blocks)
    {
        std::cout << block.size();
        for (int u : block) std::cout << ' ' << u;
        std::cout << '\n';
    }
}
```

模板题使用：Luogu P8436

题意：给定无向图，输出所有边双连通分量的顶点集合；利用 bel 按边双编号分组。点号 1..n。

所需模板：BiconnectedCore（第 163 页，6.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P8436

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    BiconnectedCore graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        std::cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    std::vector<std::vector<int>> blocks(graph.cnt);
    for (int u = 1; u <= n; u++) blocks[graph.bel[u] - 1].push_back(u);
    std::cout << blocks.size() << '\n';
    for (const auto &block : blocks)
    {
        std::cout << block.size();
        for (int u : block) std::cout << ' ' << u;
        std::cout << '\n';
    }
}
```

6.2.2 边双缩点森林（保留桥编号）

代码依赖：BiconnectedCore（第 163 页，6.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

bridge_component_forest(graph) 接收已 run 的 BiconnectedCore，返回按 1..cnt 编号的边双森林，邻接项为 (相邻分量, 原桥编号)。每条桥变成一条森林边，重边和自环不会误作桥。0 号空置，不连通输入可产生多棵树，孤立边双保留空邻接表。O(n+m) 时间、O(n) 输出空间，不修改分解结果。

```
vector<vector<pair<int, int>>> bridge_component_forest(const BiconnectedCore &graph)
{
    int cnt = graph.cnt;
    const auto &edges = graph.edges;
    const auto &bridge = graph.bridge;
    const auto &bel = graph.bel;
    vector<vector<pair<int, int>>> tree(cnt + 1);
    for (int id = 0; id < (int)edges.size(); id++)
    {
```

```
        if (!bridge[id]) continue;
        auto [u, v] = edges[id];
        u = bel[u];
        v = bel[v];
        tree[u].push_back({v, id});
        tree[v].push_back({u, id});
    }
    return tree;
}
```

应用题补充：Luogu P2860 (bridge forest)

题意：连通无向图，n≤5000、m≤10000；允许已有重边及新增重边，求最少新增边使任意两点间有两条边不相交路径。run 后边双缩点，编号 1..cnt，邻接项为 (边双号, 原桥号)。度 1 点数为 L，答案 (L+1)/2；只剩一个边双时为 0。该公式要求原图连通，不可对任意森林直接套用。O(n+m) 时间、空间。

所需模板：BiconnectedCore（第 163 页，6.2.1 节）、bridge_component_forest（第 167 页，6.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2860

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BiconnectedCore graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    auto tree = bridge_component_forest(graph);
    int leaves = 0;
    for (int u = 1; u <= graph.cnt; u++)
        if (tree[u].size() == 1) leaves++;
    cout << (leaves + 1) / 2 << '\n';
    return 0;
}
```

6.2.3 属于简单奇环的全部顶点

代码依赖：BiconnectedCore（第 163 页，6.2.1 节）、block_cut_forest（第 188 页，6.4.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

先对 BiconnectedCore 调用 run，再调用 odd_cycle_vertices(graph)，返回 answer[1..n]，值为 1 表示该点属于某个长度至少为 3 的简单奇环，answer[0]=0。允许重边、断开图及空图；自环不计入这里的奇环。某点双非二分当且仅当其中存在奇环，此时该点双每个顶点都属于某个奇环；答案为所有非二分点双的顶点并集。通过圆方森林把每条非自环边分到唯一点双：两端中森林深度较大者的父方点即该块，同深时二者父方点相同。随后只对块内边染色，避免在每个块中重复扫描割点的全部邻边。两次 DFS 均递归，分解后额外 O(n+m) 时间和空间。可重排 blocks，不能改变分解内容。六点以内所有简单图枚举奇环对拍，另有重边/自环、长链、星形和大奇环测试；POJ2942 补图驱动本地验证，在线待补。依赖 biconnected.hpp。

```cpp
// After run(). Simple odd cycles have length >= 3; self-loops are ignored.
vector<int> odd_cycle_vertices(const BiconnectedCore &graph)
{
    int n = graph.n;
    auto forest = block_cut_forest(graph);
    vector<int> parent(forest.size()), depth(forest.size());
    auto root_tree = [&](auto &&self, int u, int p) -> void
    {
        parent[u] = p;
        for (int v : forest[u])
            if (v != p)
            {
                depth[v] = depth[u] + 1;
                self(self, v, u);
            }
    };
    for (int u = 1; u <= n; u++)
        if (!depth[u])
        {
            depth[u] = 1;
            root_tree(root_tree, u, 0);
        }
```

```
    vector<vector<pair<int, int>>> edges(graph.blocks.size());
    for (auto [u, v] : graph.edges)
        if (u != v)
        {
            int block = parent[depth[u] >= depth[v] ? u : v] - n - 1;
            assert(0 <= block && block < (int)edges.size());
            edges[block].push_back({u, v});
        }
    vector<vector<int>> g(n + 1);
    vector<int> color(n + 1), answer(n + 1);
    for (int block = 0; block < (int)edges.size(); block++)
    {
        for (int u : graph.blocks[block])
        {
            g[u].clear();
            color[u] = 0;
        }
        for (auto [u, v] : edges[block])
        {
            g[u].push_back(v);
            g[v].push_back(u);
        }
        bool bipartite = true;
        auto paint = [&](auto &&self, int u) -> void
        {
            for (int v : g[u])
                if (!color[v])
                {
                    color[v] = -color[u];
                    self(self, v);
                }
                else if (color[v] == color[u])
                    bipartite = false;
        };
        int root = graph.blocks[block][0];
        color[root] = 1;
        paint(paint, root);
        if (!bipartite)
            for (int u : graph.blocks[block]) answer[u] = 1;
    }
    return answer;
}
```

应用题补充：POJ 2942 Knights of the Round Table

题意：互相憎恨的骑士不能相邻，合法圆桌至少 3 人且人数为奇数；求无法参加任何合法圆桌的人数。输入憎恨关系，顶点 1..n；对 u<v 且不憎恨的二人建补图边，不能把憎恨边直接加入。每组新建矩阵和 BiconnectedCore，先 run 再调用 odd_cycle_vertices，统计 odd[1..n] 中的 0，每组输出一行；0 0 终止且不输出。函数只返回各点能否出现在某个简单奇环，不给圆桌座次；无合法圆桌时输出 n，不是失败。POJ 原站本次不可达；输入格式和样例另与同题 UVA1364 官方 PDF 核对，未据此声称 POJ 限制或线上 AC 已复核。

所需模板：BiconnectedCore（第 163 页，6.2.1 节）、block_cut_forest（第 188 页，6.4.2 节）、odd_cycle_vertices（第 169 页，6.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://poj.org/problem?id=2942

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m && (n || m))
    {
        vector<vector<bool>> hate(n + 1, vector<bool>(n + 1));
        while (m--)
        {
            int u, v;
            cin >> u >> v;
            hate[u][v] = true;
            hate[v][u] = true;
        }
        BiconnectedCore graph(n);
        for (int u = 1; u <= n; u++)
            for (int v = u + 1; v <= n; v++)
                if (!hate[u][v]) graph.add(u, v);
        graph.run();
        auto odd = odd_cycle_vertices(graph);
        int answer = 0;
        for (int u = 1; u <= n; u++) answer += !odd[u];
        cout << answer << '\n';
    }
    return 0;
}
```

6.2.4 边双缩点的大小与内部边数

代码依赖：BiconnectedCore（第 163 页，6.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

在 BiconnectedCore.run() 后构造 EdgeCompression(graph)。n 为边双数量；size[c] 为第 c 块的原顶点数，inside[c] 为块内逻辑无向边数，分量号 1..n，0 号槽忽略。edges 每项为 {块 u, 块 v, 原边 id}，只含跨块的桥，每条桥出现一次，按原边号递增；是无向森林，不承诺端点为父子方向。重边逐条计数，自环在所属块内部计一次，不能用邻接表度数直接计边而漏掉这些约定。空图返回 n=0、长度 1 的零计数数组和空边表。时间 O(N+M)，输出空间 O(N+M)，N/M 指原图规模。

```
struct EdgeCompression
{
    int n;
    vector<int> size, inside;
    vector<array<int, 3>> edges;

    EdgeCompression(const BiconnectedCore &graph)
        : n(graph.cnt), size(n + 1), inside(n + 1)
    {
        for (int u = 1; u <= graph.n; u++) size[graph.bel[u]]++;
        for (int id = 0; id < (int)graph.edges.size(); id++)
        {
            auto [u, v] = graph.edges[id];
            u = graph.bel[u];
            v = graph.bel[v];
            if (u == v)
                inside[u]++;
            else
                edges.push_back({u, v, id});
        }
    }
};
```

应用题补充：Codeforces 732F Tourist Reform

题意：连通无向简单图，2≤n≤400000、1≤m≤400000，给每条边定向，使各点可达点数的最小值最大（含自身）。输出最优值和按原输入顺序的 m 条有向边。先求边双缩点，选 size 最大的块为根；orient_edges 定向块内边，再按 edges 中的原边编号把桥全部朝根定向。答案是最大块大小，不把缩点边端点顺序当成父子关系。O(n+m) 时间/空间；保留递归核心，需足够大栈，局部 8 MiB 栈在最大链上溢出，评测栈未知。本题是应用，不是正式缩点模板题。

所需模板：BiconnectedCore（第 163 页，6.2.1 节）、EdgeCompression（第 172 页，6.2.4 节）、Lowlink（第 180 页，6.3.3 节）、orient_edges（第 175 页，6.2.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/732/F

```cpp
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    if (!(std::cin >> n >> m)) return 0;
    BiconnectedCore core(n);
    Lowlink graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        std::cin >> u >> v;
        core.add(u, v);
        graph.add(u, v); // Same insertion order preserves logical edge IDs.
    }
    // Existing recursive cores require a sufficiently large process stack.
    // An 8 MiB stack fails at n=400000; judge stack availability is not assumed.
    core.run();
    EdgeCompression compressed(core);
    int root = 1;
    for (int c = 2; c <= compressed.n; c++)
        if (compressed.size[c] > compressed.size[root]) root = c;
    auto direction = orient_edges(graph); // Does not need graph.run().
    vector<vector<pair<int, int>>> tree(compressed.n + 1);
    for (auto [u, v, id] : compressed.edges)
    {
        tree[u].push_back({v, id});
        tree[v].push_back({u, id}); // Compressed endpoints are not parent/child.
    }
```

继续处理操作（接上页同一 main）：

```
    vector<int> parent(compressed.n + 1), order{root};
    parent[root] = -1;
    for (int i = 0; i < (int)order.size(); i++)
    {
        int u = order[i];
        for (auto [v, id] : tree[u])
        {
            if (parent[v]) continue;
            parent[v] = u;
            order.push_back(v);
            auto [a, b] = core.edges[id];
            if (core.bel[a] != v) swap(a, b);
            direction[id] = {a, b}; // Original bridge points child -> parent.
        }
    }
    std::cout << compressed.size[root] << '\n';
    for (auto [u, v] : direction) std::cout << u << ' ' << v << '\n';
    return 0;
}
```

6.2.5 无向边定向与强连通定向

代码依赖：Lowlink（第 180 页，6.3.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

orient_edges(graph) 接受 Lowlink 对象。返回长度等于逻辑边数的方向数组，第 id 项为该边定向后的 {起点, 终点}，顶点 1-based、边 id 为 0-based。每条无向边仅处理一次：DFS 树边向下，非树边指向祖先，自环保留 {u,u}，重边不合并。支持非连通图；每个边双块内定向后强连通，桥只给出某个方向，不保证跨桥互相可达。若要求整个非空图强连通，需另外 run/Run 检查 components==1 且无桥；不满足则不存在这样的方向。该函数只读取邻接表与边数，不依赖已有 DFS 状态，不修改原图，调用时无须先 run。复杂度 O(N+M) 时间、O(N+M) 额外空间，DFS 保持递归。

```
// Tree edges go down; other edges go to ancestors. Handles each logical edge once.
vector<pair<int, int>> orient_edges(const Lowlink &graph)
{
    vector<pair<int, int>> direction(graph.edges);
    vector<int> seen(graph.n + 1);
    auto dfs = [&](auto &&self, int u) -> void
    {
        seen[u] = 1;
        for (auto [v, id] : graph.g[u])
        {
            if (direction[id].first) continue;
            direction[id] = {u, v};
            if (!seen[v]) self(self, v);
        }
    };
    for (int u = 1; u <= graph.n; u++)
        if (!seen[u]) dfs(dfs, u);
    return direction;
}
```

应用题补充：Codeforces 118E Bertown roads

题意：给连通无向简单图，把每条边定向使整图强连通，无解输出 0。n≤100000、m≤300000。先用 Lowlink.run 确认连通且无桥，再调用 orient_edges；返回数组与 add 的原边编号逐项对应，每条逻辑边只加一次。任意可行方向均可。O(n+m) 时间、空间，DFS 递归；比赛应用不替代正式模板题。

所需模板：Lowlink（第 180 页，6.3.3 节）、orient_edges（第 175 页，6.2.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/118/E

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    Lowlink graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        std::cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    if (graph.components != 1 ||
        std::count(graph.bridge.begin(), graph.bridge.end(), 1))
    {
        std::cout << 0 << '\n';
        return 0;
    }
    for (auto [u, v] : orient_edges(graph)) std::cout << u << ' ' << v << '\n';
    return 0;
}
```

6.3 割点和桥

6.3.1 由点双分解计算删点连通块数

代码依赖：BiconnectedCore（第 163 页，6.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

在 BiconnectedCore.run() 后调用 removal_components(graph)，返回 (原连通块数 before, 删点结果 after)。after[u] 为独立删除原顶点 u 后剩余连通块数，点号 1..n；after[0]=0 不参与计算。before 等于边双数量减去桥数量；after[u] 等于 before-1 加包含 u 的非单点点双数量。单点块必须跳过，因此孤立点删去后会少一个连通块。空图返回 (0,{0})。不修改原分解，O(n+m) 时间、O(n) 额外空间。若只需要割点、桥和删点增量，直接用更短的 Lowlink/Low_Link；已经需要点双分解时可用本函数，避免再次 DFS。

```
// Call after run(). Each vertex is deleted independently from the original graph.
pair<int, vector<int>> removal_components(const BiconnectedCore &graph)
```

```
{
    int before = graph.cnt - count(graph.bridge.begin(), graph.bridge.end(), 1);
    vector<int> after(graph.n + 1, before - 1);
    after[0] = 0;
    for (const auto &block : graph.blocks)
        if (block.size() > 1)
            for (int u : block) after[u]++;
    return {before, after};
}
```

模板题使用：AOJ GRL_3_A

题意：连通无向简单图，按编号升序输出所有割点，N、M≤100000。输入零基转一基；BiconnectedCore.run 后取 (before,after)，仅当 after[u]>before 才输出 u-1。本例展示已有点双结果的复用；只求割点优先用更短的 Lowlink。时间、空间 O(N+M)，递归 DFS。该题只核验割点集合，完整删点数量另见 UVA10765 应用。

所需模板：BiconnectedCore（第 163 页，6.2.1 节）、removal_components（第 176 页，6.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=GRL_3_A&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BiconnectedCore g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        g.add(u + 1, v + 1);
    }
    g.run();
    auto [before, after] = removal_components(g);
    for (int u = 1; u <= n; u++)
        if (after[u] > before) cout << u - 1 << '\n';
}
```

应用题补充：UVA 10765 Doves and Bombs

题意：分别删去每个站点，按剩余连通块数降序、原编号升序输出前 m 名；n≤10000，原图连通。m 是输出名额而非边数，边表以-1 -1 结束，多测以 0 0 结束，每组末尾空行。after 是剩余总块数而非增量，不重复加 before。O(n log n+E) 时间、O(n+E) 空间。应用题不计正式模板覆盖。

所需模板：BiconnectedCore（第 163 页，6.2.1 节）、removal_components（第 176 页，6.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.org/external/107/10765.pdf

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m && n)
    {
        BiconnectedCore g(n);
        int u, v;
        while (cin >> u >> v && u != -1) g.add(u + 1, v + 1);
        g.run();
        auto [before, after] = removal_components(g);
        vector<pair<int, int>> a;
        for (int u = 1; u <= n; u++) a.push_back({-after[u], u - 1});
        sort(a.begin(), a.end());
        for (int i = 0; i < m; i++) cout << a[i].second << ' ' << -a[i].first << '\n';
        cout << '\n';
    }
}
```

6.3.2 连通无向图最少加边方案

代码依赖：BiconnectedCore（第 163 页，6.2.1 节）、bridge_component_forest（第 167 页，6.2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

先对非空连通无向图调用 BiconnectedCore.run()，再调用 bridge_augmentation(graph)。返回原图点号构成的新增边列表，不修改原图。允许新增重边，不产生自环；不适用于非连通图。已无桥时返回空列表。桥树有 L 个叶子时最少加 (L+1)/2 条边（整数除法）。递归 DFS 顺序收集叶子，奇数时复制首叶，前后半段配对。每条桥一侧的叶子是循环连续区间，半周配对必跨过该割，故所有桥被消除；每个叶子至少需要一个新边端点，得到同样的下界。分解后本函数 O(n+m) 时间与空间。依赖 bridge_component_forest.hpp。P2860 的 vector 用法已通过普通及 ASan/UBSan 本地验证；新增方案以逐边删除连通性及小图最优解枚举独立核验，该应用尚无本库在线 AC。

```
// Connected, nonempty graph after run(); new parallel edges are allowed.
vector<pair<int, int>> bridge_augmentation(const BiconnectedCore &graph)
{
    assert(graph.n > 0 && graph.cnt > 0);
    int bridges = count(graph.bridge.begin(), graph.bridge.end(), 1);
    assert(graph.cnt - bridges == 1);
    if (graph.cnt == 1) return {};
    auto tree = bridge_component_forest(graph);
    vector<int> representative(graph.cnt + 1);
    for (int u = 1; u <= graph.n; u++) representative[graph.bel[u]] = u;
    vector<int> leaves;
    auto dfs = [&](auto &&self, int u, int parent) -> void
    {
        if (tree[u].size() == 1) leaves.push_back(u);
        for (auto [v, id] : tree[u])
            if (v != parent) self(self, v, u);
    };
    dfs(dfs, 1, 0);
    if (leaves.size() % 2) leaves.push_back(leaves[0]);
    int half = leaves.size() / 2;
    vector<pair<int, int>> added;
    for (int i = 0; i < half; i++)
        added.push_back({representative[leaves[i]], representative[leaves[i + half]]});
    return added;
}
```

应用题补充：Luogu P2860 (construct augmentation)

题意：同题的构造接口：连通非空图 run 后调用 bridge_augmentation，返回最少补边方案的原点端点对，点号 1..n，可直接逐对使用；题目只输出方案长度。已有边和新增边都允许平行边。接口不修改 graph，若需更新原图须自行 add 并重新 run。O(n+m) 时间、空间；DFS 顺序收集桥树叶子后对半配对，奇数叶子补首叶。

所需模板：BiconnectedCore（第 163 页，6.2.1 节）、bridge_component_forest（第 167 页，6.2.2 节）、bridge_augmentation（第 179 页，6.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2860

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BiconnectedCore graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    cout << bridge_augmentation(graph).size() << '\n';
    return 0;
}
```

6.3.3 割点与桥

无向多重图，点号 1..n；add/Insert(u,v) 返回逻辑无向边的 0-based 编号，自环和重边均可输入。run/Run 后 cut[u] 为割点标记，bridge[id] 为桥标记，components 为原图连通块数。delta[u] 是独立删除顶点 u 及其关联边后增加的连通块数，剩余块数为 components+delta[u]；孤立点 delta=-1，普通非割点为 0，割点为正。根的增量为 DFS 子树数减 1；非根统计满足 low[child]>=dfn[u] 的子树数。n=1 的点删除后剩余 0 块。DFS 按父边号跳过，支持重边、非连通图和重复运行。复杂度 O(n+m)，递归 DFS；新增边后需重新 run。每个删点结果都针对原图，不表示逐次删除。

```
struct Lowlink
{
    int n, timer = 0, components = 0;
    vector<vector<pair<int, int>>> g;
    vector<int> dfn, low, cut, bridge, delta;
    int edges = 0;

    Lowlink(int n) : n(n), g(n + 1) {}

    int add(int u, int v)
    {
        g[u].push_back({v, edges});
        g[v].push_back({u, edges});
        return edges++;
    }
```

```
    void dfs(int u, int pe)
    {
        dfn[u] = low[u] = ++timer;
        int children = 0;
        for (auto [v, id] : g[u])
        {
            if (id == pe) continue;
            if (!dfn[v])
            {
                ++children;
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) bridge[id] = 1;
                if (pe != -1 && low[v] >= dfn[u])
                {
                    cut[u] = 1;
                    delta[u]++;
                }
            }
            else
                low[u] = min(low[u], dfn[v]);
        }
        if (pe == -1 && children > 1) cut[u] = 1;
        if (pe == -1) delta[u] = children - 1;
    }

    void run()
    {
        timer = 0;
        components = 0;
        dfn.assign(n + 1, 0);
        low = dfn;
        cut = dfn;
        delta = dfn;
        bridge.assign(edges, 0);
        for (int u = 1; u <= n; u++)
            if (!dfn[u])
            {
                components++;
                dfs(u, -1);
            }
    }
};
```

模板题使用：Luogu P3388

题意：给无向图，按编号递增输出全部割点。图可以不连通；run 后读取 cut[u] 标记。不要把 bridge 的边编号当作顶点标记。点号 1..n。

所需模板：Lowlink（第 180 页，6.3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3388

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Lowlink graph(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    vector<int> answer;
    for (int u = 1; u <= n; u++)
        if (graph.cut[u]) answer.push_back(u);
    cout << answer.size() << '\n';
    for (int u : answer) cout << u << ' ';
    cout << '\n';
}
```

6.4 圆方树

6.4.1 静态仙人掌森林最短路

Cactus(n)，0≤n<INT_MAX/2，原点 1..n。add(u,v,w) 加入无向边；输入须为无自环无重边的仙人掌森林（每条边最多属于一个简单环），不检查此结构前提。权非负，所有边权之和 ≤LLONG_MAX/4。build() 递归构造桥/环虚点森林并倍增；distance(u,v) 仅接受原点，断连返回-1，同点返回 0。查询前必须 build；加边后必须重建，支持重复 build 和复制，无动态修改或路径方案接口。编号 n+1..cnt 为环虚点；t 是从根向下的带权邻接表，成员只读。环的顶点到最上方割点距离取较短弧，虚点到该割点权 0；LCA 是虚点时，须用两个儿子在环上的较短弧替换根距离贡献。dfs 与 prepare 均递归，环上沿 par 回走仅枚举边，不模拟 DFS 栈。原图 DFS 及环枚举 O(n+m)，总预处理/空间 O((n+m)log(n+1))，查询 O(log(n+1))，递归深度 O(n)。n=0 允许 build 但无合法查询；不支持负权、非法仙人掌或修改内部数组。

```
struct Cactus
{
    using ll = long long;
    int n, m = 0, cnt, timer = 0, lg;
    vector<vector<tuple<int, int, ll>>> g;
    vector<vector<pair<int, ll>>> t;
    vector<vector<int>> up;
    vector<int> dfn, par, bel, dep, root;
    vector<ll> pref, pos, len, dis;

    Cactus(int n) : n(n), g(n + 1)
    {
        assert(0 <= n && n < INT_MAX / 2);
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(u != v && w >= 0);
        g[u].push_back({v, m, w});
        g[v].push_back({u, m, w});
        m++;
    }
```

递归构造环虚点与根距离（接上页同一结构体）：

```
    void dfs(int u, int pe)
    {
        dfn[u] = ++timer;
        for (auto [v, id, w] : g[u])
        {
            if (id == pe) continue;
            if (!dfn[v])
            {
                par[v] = u;
                pref[v] = pref[u] + w;
                dfs(v, id);
                if (!bel[v]) t[u].push_back({v, w});
            }
            else if (dfn[v] < dfn[u])
            {
                int b = ++cnt;
                len[b] = pref[u] - pref[v] + w;
                t[v].push_back({b, 0});
                for (int x = u; x != v; x = par[x])
                {
                    bel[x] = b;
                    pos[x] = pref[x] - pref[v];
                    t[b].push_back({x, min(pos[x], len[b] - pos[x])});
                }
            }
        }
    }

    void prepare(int u)
    {
        for (int k = 1; k < lg; k++)
            up[k][u] = up[k - 1][up[k - 1][u]];
        for (auto [v, w] : t[u])
        {
            up[0][v] = u;
            dep[v] = dep[u] + 1;
            root[v] = root[u];
            dis[v] = dis[u] + w;
            prepare(v);
        }
    }
```

从零重建（接上页同一结构体）：

```
    void build()
    {
        cnt = n;
        timer = 0;
        int size = 2 * n + 1;
        t.assign(size, {});
        dfn.assign(size, 0);
        par = bel = dep = root = dfn;
        pref.assign(size, 0);
        pos = len = dis = pref;
        for (int u = 1; u <= n; u++)
            if (!dfn[u]) dfs(u, -1);
        lg = 1;
        while ((1LL << lg) <= cnt) lg++;
        up.assign(lg, vector<int>(size));
        for (int u = 1; u <= n; u++)
        {
            if (dfn[u] && !par[u])
            {
                root[u] = u;
                prepare(u);
            }
        }
    }
```

查询与环上距离修正（接上页同一结构体）：

```
    // Original vertices only; disconnected pairs return -1.
    ll distance(int u, int v) const
    {
        if (root[u] != root[v]) return -1;
        ll ans = dis[u] + dis[v];
        if (dep[u] < dep[v]) swap(u, v);
        int delta = dep[u] - dep[v];
        for (int k = 0; k < lg; k++)
            if (delta >> k & 1) u = up[k][u];
        if (u == v) return ans - 2 * dis[u];
        for (int k = lg - 1; k >= 0; k--)
        {
            if (up[k][u] != up[k][v])
            {
                u = up[k][u];
                v = up[k][v];
            }
        }
        int p = up[0][u];
        if (p <= n) return ans - 2 * dis[p];
        ll arc = abs(pos[u] - pos[v]);
        return ans - dis[u] - dis[v] + min(arc, len[p] - arc);
    }
};
```

为什么需要环上修正 桥和环块构成森林，块间路径唯一。若最近公共祖先是原点，两条根距离减去公共前缀即可；若是环虚点，u、v 现为该虚点的两个原点儿子，先扣掉这两点到根的距离，再加环上两段弧长的较小值。不能直接把虚点当普通 LCA，否则会强迫路径绕经环顶割点。

复杂度与前提 每条边只属于至多一个环，因此沿 par 回走的总次数为 O(m)，不是每个返祖边各走 O(n)。两次 DFS 均为递归。输入须为非负权简单仙人掌森林，全部边权和不超过 LLONG_MAX/4；不自动校验结构，非法图可能越界。加边后必须重建。

模板题使用：P5236 【模板】静态仙人掌

题意：正权无重边仙人掌，查询两点最短距离；n,q≤10000、m≤20000、w≤100000。

所需模板：Cactus（第 183 页，6.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5236

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    Cactus solver(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        solver.add(u, v, w);
    }
    solver.build();
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        cout << solver.distance(u, v) << '\n';
    }
}
```

6.4.2 由点双构造圆方森林

代码依赖：BiconnectedCore（第 163 页，6.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

block_cut_forest(graph) 接收已经 run 的 BiconnectedCore，返回 1-based 邻接表。原点为 1..n，blocks[i] 的方点为 n+i+1；每个点双内的所有原点连向该方点。孤立点也连向自己的单点方块。不连通原图得到森林；0 号空置。O(n+m) 时间与输出空间，不修改分解结果。

```
vector<vector<int>> block_cut_forest(const BiconnectedCore &graph)
{
    int n = graph.n;
    const auto &blocks = graph.blocks;
    vector<vector<int>> tree(n + blocks.size() + 1);
    for (int i = 0; i < (int)blocks.size(); i++)
    {
        int b = n + i + 1;
        for (int u : blocks[i])
        {
            tree[u].push_back(b);
            tree[b].push_back(u);
        }
    }
    return tree;
}
```

应用题补充：Luogu P4630

题意：无向简单图，可不连通，n≤100000、m≤200000；统计存在经过 c 的简单 s-f 路径的有序三元组 (s,c,f)，三点互异。原点 1..n 权为-1，方点 n+i+1 权为 blocks[i].size()，树路径权和就是可选 c 数。sz 仅计原点，每棵树独立累计有序端点对，答案用 long long。O(n+m) 时间、空间，递归 DFS。

所需模板：BiconnectedCore（第 163 页，6.2.1 节）、block_cut_forest（第 188 页，6.4.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4630

Luogu P4630：使用代码

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BiconnectedCore graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    auto tree = block_cut_forest(graph);
    vector<int> sz(tree.size());
    auto dfs = [&](auto &&self, int u, int p) -> void
    {
        sz[u] = (u <= n);
        for (int v : tree[u])
        {
            if (v == p) continue;
            self(self, v, u);
            sz[u] += sz[v];
        }
    };
    long long ans = 0;
    auto calc = [&](auto &&self, int u, int p, int total) -> void
    {
        long long sum = total - sz[u];
        long long cnt = u <= n ? 2LL * (total - 1) : 0;
        for (int v : tree[u])
        {
            if (v == p) continue;
            cnt += 2 * sum * sz[v];
            sum += sz[v];
            self(self, v, u, total);
        }
        long long w = u <= n ? -1 : (int)tree[u].size();
        ans += w * cnt;
    };
    for (int u = 1; u <= n; u++)
    {
        if (sz[u]) continue;
        dfs(dfs, u, 0);
        calc(calc, u, 0, sz[u]);
    }
    cout << ans << '\n';
    return 0;
}
```

第七章　环计数问题

7.1　简单无向图四元环计数

count_four_cycles(n,edges) 返回 long long 计数，n≥0、点 0..n-1、m=edges.size()≤INT_MAX。输入必须是简单无向图，每条边恰好给一次，不允许重边和自环；不自动去重。允许空图、孤立点、断连，输入不变，可重复调用。计四个不同点组成的四边环，不要求诱导子图无弦；K4 有 3 个四元环，旋转和反向不重复计算。按 (度数, 编号) 严格定序，以唯一最大点 u 为起点，统计 u-v-w 中 v,w 均小于 u 的二步路径；第二条路径与前面路径配对。每轮清空实际访问的 cnt。时间 O(n+m$\sqrt{m}$)，空间 O(n+m)，无需排序或递归；不可仅按编号定序后仍声称此复杂度。cnt 只需 int，答案必须 long long；在 m≤INT_MAX 约定下，环数 ≤m(m-1)/2<2^61，不溢出。

```
long long count_four_cycles(int n, const vector<pair<int, int>> &edges)
{
    assert(n >= 0 && edges.size() <= INT_MAX);
    vector<vector<int>> g(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }
    auto less = [&](int u, int v)
    {
        return pair{g[u].size(), u} < pair{g[v].size(), v};
    };
    vector<int> cnt(n), touched;
    long long ans = 0;
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            if (!less(v, u)) continue;
            for (int w : g[v])
            {
                if (!less(w, u)) continue;
                if (cnt[w] == 0) touched.push_back(w);
                ans += cnt[w];
                cnt[w]++;
            }
        }
        for (int w : touched) cnt[w] = 0;
        touched.clear();
    }
    return ans;
}
```

模板题使用：U367189 无向图四元环计数

题意：简单无向图，n≤100000，m≤200000；允许断连，不含重边和自环。输入 1-based 端点，输出非诱导四元环数。

所需模板：count_four_cycles（第 190 页，7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/U367189

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> edges(m);
    for (auto &[u, v] : edges)
    {
        cin >> u >> v;
        u--;
        v--;
    }
    cout << count_four_cycles(n, edges) << '\n';
}
```

接口用法演示：API 零基四元环计数

演示约定：输入 n m 和 m 条 0-based 简单无向边；输出四元环数，允许 n=m=0。

所需模板：count_four_cycles（第 190 页，7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/rings-count/

```
int main()
{
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> edges(m);
    for (auto &[u, v] : edges)
        cin >> u >> v;
    cout << count_four_cycles(n, edges) << '\n';
}
```

7.2 无向简单图三元环枚举

enumerate_triangles(n,edges,visit) 使用 0..n-1 点号；edges 须为无向简单图，无自环、无重边，每条边只给一次，端点顺序任意。每个三角形恰调用一次 visit(u,v,w)，参数按度数和编号组成的次序排列，不保证编号递增。可在回调中计数、累计权值或收集方案；计数用 long long。按 (度数, 编号) 从小到大定向，标记 u 的出邻点后枚举 u 到 v 到 w 的有向路径。每个出度为 $O(\sqrt{m})$，总时间 $O(n+m\sqrt{m})$，含常数时间回调，空间 O(n+m)；空图合法。调用期间不修改输入边集。

```cpp
template <class F>
void enumerate_triangles(int n, const vector<pair<int, int>> &edges, F visit)
{
    assert(n >= 0);
    vector<int> deg(n), mark(n, -1);
    vector<vector<int>> g(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        deg[u]++, deg[v]++;
    }
    for (auto [u, v] : edges)
    {
        if (pair{deg[u], u} > pair{deg[v], v}) swap(u, v);
        g[u].push_back(v);
    }
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u]) mark[v] = u;
        for (int v : g[u])
            for (int w : g[v])
                if (mark[w] == u) visit(u, v, w);
    }
}
```

模板题使用：Library Checker enumerate_triangles

题意：给简单无向图的每个点一个权值，求所有三角形三个点权乘积之和，模 998244353。点号 0..n-1。

所需模板：enumerate_triangles（第 192 页，7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/enumerate_triangles

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> x(n);
    for (auto &v : x) cin >> v;
    vector<pair<int, int>> edges(m);
    for (auto &[u, v] : edges) cin >> u >> v;
    const long long mod = 998244353;
    long long ans = 0;
    enumerate_triangles(n,
                        edges,
                        [&](int u, int v, int w)
                        { ans = (ans + x[u] * x[v] % mod * x[w]) % mod; });
    cout << ans << '\n';
}
```

第八章　最小环

相关应用：Karp 最小平均权有向环，目标不同于最小总权环；两遍 DP 线性空间。

8.1　有向图最小平均权环：线性空间 Karp

minimum_mean_cycle(n,edges)，0≤n<INT_MAX，点 0..n-1，edges 为 tuple<int,int,T> 有向边；支持断连、负权、自环和重边，输入不变。返回 optional<pair<W,int»，无环 nullopt；有环返回分子/正分母，未约分，不返回环方案。T 为有符号至多 64 位整数时 W=int128，全部比较精确，支持 LLONG_MIN/MAX；n<2^31 使交叉乘法绝对值 <2^126。实数 T 须支持值语义，W=T；禁止 NaN/Inf，要求 $2n^2\max|w|<2^{118}$ 避免溢出，仍有舍入误差，不能泛称精确。P3199 驱动用 GCC ___float128 进行 113 位核心运算，long double 仅用于输入输出；需 GCC 支持此扩展。F0 全部为 0，Fk 为恰好 k 条边的最短行走；先滚动到 Fn，再重算 F0..Fn-1，取 min_v max_k (Fn(v)-Fk(v))/(n-k)。时间 O(n(n+m))，额外 O(n)，输入 O(m)；非强连通也成立，无需 DFS、二分或负环启发式。k 为 0..n-1，不能只比较最后一层或允许至多 k 条边。

```
template <class T>
auto minimum_mean_cycle(int n, const vector<tuple<int, int, T>> &edges)
{
    static_assert(!is_integral_v<T> || (is_signed_v<T> && sizeof(T) <= 8));
    assert(0 <= n && n < INT_MAX);
    using W = conditional_t<is_integral_v<T>, __int128_t, T>;
    using Ratio = pair<W, int>;
    const W inf = W(1LL << 60) * W(1LL << 60);
    vector<W> d(n), next(n);
    auto step = [&]()
    {
        fill(next.begin(), next.end(), inf);
        for (auto [u, v, w] : edges)
        {
            if (d[u] == inf) continue;
            W value = d[u] + w;
            if (value < next[v]) next[v] = value;
        }
        d.swap(next);
    };
    for (int k = 0; k < n; k++) step();
    auto last = d;
    fill(d.begin(), d.end(), 0);
    auto less = [&](const Ratio &a, const Ratio &b)
    {
        return a.first * b.second < b.first * a.second;
    };
```

第二遍重算并取 min-max（接上页同一函数）：

```
    vector<Ratio> best(n, {0, 0});
    for (int k = 0; k < n; k++)
    {
        for (int v = 0; v < n; v++)
        {
            if (last[v] == inf || d[v] == inf) continue;
            Ratio now{last[v] - d[v], n - k};
            if (best[v].second == 0 || less(best[v], now))
                best[v] = now;
        }
        if (k + 1 < n) step();
    }
    optional<Ratio> ans;
    for (auto r : best)
    {
        if (r.second && (!ans || less(r, *ans))) ans = r;
    }
    return ans;
}
```

为什么全点初始化成立 把各边减去最小平均值后，所有环非负且存在零环。任意起点最短行走都能删环变成少于 n 条边的路径，所以每点的 max 比值非负。到零环的最短行走接无限重复零环，取长度 n 的最短前缀，其终点使 max 为 0；因此全局 min 为 0。恢复原权后所有比值增加同一常数，公式仍成立。无环图没有长度 n 行走，返回 nullopt。

返回分数与数值类型 分母为 1..n；不保证约分，也不保证分子本身就是某个环的权和。整数类型提升到 int128 进行运算和交叉比较；小数用调用者选择的 T，普通浮点不是精确计算。P3199 采用 GCC 的 113 位浮点进行核心运算，long double 仅用于输入输出。无需保存全部 DP 层，但第二遍必须重新从全 0 开始。

模板题使用：P3199 [HNOI2009] 最小圈

题意：实数有向边权，n≤3000、m≤10000、|w|≤10^7，保证有圈；GCC ___float128 核心运算，long double 读入/输出 8 位小数。

所需模板：minimum_mean_cycle（第 194 页，8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3199

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Real = __float128;
    int n, m;
    cin >> n >> m;
    vector<tuple<int, int, Real>> edges;
    while (m--)
    {
        int u, v;
        long double w;
        cin >> u >> v >> w;
        edges.emplace_back(u - 1, v - 1, w);
    }
    auto ans = minimum_mean_cycle(n, edges);
    assert(ans);
    cout << fixed << setprecision(8) << (long double)(ans->first / ans->second) << '\n';
}
```

接口用法演示：API 整数最小平均环精确分数

演示约定：输入 n m 和 0-based 有向边 u v w，w 为 signed64；无环输出 NONE，否则输出 128 位分子和正分母，未约分。

所需模板：minimum_mean_cycle（第 194 页，8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://www2.eecs.berkeley.edu/Pubs/TechRpts/1977/29121.html

```
int main()
{
    int n, m;
    cin >> n >> m;
    vector<tuple<int, int, long long>> edges(m);
    for (auto &[u, v, w] : edges)
        cin >> u >> v >> w;
    auto ans = minimum_mean_cycle(n, edges);
    if (!ans)
    {
        cout << "NONE\n";
        return 0;
    }
    auto num = ans->first;
    if (num < 0)
    {
        cout << '-';
        num = -num;
    }
    string out;
    do
    {
        out += char('0' + num % 10);
        num /= 10;
    } while (num);
    reverse(out.begin(), out.end());
    cout << out << ' ' << ans->second << '\n';
}
```

8.2 非负权无向图最小环与原边方案

MinCycle(n), n≥0, 点 0..n-1。add(u,v,w) 返回从 0 开始的原边号, 边数 ≤INT_MAX, 0≤w≤LLONG_MAX; 允许自环和重边。solve() 从零求至少 3 个不同顶点的简单环, 无解 nullopt; 故自环忽略, 平行边只保留最轻的一条（同权留最早）, 不计二元环。Result.weight 是 int128, vertices 不重复首点, edges[i] 连接 vertices[i] 与 vertices[(i+1)%size], 边号对应 e; 并列返回任意一个, 不保证字典序。支持空图、断连、零权、重复 solve 及加边后重算, 旧结果独立; e/id 只读。Floyd 更新中间点 k 之前枚举 i<j<k, 用 d[i][j]+ 边 (i,k)+ 边 (j,k) 更新最优值, 每轮至多恢复一次路径; 严格松弛避免零权环导致方案循环。solve 为 $O(n^3+n^2)$, 构造 $O(n^2)$, add 摊还 O(1), 总空间 $O(n^2+m)$, 恢复通过 next 逐边走路径, 不是模拟 DFS。负边不支持; 无穷大 2^120 只作内部哨兵, 合法简单路径与候选和均小于 2^95。

```
struct MinCycle
{
    using I = __int128_t;
    struct Edge
    {
        int u, v;
        long long w;
    };
    struct Result
    {
        I weight;
        vector<int> vertices, edges;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> id;

    MinCycle(int n) : n(n), id(n, vector<int>(n, -1))
    {
        assert(n >= 0);
    }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
        int k = e.size();
        e.push_back({u, v, w});
        if (u != v && (id[u][v] == -1 || w < e[id[u][v]].w))
            id[u][v] = id[v][u] = k;
        return k;
    }
```

初始化与本轮候选检查（接上页同一结构体）：

```
    optional<Result> solve() const
    {
        const I inf = I(1) << 120;
        vector<vector<I>> d(n, vector<I>(n, inf));
        vector<vector<int>> next(n, vector<int>(n, -1));
        for (int i = 0; i < n; i++)
        {
            d[i][i] = 0;
            for (int j = 0; j < n; j++)
            {
                if (id[i][j] == -1) continue;
                d[i][j] = e[id[i][j]].w;
                next[i][j] = j;
            }
        }
        Result ans{inf, {}, {}};
        for (int k = 0; k < n; k++)
        {
            int u = -1, v = -1;
            for (int i = 0; i < k; i++)
            {
                if (id[i][k] == -1) continue;
                for (int j = i + 1; j < k; j++)
                {
                    if (id[j][k] == -1 || d[i][j] == inf) continue;
                    I w = d[i][j] + e[id[i][k]].w + e[id[j][k]].w;
                    if (w >= ans.weight) continue;
                    ans.weight = w;
                    u = i;
                    v = j;
                }
            }
```

方案恢复与 Floyd 更新（接上页 solve 的 k 循环）：

```
            if (u != -1)
            {
                ans.vertices = {k, u};
                ans.edges = {id[k][u]};
                while (u != v)
                {
                    int x = next[u][v];
                    ans.edges.push_back(id[u][x]);
                    ans.vertices.push_back(x);
                    u = x;
                }
                ans.edges.push_back(id[v][k]);
            }
            for (int i = 0; i < n; i++)
            {
                for (int j = 0; j < n; j++)
                {
                    if (d[i][k] == inf || d[k][j] == inf) continue;
                    I w = d[i][k] + d[k][j];
                    if (w >= d[i][j]) continue;
                    d[i][j] = w;
                    next[i][j] = next[i][k];
                }
            }
        }
        if (ans.weight == inf) return nullopt;
        return ans;
    }
};
```

检查必须在松弛之前 固定环中最大编号点 k，其两侧邻点 i、j 都小于 k。此时 d[i][j] 只允许小于 k 的中间点，与两条原边共同构成合法简单环；先做本轮 Floyd 会使路径经过 k，错误接受重复点。

恢复不能每次改进都做 每轮只记最优 i、j，结束枚举后恢复一次，所有恢复共 $O(n^2)$。next 沿真实路径逐边走，不是模拟 DFS；更新必须严格变小，尤其不能把零权等长路径改成自指。

模板题使用：P6175 无向图的最小环问题

题意：求至少 3 个不同顶点的最小权环；n≤100，m≤5000，边权 1..100000，答案 ≤10000000，可转 long long 输出；无解精确输出 No solution.。

所需模板：MinCycle（第 198 页，8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6175

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    MinCycle g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u - 1, v - 1, w);
    }
    auto ans = g.solve();
    if (ans)
        cout << (long long)ans->weight << '\n';
    else
        cout << "No solution.\n";
}
```

接口用法演示：API 无向最小环与原边编号

演示约定：输入 n m 及 0-based 无向边 u v w，w 非负 long long。无解输出 NONE；否则依次输出 128 位权值、环长、顶点序列、原边编号序列，不重复首点。

所需模板：MinCycle（第 198 页，8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/min-cycle/

```
int main()
{
    int n, m;
    cin >> n >> m;
    MinCycle g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u, v, w);
    }
    auto ans = g.solve();
    if (!ans)
    {
        cout << "NONE\n";
        return 0;
    }
    auto w = ans->weight;
    string out;
    do
    {
        out += char('0' + w % 10);
        w /= 10;
    } while (w);
    reverse(out.begin(), out.end());
    cout << out << '\n';
    cout << ans->vertices.size() << '\n';
    for (int u : ans->vertices)
        cout << u << ' ';
    cout << '\n';
    for (int id : ans->edges)
        cout << id << ' ';
    cout << '\n';
}
```

第九章　2-SAT

9.1　2-SAT 与任意可行方案

代码依赖：SCC（第 160 页，6.1.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

TwoSAT(n)，变量 1..n；add(x,a,y,b) 添加 (x=a) 或 (y=b)，强制值写 add(x,a,x,a)，蕴含 (x=a)=>(y=b) 写 add(x,!a,y,b)。solve() 返回可满足性，仅成功时读取 ans[1..n]；可重复求解和追加子句后重算，空公式可满足。不保证字典序最小。依赖同文件 SCC：Kosaraju 编号为拓扑序，故真值分量编号较大时取真；不能直接套用 Tarjan 逆拓扑编号的不等号。O(n+m) 时间和空间，DFS 递归。

```
struct TwoSAT
{
    int n;
    SCC g;
    vector<int> ans;

    TwoSAT(int n) : n(n), g(2 * n) {}

    int id(int x, bool value) const { return 2 * x - 1 + value; }

    void add(int x, bool a, int y, bool b)
    {
        g.add(id(x, !a), id(y, b));
        g.add(id(y, !b), id(x, a));
    }

    bool solve()
    {
        g.run();
        ans.assign(n + 1, 0);
        for (int x = 1; x <= n; x++)
        {
            if (g.bel[id(x, 0)] == g.bel[id(x, 1)]) return false;
            ans[x] = g.bel[id(x, 1)] > g.bel[id(x, 0)];
        }
        return true;
    }
};
```

模板题使用：Library Checker two_sat

题意：每个子句是两个文字的或，判断是否可满足并输出一组赋值。输入为 DIMACS：首行 p cnf n m，正数表示变量，负数表示取反，每个子句末尾为 0。

所需模板：SCC（第 160 页，6.1.3 节）、TwoSAT（第 203 页，9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/two_sat

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string p, cnf;
    int n, m;
    cin >> p >> cnf >> n >> m;
    TwoSAT g(n);
    for (int i = 0; i < m; i++)
    {
        int a, b, zero;
        cin >> a >> b >> zero;
        g.add(abs(a), a > 0, abs(b), b > 0);
    }
    if (!g.solve())
    {
        cout << "s UNSATISFIABLE\n";
        return 0;
    }
    cout << "s SATISFIABLE\nv";
    for (int i = 1; i <= n; i++) cout << ' ' << (g.ans[i] ? i : -i);
    cout << " 0\n";
}
```

模板题使用：Luogu P4782

题意：n 个布尔变量，m 个 (x=a) 或 (y=b) 子句，输出任意满足全部子句的赋值或 IMPOSSIBLE。变量号 1..n；n,m≤10^6。同一变量的重复文字照常 add，成功后才读取 ans。

所需模板：SCC（第 160 页，6.1.3 节）、TwoSAT（第 203 页，9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4782

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    TwoSAT sat(n);
    while (m--)
    {
        int x, a, y, b;
        std::cin >> x >> a >> y >> b;
        sat.add(x, a, y, b);
    }
    if (!sat.solve())
    {
        std::cout << "IMPOSSIBLE\n";
        return 0;
    }
    std::cout << "POSSIBLE\n";
    for (int i = 1; i <= n; i++) std::cout << sat.ans[i] << (i == n ? '\n' : ' ');
    return 0;
}
```

9.2 递归染色与回滚求最小赋值

构造 LexTwoSAT(n)，变量号 1..n。add(x,a,y,b) 加入子句 (x=a) OR (y=b)，force(x,v) 强制变量值；solve() 成功后 answer[1..n] 是按变量编号、0 优先于 1 的字典序最小赋值。失败时 answer 为空，n=0 时可满足，answer[0] 始终仅为占位。每个未确定变量先尝试 0，递归传播所有蕴含；冲突时仅撤销本次新增标记，再试 1。history 是撤销记录，不替代递归调用栈。可重复求解，追加子句会清空旧答案；marked/history 仅为内部工作区，失败时不可当作方案。内部 id(x,v)=2(x-1)+v，从 0 开始，可用异或 1 取反；不要混用另一份 SCC 2-SAT 的 1-based 字面量编号。最坏 O(n(n+m)) 时间、O(n+m) 空间，递归栈 O(n)；100000 变量测试是特定蕴含链，不代表一般大规模输入的性能保证。HDU1814 完整程序本地对拍，在线待补。

```
struct LexTwoSAT
{
    int n;
    vector<vector<int>> g;
    vector<int> marked, history, answer;

    LexTwoSAT(int n) : n(n), g(2 * n), marked(2 * n) { assert(n >= 0); }

    int id(int x, bool value) const
    {
        assert(1 <= x && x <= n);
        return 2 * (x - 1) + value;
    }

    // (x == a) OR (y == b). Internal literal IDs are 0-based.
    void add(int x, bool a, int y, bool b)
    {
        int u = id(x, a), v = id(y, b);
        g[u ^ 1].push_back(v);
        g[v ^ 1].push_back(u);
        answer.clear();
    }

    void force(int x, bool value) { add(x, value, x, value); }
```

```
    bool paint(int u)
    {
        if (marked[u ^ 1]) return false;
        if (marked[u]) return true;
        marked[u] = 1;
        history.push_back(u);
        for (int v : g[u])
            if (!paint(v)) return false;
        return true;
    }

    bool solve()
    {
        fill(marked.begin(), marked.end(), 0);
        history.clear();
        answer.clear();
        for (int x = 1; x <= n; x++)
        {
            int u = id(x, false);
            if (marked[u] || marked[u ^ 1]) continue;
            int checkpoint = history.size();
            if (!paint(u))
            {
                while ((int)history.size() > checkpoint)
                {
                    marked[history.back()] = 0;
                    history.pop_back();
                }
                if (!paint(u ^ 1)) return false;
            }
        }
        answer.assign(n + 1, 0);
        for (int x = 1; x <= n; x++) answer[x] = marked[id(x, true)];
        return true;
    }
};
```

应用题补充：HDU 1814 Peaceful Commission

题意：每党两名候选人编号 2x-1、2x，恰选一人且冲突双方不能同时入选，输出字典序最小代表序列。候选人 u 转为变量 x=(u+1)/2，偶号对应值 1；冲突 (u,v) 转为 (x≠a) OR (y≠b)，即 add(x,!a,y,!b)。变量与 answer 均为 1-based；按党号输出 2*x-1+answer[x]，每人一行。无解只输出 NIE，不能读取失败后的 answer。多组读至 EOF，每组重新构造，无 0 0 结束标记。原站题面本次不可达，题意与格式依据现有驱动及独立枚举测试，不声称已复核官方限制或线上 AC。

所需模板：LexTwoSAT（第 206 页，9.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://acm.hdu.edu.cn/showproblem.php?pid=1814

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m)
    {
        LexTwoSAT graph(n);
        while (m--)
        {
            int u, v;
            cin >> u >> v;
            int x = (u + 1) / 2;
            int y = (v + 1) / 2;
            bool a = u % 2 == 0;
            bool b = v % 2 == 0;
            graph.add(x, !a, y, !b);
        }
        if (!graph.solve())
            cout << "NIE\n";
        else
            for (int x = 1; x <= n; x++) cout << 2 * x - 1 + graph.answer[x] << '\n';
    }
    return 0;
}
```

第十章　欧拉图

10.1　有向图字典序最小欧拉路

构造 DirectedEuler(n)，n≥1，点号 1..n；add(u,v) 返回从 0 开始的逻辑边编号，支持自环与重边。run() 返回是否存在欧拉路；参数 start 默认 0，lexicographic 默认 true。start=0 自动选择最小可行起点，否则固定起点；孤立点不影响有边部分的连通性。成功后 vertices 长度为 m+1，edge_ids 长度为 m，第 i 条原边从 vertices[i] 指向 vertices[i+1]；失败清空两数组。无边时返回单点路径，自动起点为 1。可重复 run，add 会清空旧方案。默认求顶点序列字典序最小方案；关闭排序后只保证任意方案，邻接顺序可能保留上次排序。开启排序总时间 O(n+m log(m+1))，否则 O(n+m)，空间 O(n+m)。DFS 递归深度可能达到 m+1，不是 n。度数检查后，以消费边数判定是否遍历全部非孤立部分，逆后序形成连续走法。小图全部走法对拍同时验证存在性、字典序及边编号，P7771 双风格完整程序本地验证，在线待补。

```
struct DirectedEuler
{
    int n;
    vector<vector<pair<int, int>>> g;
    vector<pair<int, int>> edges;
    vector<int> in, out, next, vertices, edge_ids;

    DirectedEuler(int n) : n(n), g(n + 1), in(n + 1), out(n + 1), next(n + 1)
    {
        assert(n > 0);
    }

    int add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = edges.size();
        edges.push_back({u, v});
        g[u].push_back({v, id});
        out[u]++;
        in[v]++;
        vertices.clear();
        edge_ids.clear();
        return id;
    }

    void dfs(int u, int incoming)
    {
        while (next[u] < (int)g[u].size())
        {
            auto [v, id] = g[u][next[u]++];
            dfs(v, id);
        }
        vertices.push_back(u);
        if (incoming != -1) edge_ids.push_back(incoming);
    }
```

```
    // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
    bool run(int start = 0, bool lexicographic = true)
    {
        assert(0 <= start && start <= n);
        vertices.clear();
        edge_ids.clear();
        int source = 0, sink = 0;
        for (int u = 1; u <= n; u++)
        {
            int delta = out[u] - in[u];
            if (delta == 1 && !source)
                source = u;
            else if (delta == -1 && !sink)
                sink = u;
            else if (delta != 0)
                return false;
        }
        if (bool(source) != bool(sink)) return false;
        if (source)
        {
            if (start && start != source) return false;
            start = source;
        }
        if (!start)
        {
            start = 1;
            for (int u = 1; u <= n; u++)
                if (out[u])
                {
                    start = u;
                    break;
                }
        }
        if (lexicographic)
            for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
        fill(next.begin(), next.end(), 0);
        dfs(start, -1);
        if (edge_ids.size() != edges.size())
        {
            vertices.clear();
            edge_ids.clear();
            return false;
        }
        reverse(vertices.begin(), vertices.end());
        reverse(edge_ids.begin(), edge_ids.end());
        return true;
    }
};
```

模板题使用：Luogu P7771

题意：给有向图，求字典序最小的欧拉路径。run 默认选择可行起点并按字典序遍历；成功后 vertices 含 m+1 个顶点，失败输出 No。点号 1..n。

所需模板：DirectedEuler（第 209 页，10.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P7771

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    DirectedEuler graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    if (!graph.run())
        cout << "No\n";
    else
    {
        for (int u : graph.vertices) cout << u << ' ';
        cout << '\n';
    }
    return 0;
}
```

10.2 字典序最小单词链

代码依赖：DirectedEuler（第 209 页，10.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

word_chain(words) 返回可空的编号序列 (optional), 有解时为原输入的 0-based 单词编号顺序, 每个单词恰用一次；相邻单词首尾字母相接，单词序列字典序最小。无解返回 nullopt，空输入返回有值的空序列。单词必须非空且仅含小写 a-z，重复词视为不同编号。先对原编号按单词排序，按此顺序加边，再调用 DirectedEuler.run(0,false)；不可让底层重新按终点排序。欧拉回路从最小出现首字母开始，非回路起点由度数差确定。O(m log m) 次字符串比较及 O(L+m) 其他工作，L 为总字符数；空间 O(m)，递归栈 O(m)。依赖 directed_euler.hpp。全排列对拍、重复词/前缀词、20 万相同词及 POJ2337 完整输出本地验证，在线待补。

```
// Lowercase nonempty words. Returns original 0-based IDs; duplicates stay distinct.
optional<vector<int>> word_chain(const vector<string> &words)
{
    int n = words.size();
    vector<int> order(n);
    iota(order.begin(), order.end(), 0);
    for (const auto &word : words)
    {
        assert(!word.empty());
        for (char c : word) assert('a' <= c && c <= 'z');
    }
    sort(order.begin(),
         order.end(),
         [&](int a, int b)
         {
             if (words[a] != words[b]) return words[a] < words[b];
             return a < b;
         });
    DirectedEuler graph(26);
    for (int id : order)
        graph.add(words[id].front() - 'a' + 1, words[id].back() - 'a' + 1);
    // Preserve word order in each adjacency list; do not sort by destination.
    if (!graph.run(0, false)) return nullopt;
    vector<int> result;
    for (int id : graph.edge_ids) result.push_back(order[id]);
    return result;
}
```

应用题补充：UVA 10441 Catenyms

题意：多组给出 3..1000 个互不相同的小写单词，长度 1..20。每词恰用一次，后词首字母等于前词尾字母，输出字典序最小的点分隔词链；无解输出 ***。返回原输入的 0-based 编号，按编号取词输出，末尾不加点。单词排序不可换成按终点排序。重复词和空输入仅属更宽接口范围，不在本题范围内。

所需模板：DirectedEuler（第 209 页，10.1 节）、word_chain（第 212 页，10.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.org/external/104/10441.pdf

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<string> words(n);
        for (auto &word : words) cin >> word;
        auto order = word_chain(words);
        if (!order)
        {
            cout << "***\n";
            continue;
        }
        for (int i = 0; i < n; i++)
        {
            if (i) cout << '.';
            cout << words[(*order)[i]];
        }
        cout << '\n';
    }
    return 0;
}
```

10.3 无向图欧拉路与边方向

构造 UndirectedEuler(n)，n≥1，点号 1..n；add(u,v) 返回从 0 开始的逻辑边号，自环和重边均保留。run() 返回是否存在欧拉路，参数 start 默认 0（自动），lexicographic 默认 true。自动选最小可行起点；指定起点须满足奇度条件且位于有边部分。成功得到 m+1 个 vertices 和 m 个 edge_ids，失败清空两数组。第 i 条边方向是 vertices[i] 到 vertices[i+1]，与 edges[edge_ids[i]] 比较可判断是否反向，自环任选方向。used 按逻辑边号标记，使两条半边共同只消费一次；自环贡献度数 2。孤立点不影响有边部分；无边图返回指定点或点 1 的单点序列。默认求顶点序列字典序最小方案；关闭排序只保证任意方案，可能保留上次排序。可重复 run，加边清空旧方案。排序开启时 O(n+m log(m+1))，否则 O(n+m)，空间 O(n+m)，DFS 递归深度可能达到 m+1。全小图走法、原边号与方向独立验证，P2731 字典序顶点序列已线上 AC；边编号、失败输入及其他模式仍由本地验证。

```
struct UndirectedEuler
{
    int n;
    vector<vector<pair<int, int>>> g;
    vector<pair<int, int>> edges;
    vector<int> next, used, vertices, edge_ids;

    UndirectedEuler(int n) : n(n), g(n + 1), next(n + 1) { assert(n > 0); }

    int add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = edges.size();
        edges.push_back({u, v});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        vertices.clear();
        edge_ids.clear();
        return id;
    }

    void dfs(int u, int incoming)
    {
        while (next[u] < (int)g[u].size())
        {
            auto [v, id] = g[u][next[u]++];
            if (used[id]) continue;
            used[id] = 1;
            dfs(v, id);
        }
        vertices.push_back(u);
        if (incoming != -1) edge_ids.push_back(incoming);
    }
```

```
    // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
    bool run(int start = 0, bool lexicographic = true)
    {
        assert(0 <= start && start <= n);
        vertices.clear();
        edge_ids.clear();
        vector<int> odd;
        for (int u = 1; u <= n; u++)
            if (g[u].size() % 2) odd.push_back(u);
        if (!odd.empty() && odd.size() != 2) return false;
        if (!odd.empty())
        {
            if (start && start != odd[0] && start != odd[1]) return false;
            if (!start) start = odd[0];
        }
        if (!start)
        {
            start = 1;
            for (int u = 1; u <= n; u++)
                if (!g[u].empty())
                {
                    start = u;
                    break;
                }
        }
        if (lexicographic)
            for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
        fill(next.begin(), next.end(), 0);
        used.assign(edges.size(), 0);
        dfs(start, -1);
        if (edge_ids.size() != edges.size())
        {
            vertices.clear();
            edge_ids.clear();
            return false;
        }
        reverse(vertices.begin(), vertices.end());
        reverse(edge_ids.begin(), edge_ids.end());
        return true;
    }
};
```

模板题使用：Luogu P2731

题意：USACO Training：无向多重图保证有欧拉路，输出字典序最小顶点序列。m≤1024，点号 1..500；直接构造 500 个点，不把孤立点 1 强设为起点。run() 自动选较小奇点，若全偶选最小非孤立点，并按邻点排序。输出 vertices，共 m+1 项；本题不输出 edge_ids，边编号与方向另验。DFS 递归，最深可达 m+1。

所需模板：UndirectedEuler（第 214 页，10.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2731

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> m;
    UndirectedEuler graph(500);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    if (!graph.run()) return 0;
    for (int u : graph.vertices) cout << u << '\n';
    return 0;
}
```

10.4 欧拉回路与指定端点定向

代码依赖：Dinic（第 227 页，11.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

mixed_euler_orientation(n,edges,start,finish) 返回可空的定向边序列。每条输入为 {u,v,type}，type=1 固定 u 到 v，type=0 可反转；点号 1..n，n≥1。省略两个端点或均传 0 时请求任意起点的欧拉回路；两个正端点请求指定起终点的欧拉路径，相等时为指定起点回路。无解返回 nullopt，有解返回与原边顺序对应的端点对；无边图可返回有值空序列。忽略无关孤立点，但正端点须在有边部分。先任意定向可反转边，扣除目标出入度差；每条反转将两端差值分别改变 -2 和 +2，奇数差无解，否则用容量 1 的反转边流调整。检查底层无向连通性后，满流即得到所需方向。最大流图 n+2 点、至多 m+n 条正向边，总时间可按 Dinic 的 $O((n+2)^2(m+n))$ 上界计，空间 O(n+m)。依赖 flow.hpp；返回边交给 DirectedEuler 可生成连续走法，指定端点时保持指定起点。定向不保证字典序最小。

```
// Edge = {u,v,type}: type 1 is fixed u->v; type 0 may be reversed.
// start=finish=0 requests a circuit; positive endpoints request a specified trail.
optional<vector<pair<int, int>>> mixed_euler_orientation(
    int n, const vector<array<int, 3>> &edges, int start = 0, int finish = 0)
{
    assert(n > 0 && 0 <= start && start <= n && 0 <= finish && finish <= n);
    assert(bool(start) == bool(finish));
    vector<int> parent(n + 1), id;
    vector<long long> degree(n + 1), balance(n + 1);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int u) -> int
    {
        if (parent[u] != u) parent[u] = self(self, parent[u]);
        return parent[u];
    };
    Dinic flow(n + 2);
    vector<pair<int, int>> direction;
    for (auto [u, v, type] : edges)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(type == 0 || type == 1);
        parent[find(find, u)] = find(find, v);
        degree[u]++;
        degree[v]++;
        balance[u]++;
        balance[v]--;
        id.push_back(type == 0 ? flow.add(u, v, 1) : -1);
        direction.push_back({u, v});
    }
```

```
    if (!edges.empty())
    {
        int root = find(find, edges[0][0]);
        for (int u = 1; u <= n; u++)
            if (degree[u] && find(find, u) != root) return nullopt;
        if (start && (!degree[start] || !degree[finish])) return nullopt;
    }
    if (start)
    {
        balance[start]--;
        balance[finish]++;
    }
    int source = n + 1, sink = n + 2;
    long long need = 0;
    for (int u = 1; u <= n; u++)
    {
        if (balance[u] % 2) return nullopt;
        if (balance[u] > 0)
        {
            flow.add(source, u, balance[u] / 2);
            need += balance[u] / 2;
        }
        else if (balance[u] < 0)
            flow.add(u, sink, -balance[u] / 2);
    }
    if (flow.flow(source, sink) != need) return nullopt;
    for (int i = 0; i < (int)edges.size(); i++)
        if (id[i] != -1 && flow.used(id[i]))
            swap(direction[i].first, direction[i].second);
    return direction;
}
```

接口用法演示：混合欧拉路径：定向与走法（自定义协议）

演示约定：自定义接口演示，非 OJ 题目。首行 T；每组 mode n m start finish，再读 m 条 u v type。点号 1..n，type=1 固定方向、0 可反转。C 输入端点 0 0 求回路；A 也输入 0 0，由 mixed_euler_trail 选择端点；P 指定两个正端点，相等时是指定起点回路。optional 有值空数组是合法空方案。返回方向须按原边顺序加入 DirectedEuler，再 run(start,false) 恢复连续走法。成功输出 YES、VERTICES 及 EDGES 证书，原边号由内部 0-based 加一；无解输出 NO。零边 C/A 输出点 1，P s s 输出点 s，不同正端点无解。非空图忽略无关孤立点，指定端点须在有边部分。1≤T≤10000、1≤n≤200、0≤m≤800 仅为演示边界。定向采用 Dinic 一般上界，任意端点最多求两次；欧拉恢复 O(n+m)，DFS 深度可达 m+1。不保证所有定向中的全局字典序最小。

所需模板：Dinic（第 227 页，11.1.2 节）、mixed_euler_orientation（第 217 页，10.4 节）、mixed_euler_trail（第 220 页，10.5 节）、DirectedEuler（第 209 页，10.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/mixed_euler.hpp

混合欧拉路径：定向与走法（自定义协议）：使用代码

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    assert(1 <= tests && tests <= 10000);
    while (tests--)
    {
        char mode;
        int n, m, start, finish;
        cin >> mode >> n >> m >> start >> finish;
        assert(1 <= n && n <= 200 && 0 <= m && m <= 800);
        assert(mode == 'C' || mode == 'A' || mode == 'P');
        assert(mode == 'P' ? 1 <= start && start <= n && 1 <= finish && finish <= n
                           : start == 0 && finish == 0);
        vector<array<int, 3>> edges(m);
        for (auto &[u, v, type] : edges) cin >> u >> v >> type;
        optional<vector<pair<int, int>>> direction;
        if (mode == 'A')
            direction = mixed_euler_trail(n, edges);
        else if (mode == 'C')
            direction = mixed_euler_orientation(n, edges);
        else
            direction = mixed_euler_orientation(n, edges, start, finish);
        if (!direction)
        {
            cout << "NO\n";
            continue;
        }
        // A present empty vector is a valid zero-edge circuit, not failure.
        DirectedEuler trail(n);
        for (auto [u, v] : *direction) trail.add(u, v);
        // Preserve a requested start. The orientation is not globally lexicographic.
        if (!trail.run(start, false)) return 1;
        cout << "YES\nVERTICES " << trail.vertices.size();
        for (int u : trail.vertices) cout << ' ' << u;
        cout << "\nEDGES " << trail.edge_ids.size();
        // add() used original input order; its 0-based IDs become 1-based here.
        for (int id : trail.edge_ids) cout << ' ' << id + 1;
        cout << '\n';
    }
    return 0;
}
```

10.5 任意起终点的混合欧拉路径

代码依赖：mixed_euler_orientation（第 217 页，10.4 节）。抄写时一并准备；标准库类型不列入算法依赖。

mixed_euler_trail(n,edges) 返回任意可行欧拉路径的定向边序列，格式同上一节。忽略边原方向统计度数，自环计 2；若奇度点为 0 个，请求回路；若恰有 2 个，尝试这两个端点的两种顺序；否则无解。最多两次定向求解。返回方向可交给 DirectedEuler 自动选择起点，恢复顶点与原边编号序列。枚举所有可反转边方向、全部固定端点及自由端点对拍；逐边检查固定边不变和连续走法，另有 10 万点长流量路径测试。POJ1637 双驱动本地验证，仅检查回路存在性；完整方向和开放路径依靠本地证据，在线待补。

```
// Any endpoints: zero odd degrees requests a circuit; two odd degrees tries both
// orders.
optional<vector<pair<int, int>>> mixed_euler_trail(int n,
                                                   const vector<array<int, 3>> &edges)
{
    assert(n > 0);
    vector<long long> degree(n + 1);
    vector<int> odd;
    for (auto [u, v, type] : edges)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        degree[u]++;
        degree[v]++;
    }
    for (int u = 1; u <= n; u++)
        if (degree[u] % 2) odd.push_back(u);
    if (odd.empty()) return mixed_euler_orientation(n, edges);
    if (odd.size() != 2) return nullopt;
    auto answer = mixed_euler_orientation(n, edges, odd[0], odd[1]);
    if (answer) return answer;
    return mixed_euler_orientation(n, edges, odd[1], odd[0]);
}
```

联合使用示例见第 219 页（混合欧拉路径：定向与走法（自定义协议）），其中直接调用本模板。

第十一章　网络流

11.1　最大流

11.1.1　HLPP 最高标号预流推进与流方案

HLPP(n)，2≤n<INT_MAX/2，点 1..n。add(u,v,cap) 返回偶数残量边号 id，允许自环、重边、反向边和零容量；原边数 <INT_MAX/2，全部原容量之和 ≤LLONG_MAX。flow(s,t) 要求 s≠t，每次从零重算并返回总最大流，支持重复调用、换源汇和加边后重算；与 Dinic 返回增量的约定不同。used(id) 读原边净流，min_cut() 返回源侧 0/1 数组（下标 0 不用），只在完整 flow 后有效，加边后须重算。保证非源汇流守恒，多余预流确实退回源点；不止计算最大预流值。e/g 只读，不支持外部改容量或限流。初始反向 BFS 标号，最高标号单链桶、当前弧和 gap；没有周期全局重标号。不可达点初高 n+1，仍须放流。head/next 是活跃点桶链表，不是 DFS 栈；本算法无需 DFS。给出保守 $O(n^2(n+m))$ 时间、O(n+m) 空间界；不据经验或 AC 声称线性。

```
struct HLPP
{
    using ll = long long;
    struct Edge
    {
        int from, to;
        ll cap, initial;
    };
    int n, s, t, top;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> h, count, cur, head, next;
    vector<ll> ex;

    HLPP(int n) : n(n), g(n + 1)
    {
        assert(n >= 2 && n < INT_MAX / 2);
    }

    int add(int u, int v, ll cap)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && cap >= 0);
        int id = e.size();
        e.push_back({u, v, cap, cap});
        e.push_back({v, u, 0, 0});
        g[u].push_back(id);
        g[v].push_back(id ^ 1);
        return id;
    }
```

活跃点、推流与 gap 重标号（接上页同一结构体）：

```
    void activate(int u)
    {
        if (u == s || u == t || !ex[u]) return;
        next[u] = head[h[u]];
        head[h[u]] = u;
        top = max(top, h[u]);
    }

    void push(int id, ll f)
    {
        auto &a = e[id];
        e[id ^ 1].cap += f;
        a.cap -= f;
        ex[a.from] -= f;
        bool empty = ex[a.to] == 0;
        ex[a.to] += f;
        if (empty) activate(a.to);
    }

    void relabel(int u)
    {
        int old = h[u];
        count[old]--;
        h[u] = 2 * n;
        for (int id : g[u])
            if (e[id].cap) h[u] = min(h[u], h[e[id].to] + 1);
        count[h[u]]++;
        cur[u] = 0;
        if (old >= n || count[old]) return;
        for (int v = 1; v <= n; v++)
        {
            if (v != s && h[v] > old && h[v] < n)
            {
                count[h[v]]--;
                h[v] = n + 1;
                count[h[v]]++;
                cur[v] = 0;
            }
        }
        fill(head.begin(), head.end(), -1);
        top = -1;
        for (int v = 1; v <= n; v++)
            if (v != u) activate(v);
    }
```

从零初始化与反向 BFS（接上页同一结构体）：

```
    // Recompute from zero, including after adding edges or changing terminals.
    ll flow(int source, int sink)
    {
        assert(1 <= source && source <= n && 1 <= sink && sink <= n);
        assert(source != sink);
        s = source;
        t = sink;
        for (auto &a : e) a.cap = a.initial;
        h.assign(n + 1, n + 1);
        cur.assign(n + 1, 0);
        ex.assign(n + 1, 0);
        next.assign(n + 1, -1);
        head.assign(2 * n + 1, -1);
        count.assign(2 * n + 1, 0);
        queue<int> q;
        h[t] = 0;
        q.push(t);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int id : g[u])
            {
                int v = e[id].to;
                if (e[id ^ 1].cap && h[v] == n + 1)
                {
                    h[v] = h[u] + 1;
                    q.push(v);
                }
            }
        }
        h[s] = n;
        for (int u = 1; u <= n; u++) count[h[u]]++;
        top = -1;
        for (int id : g[s])
            if (e[id].to != s && e[id].cap) push(id, e[id].cap);
```

最高标号放流（接上页同一结构体）：

```
        while (top >= 0)
        {
            if (head[top] == -1)
            {
                top--;
                continue;
            }
            int u = head[top];
            head[top] = next[u];
            while (ex[u])
            {
                if (cur[u] == int(g[u].size()))
                {
                    relabel(u);
                    continue;
                }
                int id = g[u][cur[u]];
                if (e[id].cap && h[u] == h[e[id].to] + 1)
                    push(id, min(ex[u], e[id].cap));
                else
                    cur[u]++;
            }
        }
        return ex[t];
    }
```

原边流量与最小割（接上页同一结构体）：

```
    ll used(int id) const
    {
        return e[id].initial - e[id].cap;
    }

    vector<int> min_cut() const
    {
        vector<int> seen(n + 1);
        queue<int> q;
        seen[s] = 1;
        q.push(s);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int id : g[u])
            {
                int v = e[id].to;
                if (e[id].cap && !seen[v])
                {
                    seen[v] = 1;
                    q.push(v);
                }
            }
        }
        return seen;
    }
};
```

预流不能冒充流方案 不可达汇点的活跃点仍须继续放流，直到多余流退回源点；否则虽然 ex[t] 可能已是最大流值，used 仍可能违反中间点流守恒。完成后，每对残量容量之和等于原容量，各原边流量合法；源侧残量可达集给出同值最小割。

桶与 gap head[h] 和 next[u] 是活跃点单链桶，不是 DFS 栈。点仅在多余流从 0 变正时加入，取出后放流至 0；gap 改变多个点高度后重建桶，并排除当前正在处理的点，避免重复入桶或旧高度记录。初始 BFS 只做一次，反向遍历必须看反边残量；没有周期全局重标号。

调用约定 flow 每次清空旧流并重算，因此允许换源汇；不返回增量，也不支持限流。used 只接收 add 返回的偶数边号，min_cut 只在完整求流后有效。容量总和不超过 LLONG_MAX，不能只保证最大流值未溢出而忽略初始化预流。

模板题使用：P4722 【模板】最大流加强版 / 预流推进

题意：n≤1200、m≤120000，有向容量，输出最大流；允许重边/反向边。

所需模板：HLPP（第 221 页，11.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4722

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    HLPP g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        g.add(u, v, c);
    }
    cout << g.flow(s, t) << '\n';
}
```

接口用法演示：API 最大流方案与最小割

演示约定：输入 n m s t 和 m 条边；输出总流量、每条原边流量，最后输出 n 位源侧标记。

所需模板：HLPP（第 221 页，11.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://www.luogu.com.cn/problem/P4722

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    HLPP g(n);
    vector<int> ids;
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long cap;
        cin >> u >> v >> cap;
        ids.push_back(g.add(u, v, cap));
    }
    cout << g.flow(s, t) << '\n';
    for (int id : ids) cout << g.used(id) << '\n';
    auto cut = g.min_cut();
    for (int u = 1; u <= n; u++) cout << cut[u];
    cout << '\n';
}
```

11.1.2 最大流与方案

Dinic(n)，点号 1..n；add(u,v,c) 返回偶数原边编号；flow(s,t,limit) 返回本次新增流；used(id) 查询原边净流，cut(s) 返回残量可达点集。允许重边、自环，容量非负，s≠t；重复求流须使用相同源汇。只有求尽最大流后 cut 才是最小割。一般 $O(V^2E)$，递归增广深度 O(V)。总流不得超过 long long 上限。get_edge(id) 返回 tuple(起点, 终点, 总容量, 当前流)，change_edge(id,cap,f) 修改总容量与流，要求 id 为 add 返回的偶数原边编号且 0≤f≤cap；正向残量变为 cap-f，反向为 f。仅保证此残量对一致，不自动维持所有顶点的流量守恒；继续 flow 前必须由调用方保证全局可行流及同源汇。增容且保留 f 合法，缩容低于已有流时不能只截断该边。要从零重算须将所有原边一起重置为零流。两个新接口为 O(1)，尚只有本地验证。

```
struct Dinic
{
    using ll = long long;

    struct Edge
    {
        int from, to;
        ll cap, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> dep, cur;

    Dinic(int n) : n(n), g(n + 1), dep(n + 1), cur(n + 1) {}

    int add(int u, int v, ll c)
    {
        assert(c >= 0);
        int id = (int)e.size();
        e.push_back({u, v, c, c});
        e.push_back({v, u, 0, 0});
        g[u].push_back(id);
        g[v].push_back(id + 1);
        return id;
    }
```

```
    bool bfs(int s, int t)
    {
        fill(dep.begin(), dep.end(), -1);
        queue<int> q;
        dep[s] = 0;
        q.push(s);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int id : g[u])
                if (e[id].cap && dep[e[id].to] < 0)
                {
                    dep[e[id].to] = dep[u] + 1;
                    q.push(e[id].to);
                }
        }
        return dep[t] >= 0;
    }

    ll dfs(int u, int t, ll f)
    {
        if (u == t) return f;
        for (int &i = cur[u]; i < (int)g[u].size(); i++)
        {
            int id = g[u][i], v = e[id].to;
            if (!e[id].cap || dep[v] != dep[u] + 1) continue;
            ll d = dfs(v, t, min(f, e[id].cap));
            if (d)
            {
                e[id].cap -= d;
                e[id ^ 1].cap += d;
                return d;
            }
        }
        return 0;
    }
```

```
    // Returns additional flow; reuse only with the same source/sink.
    ll flow(int s, int t, ll limit = LLONG_MAX)
    {
        assert(s != t && limit >= 0);
        ll ans = 0;
        while (ans < limit && bfs(s, t))
        {
            fill(cur.begin(), cur.end(), 0);
            while (ans < limit)
            {
                ll d = dfs(s, t, limit - ans);
                if (!d) break;
                ans += d;
            }
        }
        return ans;
    }

    ll used(int id) const { return e[id].initial - e[id].cap; }

    tuple<int, int, ll, ll> get_edge(int id) const
    {
        assert(0 <= id && id < (int)e.size() && !(id & 1));
        return {e[id].from, e[id].to, e[id].initial, used(id)};
    }

    // Changes one residual pair; caller must preserve global flow conservation.
    void change_edge(int id, ll cap, ll f)
    {
        assert(0 <= id && id < (int)e.size() && !(id & 1));
        assert(0 <= f && f <= cap);
        e[id].initial = cap;
        e[id].cap = cap - f;
        e[id ^ 1].initial = 0;
        e[id ^ 1].cap = f;
    }

    vector<int> cut(int s)
    {
        bfs(s, s);
        vector<int> ans;
        for (int u = 1; u <= n; u++)
            if (dep[u] >= 0) ans.push_back(u);
        return ans;
    }
};
```

模板题使用：Luogu P3376

题意：给定有向容量网络与源点、汇点，求最大流。输入点号为 1..n，平行边逐条加入。

所需模板：Dinic（第 227 页，11.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3376

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    Dinic g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        g.add(u, v, c);
    }
    cout << g.flow(s, t) << '\n';
}
```

接口用法演示：右侧容量二分图匹配与方案（接口演示）

演示约定：输入 n,m,e，m 个非负 long long 右容量，再输入 e 条左右各自 0-based 的可配对边。每个左点最多使用一次；输出最大配对数和对应 (u,v)。支持空侧、零容量、重边，单容量可达 LLONG_MAX；总流至多 n，无需累加容量。保存 add 边号，用 used 还原方案。一次性建图求解，不把 flow 的新增量误作重跑总量。

所需模板：Dinic（第 227 页，11.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：容量匹配来源与接口核验

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, e;
    cin >> n >> m >> e;
    int s = n + m + 1;
    int t = s + 1;
    Dinic g(t);
    for (int u = 0; u < n; u++)
        g.add(s, u + 1, 1);
    for (int v = 0; v < m; v++)
    {
        long long c;
        cin >> c;
        g.add(n + v + 1, t, c);
    }
    vector<array<int, 3>> edges;
    for (int i = 0; i < e; i++)
    {
        int u, v;
        cin >> u >> v;
        int id = g.add(u + 1, n + v + 1, 1);
        edges.push_back({u, v, id});
    }
    cout << g.flow(s, t) << '\n';
    for (auto [u, v, id] : edges)
        if (g.used(id))
            cout << u << ' ' << v << '\n';
}
```

应用题补充：Luogu P2763 / 试题库问题

题意：k 种题型、n 道题，给出每类需求和各题可属类别；每题最多选一次，输出恰好满足各类需求的方案，或 No Solution!。2≤k≤20，k≤n≤1000；不是独立匹配模板题。题号/类别均 1-based；源到题容量 1，题到类容量 1，类到汇为需求，满流即各类满足。need 逐项截到 n+1 再累加，超过 n 的需求必无解，可避免超大容量求和溢出。

所需模板：Dinic（第 227 页，11.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2763

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int k, n;
    cin >> k >> n;
    int s = n + k + 1;
    int t = s + 1;
    Dinic g(t);
    long long need = 0;
    for (int v = 1; v <= k; v++)
    {
        long long c;
        cin >> c;
        need += min(c, 1LL * n + 1);
        g.add(n + v, t, c);
    }
    vector<vector<pair<int, int>>> ids(k + 1);
    for (int u = 1; u <= n; u++)
    {
        g.add(s, u, 1);
        int p;
        cin >> p;
        while (p--)
        {
            int v;
            cin >> v;
            int id = g.add(u, n + v, 1);
            ids[v].push_back({u, id});
        }
    }
    if (g.flow(s, t) != need)
    {
        cout << "No Solution!\n";
        return 0;
    }
    for (int v = 1; v <= k; v++)
    {
        cout << v << ':';
        for (auto [u, id] : ids[v])
            if (g.used(id))
                cout << ' ' << u;
        cout << '\n';
    }
}
```

11.1.3 最大流中单位容量边的可行性与必选性

代码依赖：Dinic（第 227 页，11.1.2 节）、TarjanSCC（第 150 页，6.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

unit_flow_edges(g,ids) 在完整普通最大流的残量图上，查询每条指定边的流量是否在至少一个/每一个整数最大流中等于 1，返回 {possible,forced}。ids 必须是 Dinic.add 返回的原始偶数边号，且每条被查询边初始容量恰为 1；其他边允许任意非负 long long 容量，最大流值须可表示。结果保持查询顺序，支持重复编号。已使用或两端残量 SCC 相同则可行，已使用且不同 SCC 则必选。这里比较的是完整流量方案，允许不改变源汇净流量的环流；单位自环可以取 0 或 1，不应解释成必经的简单 s-t 路径。不是最小割边分类，也不把容量大于 1 的边强行按布尔流处理。O(n+m+ 查询数) 时间与空间，Tarjan 保留递归；分析不修改 g。

```
vector<array<bool, 2>> unit_flow_edges(const Dinic &g, const vector<int> &ids)
{
    TarjanSCC d(g.n);
    for (auto e : g.e)
        if (e.cap) d.add(e.from, e.to);
    d.run();
    vector<array<bool, 2>> ans;
    for (int id : ids)
    {
        assert(0 <= id && id < (int)g.e.size() && id % 2 == 0);
        auto e = g.e[id];
        assert(e.initial == 1);
        bool used = g.used(id) == 1, same = d.bel[e.from] == d.bel[e.to];
        ans.push_back({used || same, used && !same});
    }
    return ans;
}
```

应用题补充：最大流单位边 Y/N/?（本地接口示例）

题意：输入 n、m、s、t、q，随后 m 条有向容量边和 q 个原边序号；被询问边容量须为 1。先输出最大流值，再输出 Y（所有方案取 1）、N（没有方案取 1）、?（两种都可能）。用 add 返回值保存残量边号。本地示例对应附件 2.14 的分类内核，不代替原比赛题的排列约束建图。

所需模板：Dinic（第 227 页，11.1.2 节）、TarjanSCC（第 150 页，6.1.1 节）、unit_flow_edges（第 233 页，11.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://github.com/gsh20040816/CPCtemplate/issues/11

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t, q;
    cin >> n >> m >> s >> t >> q;
    Dinic g(n);
    vector<int> edge(m), ids(q);
    for (int &id : edge)
    {
        int u, v;
        long long cap;
        cin >> u >> v >> cap;
        id = g.add(u, v, cap);
    }
    for (int &id : ids)
    {
        int i;
        cin >> i;
        id = edge[i - 1];
    }
    cout << g.flow(s, t) << '\n';
    for (auto [possible, forced] : unit_flow_edges(g, ids))
        cout << (forced ? 'Y' : possible ? '?' : 'N');
    cout << '\n';
}
```

应用题补充：QOJ 10424 / NERC 2024 K

题意：两套排列各给出同一 k 元素集合的部分子序列，-1 表示未知；输出每个元素必选 Y、必不选 N 或可选?，无解输出 Inconsistent。已知元素切分两侧容量组，每个元素对应一条单位边；最大流必须等于 k，再作残量 SCC 分类。n 及所有测试的 n 之和至 2000。比赛应用，不计作独立模板题覆盖。

所需模板：Dinic（第 227 页，11.1.2 节）、TarjanSCC（第 150 页，6.1.1 节）、unit_flow_edges（第 233 页，11.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://qoj.ac/problem/10424

```
bool blocks(const vector<int> &p,
            const vector<int> &s,
            vector<int> &id,
            vector<int> &cap)
{
    int n = p.size(), k = s.size(), last = -1, prev = -1;
    vector<int> pos(n + 1);
    for (int i = 0; i < n; i++) pos[p[i]] = i;
    id.assign(n + 1, -1);
    cap.clear();
    for (int i = 0; i <= k; i++)
    {
        if (i < k && s[i] == -1) continue;
        int at = i == k ? n : pos[s[i]];
        if (at <= last || at - last - 1 < i - prev - 1) return false;
        int g = cap.size();
        cap.push_back(i - prev - 1);
        for (int j = last + 1; j < at; j++) id[p[j]] = g;
        if (i < k)
        {
            id[s[i]] = cap.size();
            cap.push_back(1);
        }
        last = at;
        prev = i;
    }
    return true;
}
```

使用入口（接上页配置函数）：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n, k;
        cin >> n;
        vector<int> p(n), q(n);
        for (int &x : p) cin >> x;
        for (int &x : q) cin >> x;
        cin >> k;
        vector<int> a(k), b(k), l, r, x, y;
        for (int &v : a) cin >> v;
        for (int &v : b) cin >> v;
        if (!blocks(p, a, l, x) || !blocks(q, b, r, y))
        {
            cout << "Inconsistent\n";
            continue;
        }
        int m = x.size(), source = m + y.size() + 1, sink = source + 1;
        Dinic g(sink);
        for (int i = 0; i < m; i++) g.add(source, i + 1, x[i]);
        for (int i = 0; i < (int)y.size(); i++) g.add(m + i + 1, sink, y[i]);
        vector<int> ids;
        for (int v = 1; v <= n; v++) ids.push_back(g.add(l[v] + 1, m + r[v] + 1, 1));
        if (g.flow(source, sink) != k)
        {
            cout << "Inconsistent\n";
            continue;
        }
        for (auto [possible, forced] : unit_flow_edges(g, ids))
            cout << (forced ? 'Y' : possible ? '?' : 'N');
        cout << '\n';
    }
}
```

11.1.4 递归 ISAP 最大流

Isap(n)，点号 1..n；add(u,v,cap) 返回偶数原边编号，used(id) 读取流量；flow(s,t,limit) 返回本次新增流，默认 LLONG_MAX，重复调用保持相同源汇。cut(s) 返回残量可达点集，只有求尽后才是最小割。容量非负，总最大流 ≤LLONG_MAX，允许自环、重边与反向原边；重建对象从零重算。e[id] 含 from/to/cap/initial，其中 cap 是残量容量；e/g 可读，任意改动须维护全局可行流。反向残量 BFS 初始化距离，当前弧、重标号和 gap 剪枝，DFS 保持递归，最坏 $O(V^2E)$，空间 O(V+E)、递归深度 O(V)。bfs/dfs 为内部方法，不能用其临时标签代替 cut。与 Dinic 为独立备选，不承诺更快；两个方向有容量时分别 add，逐边方案不压成有符号净流。

```
struct Isap
{
    using ll = long long;

    struct Edge
    {
        int from, to;
        ll cap, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> dep, cur, gap;

    Isap(int n) : n(n), g(n + 1), dep(n + 1), cur(n + 1), gap(n + 1)
    {
    }

    int add(int u, int v, ll cap)
    {
        assert(cap >= 0);
        int id = (int)e.size();
        e.push_back({u, v, cap, cap});
        e.push_back({v, u, 0, 0});
        g[u].push_back(id);
        g[v].push_back(id ^ 1);
        return id;
    }

    ll used(int id) const
    {
        return e[id].initial - e[id].cap;
    }
```

接上页同一结构体：

```
    void bfs(int t)
    {
        fill(dep.begin(), dep.end(), n);
        fill(gap.begin(), gap.end(), 0);
        queue<int> q;
        dep[t] = 0;
        q.push(t);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int id : g[u])
            {
                int v = e[id].to;
                if (!e[id ^ 1].cap || dep[v] != n) continue;
                dep[v] = dep[u] + 1;
                q.push(v);
            }
        }
        for (int u = 1; u <= n; u++) gap[dep[u]]++;
    }

    ll dfs(int u, int s, int t, ll f)
    {
        if (u == t) return f;
        for (int &i = cur[u]; i < (int)g[u].size(); i++)
        {
            int id = g[u][i], v = e[id].to;
            if (!e[id].cap || dep[u] != dep[v] + 1) continue;
            ll take = dfs(v, s, t, min(f, e[id].cap));
            if (take)
            {
                e[id].cap -= take;
                e[id ^ 1].cap += take;
                return take;
            }
            if (dep[s] == n) return 0;
        }
        int old = dep[u], best = n;
        for (int id : g[u])
            if (e[id].cap) best = min(best, dep[e[id].to] + 1);
        dep[u] = best;
        cur[u] = 0;
        gap[best]++;
        if (--gap[old] == 0) dep[s] = n;
        return 0;
    }
```

接上页同一结构体：

```
    // Returns additional flow; reuse only with the same source/sink.
    ll flow(int s, int t, ll limit = LLONG_MAX)
    {
        assert(s != t && limit >= 0);
        bfs(t);
        fill(cur.begin(), cur.end(), 0);
        ll ans = 0;
        while (ans < limit && dep[s] < n)
            ans += dfs(s, s, t, limit - ans);
        return ans;
    }

    vector<int> cut(int s) const
    {
        vector<int> vis(n + 1), ans;
        queue<int> q;
        vis[s] = 1;
        q.push(s);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            ans.push_back(u);
            for (int id : g[u])
            {
                int v = e[id].to;
                if (!e[id].cap || vis[v]) continue;
                vis[v] = 1;
                q.push(v);
            }
        }
        return ans;
    }
};
```

模板题使用：Luogu P3376

题意：给定 n,m,s,t 和 1-based 有向容量边，输出最大流。n≤200、m≤5000、0≤ 容量 <2^31；s≠t，重边逐条加入。总流可能超 int，使用 long long。这里只验证正式题目 add/flow 入口，限流、方案及 cut 另有独立本地核验。

所需模板：Isap（第 237 页，11.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3376

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    Isap g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        g.add(u, v, c);
    }
    cout << g.flow(s, t) << '\n';
}
```

11.1.5 最大流方案唯一性

代码依赖：Dinic（第 227 页，11.1.2 节）、TarjanSCC（第 150 页，6.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

flow_unique(g) 只读检查 Dinic 已求尽的可行最大流，其原边流量向量是否唯一；源汇不同，容量非负整数，最大流可由 long long 表示。允许重边、自环、零容量和不连通图，包含独立环流；与最小割唯一性不同。残量 SCC 内按原边对各计一次，正容量自环亦计一条；某分量内部原边数不少于顶点数即多解，否则唯一。正反残量同时存在的同一边不算两条，不把无效的往返抵消视为新方案。复杂度 O(n+m)，空间 O(n+m)，依赖 Dinic、TarjanSCC，DFS 保留递归。不修改 g，代表方案仍由 used 读取；不返回第二个方案。须先求完流，不能拿尚未求完的网络当最大流结论。

```
bool flow_unique(const Dinic &g)
{
    TarjanSCC d(g.n);
    for (auto e : g.e)
        if (e.cap) d.add(e.from, e.to);
    d.run();
    vector<int> left(d.cnt + 1);
    for (int u = 1; u <= g.n; u++)
        left[d.bel[u]]++;
    for (int id = 0; id < (int)g.e.size(); id += 2)
    {
        auto e = g.e[id];
        if (!e.cap && !g.e[id ^ 1].cap) continue;
        if (d.bel[e.from] != d.bel[e.to]) continue;
        if (--left[d.bel[e.from]] == 0) return false;
    }
    return true;
}
```

接口用法演示：最大流方案唯一性（接口演示）

演示约定：输入 n,m,s,t 及 1-based 有向边 (u,v, 非负 long long 容量)，先求最大流，再输出流值、UNIQUE 或 MULTIPLE，以及按输入顺序的各原边流量。包括所有独立环流，重边分别计量；不是最小割唯一性。s≠t，总最大流 ≤LLONG_MAX。仅接口示例，尚无在线判题记录。

所需模板：Dinic（第 227 页，11.1.2 节）、TarjanSCC（第 150 页，6.1.1 节）、flow_unique（第 241 页，11.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：最大流唯一性的合同与证明

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    Dinic g(n);
    vector<int> ids;
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        ids.push_back(g.add(u, v, c));
    }
    cout << g.flow(s, t) << '\n';
    cout << (flow_unique(g) ? "UNIQUE" : "MULTIPLE") << '\n';
    for (int id : ids)
        cout << g.used(id) << '\n';
}
```

11.2 最小割

11.2.1 正容量最小割的可行边与必选边

代码依赖：Dinic（第 227 页，11.1.2 节）、TarjanSCC（第 150 页，6.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

mincut_edges(g,s,t) 只读分析 Dinic 已经完整求出的普通 s→t 最大流；不得传入限流提前停止或人为修改过的残量网络。所有原边初始容量必须严格为正，最大流值须可由 long long 表示。返回逐条原边的 {possible,forced}，对应 Dinic 原始偶数边号 id 的结果是 ans[id/2]。只有饱和边可能属于最小割：两端残量 SCC 不同则可行；起点与 s 同 SCC、终点与 t 同 SCC 则必选。支持平行边和自环，源汇不同，点号 1..g.n。O(n+m) 时间与空间，Tarjan 递归栈 O(n)。零容量边没有正流对应的反向残量边，不能套用本函数的同 SCC 简化条件。每次修改图或重新增广后都需重新分析。

```
vector<array<bool, 2>> mincut_edges(const Dinic &g, int s, int t)
{
    TarjanSCC d(g.n);
    for (auto e : g.e)
        if (e.cap) d.add(e.from, e.to);
    d.run();
    vector<array<bool, 2>> ans(g.e.size() / 2);
    for (int id = 0; id < (int)g.e.size(); id += 2)
    {
        auto e = g.e[id];
        assert(e.initial > 0);
        if (e.cap) continue;
        ans[id / 2][0] = d.bel[e.from] != d.bel[e.to];
        ans[id / 2][1] = d.bel[e.from] == d.bel[s] && d.bel[e.to] == d.bel[t];
    }
    return ans;
}
```

应用题补充：Luogu P4126 / AHOI2009 最小割

题意：求每条正容量有向边是否出现在至少一个/每一个最小割中。先不设额外限制地跑完最大流，再按原边输入顺序输出两项布尔值。容量用 long long；平行道路分别加边。此题源自 AHOI，作为应用补充单列。

所需模板：Dinic（第 227 页，11.1.2 节）、TarjanSCC（第 150 页，6.1.1 节）、mincut_edges（第 243 页，11.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4126

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    Dinic g(n);
    while (m--)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        g.add(u, v, c);
    }
    g.flow(s, t);
    for (auto [possible, forced] : mincut_edges(g, s, t))
        cout << possible << ' ' << forced << '\n';
}
```

11.2.2 最大权闭合子图与选点方案

代码依赖：Dinic（第 227 页，11.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

maximum_closure(weight,dependencies) 返回 (最大权值, 选中点列表), weight 为 0-based vector<long long>, 图中点号 1..n。依赖边 (u,v) 表示选 u 必须选 v, 允许环、自环与重边; 允许空选集, 因此答案不小于 0。依赖 flow.hpp 中的 Dinic。正权连源、负权连汇，依赖边容量取正权总和加 1；要求所有正权之和严格小于 LLONG_MAX。负权先提升到 int128 再取负，并截断到依赖边容量，因此 LLONG_MIN 也可处理。返回残量图源点可达的原顶点，权值为正权总和减最小割；超大负权边不会出现在最优割中，截断不改变原问题最优值。每次调用重建网络。复杂度沿用 Dinic：一般 $O(V^2E)$，空间 O(V+E)，增广保持递归。返回一种最优闭合集，不承诺最大或最小选点个数。

```
pair<long long, vector<int>> maximum_closure(const vector<long long> &weight,
                                             const vector<pair<int, int>> &dependencies)
{
    using I = __int128_t;
    int n = weight.size(), s = n + 1, t = n + 2;
    I positive = 0;
    for (long long w : weight)
        if (w > 0) positive += w;
    assert(positive < LLONG_MAX);
    long long inf = (long long)positive + 1;
    Dinic graph(n + 2);
    for (int u = 1; u <= n; u++)
    {
        long long w = weight[u - 1];
        if (w > 0)
            graph.add(s, u, w);
        else if (w < 0)
            graph.add(u, t, (long long)min(-I(w), I(inf)));
    }
    for (auto [u, v] : dependencies)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        graph.add(u, v, inf);
    }
    long long answer = (long long)positive - graph.flow(s, t);
    vector<int> selected;
    for (int u : graph.cut(s))
        if (u <= n) selected.push_back(u);
    return {answer, selected};
}
```

应用题补充：Luogu P2762 太空飞行计划问题

题意：选实验获得收益，配置仪器支付费用，输出最优实验/仪器编号和净收益。m、n 至 50，单项费用为正且小于 2^{31}，累计用 long long。逐行读取变长依赖，istringstream 兼容 CRLF；实验 i 依赖仪器 m+j。应用用法，不计非比赛模板覆盖。

所需模板：Dinic（第 227 页，11.1.2 节）、maximum_closure（第 245 页，11.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2762

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m, n;
    cin >> m >> n;
    vector<long long> w(m + n);
    vector<pair<int, int>> dep;
    string line;
    getline(cin, line);
    for (int i = 1; i <= m; i++)
    {
        getline(cin, line);
        istringstream in(line);
        in >> w[i - 1];
        int j;
        while (in >> j) dep.push_back({i, m + j});
    }
    for (int j = 0; j < n; j++)
    {
        long long cost;
        cin >> cost;
        w[m + j] = -cost;
    }
    auto [best, selected] = maximum_closure(w, dep);
    for (int u : selected)
        if (u <= m) cout << u << ' ';
    cout << '\n';
    for (int u : selected)
        if (u > m) cout << u - m << ' ';
    cout << '\n' << best << '\n';
}
```

11.2.3 Gomory–Hu 最小割树

代码依赖：Dinic（第 227 页，11.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

gomory_hu(n,edges) 返回 n-1 条 (u,v, 容量) 树边；n≥1，原图点号 0..n-1，输入为无向边 tuple 列表，重边保留、自环忽略、容量非负。非自环边总容量须严格小于 LLONG_MAX。两点最小割是树路径最小边权；删去树边得到的两侧也是原图该边端点的最小割。允许非连通图和零权树边，n=1 返回空树。运行 n-1 次 Dinic，一般上界 $O(n^3m+n^2)$，额外空间 O(n+m)，DFS 增广保持递归。P4897 输入 n 表示 n+1 个点，驱动传 n+1。双版本 P4897 已 AC，最长单点分别 252、185 ms。

```
vector<tuple<int, int, long long>>
gomory_hu(int n, const vector<tuple<int, int, long long>> &edges)
{
    assert(n >= 1);
    vector<int> parent(n);
    vector<long long> value(n);
    for (int s = 1; s < n; s++)
    {
        int t = parent[s];
        Dinic flow(n);
        for (auto [u, v, w] : edges)
            if (u != v)
            {
                flow.add(u + 1, v + 1, w);
                flow.add(v + 1, u + 1, w);
            }
        long long cut = flow.flow(s + 1, t + 1);
        vector<bool> side(n);
        for (int u : flow.cut(s + 1)) side[u - 1] = true;
        for (int v = 1; v < n; v++)
            if (v != s && parent[v] == t && side[v]) parent[v] = s;
        if (side[parent[t]])
        {
            parent[s] = parent[t];
            parent[t] = s;
            value[s] = value[t];
            value[t] = cut;
        }
        else
            value[s] = cut;
    }
    vector<tuple<int, int, long long>> tree;
    for (int v = 1; v < n; v++) tree.push_back({v, parent[v], value[v]});
    return tree;
}
```

模板题使用：Luogu P4897

题意：给无向图，多次询问不同两点的最小割。点号为 0..n，因此建树传 n+1。再预处理所有点对的答案，查询矩阵额外占 O(n^2) 时间和空间。

所需模板：Dinic（第 227 页，11.1.2 节）、gomory_hu（第 247 页，11.2.3 节）、cut_tree_values（第 249 页，11.2.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4897

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    vector<tuple<int, int, long long>> edges;
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        scanf("%d%d%lld", &u, &v, &w);
        edges.push_back({u, v, w});
    }
    auto tree = gomory_hu(n + 1, edges);
    auto answer = cut_tree_values(n + 1, tree);
    int q;
    scanf("%d", &q);
    while (q--)
    {
        int u, v;
        scanf("%d%d", &u, &v);
        printf("%lld\n", answer[u][v]);
    }
    return 0;
}
```

11.2.4 最小割树全点对查询

cut_tree_values(n,tree) 对合法的连通无环加权树计算全点对路径最小边权；点号 0..n-1、n≥1、恰有 n-1 条边，边权为非负 long long。输入可直接使用 gomory_hu 的返回值。结果 ans[u][v] 在 u≠v 时为两点最小割；对角线为 LLONG_MAX 哨兵，不能解释成同点最小割。n=1 返回单个哨兵。$O(n^2)$ 时间与空间，递归 DFS 深度 O(n)，查询 O(1)。只需要标准库，不依赖流模板；不是任意图的查询函数。

```
vector<vector<long long>>
cut_tree_values(int n, const vector<tuple<int, int, long long>> &tree)
{
    assert(n >= 1 && tree.size() == (size_t)n - 1);
    vector<vector<pair<int, long long>>> g(n);
    for (auto [u, v, w] : tree)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    vector<vector<long long>> answer(n, vector<long long>(n, LLONG_MAX));
    for (int s = 0; s < n; s++)
    {
        auto dfs = [&](auto &&self, int u, int parent, long long value) -> void
        {
            answer[s][u] = value;
            for (auto [v, w] : g[u])
                if (v != parent) self(self, v, u, min(value, w));
        };
        dfs(dfs, s, -1, LLONG_MAX);
    }
    return answer;
}
```

联合使用示例见第 248 页（Luogu P4897），其中直接调用本模板。

11.3 费用流

11.3.1 势能最小费用最大流

MinCostFlow(n),点号 1..n;add(u,v,cap,cost) 返回边编号,flow(s,t,limit) 返回新增流量及 128 位费用,used(id) 返回原边流量。允许负费用，初始残量网络必须没有负环；检测到负环抛异常。Bellman-Ford 初始化 O(VE)，每次增广 O(E log V)；增广次数不能简单假设为 O(V)。另要求 $\sum(\text{cap}\times|\text{cost}|)<2^{120}$，防止总费用超过 int128。slope(s,t,limit) 返回从 (0,0) 开始的本次新增流量/费用折点，相邻同斜率段合并，费用为 int128。无增广时只返回起点；每个整数流量的费用可在线性段内插值。重复调用仍用同源汇，返回值不是历史累计值。flow 不存储曲线；slope 额外空间为折点数。内部 run 不是用户接口。新曲线接口仅本地验证，P3381 驱动只输出终点，不能据此验证全部中间折点。

```
struct MinCostFlow
{
    using ll = long long;
    using i128 = __int128_t;

    struct Edge
    {
        int from, to;
        ll cap, cost, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;

    MinCostFlow(int n) : n(n), g(n + 1) {}

    int add(int u, int v, ll cap, ll cost)
    {
        assert(cap >= 0 && cost != LLONG_MIN);
        int id = (int)e.size();
        e.push_back({u, v, cap, cost, cap});
        e.push_back({v, u, 0, -cost, 0});
        g[u].push_back(id);
        g[v].push_back(id + 1);
        return id;
    }

    ll used(int id) const { return e[id].initial - e[id].cap; }
```

```
    // Negative costs allowed; initial residual graph must have no negative cycle.
    pair<ll, i128> run(int s, int t, ll limit, vector<pair<ll, i128>> *curve)
    {
        assert(s != t && limit >= 0);
        const i128 inf = i128(1) << 120;
        vector<i128> h(n + 1, 0), d(n + 1);
        vector<int> pre(n + 1);
        // Super-source Bellman-Ford: feasible potentials for every vertex.
        for (int it = 0; it < n; it++)
        {
            bool changed = false;
            for (auto &a : e)
                if (a.cap && h[a.to] > h[a.from] + a.cost)
                {
                    h[a.to] = h[a.from] + a.cost;
                    changed = true;
                }
            if (!changed) break;
            if (it == n - 1) throw invalid_argument("negative cycle");
        }
        ll f = 0;
        i128 cost = 0;
        optional<i128> last;
        while (f < limit)
        {
            fill(d.begin(), d.end(), inf);
            d[s] = 0;
            priority_queue<pair<i128, int>,
                           vector<pair<i128, int>>,
                           greater<pair<i128, int>>>
                q;
            q.push({0, s});
            while (!q.empty())
            {
                auto [du, u] = q.top();
                q.pop();
                if (du != d[u]) continue;
                for (int id : g[u])
                {
                    auto &a = e[id];
                    i128 nd = du + a.cost + h[u] - h[a.to];
                    if (a.cap && nd < d[a.to])
                    {
                        d[a.to] = nd;
                        pre[a.to] = id;
                        q.push({nd, a.to});
                    }
                }
            }
```

```
            if (d[t] == inf) break;
            for (int u = 1; u <= n; u++)
                if (d[u] != inf) h[u] += d[u];
            ll take = limit - f;
            for (int u = t; u != s; u = e[pre[u]].from) take = min(take, e[pre[u]].cap);
            for (int u = t; u != s; u = e[pre[u]].from)
            {
                int id = pre[u];
                e[id].cap -= take;
                e[id ^ 1].cap += take;
                cost += i128(take) * e[id].cost;
            }
            f += take;
            if (curve)
            {
                i128 unit = h[t] - h[s];
                if (last && *last == unit) curve->pop_back();
                curve->push_back({f, cost});
                last = unit;
            }
        }
        return {f, cost};
    }

    pair<ll, i128> flow(int s, int t, ll limit = LLONG_MAX)
    {
        return run(s, t, limit, nullptr);
    }

    vector<pair<ll, i128>> slope(int s, int t, ll limit = LLONG_MAX)
    {
        vector<pair<ll, i128>> curve{{0, 0}};
        run(s, t, limit, &curve);
        return curve;
    }
};
```

模板题使用：Luogu P3381

题意：给有向容量费用网络，求最大流及对应最小费用。点号 1..n，add 参数依次为起点、终点、容量、单位费用。题目保证费用答案不超过 2^31-1，因此这里可将返回的 int128 转为 long long；一般题目不能照搬转换。

所需模板：MinCostFlow（第 250 页，11.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3381

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    MinCostFlow graph(n);
    while (m--)
    {
        int u, v;
        long long capacity, cost;
        cin >> u >> v >> capacity >> cost;
        graph.add(u, v, capacity, cost);
    }
    auto [flow, cost] = graph.flow(s, t);
    cout << flow << ' ' << (long long)cost << '\n';
}
```

11.3.2 SPFA 最小费用最大流

SpfaFlow(n)，点号 1..n；add(u,v,cap,cost) 返回偶数原边编号，used(id) 查询原边流量。flow(s,t,limit) 返回本次新增流量及 int128 费用；默认 limit=LLONG_MAX，s≠t。允许重边、自环、零容量和负费用，调用前残量网络不得有负环；先用全点入队 SPFA 检查全图，含不可达负环也抛 invalid_argument，检查阶段不修改流量。重复调用须保持同源汇，返回值不累计历史。容量非负、费用不能是 LLONG_MIN，总流 ≤LLONG_MAX，$\sum(\text{cap}\times|\text{cost}|)<2^{120}$。每次增广用 SPFA 最短路，复杂度 O((A+1)VE)，A 为实际增广次数，不能假设等于点数；空间 O(V+E)。通常优先势能 Dijkstra，本版作为更直接的 SPFA 备选。e/g 可读，不能任意修改残量；shortest 为内部方法，0 仅是全点初始化标记，不是合法顶点。

```
struct SpfaFlow
{
    using ll = long long;
    using I = __int128_t;

    struct Edge
    {
        int from, to;
        ll cap, cost, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;

    SpfaFlow(int n) : n(n), g(n + 1) {}

    int add(int u, int v, ll cap, ll cost)
    {
        assert(cap >= 0 && cost != LLONG_MIN);
        int id = (int)e.size();
        e.push_back({u, v, cap, cost, cap});
        e.push_back({v, u, 0, -cost, 0});
        g[u].push_back(id);
        g[v].push_back(id + 1);
        return id;
    }

    ll used(int id) const { return e[id].initial - e[id].cap; }
```

接上页同一结构体：

```
    // Internal: s=0 seeds every vertex to detect any residual negative cycle.
    void shortest(int s, vector<I> &d, vector<int> &pre)
    {
        const I inf = I(1) << 120;
        d.assign(n + 1, inf);
        pre.assign(n + 1, -1);
        vector<int> len(n + 1), in(n + 1);
        queue<int> q;
        for (int u = 1; u <= n; u++)
            if (!s || u == s)
            {
                d[u] = 0;
                in[u] = 1;
                q.push(u);
            }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            in[u] = 0;
            for (int id : g[u])
            {
                auto &a = e[id];
                if (!a.cap || d[a.to] <= d[u] + a.cost) continue;
                d[a.to] = d[u] + a.cost;
                pre[a.to] = id;
                len[a.to] = len[u] + 1;
                if (len[a.to] >= n) throw invalid_argument("negative cycle");
                if (!in[a.to])
                {
                    in[a.to] = 1;
                    q.push(a.to);
                }
            }
        }
    }
```

接上页同一结构体：

```
    // Returns additional flow/cost; reuse only with the same source/sink.
    pair<ll, I> flow(int s, int t, ll limit = LLONG_MAX)
    {
        assert(s != t && limit >= 0);
        vector<I> d;
        vector<int> pre;
        shortest(0, d, pre);
        ll f = 0;
        I cost = 0;
        while (f < limit)
        {
            shortest(s, d, pre);
            if (pre[t] < 0) break;
            ll take = limit - f;
            for (int u = t; u != s; u = e[pre[u]].from) take = min(take, e[pre[u]].cap);
            for (int u = t; u != s; u = e[pre[u]].from)
            {
                int id = pre[u];
                e[id].cap -= take;
                e[id ^ 1].cap += take;
            }
            f += take;
            cost += I(take) * d[t];
        }
        return {f, cost};
    }
};
```

模板题使用：Luogu P3381

题意：求最小费用最大流。点号 1..n，输入容量与费用均为 0..1000，最多 5000 点、50000 边；本题保证最大流及最小费用 $\le 2^{31}-1$，可把返回费用转 long long 输出。这里实际使用 SPFA 增广，通用接口允许负边但禁止负环；有负环的费用流须用另一套算法。

所需模板：SpfaFlow（第 254 页，11.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3381

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    SpfaFlow g(n);
    while (m--)
    {
        int u, v;
        long long cap, cost;
        cin >> u >> v >> cap >> cost;
        g.add(u, v, cap, cost);
    }
    auto [flow, cost] = g.flow(s, t);
    cout << flow << ' ' << (long long)cost << '\n';
}
```

11.3.3 允许负环的最小费用最大流

代码依赖：MinCostFlow（第 250 页，11.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

NegativeCostFlow(n)，点号 1..n；add(u,v,cap,cost) 返回从 0 开始的原边编号。容量非负，所有原边容量总和 ≤LLONG_MAX；费用为 long long 且不等于 LLONG_MIN。允许重边、自环及任意位置的负环。solve(s, t, limit) 只调用一次，limit 默认 LLONG_MAX，s≠t、limit≥0，返回 (不超过 limit 的最大流量, 该流量下的最小费用)，费用为 int128。limit=0 可求最小费用循环流；不足 limit 时返回实际最大流。成功后 used(id) 返回原边流量，自动还原负边退流。先满流负费用边并建立反向正费用退流边，再用超级源汇修复平衡；初始残量费用非负，修复后无负环。删除辅助边后继续最短增广，包含不经过源汇的负环收益。依赖 MinCostFlow，使用两次势能预处理；若总增广次数为 K，时间 O(VE + K(E log(E+1) + V))，空间 O(V+E)，整数容量下为伪多项式，不是费用缩放算法。节点和残量边数须可由 int 表示。

```
struct NegativeCostFlow
{
    using ll = long long;
    using I = __int128_t;
    int n;
    MinCostFlow g;
    vector<ll> balance, cap;
    vector<int> ids;
    vector<bool> reversed;
    ll total = 0;
    I cost = 0;
    bool solved = false;

    NegativeCostFlow(int n) : n(n), g(n + 2), balance(n + 1) {}

    int add(int u, int v, ll c, ll w)
    {
        assert(!solved && 1 <= u && u <= n && 1 <= v && v <= n);
        assert(0 <= c && c <= LLONG_MAX - total && w != LLONG_MIN);
        total += c;
        int id = ids.size();
        cap.push_back(c);
        reversed.push_back(w < 0);
        if (w < 0)
        {
            balance[u] -= c;
            balance[v] += c;
            cost += I(c) * w;
            ids.push_back(g.add(v, u, c, -w));
        }
        else
            ids.push_back(g.add(u, v, c, w));
        return id;
    }
```

需求修复、增广与原边流量还原（接上页同一结构体）：

```
    pair<ll, I> solve(int s, int t, ll limit = LLONG_MAX)
    {
        assert(!solved && 1 <= s && s <= n && 1 <= t && t <= n && s != t);
        assert(limit >= 0);
        solved = true;
        int first = g.e.size(), back = g.add(t, s, min(total, limit), 0);
        ll need = 0;
        for (int u = 1; u <= n; u++)
            if (balance[u] > 0)
            {
                g.add(n + 1, u, balance[u], 0);
                need += balance[u];
            }
            else if (balance[u] < 0)
                g.add(u, n + 2, -balance[u], 0);
        auto [sent, correction] = g.flow(n + 1, n + 2, need);
        assert(sent == need);
        ll base = g.used(back);
        cost += correction;
        for (int id = first; id < (int)g.e.size(); id++) g.e[id].cap = 0;
        auto [more, extra] = g.flow(s, t, limit - base);
        cost += extra;
        return {base + more, cost};
    }

    ll used(int id) const
    {
        assert(solved && 0 <= id && id < (int)ids.size());
        ll f = g.used(ids[id]);
        return reversed[id] ? cap[id] - f : f;
    }
};
```

模板题使用：Luogu P7173

题意：求允许负费用环的最小费用最大流，独立于源汇的负环也计入费用。无额外流量上限时调用 solve(s,t)。题目容量和费用界保证最终费用可转为 long long；一般模板返回 int128，不能任意照搬此转换。

所需模板：MinCostFlow（第 250 页，11.3.1 节）、NegativeCostFlow（第 258 页，11.3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P7173

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    NegativeCostFlow g(n);
    while (m--)
    {
        int u, v;
        long long cap, cost;
        cin >> u >> v >> cap >> cost;
        g.add(u, v, cap, cost);
    }
    auto [flow, cost] = g.solve(s, t);
    cout << flow << ' ' << (long long)cost << '\n';
}
```

11.3.4 zkw 最小费用最大流

ZkwFlow(n)，点号 1..n，s≠t。add(u,v,cap,cost) 返回偶数原边编号，used(id) 查询原边流量；flow(s,t,limit) 返回本次新增流量及 int128 费用，默认 limit=LLONG_MAX，重复调用保持同源汇。允许重边、自环、零容量和负费用，但调用前残量图不得含负环；全图 Bellman-Ford 建立可行势并检查不可达负环，失败抛 invalid_argument 且不改流量，limit=0 仍检查。容量非负、cost≠LLONG_MIN，总流 ≤LLONG_MAX，$\sum(\text{cap}\times|\text{cost}|)<2^{120}$。递归沿零约化费用边增广，失败后按可达集最小跨边松弛量调势。实际增广 A 次，O((A+1)V(V+E))，空间 O(V+E)、递归深度 O(V)，不保证快于势能 Dijkstra 或 SPFA。init/dfs/relabel 为内部方法；e/g 可读，任意改动须维持成对残量与全局最小费用可行流。无负环前提下加边后续流也须重新满足此条件。返回顺序为流量、费用，与 kuangbin 原稿相反。

```
struct ZkwFlow
{
    using ll = long long;
    using I = __int128_t;

    struct Edge
    {
        int from, to;
        ll cap, cost, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<I> h;
    vector<int> vis;

    ZkwFlow(int n) : n(n), g(n + 1), h(n + 1), vis(n + 1)
    {
    }

    int add(int u, int v, ll cap, ll cost)
    {
        assert(cap >= 0 && cost != LLONG_MIN);
        int id = (int)e.size();
        e.push_back({u, v, cap, cost, cap});
        e.push_back({v, u, 0, -cost, 0});
        g[u].push_back(id);
        g[v].push_back(id ^ 1);
        return id;
    }

    ll used(int id) const
    {
        return e[id].initial - e[id].cap;
    }
```

接上页同一结构体：

```
    // Internal: a feasible potential on every residual component.
    void init()
    {
        fill(h.begin(), h.end(), 0);
        for (int it = 0; it < n; it++)
        {
            bool changed = false;
            for (auto a : e)
                if (a.cap && h[a.to] > h[a.from] + a.cost)
                {
                    h[a.to] = h[a.from] + a.cost;
                    changed = true;
                }
            if (!changed) return;
            if (it == n - 1) throw invalid_argument("negative cycle");
        }
    }

    ll dfs(int u, int t, ll f)
    {
        if (u == t) return f;
        vis[u] = 1;
        for (int id : g[u])
        {
            int v = e[id].to;
            if (!e[id].cap || vis[v]) continue;
            if (I(e[id].cost) + h[u] != h[v]) continue;
            ll take = dfs(v, t, min(f, e[id].cap));
            if (!take) continue;
            e[id].cap -= take;
            e[id ^ 1].cap += take;
            return take;
        }
        return 0;
    }
```

接上页同一结构体：

```
    bool relabel()
    {
        const I inf = I(1) << 120;
        I d = inf;
        for (auto a : e)
            if (a.cap && vis[a.from] && !vis[a.to])
                d = min(d, I(a.cost) + h[a.from] - h[a.to]);
        if (d == inf) return false;
        for (int u = 1; u <= n; u++)
            if (vis[u]) h[u] -= d;
        return true;
    }

    // Returns additional flow/cost; reuse only with the same source/sink.
    pair<ll, I> flow(int s, int t, ll limit = LLONG_MAX)
    {
        assert(s != t && limit >= 0);
        init();
        ll f = 0;
        I cost = 0;
        while (f < limit)
        {
            fill(vis.begin(), vis.end(), 0);
            ll take = dfs(s, t, limit - f);
            if (take)
            {
                f += take;
                cost += I(take) * (h[t] - h[s]);
            }
            else if (!relabel()) break;
        }
        return {f, cost};
    }
};
```

模板题使用：Luogu P3381

题意：有向网络，输入 n,m,s,t 和每条边的起终点、容量、单位费用，输出最大流及其最小费用。点号 1..n，s≠t；n≤5000、m≤50000、容量/费用均 0..1000 且无自环，题面保证流量和费用 ≤2^31−1，因此此驱动可将 int128 费用转换输出。负边、续流与 used 由独立本地测试覆盖，不把它们算作本题验证范围。

所需模板：ZkwFlow（第 261 页，11.3.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3381

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    ZkwFlow g(n);
    while (m--)
    {
        int u, v;
        long long cap, cost;
        cin >> u >> v >> cap >> cost;
        g.add(u, v, cap, cost);
    }
    auto [flow, cost] = g.flow(s, t);
    cout << flow << ' ' << (long long)cost << '\n';
}
```

11.4 上下界网络流

11.4.1 上下界可行循环流

代码依赖：Dinic（第 227 页，11.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

BoundedCirculation(n)，点号 1..n。add(u,v,lo,hi) 返回原边编号，0≤lo≤hi≤LLONG_MAX，所有下界总和须 ≤LLONG_MAX。solve() 仅调用一次，返回可行性；成功后 used(id) 读取边流量。每点要求流量守恒，属于循环流；带源汇最大流用 BoundedMaxFlow。hi-lo 是残量容量，下界引起 balance；各边上界不必求和，只要求下界总和保证 balance 和超级源总需求不溢出。依赖 Dinic。

```
struct BoundedCirculation
{
    using ll = long long;
    int n;
    Dinic g;
    vector<ll> balance, lower;
    vector<int> ids;
    bool solved = false, feasible = false;

    BoundedCirculation(int n) : n(n), g(n + 2), balance(n + 1) {}

    int add(int u, int v, ll lo, ll hi)
    {
        assert(!solved && 0 <= lo && lo <= hi);
        int id = ids.size();
        ids.push_back(g.add(u, v, hi - lo));
        lower.push_back(lo);
        balance[u] -= lo;
        balance[v] += lo;
        return id;
    }

    bool solve()
    {
        assert(!solved);
        solved = true;
        ll need = 0;
        for (int u = 1; u <= n; u++)
            if (balance[u] > 0)
            {
                g.add(n + 1, u, balance[u]);
                need += balance[u];
            }
            else if (balance[u] < 0)
                g.add(u, n + 2, -balance[u]);
        return feasible = (g.flow(n + 1, n + 2) == need);
    }

    ll used(int id) const
    {
        assert(feasible);
        return lower[id] + g.used(ids[id]);
    }
};
```

模板题使用：LibreOJ 115

题意：无源汇有上下界可行流：每个点流入等于流出，按输入顺序输出每条边的合法流量，无解输出 NO。点号 1..n，n 不超过 200、m 不超过 10200，0≤lo≤hi<3000。add 返回 0 起原始边编号，used(i) 不是 Dinic 残量边号；solve 只能调用一次，且只有可行时才可读取 used。自环、重边逐条保留，不添加源汇回边。

所需模板：Dinic（第 227 页，11.1.2 节）、BoundedCirculation（第 265 页，11.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://loj.ac/p/115

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BoundedCirculation graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long lo, hi;
        cin >> u >> v >> lo >> hi;
        graph.add(u, v, lo, hi);
    }
    if (!graph.solve())
    {
        cout << "NO\n";
        return 0;
    }
    cout << "YES\n";
    for (int i = 0; i < m; i++) cout << graph.used(i) << '\n';
}
```

11.4.2 有源汇上下界最值流与边流量

代码依赖：BoundedCirculation（第 265 页，11.4.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

BoundedMaxFlow(n)，点号 1..n；add(u,v,lo,hi) 返回从 0 开始的原边编号，0≤lo≤hi，所有原边上界总和≤LLONG_MAX。solve(s,t) 求最大净流，minimum(s,t) 求最小净流；同一对象只能任选其一调用一次，s≠t。minimum 交换源汇求最大值后取反，原边方向及 used 不变；返回 optional<long long>，nullopt 表示无可行流。目标为源点流出减流入的净流量，两种查询都允许负答案，不能把 -1 直接当无解；minimum 不额外强制净流量非负。只在成功后读取 used(id) 的原边方案。先加 t→s 与 s→t 两条辅助边兼容负净流，再求可行循环流；保存净值后清空全部辅助残量边，最后在原图从 s 到 t 增广。最优值与增加量均受原边上界总和控制。允许自环、重边；构建和清理 O(n+m)，主体为两次 Dinic，额外空间 O(n+m)。依赖 BoundedCirculation 与 Dinic，二者内部字段只读，不应在求解后再加边或改流。

```
struct BoundedMaxFlow
{
    using ll = long long;
    BoundedCirculation b;
    ll total = 0;
    int m = 0;

    BoundedMaxFlow(int n) : b(n) {}

    int add(int u, int v, ll lo, ll hi)
    {
        assert(0 <= lo && lo <= hi && hi <= LLONG_MAX - total);
        total += hi;
        b.add(u, v, lo, hi);
        return m++;
    }

    optional<ll> solve(int s, int t)
    {
        assert(!b.solved && 1 <= s && s <= b.n && 1 <= t && t <= b.n && s != t);
        int first = b.g.e.size();
        int back = b.add(t, s, 0, total);
        int forward = b.add(s, t, 0, total);
        if (!b.solve()) return nullopt;
        __int128 ans = (__int128)b.used(back) - b.used(forward);
        for (int id = first; id < (int)b.g.e.size(); id += 2) b.g.change_edge(id, 0, 0);
        ans += b.g.flow(s, t);
        assert(-(__int128)total <= ans && ans <= total);
        return (ll)ans;
    }
```

最小净流与原边流量（接上页同一结构体）：

```
    optional<ll> minimum(int s, int t)
    {
        auto ans = solve(t, s);
        if (ans) *ans = -*ans;
        return ans;
    }

    ll used(int id) const
    {
        assert(0 <= id && id < m);
        return b.used(id);
    }
};
```

应用题补充：Luogu P5192 Shoot the Bullet

题意：每天至多拍 D 张，每次独立拍摄有 [L,R] 限制，每位目标至少拍 G 张，求最多拍摄数。源连天、天连目标、目标连汇；重复目标记录分别加边。目标原编号加 1 后再偏移 n。题目总拍摄数至多 365×30000，用作目标到汇的容量上界。读至 EOF，每组输出后空一行。本题净流非负，因此无解可输出 -1；一般接口允许负净流。作为附件建图应用单列。

所需模板：Dinic（第 227 页，11.1.2 节）、BoundedCirculation（第 265 页，11.4.1 节）、BoundedMaxFlow（第 267 页，11.4.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5192

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m)
    {
        int s = n + m + 1, t = s + 1;
        BoundedMaxFlow f(t);
        for (int j = 1; j <= m; j++)
        {
            long long g;
            cin >> g;
            f.add(n + j, t, g, 365LL * 30000);
        }
        for (int i = 1; i <= n; i++)
        {
            int c, d;
            cin >> c >> d;
            f.add(s, i, 0, d);
            while (c--)
            {
                int j, lo, hi;
                cin >> j >> lo >> hi;
                f.add(i, n + j + 1, lo, hi);
            }
        }
        auto ans = f.solve(s, t);
        cout << (ans ? *ans : -1) << "\n\n";
    }
}
```

应用题补充：有源汇上下界最小净流（本地接口示例）

题意：输入带上下界的有向网络，求源点流出减流入的最小净流量，并输出每条原边流量。无解输出 NO；有解输出 YES、最小值及方案。净值可以为负，因此不能用 -1 作为无解哨兵；本地接口未附加“净流非负”约束。

所需模板：Dinic（第 227 页，11.1.2 节）、BoundedCirculation（第 265 页，11.4.1 节）、BoundedMaxFlow（第 267 页，11.4.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://github.com/gsh20040816/CPCtemplate/issues/11

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    BoundedMaxFlow g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long lo, hi;
        cin >> u >> v >> lo >> hi;
        g.add(u, v, lo, hi);
    }
    auto ans = g.minimum(s, t);
    if (!ans)
        cout << "NO\n";
    else
    {
        cout << "YES " << *ans << '\n';
        for (int i = 0; i < m; i++) cout << g.used(i) << '\n';
    }
}
```

11.5 Stoer–Wagner 算法

11.5.1 无向全局最小割

solve(w)，n≥2；w 为 n×n 对称非负权矩阵，重边先累加，主对角置零。返回 {割权, 一侧顶点列表}，编号 0..n-1；两侧非空，不要求原图连通，不连通时割权为 0。并列最优割返回其中一种，不承诺点数或字典序。矩阵按值传入，原图不变。$O(n^3)$ 时间、$O(n^2)$ 空间；无向边总权（每条只计一次）严格小于 2^120，内部 int128。负权不适用。已逐条核验返回点集在原图上的跨割费用，暂无专门 OJ AC。

```
struct StoerWagner
{
    using I = __int128_t;

    // Symmetric nonnegative adjacency matrix; returns weight and one cut side.
    static pair<I, vector<int>> solve(vector<vector<I>> w)
    {
        int n = w.size();
        assert(n >= 2);
        vector<int> active(n);
        iota(active.begin(), active.end(), 0);
        vector<vector<int>> group(n);
        for (int i = 0; i < n; i++) group[i] = {i};
        I best = I(1) << 120;
        vector<int> side;
        while (active.size() > 1)
        {
            vector<I> dis(n);
            vector<bool> used(n);
            int prev = -1;
            for (int step = 0; step < (int)active.size(); step++)
            {
                int sel = -1;
                for (int v : active)
                    if (!used[v] && (sel == -1 || dis[v] > dis[sel])) sel = v;
```

本轮割、合并与继续扩张（接上页同一函数）：

```
                if (step + 1 == (int)active.size())
                {
                    if (dis[sel] < best)
                    {
                        best = dis[sel];
                        side = group[sel];
                    }
                    for (int v : active)
                        if (v != prev && v != sel)
                            w[prev][v] = w[v][prev] = w[prev][v] + w[sel][v];
                    group[prev].insert(
                        group[prev].end(), group[sel].begin(), group[sel].end());
                    active.erase(find(active.begin(), active.end(), sel));
                    break;
                }
                used[sel] = true;
                prev = sel;
                for (int v : active)
                    if (!used[v]) dis[v] += w[sel][v];
            }
        }
        return {best, side};
    }
};
```

模板题使用：Luogu P5632

题意：无向连通正权图的全局最小割，n 至 600、边权总和至 10^9；矩阵按无向边双向累加、自环忽略。核心返回费用与 0-based 割侧 side，本题只输出费用。核心要求 n≥2；单点没有非平凡割，驱动额外约定输出 0。

所需模板：StoerWagner（第 271 页，11.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5632

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using I = StoerWagner::I;
    vector<vector<I>> w(n, vector<I>(n));
    while (m--)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        u--;
        v--;
        if (u == v) continue;
        w[u][v] += c;
        w[v][u] += c;
    }
    // A singleton has no nontrivial cut; use zero by convention.
    if (n == 1)
    {
        cout << 0 << '\n';
        return 0;
    }
    auto [ans, side] = StoerWagner::solve(w);
    // The statement bounds the sum of edge weights by 1e9.
    cout << (long long)ans << '\n';
}
```

第十二章　图的匹配

12.1　二分图最大匹配

12.1.1　Hopcroft–Karp 最大匹配与最小点覆盖

左右两侧分别编号 1..n、1..m，可有空侧及重边；add(u,v) 加边，solve() 返回最大匹配总数，l[u] 与 r[v] 为匹配端点，0 表示未匹配。保留已有合法匹配，可加边后再次 solve；不支持删边或手动更改匹配数组。BFS 用 dep[0] 记录最短终点层，递归 DFS 只沿最短层增广，并使用当前弧。V=n+m，单次 solve 最坏 $O((V+E)\sqrt{V})$，空间 O(V+E)。cover() 在 solve 后、下一次改图前调用，返回 (左侧点列表, 右侧点列表)，为最小点覆盖；其补集是最大独立集。覆盖与匹配方案均可直接输出。

```
struct BipartiteMatching
{
    int n, m;
    vector<vector<int>> g;
    vector<int> l, r, dep, cur;

    BipartiteMatching(int n, int m)
        : n(n), m(m), g(n + 1), l(n + 1), r(m + 1), dep(n + 1), cur(n + 1)
    {
    }

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= m);
        g[u].push_back(v);
    }

    bool bfs()
    {
        queue<int> q;
        fill(dep.begin(), dep.end(), n + 1);
        fill(cur.begin(), cur.end(), 0);
        for (int u = 1; u <= n; u++)
        {
            if (!l[u])
            {
                dep[u] = 0;
                q.push(u);
            }
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            if (dep[u] >= dep[0]) continue;
            for (int v : g[u])
            {
                int next = r[v];
                if (dep[next] != n + 1) continue;
                dep[next] = dep[u] + 1;
```

```
                if (next) q.push(next);
            }
        }
        return dep[0] != n + 1;
    }

    bool dfs(int u)
    {
        if (!u) return true;
        for (int &i = cur[u]; i < (int)g[u].size(); i++)
        {
            int v = g[u][i];
            if (dep[r[v]] == dep[u] + 1 && dfs(r[v]))
            {
                l[u] = v;
                r[v] = u;
                return true;
            }
        }
        dep[u] = n + 1;
        return false;
    }

    // Retains an existing matching; edges may be added between calls.
    int solve()
    {
        int answer = 0;
        for (int u = 1; u <= n; u++) answer += (l[u] != 0);
        while (bfs())
        {
            for (int u = 1; u <= n; u++)
            {
                if (!l[u] && dfs(u)) answer++;
            }
        }
        return answer;
    }

    // Call after solve(), before adding more edges. Returns left/right vertex IDs.
    pair<vector<int>, vector<int>> cover() const
    {
        vector<int> left, right;
        for (int u = 1; u <= n; u++)
        {
            if (dep[u] == n + 1) left.push_back(u);
        }
        for (int v = 1; v <= m; v++)
        {
            if (r[v] && dep[r[v]] != n + 1) right.push_back(v);
        }
        return {left, right};
    }
};
```

模板题使用：Luogu P3386

题意：给定二分图，求最大匹配数。左右两侧分别编号 1..n、1..m，add(u,v) 的 u 属于左侧，v 属于右侧；solve 返回匹配数。此题不输出匹配或最小点覆盖方案。

所需模板：BipartiteMatching（第 274 页，12.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3386

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m, e;
    std::cin >> n >> m >> e;
    BipartiteMatching graph(n, m);
    while (e--)
    {
        int u, v;
        std::cin >> u >> v;
        graph.add(u, v);
    }
    std::cout << graph.solve() << '\n';
}
```

应用题补充：CSES 1709 / Coin Grid

题意：n≤100 的方阵中 o 为硬币，每次清空一行或一列，输出最少操作数及方案。行列作为二分图两侧，硬币为边；solve 后 cover 给出最小点覆盖，左点输出 1 行号，右点输出 2 列号。

所需模板：BipartiteMatching（第 274 页，12.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1709/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    BipartiteMatching g(n, n);
    for (int i = 1; i <= n; i++)
    {
        string row;
        cin >> row;
        for (int j = 1; j <= n; j++)
        {
            if (row[j - 1] == 'o') g.add(i, j);
        }
    }
    cout << g.solve() << '\n';
    auto [a, b] = g.cover();
    for (int u : a) cout << 1 << ' ' << u << '\n';
    for (int v : b) cout << 2 << ' ' << v << '\n';
}
```

12.1.2 二分图最大匹配的可行边与必选边

代码依赖：BipartiteMatching（第 274 页，12.1.1 节）、TarjanSCC（第 150 页，6.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

matching_edges(n,m,edges) 接收左右两侧点号各为 1..n、1..m 的边表，返回 (最大匹配大小, 逐边分类)。每项为 {possible,forced}：是否属于至少一个/每一个最大匹配；分类保持输入顺序，平行边作为不同边处理。允许空侧、空图和非完备匹配。先用 BipartiteMatching 求解，再建含源汇的残量图跑 TarjanSCC；已匹配或两端同 SCC 为可行，已匹配且两端不同 SCC 为必选。不能在非完备匹配时省略源汇，也不能把可行条件写成已匹配且同 SCC。时间 $O(E\sqrt{(n+m)}+n+m)$，空间 O(n+m+E)，节点与残量边数须可用 int 表示。HK 与 SCC 都保留递归，栈深可为 O(n+m)。

```
pair<int, vector<array<bool, 2>>>
matching_edges(int n, int m, const vector<pair<int, int>> &edges)
{
    BipartiteMatching b(n, m);
    for (auto [u, v] : edges)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= m);
        b.add(u, v);
    }
    int size = b.solve(), s = n + m + 1, t = s + 1;
    TarjanSCC d(t);
    for (int u = 1; u <= n; u++)
        if (b.l[u])
            d.add(u, s);
        else
            d.add(s, u);
    for (int v = 1; v <= m; v++)
        if (b.r[v])
            d.add(t, n + v);
        else
            d.add(n + v, t);
    vector<bool> chosen(n + 1), used(edges.size());
    for (int i = 0; i < (int)edges.size(); i++)
    {
        auto [u, v] = edges[i];
        used[i] = b.l[u] == v && !chosen[u];
        if (used[i])
        {
            chosen[u] = true;
            d.add(n + v, u);
        }
        else
            d.add(u, n + v);
    }
    d.run();
    vector<array<bool, 2>> ans(edges.size());
    for (int i = 0; i < (int)edges.size(); i++)
    {
        auto [u, v] = edges[i];
        bool same = d.bel[u] == d.bel[n + v];
        ans[i] = {used[i] || same, used[i] && !same};
    }
    return {size, ans};
}
```

应用题补充：二分图最优匹配边分类（本地接口示例）

题意：输入左右点数、边数及 1-based 边表，输出最大匹配大小，再按原边顺序输出可行/必选标记。非完备匹配和平行边都允许；本例演示接口，不是 OJ 提交记录。

所需模板：BipartiteMatching（第 274 页，12.1.1 节）、TarjanSCC（第 150 页，6.1.1 节）、matching_edges（第 278 页，12.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://oi-wiki.org/graph/graph-matching/bigraph-match/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    vector<pair<int, int>> edges(k);
    for (auto &[u, v] : edges) cin >> u >> v;
    auto [size, labels] = matching_edges(n, m, edges);
    cout << size << '\n';
    for (auto [possible, forced] : labels) cout << possible << ' ' << forced << '\n';
}
```

12.1.3 二分图最大独立集方案

代码依赖：BipartiteMatching（第 274 页，12.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

independent_set(g) 接收已建好的 BipartiteMatching，内部调用 solve 再取最小点覆盖的补集，返回左右两侧的原点号列表，分别从 1 计、升序排列。允许空侧、孤立点、重边与断开图；返回一种最大基数独立集，不保证字典序或最大权。只适用于已知左右划分的二分图，不能直接输入一般图或同侧边。

调用会更新 g 的匹配与搜索状态，保留合法旧匹配并继续增广；调用后可立即读 g.cover()。允许继续 add 后再次调用，不可删除边或手改匹配。n,m 非负且小于 INT_MAX，内存足够。最大独立集大小等于 n+m 减最大匹配数；只求数量可直接使用该式，求和时先转 long long。

依赖 BipartiteMatching；总时间 $O((V+E)\sqrt{(V+1)})$，V=n+m，额外时间与空间 O(V)。HK 增广保持递归，深度最坏 O(V)。骑士攻击使格子黑白染色反转，故可应用此函数；障碍格不要建成孤立点。

```
// Maximum independent set; left/right IDs are separate and 1-based.
pair<vector<int>, vector<int>> independent_set(BipartiteMatching &g)
{
    g.solve();
    auto [a, b] = g.cover();
    vector<bool> x(g.n + 1, true), y(g.m + 1, true);
    for (int u : a) x[u] = false;
    for (int v : b) y[v] = false;
    vector<int> left, right;
    for (int u = 1; u <= g.n; u++)
    {
        if (x[u]) left.push_back(u);
    }
    for (int v = 1; v <= g.m; v++)
    {
        if (y[v]) right.push_back(v);
    }
    return {left, right};
}
```

应用题补充：Luogu P3355 / 骑士共存问题

题意：n×n 棋盘有障碍，求互不攻击骑士的最大数量。n≤200；可用格按 (x+y) 奇偶分成两侧并单独编号，只从奇色连向八个合法骑士落点。障碍不参与编号，输出两侧独立集大小之和。

所需模板：BipartiteMatching（第 274 页，12.1.1 节）、independent_set（第 279 页，12.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3355

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> id(n + 1, vector<int>(n + 1));
    while (m--)
    {
        int x, y;
        cin >> x >> y;
        id[x][y] = -1;
    }
    int l = 0, r = 0;
    for (int x = 1; x <= n; x++)
        for (int y = 1; y <= n; y++)
            if (id[x][y] != -1)
                id[x][y] = (x + y) % 2 ? ++l : ++r;
    BipartiteMatching g(l, r);
    vector<pair<int, int>> moves = {
        {1, 2}, {1, -2}, {-1, 2}, {-1, -2},
        {2, 1}, {2, -1}, {-2, 1}, {-2, -1}
    };
    for (int x = 1; x <= n; x++)
        for (int y = 1; y <= n; y++)
            if ((x + y) % 2 && id[x][y] != -1)
                for (auto [dx, dy] : moves)
                {
                    int u = x + dx, v = y + dy;
                    if (1 <= u && u <= n && 1 <= v && v <= n && id[u][v] != -1)
                        g.add(id[x][y], id[u][v]);
                }
    auto [a, b] = independent_set(g);
    cout << a.size() + b.size() << '\n';
}
```

接口用法演示：二分图最大独立集原点方案（接口演示）

演示约定：输入左点数 n、右点数 m、边数 e 及左右各自 1-based 边端点，允许空侧、孤立点和重边。首行输出最优大小，后两行分别输出“左侧数量左点号列表”和“右侧数量右点号列表”。两侧同号是不同点。

所需模板：BipartiteMatching（第 274 页，12.1.1 节）、independent_set（第 279 页，12.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/graph-matching/bigraph-match/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, e;
    cin >> n >> m >> e;
    BipartiteMatching g(n, m);
    while (e--)
    {
        int u, v;
        cin >> u >> v;
        g.add(u, v);
    }
    auto [a, b] = independent_set(g);
    cout << a.size() + b.size() << '\n';
    cout << a.size();
    for (int u : a) cout << ' ' << u;
    cout << '\n';
    cout << b.size();
    for (int v : b) cout << ' ' << v;
    cout << '\n';
}
```

12.2 二分图最大权匹配

12.2.1 矩形二分图最小权匹配

solve(cost) 输入 n×m 完整权值矩阵，0≤n≤m；每行匹配不同列，返回最小费用及列号（0-based）。允许负费用，使用 int128 势能；$O(n^2m)$。禁止边不能用未经证明的有限大常数随意替代，当前接口仅表示完整二分图。

```
struct Hungarian
{
    using I = __int128_t;

    // Minimum cost injection from n rows to m columns, n<=m. Zero-based matrix.
    static pair<I, vector<int>> solve(const vector<vector<long long>> &cost)
    {
        int n = cost.size();
        if (!n) return {0, {}};
        int m = cost[0].size();
        assert(n <= m);
        vector<I> u(n + 1), v(m + 1);
        vector<int> p(m + 1), way(m + 1);
        const I inf = I(1) << 120;
        for (int i = 1; i <= n; i++)
        {
            assert((int)cost[i - 1].size() == m);
```

```
            p[0] = i;
            int j0 = 0;
            vector<I> dis(m + 1, inf);
            vector<bool> used(m + 1);
            do
            {
                used[j0] = true;
                int i0 = p[j0], j1 = 0;
                I delta = inf;
                for (int j = 1; j <= m; j++)
                    if (!used[j])
                    {
                        I cur = I(cost[i0 - 1][j - 1]) - u[i0] - v[j];
                        if (cur < dis[j])
                        {
                            dis[j] = cur;
                            way[j] = j0;
                        }
                        if (dis[j] < delta)
                        {
                            delta = dis[j];
                            j1 = j;
                        }
                    }
                for (int j = 0; j <= m; j++)
                    if (used[j])
                    {
                        u[p[j]] += delta;
                        v[j] -= delta;
                    }
                    else
                        dis[j] -= delta;
                j0 = j1;
            } while (p[j0]);
            do
            {
                int j1 = way[j0];
                p[j0] = p[j1];
                j0 = j1;
            } while (j0);
        }
        vector<int> ans(n);
        for (int j = 1; j <= m; j++)
            if (p[j]) ans[p[j] - 1] = j - 1;
        return {-v[0], ans};
    }
};
```

模板题使用：Library Checker assignment

题意：给完整方阵，为每一行分配一个不同的列，使费用和最小。返回列号是 0-based，按行顺序输出。题目 n≤500、单项绝对值 ≤10^9，总费用可由 int128 转为 long long；缺边问题不能直接套本例。

所需模板：Hungarian（第 281 页，12.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/assignment

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<long long>> cost(n, vector<long long>(n));
    for (auto &row : cost)
        for (auto &x : row) cin >> x;
    auto [value, columns] = Hungarian::solve(cost);
    cout << (long long)value << '\n';
    for (int j : columns) cout << j << ' ';
    cout << '\n';
}
```

12.2.2 二分图最大权匹配（缺边与可选匹配）

WeightedMatching(n,m)，两侧编号 1..n、1..m。add(x,y,w) 加边，重边保留最大权；边权支持完整 long long 范围，内部势能与答案为 int128。solve() 要求全部左点匹配不同右点，无解返回 nullopt；n=m 时即最大权完美匹配。solve(true) 允许点不匹配，只最大化总权，不优先最大化边数。两种模式均从空匹配重算；l[x]、r[y] 为方案，0 表示未匹配，无解时全部清零。缺边用独立于合法边权的 -2^120 哨兵，仅用 add 修改；不把缺边当零权边。允许空侧。全匹配 $O(n^2m)$，可选模式增加 n 个虚拟零权右点，为 $O(n^2(m+n))$；空间 O(nm+n+m)。按最短松弛量扩展交错树：每次至少访问一个新右点，最多 n 次后增广；无有限松弛量即无解。

```
struct WeightedMatching
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 120;
    int n, m;
    vector<vector<I>> w;
    vector<int> l, r;

    WeightedMatching(int n, int m)
        : n(n), m(m), w(n + 1, vector<I>(m + 1, -inf)), l(n + 1), r(m + 1)
    {
    }

    void add(int x, int y, long long weight)
    {
        assert(1 <= x && x <= n && 1 <= y && y <= m);
        w[x][y] = max(w[x][y], I(weight));
    }

    optional<I> solve(bool allow_unmatched = false)
    {
        fill(l.begin(), l.end(), 0);
        fill(r.begin(), r.end(), 0);
```

```
        if (!allow_unmatched && n > m) return nullopt;
        int cols = m + (allow_unmatched ? n : 0);
        vector<I> u(n + 1), v(cols + 1);
        vector<int> p(cols + 1), way(cols + 1);
        for (int i = 1; i <= n; i++)
        {
            p[0] = i;
            int j0 = 0;
            vector<I> dis(cols + 1, inf);
            vector<bool> used(cols + 1);
            do
            {
                used[j0] = true;
                int x = p[j0], j1 = 0;
                I delta = inf;
                for (int j = 1; j <= cols; j++)
                {
                    if (used[j]) continue;
                    I weight = j <= m ? w[x][j] : 0;
                    if (weight != -inf)
                    {
                        I cur = -weight - u[x] - v[j];
                        if (cur < dis[j])
                        {
                            dis[j] = cur;
                            way[j] = j0;
                        }
                    }
                    if (dis[j] < delta)
                    {
                        delta = dis[j];
                        j1 = j;
                    }
                }
                if (delta == inf) return nullopt;
                for (int j = 0; j <= cols; j++)
                {
                    if (used[j])
                    {
                        u[p[j]] += delta;
                        v[j] -= delta;
                    }
                    else if (dis[j] != inf)
                        dis[j] -= delta;
                }
                j0 = j1;
            } while (p[j0]);
            do
            {
                int j1 = way[j0];
                p[j0] = p[j1];
                j0 = j1;
            } while (j0);
        }
        I answer = 0;
```

```
        for (int j = 1; j <= m; j++)
        {
            if (!p[j]) continue;
            l[p[j]] = j;
            r[j] = p[j];
            answer += w[p[j]][j];
        }
        return answer;
    }
};
```

模板题使用：Luogu P6577

题意：求二分图最大权完美匹配，两侧均为 n 点，题目保证存在完美匹配。输出 r[j]，即右部 j 对应的左部编号，均为 1-based；题目权值与规模保证总权值可转为 long long。

所需模板：WeightedMatching（第 283 页，12.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6577

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    WeightedMatching graph(n, n);
    while (m--)
    {
        int x, y;
        long long w;
        std::cin >> x >> y >> w;
        graph.add(x, y, w);
    }
    auto answer = graph.solve();
    assert(answer);
    std::cout << (long long)*answer << '\n';
    for (int j = 1; j <= n; j++) std::cout << graph.r[j] << (j == n ? '\n' : ' ');
}
```

12.2.3 按匹配基数求最大权与顶标证书

AssignmentSpectrum(n,m)，两侧 0-based，n,m≥0。add(x,y,w) 接受完整 signed64 权，重边保留最大权，缺边不当零权。solve(limit=INT_MAX) 从空重算，返回 k=min(limit, 最大匹配数)，limit≥0；best[j] 为全图恰好 j 条边的最大权，0≤j≤k，best[0]=0。负增益也继续，best 不必单调。l/r 为最后 k 条的互逆方案，-1 未匹配；要 j 条方案重新 solve(j)。顶标 lx,ly 和 level 仅对最后基数有效：令 α=lx-level、β=ly，则 α,β 非负，每条边 $\alpha+\beta+\text{level}\geq w$，匹配边取等，未匹配点对应 α 或 β 为 0；$\text{sum}(\alpha)+\text{sum}(\beta)+k*\text{level}$ 等于 best[k]。答案与顶标为 int128；加边后须重新 solve，其他成员只读。多源松弛始终从所有未匹配左点出发，故每个基数均全局最优。时间 O((k+1)nm+n+m)、空间 O(nm+n+m)，适合小侧或稠密图。

```
struct AssignmentSpectrum
{
    using I = __int128_t;
    static constexpr I inf = I(1) << 120;
    int n, m;
    vector<vector<I>> w;
    vector<I> lx, ly, best;
    vector<int> l, r;
    I level = 0;

    AssignmentSpectrum(int n, int m)
        : n(n), m(m), w(n, vector<I>(m, -inf))
    {
    }

    void add(int x, int y, long long value)
    {
        assert(0 <= x && x < n && 0 <= y && y < m);
        w[x][y] = max(w[x][y], I(value));
    }
```

重置状态与多源松弛（接上页同一结构体）：

```
    int solve(int limit = INT_MAX)
    {
        assert(limit >= 0);
        level = 0;
        for (const auto &row : w)
            for (I value : row)
                level = max(level, value);
        lx.assign(n, level);
        ly.assign(m, 0);
        l.assign(n, -1);
        r.assign(m, -1);
        best.assign(1, 0);
        int bound = min({n, m, limit});
        for (int k = 0; k < bound; k++)
        {
            vector<bool> sx(n), sy(m);
            vector<I> slack(m, inf);
            vector<int> from(m, -1);
            auto relax = [&](int x)
            {
                sx[x] = true;
                for (int y = 0; y < m; y++)
                {
                    if (sy[y] || w[x][y] == -inf)
                        continue;
                    I value = lx[x] + ly[y] - w[x][y];
                    if (value < slack[y])
                    {
                        slack[y] = value;
                        from[y] = x;
                    }
                }
            };
            for (int x = 0; x < n; x++)
                if (l[x] == -1)
                    relax(x);
```

继续 solve：调整顶标与翻转增广路径（接上页同一结构体）：

```
            int y;
            while (true)
            {
                y = -1;
                for (int j = 0; j < m; j++)
                    if (!sy[j] && (y == -1 || slack[j] < slack[y]))
                        y = j;
                if (y == -1 || slack[y] == inf)
                    return k;
                I delta = slack[y];
                level -= delta;
                for (int x = 0; x < n; x++)
                    if (sx[x])
                        lx[x] -= delta;
                for (int j = 0; j < m; j++)
                {
                    if (sy[j])
                        ly[j] += delta;
                    else if (slack[j] != inf)
                        slack[j] -= delta;
                }
                sy[y] = true;
                if (r[y] == -1)
                    break;
                relax(r[y]);
            }
            I answer = best.back();
            while (y != -1)
            {
                int x = from[y];
                int old = l[x];
                answer += w[x][y];
                if (old != -1)
                    answer -= w[x][old];
                l[x] = y;
                r[y] = x;
                y = old;
            }
            best.push_back(answer);
        }
        return bound;
    }
};
```

接口用法演示：按基数匹配与对偶证书

演示约定：输入 n,m,e,limit 与 e 条 0-based 边 x,y,w，输出实际基数 k、best[0..k]、左侧配对 l（-1 未匹配）、level、lx、ly。本演示限定两侧 ≤1000 且 |w|≤10^9，输出可安全转 long long；核心支持全 signed64 输入及 int128 结果。证书仅对应最后基数；没有正式题目或线上 AC 声明。

所需模板：AssignmentSpectrum（第 286 页，12.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/hh2048/XCPC

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, e, limit;
    cin >> n >> m >> e >> limit;
    AssignmentSpectrum matching(n, m);
    while (e--)
    {
        int x, y;
        long long w;
        cin >> x >> y >> w;
        matching.add(x, y, w);
    }
    int k = matching.solve(limit);
    // 本演示限定 n,m<=1000，边权绝对值<=10^9，可安全转 long long。
    cout << k << '\n';
    for (auto value : matching.best)
        cout << (long long)value << ' ';
    cout << '\n';
    for (int y : matching.l)
        cout << y << ' ';
    cout << '\n';
    cout << (long long)matching.level << '\n';
    for (auto value : matching.lx)
        cout << (long long)value << ' ';
    cout << '\n';
    for (auto value : matching.ly)
        cout << (long long)value << ' ';
    cout << '\n';
    return 0;
}
```

应用题补充：AtCoder ABC247 G / Dream Team

题意：N≤30000，每人有 1..150 的大学 A 和学科 B、1..10^9 的强度 C。选人须使 A、B 各不重复，输出最大人数 k 与每种人数 1..k 的最大总强度。同属性对保留最大权；答案 ≤150×10^9，可转 long long。缺边不能填 0，各基数答案可能下降。比赛应用，已核对官方题面与样例，尚无本项目线上 AC 或排名。

所需模板：AssignmentSpectrum（第 286 页，12.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://atcoder.jp/contests/abc247/tasks/abc247_g

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    AssignmentSpectrum matching(150, 150);
    while (n--)
    {
        int a, b;
        long long c;
        cin >> a >> b >> c;
        matching.add(a - 1, b - 1, c);
    }
    int k = matching.solve();
    cout << k << '\n';
    for (int i = 1; i <= k; i++)
        cout << (long long)matching.best[i] << '\n';
    return 0;
}
```

模板题使用：Luogu P6577 / 二分图最大权完美匹配

题意：左右均 n≤500 点，m≤n^2，无重边，权值范围 [-19980731,19980731]，保证完美匹配。输出最大总权及每个右点对应的左点编号；调用前点号减一，输出 r 加一。答案绝对值 ≤500×19980731，可转 long long。该题只验证最终完美匹配，不能代替中间基数和顶标检查；新驱动尚无线上 AC。

所需模板：AssignmentSpectrum（第 286 页，12.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6577

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    AssignmentSpectrum matching(n, n);
    while (m--)
    {
        int x, y;
        long long w;
        cin >> x >> y >> w;
        matching.add(x - 1, y - 1, w);
    }
    matching.solve();
    cout << (long long)matching.best[n] << '\n';
    for (int x : matching.r)
        cout << x + 1 << ' ';
    cout << '\n';
    return 0;
}
```

12.3 一般图最大匹配

12.3.1 一般图最大基数匹配：带花树

Blossom(n)，点号 0..n-1；add(u,v) 加无向边，忽略自环，允许重边。solve() 从空匹配重新计算，返回匹配边数；mate[u] 为配对点，未匹配为 -1。结果支持直接输出方案。缩花维护交错树，单次增广 $O(n^2+m)$，总计 $O(n^3+nm)$，简单图为 $O(n^3)$；空间 $O(n+m)$。重复调用 solve 或加边后重算均可。不处理边权，不能代替最大权匹配。

```
struct Blossom
{
    int n;
    vector<vector<int>> g;
    vector<int> mate, pre, base, q;
    vector<bool> outer, flower, seen;

    Blossom(int n)
        : n(n), g(n), mate(n, -1), pre(n), base(n), outer(n), flower(n), seen(n)
    {
    }

    void add(int u, int v)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        if (u == v) return;
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int lca(int u, int v)
    {
        fill(seen.begin(), seen.end(), false);
        while (true)
        {
            u = base[u];
            seen[u] = true;
            if (mate[u] == -1) break;
            u = pre[mate[u]];
        }
        while (!seen[base[v]]) v = pre[mate[base[v]]];
        return base[v];
    }

    void mark(int u, int b, int v)
    {
        while (base[u] != b)
        {
            flower[base[u]] = true;
            flower[base[mate[u]]] = true;
            pre[u] = v;
            v = mate[u];
            u = pre[v];
        }
    }

    bool augment(int root)
    {
```

```
        fill(outer.begin(), outer.end(), false);
        fill(pre.begin(), pre.end(), -1);
        iota(base.begin(), base.end(), 0);
        q.assign(1, root);
        outer[root] = true;
        for (int head = 0; head < (int)q.size(); head++)
        {
            int u = q[head];
            for (int v : g[u])
            {
                if (base[u] == base[v] || mate[u] == v) continue;
                if (v == root || (mate[v] != -1 && pre[mate[v]] != -1))
                {
                    int b = lca(u, v);
                    fill(flower.begin(), flower.end(), false);
                    mark(u, b, v);
                    mark(v, b, u);
                    for (int x = 0; x < n; x++)
                    {
                        if (!flower[base[x]]) continue;
                        base[x] = b;
                        if (!outer[x])
                        {
                            outer[x] = true;
                            q.push_back(x);
                        }
                    }
                }
                else if (pre[v] == -1)
                {
                    pre[v] = u;
                    if (mate[v] == -1)
                    {
                        while (v != -1)
                        {
                            int p = pre[v];
                            int next = mate[p];
                            mate[v] = p;
                            mate[p] = v;
                            v = next;
                        }
                        return true;
                    }
                    int x = mate[v];
                    outer[x] = true;
                    q.push_back(x);
                }
            }
        }
        return false;
    }

    // Rebuild the maximum matching; mate[u] is -1 for an unmatched vertex.
    int solve()
```

```
    {
        fill(mate.begin(), mate.end(), -1);
        int answer = 0;
        for (int u = 0; u < n; u++)
        {
            if (mate[u] == -1 && augment(u)) answer++;
        }
        return answer;
    }
};
```

模板题使用：Library Checker general_matching

题意：给简单无向图，输出最大匹配的边数及匹配边。点号 0..n-1；solve 后 mate[u]=-1 表示未匹配，只在 u<mate[u] 时输出，避免同一条边打印两次。

所需模板：Blossom（第 292 页，12.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/general_matching

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Blossom graph(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    cout << graph.solve() << '\n';
    for (int u = 0; u < n; u++)
        if (u < graph.mate[u]) cout << u << ' ' << graph.mate[u] << '\n';
}
```

第十三章　Prüfer 序列

13.1　Prüfer 双射与有标号树计数

本节的树是顶点标号固定为 $1,\dots,n$ 的简单无向树，不把同构的不同标号树合并。先设 $n\ge 2$。反复删除当前**最小标号叶子**，记录它唯一的邻点，直到剩两个点；得到长度 $n-2$、每项属于 $[n]$ 的 Prüfer 序列。$n=2$ 对应唯一空序列；$n=1$ 另有唯一空边树，度数为 0，不代入含 $n-2$ 的公式。

13.1.1　双射为何成立

解码时保留所有尚未删去的标号。读下一项 u，取当前未删除且没有出现在**整个剩余序列**中的最小标号 v，连边 uv，删除 v 并消耗这一项 u；最后连接剩下的两个标号。当前有 t 个点、$t-2$ 个剩余序列项，所以至少两个标号没有出现，可选点总存在。u 自己仍在序列中，故不会选出自环。每个被删点只连接到更晚删去的点，最终得到一棵连通的 $n-1$ 边树。

关键不变式是：在当前剩余树中，标号 i 在剩余序列里的出现次数等于 d_i-1。每个点最后一条关联边在它作为叶子被删除时移除，或在最后两点之间保留到结束；这一条不记录该点。此前每删除一个邻点，就记录一次它。每一步删除一个叶子，会同时使邻点的度数和出现次数减少 1，因此不变式继续成立。反过来看解码得到的剩余树，每个活跃点有一条最终连接边，再加上剩余序列每次出现它时接来的边；其度数同样是剩余出现次数加 1。所以未出现的活跃标号恰好是叶子；两边都取最小者，编码和解码逐步互为逆操作。特别地，原树满足

$$d_i = 1 + \#\{\text{序列中等于}i\text{ 的项}\}.$$

本节给出计数依据，不新增可调用的编码/解码 API。

13.1.2　Cayley、指定度数与叶子数

任意长度 $n-2$ 的序列都对应唯一树，故树总数为 n^{n-2}。给定整数度数 $d_1,\dots,d_n$，若某个 $d_i<1$ 或 $\sum_i d_i\neq 2n-2$，答案为 0；否则各标号出现次数已确定，答案是多项式系数

$$\frac{(n-2)!}{\prod_{i=1}^{n}(d_i-1)!}.$$

例如 $n=4$，度数 $(3,1,1,1)$ 给出唯一指定中心的星形树，度数 $(2,2,1,1)$ 则有 2 棵，而不是把两名度为 2 的顶点视作不可区分。

叶子恰是没有出现的标号。恰有 ℓ 个叶子的树数为

$$\binom{n}{\ell}\,(n-\ell)!\,S(n-2,n-\ell),$$

其中 S 为第二类 Stirling 数：先选叶子，再把序列位置满射到其余标号。$n\ge 3$ 时合法范围为 $2\le\ell\le n-1$；其他叶子数为 0。$n=2,\ell=2$ 时用 $S(0,0)=1$ 得 1。若把叶子定义为度数 1，单点树没有叶子，仍单独处理。这里是整数计数恒等式；取模时不能无条件把阶乘分母换成逆元。素数模的预处理与下标限制见第 B.1.1 节（第 331 页），一般模数条件见数学分册“模运算的适用条件”。有根标号树及其生成函数见数学分册“隐式生成函数与拉格朗日反演”，不重复另造树计数模板。

13.1.3 加权 Cayley 与连接已有连通块

若完整图的边权有特殊乘积形式 $w_{ij} = x_i x_j$，则

$$\sum_T \prod_{ij \in T} w_{ij} = \sum_T \prod_i x_i^{d(i)} = \left(\prod_i x_i\right)\left(\sum_i x_i\right)^{n-2}.$$

这里 $d(i)$ 表示点 i 的度数。第二步利用出现次数恒等式，把每棵树变成公共因子 $\prod_i x_i$ 乘以其序列各项的权；对所有序列求和就是展开右侧的幂。证明没有除以任何 x_i，所以零权、负权以及任意正模数都可按多项式恒等式使用。$n = 2$ 的零次幂是常数 1，即使 $\sum_i x_i = 0$ 也如此。这只适用于上述乘积边权；缺边图或一般边权应使用矩阵树定理，见第 14.4 节（第 305 页）及第 14.2 节（第 299 页）。代数权值和为零也可能来自抵消，不能据此断言没有树。

设一个固定图有 $k \geq 2$ 个连通块，大小为正整数 $s_1, \ldots, s_k$，总顶点数 $N = \sum_i s_i$。允许任选不同连通块的顶点对加边，使全图连通，且恰加 $k-1$ 条边。缩点后新增边必须组成树；一条块间边 ij 有 $s_i s_j$ 种端点选择。因此不同**无序新增边集**的数量是

$$N^{k-2} \prod_{i=1}^{k} s_i.$$

$k = 1$ 时无需加边，答案为 1，不计算负指数。原图的内部边固定，不计内部生成树选择。若禁止部分端点对、给每条新增边限制，或数加边的执行顺序，就不能直接使用此式。有标号树也不能简单除以 $n!$ 得到无标号树数，因为不同树的自同构数量不同。

双射背景见 Stanley, Topics in Algebraic Combinatorics，导航与基础公式见 固定版本 OI Wiki Prüfer 序列。以上用最小叶子的一致约定推导；不将本节计数说明当作整个上游页面的实现覆盖。

第十四章　矩阵树定理

14.1　带权矩阵树：无向树与内外向树形图

代码依赖：ModInt（第 327 页，A.1.1 节）、det_prime（第 333 页，C.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

声明 MatrixTree<mod>，调用 count(n, edges, root, kind) 返回所有合法树的边权乘积之和，mod 必须是素数。Edge=(u,v,Z)，点号 0..n-1，n≥1；Kind::undirected 为无向生成树，toward_root 为每点沿边能到根，away_from_root 为根沿边能到每点。后两种使用输入边方向，不能混淆。重边按不同边计数，自环忽略；单点答案 1。支持模意义下的零权和负权，结果 0 不一定表示不存在树。无向图删除哪个根行列均可；有向图 root 决定答案。依赖 det_prime 的素数模行列式，$O(n^3+m)$ 时间、$O(n^2)$ 额外空间。

```
template <int mod> struct MatrixTree
{
    using Z = ModInt<mod>;
    using Edge = tuple<int, int, Z>;
    enum class Kind
    {
        undirected,
        toward_root,
        away_from_root
    };

    // Sum of products of edge weights; vertices 0..n-1, prime modulus.
    static Z count(int n, const vector<Edge> &edges, int root, Kind kind)
    {
        assert(n >= 1 && root >= 0 && root < n);
        vector<vector<Z>> lap(n - 1, vector<Z>(n - 1));
        auto add = [&](int u, int v, Z w)
        {
            if (u == root) return;
            int i = u - (u > root), j = v - (v > root);
            lap[i][i] = lap[i][i] + w;
            if (v != root) lap[i][j] = lap[i][j] - w;
        };
        for (auto [u, v, w] : edges)
        {
            assert(u >= 0 && u < n && v >= 0 && v < n);
            if (u == v) continue;
            if (kind == Kind::away_from_root) swap(u, v);
            add(u, v, w);
            if (kind == Kind::undirected) add(v, u, w);
        }
        return det_prime<mod>(move(lap));
    }
};
```

模板题使用：Luogu P6178

题意：求所有生成树边权乘积之和，模 1000000007。type=0 为无向图，type=1 为以输入点 1 为根的外向树；输入点号减 1 后根为 0。重边分别计数，自环不贡献。

所需模板：ModInt（第 327 页，A.1.1 节）、det_prime（第 333 页，C.1.1 节）、MatrixTree（第 297 页，14.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6178

```
int main()
{
    using M = MatrixTree<1000000007>;
    int n, m, type;
    scanf("%d%d%d", &n, &m, &type);
    vector<M::Edge> edges;
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        scanf("%d%d%lld", &u, &v, &w);
        edges.push_back({u - 1, v - 1, w});
    }
    auto kind = type ? M::Kind::away_from_root : M::Kind::undirected;
    printf("%d\n", M::count(n, edges, 0, kind).v);
}
```

14.2 任意正模数带权矩阵树

代码依赖：determinant_mod（第 335 页，C.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

count(n, edges, root, kind, mod) 返回树的边权乘积之和对 mod 的最小非负余数。mod 为任意正 signed 64 位数，边权支持完整 signed 64 位。Kind 的 undirected、toward_root、away_from_root 分别表示无向、内向和外向树；点号 0..n-1。自环忽略，重边按不同边计数，单点返回 1%mod，mod=1 返回零。先在 128 位中构造模拉普拉斯余子式，再调用 determinant_mod；$O(n^3 \log \mathrm{mod} + m)$ 时间、$O(n^2)$ 额外空间。不使用模逆元。已知模数是素数时一般优先使用 MatrixTree<mod>。

```
struct MatrixTreeMod
{
    using ll = long long;
    using Edge = tuple<int, int, ll>;
    enum class Kind
    {
        undirected,
        toward_root,
        away_from_root
    };

    static ll count(int n, const vector<Edge> &edges, int root, Kind kind, ll mod)
    {
        assert(n >= 1 && root >= 0 && root < n && mod >= 1);
        vector<vector<ll>> lap(n - 1, vector<ll>(n - 1));
        auto add = [&](int u, int v, ll w)
        {
            if (u == root) return;
            int i = u - (u > root), j = v - (v > root);
            lap[i][i] = ((__int128)lap[i][i] + w) % mod;
            if (v != root) lap[i][j] = ((__int128)lap[i][j] + mod - w) % mod;
        };
        for (auto [u, v, w] : edges)
        {
            assert(u >= 0 && u < n && v >= 0 && v < n);
            if (u == v) continue;
            w %= mod;
            if (w < 0) w += mod;
            if (kind == Kind::away_from_root) swap(u, v);
            add(u, v, w);
            if (kind == Kind::undirected) add(v, u, w);
        }
        return determinant_mod(move(lap), mod);
    }
};
```

模板题使用：Library Checker Counting Spanning Trees (Undirected)

题意：无向多重图生成树计数，模 998244353；点号 0..n-1，1≤n≤500、0≤m≤500000，允许自环和重边。每条原边权重设 1，自环不参与树，重边是不同选择。删去任意根的拉普拉斯行列，这里取根 0；n=1 的空树计 1，断连计 0。此接口也支持合数模数，复杂度 $O(n^3 \log \mathrm{mod})$，本题固定质数亦可选更快的 MatrixTree 版本。

所需模板：determinant_mod（第 335 页，C.1.2 节）、MatrixTreeMod（第 299 页，14.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/counting_spanning_tree_undirected

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<MatrixTreeMod::Edge> e;
    e.reserve(m);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        e.emplace_back(u, v, 1);
    }
    cout << MatrixTreeMod::count(n, e, 0, MatrixTreeMod::Kind::undirected, 998244353)
         << '\n';
}
```

模板题使用：Library Checker Counting Spanning Trees (Directed)

题意：有向多重图生成树计数，模 998244353，要求指定根 r 能到达所有点，使用 away_from_root。n 不超过 500、m 不超过 500000，点号及根为 0 起；每条原边权 1，自环忽略，重边分别计数。每个非根点恰有一个入边，根无入边；toward_root 表示所有点到达根，不能替代。n=1 计空树 1，无外向生成树计 0。

所需模板：determinant_mod（第 335 页，C.1.2 节）、MatrixTreeMod（第 299 页，14.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/counting_spanning_tree_directed

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, r;
    cin >> n >> m >> r;
    vector<MatrixTreeMod::Edge> e;
    e.reserve(m);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        e.emplace_back(u, v, 1);
    }
    cout << MatrixTreeMod::count(
                n, e, r, MatrixTreeMod::Kind::away_from_root, 998244353)
         << '\n';
}
```

14.3 精确整数带权生成树计数

代码依赖：determinant_exact（第 337 页，C.1.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

MatrixTreeExact<Z>::count(n,edges,root,kind)，1≤n<INT_MAX、点号 0..n-1；Edge 为 tuple<int,int,Z>。kind 为 undirected、toward_root 或 away_from_root，分别计无向生成树、朝根汇聚树、从根发散树。返回各树边权乘积之和；负权和零权合法，答案 0 可能是抵消。重边按独立选择计数，自环忽略；单点返回空树权 1。无向结果与根无关，有向根必须指定。构造出度拉普拉斯，发散模式先反边，删根行列后调用 determinant_exact。$O(m+n^3)$ 次整数算术、$O(n^2)$ 个整数工作对象，不含调用者边表；大整数位成本另计。Z 要求同精确行列式。

```
template<class Z>
struct MatrixTreeExact
{
    using Edge = tuple<int, int, Z>;
    enum class Kind
    {
        undirected,
        toward_root,
        away_from_root
    };

    static Z count(int n, const vector<Edge> &edges, int root, Kind kind)
    {
        assert(n >= 1 && root >= 0 && root < n);
        vector<vector<Z>> lap(n - 1, vector<Z>(n - 1));
        auto add = [&](int u, int v, const Z &w)
        {
            if (u == root)
                return;
            int i = u - (u > root);
            int j = v - (v > root);
            lap[i][i] += w;
            if (v != root)
                lap[i][j] -= w;
        };
        for (auto [u, v, w] : edges)
        {
            assert(0 <= u && u < n && 0 <= v && v < n);
            if (u == v)
                continue;
            if (kind == Kind::away_from_root)
                swap(u, v);
            add(u, v, w);
            if (kind == Kind::undirected)
                add(v, u, w);
        }
        return determinant_exact(move(lap));
    }
};
```

接口用法演示：带权矩阵树的精确整数答案

演示约定：T 组；每组 n,m,root,type 及 m 条 0-based 边 u,v,w。type=0 无向、1 朝根、2 离根。权值为有符号十进制整数；输出各树权积之和。重边独立、自环忽略、单点为 1；负权可抵消成 0。仅 API 用法。

所需模板：determinant_exact（第 337 页，C.1.3 节）、MatrixTreeExact（第 302 页，14.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

额外依赖（不属于标准头文件）：`boost/multiprecision/cpp_int.hpp`，须由编译环境提供并显式包含。

来源：参考文档

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Z = boost::multiprecision::cpp_int;
    using Tree = MatrixTreeExact<Z>;
    int t;
    cin >> t;
    while (t--)
    {
        int n, m, root, type;
        cin >> n >> m >> root >> type;
        vector<Tree::Edge> edges(m);
        for (auto &[u, v, w] : edges)
            cin >> u >> v >> w;
        auto kind = Tree::Kind::undirected;
        if (type == 1)
            kind = Tree::Kind::toward_root;
        if (type == 2)
            kind = Tree::Kind::away_from_root;
        cout << Tree::count(n, edges, root, kind) << '\n';
    }
    return 0;
}
```

接口用法演示：不取模生成树计数（kuangbin 来源模型）

演示约定：T 组；每组 n,m 及 m 条 1-based 无向边。按原稿布尔邻接语义去掉自环、合并重复和反向边，再输出精确树数。核心接口本身不合并重边。原稿标注 SPOJ104，但完整官方题面及资源约束未核验，故仅登记来源模型/API，不声称正式题目或线上 AC。

所需模板：determinant_exact（第 337 页，C.1.3 节）、MatrixTreeExact（第 302 页，14.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

额外依赖（不属于标准头文件）：`boost/multiprecision/cpp_int.hpp`，须由编译环境提供并显式包含。

来源：参考文档

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Tree = MatrixTreeExact<boost::multiprecision::cpp_int>;
    int t;
    cin >> t;
    while (t--)
    {
        int n, m;
        cin >> n >> m;
        set<pair<int, int>> seen;
        while (m--)
        {
            int u, v;
            cin >> u >> v;
            u--;
            v--;
            if (u > v)
                swap(u, v);
            if (u != v)
                seen.insert({u, v});
        }
        vector<Tree::Edge> edges;
        for (auto [u, v] : seen)
            edges.emplace_back(u, v, 1);
        cout << Tree::count(n, edges, n - 1, Tree::Kind::undirected) << '\n';
    }
    return 0;
}
```

14.4 带权矩阵树定理与方向约定

实现见第 14.1 节（第 297 页）。返回的是每棵树的边权乘积，再对所有树求和，并对素数取模。无权计数只需把所有边权设为 1。重边是不同选择，自环不属于任何生成树；单点图的空树乘积为 1。零权和负权都按域上的运算处理，因此结果为零可能来自抵消，不能作为连通性判定。

无向边 uv 权值 w 对拉普拉斯矩阵贡献为 L_{uu}, L_{vv} 各加 w，L_{uv}, L_{vu} 各减 w。删除 root 行和列后求行列式，与 root 的选择无关。对于每个点都能沿有向边走到根的树形图，边 $u \to v$ 贡献 $L_{uu} \mathrel{+}= w$、$L_{uv} \mathrel{-}= w$；删除根行列后得到内向树形图的权值和。外向树形图先反转所有边，再使用同一构造。

例如只有一条边 $0 \to 1$ 时，内向根 1 的答案为该边权，内向根 0 为零；外向根 0 的答案为该边权。P6178 的有向模式（type=1）指定以 1 为根的外向树；type=0 为无向图，完整用法见第 14.1 节（第 298 页）；驱动把点号减一，有向模式选择 `away_from_root`。只直接构造 $(n-1)\times(n-1)$ 余子式，时间 $O(n^3+m)$、额外空间 $O(n^2)$。行列式复用本书的素数模高斯消元，不支持将 mod 随意改成合数。

14.4.1 为什么这个余子式恰好计数

先证内向情形。删去根 r 的坐标，令 $e_r = 0$，其他 e_i 为单位行向量。余子式第 i 行是 $\sum_{i\to j} w_{ij}(e_i - e_j)$。按行的多线性展开，每项从每个非根点恰选一条出边，权为所选边权之积。若这些边含有有向环，环上向量 $e_i - e_j$ 相加为零，该项行列式为零。若无环，有限图中每条路径只能在没有出边的根终止。按到根距离从小到大，同时重排行列，得到对角全为 1 的三角矩阵，行列式为 1。因此只有内向生成树保留下来，每棵树的权恰计一次。

自环的向量为零；重边在展开时是不同选择；单点对应空行列式与空乘积 1。无向边替换成权相同的两条反向弧，每棵无向生成树恰有一种朝根定向；有向边全部反转则把外向树变成内向树。整个论证是整数系数多项式恒等式，没有除法，故允许负权、零权及任意正模数。素数限制来自消元实现，不来自矩阵树定理本身。

证明依据行列式多线性与有向树的出度刻画，参见 MIT 18.212 第 29 讲；这里直接按选边后的环与根路径给出主余子式的证明，不扩展为任意非主余子式公式。

14.4.2 既有接口与验证记录

测试独立枚举 $n-1$ 条边的所有选择，检查连通性或根向约束，用任意精度整数计算边权乘积之和，再分别对多个素数取模。两套实现覆盖所有根、三种方向、自环重边、负权及 64 位权值边界，并通过 220 点完全图的加权 Cayley 公式检查。P6178 完整驱动另测 300 点、十万条边以及输入输出约定。普通与 ASan/UBSan 均通过。两套代码已在 P6178 通过全部 11 个测试点：动态版记录 297619886、传统版记录 297620263。单点最大用时分别为 64 ms、62 ms，内存峰值为 2.85 MB、2.40 MB。在线测试覆盖无向与根为 1 的外向树；其他根、内向树和负权仍使用本地证据。定理背景可参考 OI Wiki 矩阵树定理。

14.5 任意正模数的矩阵树计数

实现见第 14.2 节（第 299 页）。拉普拉斯余子式表示生成树权值和的恒等式本身不要求模数为素数；需要换掉的是依赖模逆元的行列式算法。本节沿用第 14.4 节（第 305 页）的三种构造，但使用欧几里得行列式消元。输入边权和正模数均支持 signed 64 位，接口名为 `MatrixTreeMod::count`。行列式实现见第 C.1.2 节（第 335 页），时间 $O(n^3 \log(m+1) + |E|)$、空间 $O(n^2)$；此处 m 为模数。

先把边权正规化到 $[0, m)$，每次矩阵元素加减仍先扩展至 128 位，再取模存回 64 位。重边的权值和可能超过 64 位范围，不能先累加到普通整数中。只构造删除根后的余子式，并将其存储直接交给行列式函数。单点返回 $1 \bmod m$，模 1 返回零。负权与零权仍按代数权值处理，返回零不能直接解释为没有生成树。

独立参考程序逐个枚举边集，用任意精度整数计算权值和。已检查多个合数模数、所有根与三种方向、断图、重边自环及 64 位极值权重；另以大权值完全图的 Cayley 公式验证 160 点规模。普通与 ASan/UBSan 均通过，尚无该接口的在线 AC。此前 P6178 的 AC 属于素数模版本，不自动转移到本接口。

第十五章　LGV 引理

15.1　LGV：DAG 路径权值行列式

代码依赖：ModInt（第 327 页，A.1.1 节）、det_prime（第 333 页，C.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

dag_path_determinant<mod>(g,sources,sinks) 返回路径权值矩阵的行列式，mod 必须为素数。g[u] 的项为 (v,ModInt<mod> 权值)，点号 0..n-1；两个端点序列等长 k，各自不能重复，但可相互重叠。允许重边、零权、负权；长度为 0 的路径权为 1。整张图有环则返回 nullopt，空端点族在 DAG 上返回 1。由 LGV，它是顶点不交路径族按汇点排列带符号的权值和；只有能证明非恒等排列均不可能时，才是对应配对的普通计数。顺序不能随意排序，模结果为 0 也不能据此断言不存在路径族。时间 $O((k+1)(n+m)+k^3+k \log mod)$，额外空间 $O(n+k^2)$，依赖 det_prime 和 ModInt。

```
template <int mod>
optional<ModInt<mod>>
dag_path_determinant(const vector<vector<pair<int, ModInt<mod>>>> &g,
                     const vector<int> &sources,
                     const vector<int> &sinks)
{
    using Z = ModInt<mod>;
    int n = g.size(), k = sources.size();
    assert((int)sinks.size() == k);
    vector<int> in(n), order;
    vector<bool> seen_source(n), seen_sink(n);
    for (int i = 0; i < k; i++)
    {
        assert(0 <= sources[i] && sources[i] < n);
        assert(0 <= sinks[i] && sinks[i] < n);
        assert(!seen_source[sources[i]] && !seen_sink[sinks[i]]);
        seen_source[sources[i]] = seen_sink[sinks[i]] = true;
    }
    for (const auto &edges : g)
        for (auto [v, w] : edges)
        {
            assert(0 <= v && v < n);
            in[v]++;
        }
    for (int u = 0; u < n; u++)
        if (!in[u]) order.push_back(u);
    for (int i = 0; i < (int)order.size(); i++)
        for (auto [v, w] : g[order[i]])
            if (!--in[v]) order.push_back(v);
    if ((int)order.size() != n) return nullopt;
```

```
    vector<vector<Z>> a(k, vector<Z>(k));
    for (int i = 0; i < k; i++)
    {
        vector<Z> dp(n);
        dp[sources[i]] = 1;
        for (int u : order)
            for (auto [v, w] : g[u]) dp[v] = dp[v] + dp[u] * w;
        for (int j = 0; j < k; j++) a[i][j] = dp[sinks[j]];
    }
    return det_prime<mod>(move(a));
}
```

应用题补充：DAG 路径行列式（本地接口示例）

题意：输入 n,m,k、m 条有向带权边、k 个起点和 k 个终点，均为 0-based。输出模 998244353 的带符号路径族权值和，整图有环输出 CYCLE。端点顺序原样保留，不能把负号当作错误去掉。

所需模板：ModInt（第 327 页，A.1.1 节）、det_prime（第 333 页，C.1.1 节）、dag_path_determinant（第 307 页，15.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://github.com/gsh20040816/CPCtemplate/issues/11

```
int main()
{
    using Z = ModInt<998244353>;
    int n, m, k;
    cin >> n >> m >> k;
    vector<vector<pair<int, Z>>> g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g[u].push_back({v, Z(w)});
    }
    vector<int> sources(k), sinks(k);
    for (int &u : sources) cin >> u;
    for (int &u : sinks) cin >> u;
    auto ans = dag_path_determinant<998244353>(g, sources, sinks);
    if (!ans)
        cout << "CYCLE\n";
    else
        cout << ans->v << '\n';
}
```

第十六章　支配树

16.1　支配树：Lengauer–Tarjan

DominatorTree(n)，点号 1..n；add(u,v) 加有向边，允许重边、自环。build(root) 后 idom 为原点号直接支配点，根指向自身，不可达为 0。仅对 root 可达点定义支配关系：所有 root 到 v 的路径均经过 u，等价于 u 在支配树上是 v 的祖先。timer 是可达点数，vertex[1..timer] 为 DFS 顺序，dfn 为逆映射；逆序向 idom 累加可得支配点数，根不向自身累加。g 保留原点号；pred、bucket、semi 等辅助数组使用 DFS 编号。可更换根重建；加边使预处理失效，需重新 build。简单 LT 时间 O((n+m) log(n+1))，空间及递归栈 O(n+m) 与 O(n)，n+1、边数须可由 int 表示。一般图与 DAG 均适用；可由 idom 建树组合 LCA。

```
struct DominatorTree
{
    int n, timer = 0;
    vector<vector<int>> g, pred, bucket;
    vector<int> dfn, vertex, parent, semi, label, ancestor, dom, idom;

    DominatorTree(int n) : n(n), g(n + 1) { assert(n > 0); }

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back(v);
        idom.clear();
    }

    void dfs(int u)
    {
        dfn[u] = ++timer;
        vertex[timer] = u;
        for (int v : g[u])
            if (!dfn[v])
            {
                dfs(v);
                parent[dfn[v]] = dfn[u];
            }
    }

    void compress(int v)
    {
        int a = ancestor[v];
        if (ancestor[a])
        {
            compress(a);
            if (semi[label[a]] < semi[label[v]]) label[v] = label[a];
            ancestor[v] = ancestor[a];
        }
    }
```

路径查询、半支配点与直接支配点计算（接上页同一结构体）：

```
    int eval(int v)
    {
        compress(v);
        return label[v];
    }

    void build(int root)
    {
        assert(1 <= root && root <= n);
        timer = 0;
        dfn.assign(n + 1, 0);
        vertex.assign(n + 1, 0);
        parent.assign(n + 1, 0);
        ancestor.assign(n + 1, 0);
        dom.assign(n + 1, 0);
        idom.assign(n + 1, 0);
        pred.assign(n + 1, {});
        bucket.assign(n + 1, {});
        semi.resize(n + 1);
        iota(semi.begin(), semi.end(), 0);
        label = semi;
        dfs(root);
        for (int u = 1; u <= n; u++)
            if (dfn[u])
                for (int v : g[u]) pred[dfn[v]].push_back(dfn[u]);
        for (int i = timer; i >= 2; i--)
        {
            for (int p : pred[i]) semi[i] = min(semi[i], semi[eval(p)]);
            bucket[semi[i]].push_back(i);
            ancestor[i] = parent[i];
            for (int v : bucket[parent[i]])
            {
                int u = eval(v);
                dom[v] = semi[u] < semi[v] ? u : parent[i];
            }
            bucket[parent[i]].clear();
        }
        for (int i = 2; i <= timer; i++)
        {
            if (dom[i] != semi[i]) dom[i] = dom[dom[i]];
            idom[vertex[i]] = vertex[dom[i]];
        }
        idom[root] = root;
    }
};
```

模板题使用：Library Checker dominatortree

题意：从指定起点求每个点的直接支配点。输入输出是 0-based，调用模板时加 1；输出减 1 后，不可达点为 -1，根仍指向自身。

所需模板：DominatorTree（第 309 页，16.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/dominatortree

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    DominatorTree t(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        t.add(u + 1, v + 1);
    }
    t.build(s + 1);
    for (int u = 1; u <= n; u++) cout << t.idom[u] - 1 << (u == n ? '\n' : ' ');
}
```

模板题使用：Luogu P5180

题意：从点 1 出发，输出各点支配的可达点数，包含自己。按 DFS 序逆序向直接支配点累加，根不向自身累加；不可达点输出 0。

所需模板：DominatorTree（第 309 页，16.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5180

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    DominatorTree t(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        t.add(u, v);
    }
    t.build(1);
    vector<int> ans(n + 1);
    for (int i = t.timer; i >= 1; i--)
    {
        int u = t.vertex[i];
        ans[u]++;
        if (u != 1) ans[t.idom[u]] += ans[u];
    }
    for (int u = 1; u <= n; u++) cout << ans[u] << (u == n ? '\n' : ' ');
```

```
}
```

16.2 DAG 支配树：拓扑序与在线倍增 LCA

DagDominator(n)，1≤n<INT_MAX，点号 1..n；add(u,v)，允许重边。build(root) 检查全图 DAG 并返回 bool，任意分量有环（含自环）都返回 false，同时清空 idom/dep/up/order，不能使用半成品。成功后 idom[root]=root，不可达点 idom=dep=0；order 为全图拓扑序。仅从可达前驱累积 LCA，不可达前驱不能影响结果。lca(u,v) 仅在成功 build 后对可达点使用，0 可作空累积值，lca(0,v)=v。idom 是直接支配点，可按逆 order 向父亲累加子树大小，跳过不可达点和根的自累加。可更换 root 重建；加边后旧结果失效，须重新 build。g 只通过 add 改动，结果数组只读。时间 O((n+m)log(n+1))，空间 O(nlog(n+1)+m)，入度须可由 int 表示。一般图请使用 DominatorTree；多入口按题意添加虚拟根，不可擅自把所有无入度点当同一个入口。

```
struct DagDominator
{
    int n, lg = 1;
    vector<vector<int>> g, up;
    vector<int> idom, dep, order;

    DagDominator(int n) : n(n), g(n + 1)
    {
        assert(n > 0);
        while ((1LL << lg) <= n)
            lg++;
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
    }

    int lca(int u, int v) const
    {
        if (!u || !v)
            return u + v;
        if (dep[u] < dep[v])
            swap(u, v);
        for (int k = lg - 1; k >= 0; k--)
            if (dep[up[k][u]] >= dep[v])
                u = up[k][u];
        if (u == v)
            return u;
        for (int k = lg - 1; k >= 0; k--)
            if (up[k][u] != up[k][v])
            {
                u = up[k][u];
                v = up[k][v];
            }
        return up[0][u];
    }
```

全图拓扑检查与逐点构建支配树（接上页同一结构体）：

```
    bool build(int root)
    {
        assert(1 <= root && root <= n);
        idom.clear();
        dep.clear();
        up.clear();
        order.clear();
        vector<int> deg(n + 1);
        for (int u = 1; u <= n; u++)
            for (int v : g[u])
                deg[v]++;
        for (int u = 1; u <= n; u++)
            if (!deg[u])
                order.push_back(u);
        for (int i = 0; i < (int)order.size(); i++)
            for (int v : g[order[i]])
                if (--deg[v] == 0)
                    order.push_back(v);
        if ((int)order.size() != n)
        {
            order.clear();
            return false;
        }
        idom.assign(n + 1, 0);
        dep.assign(n + 1, 0);
        up.assign(lg, vector<int>(n + 1));
        idom[root] = root;
        for (int u : order)
        {
            if (!idom[u])
                continue;
            dep[u] = u == root ? 1 : dep[idom[u]] + 1;
            up[0][u] = idom[u];
            for (int k = 1; k < lg; k++)
                up[k][u] = up[k - 1][up[k - 1][u]];
            for (int v : g[u])
                idom[v] = lca(idom[v], u);
        }
        return true;
    }
};
```

接口用法演示：DAG 直接支配点与全图环检查

演示约定：输入 n,m,root 与 m 条 1-based 有向边。若任意分量有环，输出-1；否则输出每点 idom，根指向自身，不可达为 0。本例演示纯 DAG 接口，成功仅表示全图无环，不能说明每点都从 root 可达。

所需模板：DagDominator（第 313 页，16.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/graph/dominator-tree/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, root;
    cin >> n >> m >> root;
    DagDominator d(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        d.add(u, v);
    }
    if (!d.build(root))
    {
        cout << -1 << '\n';
        return 0;
    }
    for (int u = 1; u <= n; u++)
        cout << d.idom[u] << (u == n ? '\n' : ' ');
    return 0;
}
```

模板题使用：Library Checker dominatortree（DAG/LT 组合）

题意：一般有向图指定起点求直接支配点，n,m≤200000，输入输出 0-based；根指向自身，不可达输出-1。先调用 DagDominator，若非 DAG 则把图移交 DominatorTree 求解。本用法验证两种算法组合，不能声称 DAG 算法单独支持有环图，既有 LT 线上记录不转移到新程序。

所需模板：DagDominator（第 313 页，16.2 节）、DominatorTree（第 309 页，16.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/dominatortree

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, root;
    cin >> n >> m >> root;
    root++;
    DagDominator d(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        d.add(u + 1, v + 1);
    }
    vector<int> ans;
    if (d.build(root))
        ans = move(d.idom);
    else
    {
        DominatorTree t(n);
        t.g = move(d.g);
        t.build(root);
        ans = move(t.idom);
    }
    for (int u = 1; u <= n; u++)
        cout << ans[u] - 1 << (u == n ? '\n' : ' ');
    return 0;
}
```

应用题补充：Luogu P2597 / ZJOI2012 灾难

题意：输入无环食物网，n≤65534，每行列出该生物的食物，0 结束；食物向捕食者连边。为所有生产者添加虚拟根 n+1，build 后逆拓扑累加支配树子树大小。输出每种生物灭绝后额外灭绝的数量，不含自身。竞赛应用，不能替代独立模板题；尚无本项目线上 AC。

所需模板：DagDominator（第 313 页，16.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2597

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    int root = n + 1;
    DagDominator d(root);
    for (int v = 1; v <= n; v++)
    {
        int u;
        cin >> u;
        if (!u)
            d.add(root, v);
        while (u)
        {
            d.add(u, v);
            cin >> u;
        }
    }
    if (!d.build(root))
        return 0;
    vector<int> size(root + 1, 1);
    for (int i = (int)d.order.size() - 1; i >= 0; i--)
    {
        int u = d.order[i];
        if (u != root)
            size[d.idom[u]] += size[u];
    }
    for (int u = 1; u <= n; u++)
        cout << size[u] - 1 << '\n';
    return 0;
}
```

第十七章　配套知识与验证范围

17.1　树路径交：投影与单点交

核心见第 23 页，P3398 完整用法见第 24 页。将 x 投影到树路径 a–b，投影是 $\mathrm{lca}(a,b)$、$\mathrm{lca}(a,x)$、$\mathrm{lca}(b,x)$ 中深度最大者。三条两点间路径在同一个分叉点相遇；按同一根计算 LCA 即得这个分叉点，与顶点编号无关。设 c,d 的投影是 p,q。若 $p\neq q$，路径 c–d 必经过 p–q，后者正是两路径的交集。若 $p=q$，两端可能在同一条旁支，不能直接认为有交；须检查 $d(c,p)+d(p,d)=d(c,d)$。通过时为单点交，否则为空。这里距离和 depth 都按边数，不能换成带权根距离；零权边会破坏上述距离等式的判别。

接口返回交集端点和顶点数；空交是 $\{0,0,0\}$，公共边数为 $\max(0,\mathrm{vertices}-1)$。P3398 仅输出是否相交，不验证具体端点。本地接口测试穷举至五点的全部标号树、每个根及四端点，独立构造两条路径的顶点集合求交，核验数量与返回端点间的整条路径。完整驱动另验 146 棵小树、100 棵随机换号树，以及十万点链和十万次查询；两种编译模式均通过。EulerLCA 预处理 $O(n\log n)$、每次路径交 $O(1)$；若换用 HLD 回调则为 $O(\log n)$。完整驱动以 C++20 O2 在P3398 记录 299972443通过全部十个测试点，最慢单点 134 ms、峰值 31.89 MB；该 AC 仅对应布尔相交判定，不涵盖端点、精确数量或带权距离。

17.2 非负权树的点集直径摘要

代码见第 9 页。一个非空点集只需保存任意一对直径端点 a, b，即可回答任意树上点 x 到该点集的最远距离：$\max(d(x,a), d(x,b))$。边权非负，包括零权；该结论不能推广到一般图或负权树。

证明可将点 u 投影到路径 a–b，记投影距 a 为 t，离路径的距离为 h，直径长为 D。由于 $d(a,u), d(b,u) \le D$，有 $h \le \min(t, D-t)$。再将查询点投影到同一路径，投影距 a 为 s，离路径的距离为 g。经两个投影点连接得到 $d(x,u) \le g + h + |s-t| \le g + \max(s, D-s)$，右侧恰为 $\max(d(x,a), d(x,b))$。因此插入新点只需比较到两个旧端点的距离；合并点集时，只插入另一个摘要的两个端点即可。并列直径可任选，不应假设端点编号固定或合并后的端点与合并顺序无关。

Euler LCA 现在同时保存根距离，但其原有 distance 接口仍返回边数。带权距离请显式调用 weighted_distance / Weighted_Distance；采用 $(D_u - D_z) + (D_v - D_z)$ 避免不必要的两倍根距离中间值。所有路径权和须能用 long long 表示。独立验证穷举小点集及合并，逐点检查最远距离与返回顶点；带权路径用独立树遍历参照，并覆盖全部根、零边权与 `LLONG_MAX` 单边。原有 Euler LCA 全小树回归，以及 50 万点、边权 10^9 的递归链也通过普通与 ASan/UBSan 检查。

离线添叶仍是固定树度量。 CF379F New Year Tree 的完整用法见第 11 页。先把全部添叶操作建成最终树，再用 HLD 求距离；只有旧树外接新叶，不会改变两个旧点之间的唯一简单路径。因此可以按出现顺序向摘要插点，尚未出现的点只帮助预处理距离，不提前进入点集。初始四点也必须插入；每次两个新点都插完，再输出 length。$q \le 500000$ 时最终有 $4 + 2q$ 个点，采用 HLD 的 $O(q)$ 空间、$O(q \log q)$ 总时间。递归 DFS 保留；本地放大测试栈，最大实例包含五十万层链。250 组随机合法添叶过程逐步重建当前树，以独立两次 BFS 求直径，另验满规模链；普通与 ASan/UBSan 均通过。这份用法仅覆盖 insert 和 length；任意点集合并、空集和最远点见接口独立测试。2026-09-29 洛谷提交入口返回 Codeforces RemoteJudge 暂不可用，故不登记此题线上 AC。

杭州 2023 F 中，点权互异。令 g 为全树点权的 mex，只需按点权 $0, 1, \ldots, g-1$ 建立前缀摘要。某个前缀全部位于以 x 为中心、半径 k 的范围内，当且仅当该摘要的最远距离不超过 k。二分最长满足条件的前缀长度即为答案；$g = 0$ 时直接为零。题面树边带权，不能用边数距离。完整驱动在 `verify/qoj/8235.*.cpp`，本地使用实际权距与 mex 集合参照，$n = q = 500000$ 的链和星形树保持递归 DFS。两套驱动各 50 个正式测试和 1 个额外测试全部在线 AC：动态版 666 ms、198076 KB，传统版 588 ms、198196 KB。OJ 验证题面正权边、前缀插入和最远距离判定；零权边、完整 64 位距离与任意摘要合并仍由独立本地验证支持。

17.3 函数图的入环链与到达查询

代码见第 155 页。三份完整用法分别见第 158 页（CSES1750 跳转）、第 159 页（CSES1160 最少步数）、第 159 页（P2921 不同访问点数）。题目点号均从 1 开始，构造时统一减 1；只有跳转返回的顶点需要加 1，最少步数的 0 和不可达的-1 应原样输出。P2921 只需 depth 加环长，不代表它测试了跳转或两点可达查询。本轮完整驱动穷举至四点的全部 288 个后继映射，独立沿后继走到首次重复，核验步数、跳转与访问数；另验随机映射和题目最大规模。CSES1750 的题面步数上限为 10^9，完整 unsigned64 范围另由本地测试覆盖，不能由该题 AC 推出。

每个点有且仅有一个后继，故每个弱连通分量由一个有向环和若干指向环的树组成。先不断移除入度为零的点，剩下的点恰在环上；沿后继枚举各环。反向处理移除序列时，后继的环编号、入环点和入环距离均已确定，可直接传给当前点。这里使用的是入度剥离队列，不是用显式栈模拟 DFS。

跳转 k 步时，若 k 小于入环距离，用普通倍增；否则先到入环点，减去入环步数，再对环长取模。因此即使 k 达到无符号 64 位上界，也只需 $O(\log n)$ 层倍增表。环上位置相加使用无符号宽整数，避免两个大下标的 int 中间值溢出。

判断从 u 到 v 的最少步数时：不同环编号必不可达。若 v 在入环树上，先检查 u 的深度至少为 v 的深度，再跳过深度差，看是否恰好到 v，从而排除同分量的不同分支。若 v 在环上，答案为 u 的入环步数加上沿后继方向的环上距离。不可达返回 -1，从自己到自己为零步。字段 position 只对环点有意义，不要将树点的 -1 当作环坐标。

独立参照逐步行走直至首次重复，同时记录每个点的第一次出现位置。穷举不超过 5 个点的全部后继映射，并检查跳转、最少步数、入环点与环长；另验证同对象重建、空图、50 万点长链连环、50 万个自环和完整无符号 64 位步数。普通与 ASan/UBSan 均通过。完整驱动对应CSES 1750、CSES 1160 和 P2921，均使用 vector 版驱动。前两题检查跳转与可达步数，后一题仅使用“入环步数 + 环长”，不能替代查询接口的验证。P2921 的当前程序已取得线上 AC；两道 CSES 查询题的线上评测仍待完成。

17.4 杭州 2023 H：随机事件顺序与函数图

Sugar Sweet II 的完整驱动为 `verify/qoj/8237.*.cpp`，复用函数图分解与逆阶乘表。事件 u 发生前，自己的数量仍为 a_u；目标 $v=b_u$ 只可能为 a_v 或 a_v+w_v。因此 $a_u<a_v$ 时必定发生，$a_u\ge a_v+w_v$ 时必定不发生。其余情况下，事件 u 成功当且仅当事件 v 更早执行且成功。

将两类确定事件的后继改为自己，另保存哪些是确定成功的终点。未修改的依赖环没有成功起点：最早执行的环上事件必失败，后续也无法形成成功链。自环同理，自己不可能严格少于自己。沿函数图到达确定失败终点或纯依赖环的节点，成功概率都是零。若到达确定成功终点，且入环距离为 d，沿途 $d+1$ 个事件必须按唯一的顺序出现。均匀随机全排列对这 $d+1$ 个事件诱导均匀相对顺序，因此成功概率为 $1/(d+1)!$。这些顺序约束有关联，不能将每条依赖当作独立的二分之一概率。期望为 $a_u+w_u/(d+1)!$，在模 10^9+7 下计算。

题面所有增量为正，$\sum n\le 500000$；阶乘表跨测试组只扩充不缩小，避免大量单点测试时反复求逆。当前驱动复用带倍增表的通用函数图，时间、空间为 $O(n\log n)$；概率传播本身只读取 entry 和 depth，不调用跳转与最少步数接口。

验证穷举小输入及事件全排列，逐步修改真实糖果数量，最后汇总精确总和并除以排列数。另外检查 50 万点的依赖链、纯环，以及 50 万组单点测试，普通与 ASan/UBSan 均通过。两套驱动各 31 个正式测试与 2 个额外测试全过：动态版 162 ms、84608 KB，传统版 158 ms、84612 KB。本应用不能验证函数图的 advance 或 steps 接口。

17.5 杭州 2023 G：开放时间的单位边最短路

驱动 `verify/qoj/8236.*.cpp` 直接枚举网格邻居，不建立四倍规模的边表。初始蛇身从头到尾编号 $0,\dots,k-1$，第 $i\ge 1$ 节所在点的最早进入时间为 $k-i$，头与其他空格为零。旧蛇尾允许在同一步进入，所以尾格的限制必须是 1。长度为 4 的满 2×2 网格样例距离为 $0,3,2,1$，平方和为 14。

每步只能删除或移动一个尾节，故这些时间限制必要。最短路可以取不重复顶点的路径；行走前把途中等待的总时间全部用于缩短蛇身，随后沿路径连续移动。进入初始蛇身第 i 节时，已删除的尾节数量至少为 $k-i$；新形成的蛇身不会与无重复顶点的路径相撞。因此这些限制也充分。等待超过 $k-1$ 没有必要，因为此时全部初始蛇身均已释放。

转移为 $d_v=\max(d_u+1,\mathrm{open}_v)$。按距离递增处理顶点时，第一个发现 v 的前驱就给出最小候选值，故不需要后续松弛。候选等于 d_u+1 的顶点进入普通 FIFO；其余进入对应开放时间的桶。每次取两者中距离较小的一项。桶指针只能扫描到下一个 FIFO 距离，否则可能越过之后才被发现的较早开放时间。令 $R=\max \mathrm{open}$，复杂度 $O(V+E+R)$，空间 $O(V+R)$。此性质依赖所有边权为 1；一般非负权仍需优先队列。

本地以独立堆 Dijkstra 对拍，并穷举 2×3 网格障碍和全部 744 种初始蛇形，对真实蛇身序列逐步执行移动与缩尾 BFS，核对每个点的最早到达时间。另验证正式样例、满长链和 3000×3000 空网格及隔断网格；普通与 ASan/UBSan 均通过。答案用 unsigned long long 累加距离平方，自然实现模 2^{64}。两套驱动在 QOJ8236 各通过 58 个正式与 3 个额外测试：动态版 324 ms、123080 KB，传统版 294 ms、122768 KB。线上证据对应单位网格、源点时间为零及初始蛇身开放时间；一般有向图与非零源点时间由本地对拍覆盖。

17.6 SCC 模板题与速度榜口径

QOJ906 来自 Library Checker，要求输出强连通分量的拓扑序。本库 Tarjan 编号按逆拓扑序产生，驱动将输入点号加一，再按分量编号从大到小输出，输出点号减一。CCF_NOI 的记录 2940638 通过全部 11 个测试，203 ms、58636 KB。验证覆盖 add、run、bel 和完整分量输出，不调用 dag。本地独立可达性参考核对分量等价关系及每条跨分量边方向，并测试 50 万点链、环和多分量。普通与 ASan/UBSan 均通过；本地测试线程使用 1 GiB 栈容纳 sanitizer 扩大的递归栈帧，提交代码保持普通 main 和递归 DFS。

2026-09-12 核对速度榜时，页面第 66 行处于 203 ms 的第 65–67 行并列区间。但该表合并同一用户的提交，不能将行号与满分提交数 578 配对。以 CCF_NOI 逐页读取页面链接的满分列表，共取得 143 条，其中 93 条严格更快、3 条同为 203 ms。可见范围内按耗时并列排名为 94 out of 143，位置区间为 94–96。统计页显示所有提交 1053、满分提交 578；与可见列表的计数差异原因尚未核实，因此全站排名仍为待查。记录清单与证据见 `docs/ranking-audits.json`，不得将该局部排名改写为全站排名。

17.7 SPFA 费用流与残量方案

SPFA 备选每次在残量图上直接求最短路，保留负边、限流和继续增广，但不允许负环。每次调用先将全部顶点以零距离入队检查，包括源点不可达的负环；因为不连通的负环同样能降低总费用，不能忽略。检查时不改流量，发现负环抛异常。记录严格改进路径的边数，达到顶点数即拒绝；不要用松弛次数冒充路径边数。零费用环不发生严格改进，不会反复改写前驱。

沿最短残量路增广保持当前流量下的最小费用性。反边可以撤销已经送过的流，最终方案应通过原边编号查询 used，不能累加历史路径当作最终流量。flow 返回本次新增流量与费用，重复调用须同源汇；传零限流仍检查负环。最小费用流允许负边与允许负环是两种不同的契约，后者应使用专门算法。

容量为 long long，距离及费用为 int128；费用不能为 LLONG_MIN，因反边须取相反数。另要求总流不超过 long long，且 $\sum cap\,|cost|<2^{120}$。P3381 另保证答案不超过 $2^{31}-1$，其示例才可转换为 long long 输出。SPFA 版实际增广 A 次的复杂度为 $O((A+1)VE)$，不能假设 $A=O(V)$；通常优先势能 Dijkstra，本版保留为直接的备选写法。

两版通过 4000 组独立流量枚举及边界测试，完整驱动通过最大规模本地检查。验证细节见仓库 docs/FLOW-ATTACHMENT.md；本地通过不等于线上 AC 或速度排名。

17.8 2-SAT 编号方向与双连通定义

TwoSAT 的 add(x,a,y,b) 表示 $(x = a) \vee (y = b)$。强制值写 add(x,a,x,a)，蕴含 $(x = a) \Rightarrow (y = b)$ 写 add(x,!a,y,b)。本库使用 Kosaraju 拓扑编号，真文字所在分量编号更大时取真；附件 Tarjan 的分量编号为逆拓扑序，比较方向相反。不能更换 SCC 实现而不检查赋值的不等号。solve 可在追加子句后重算，仅成功时读取 ans；一般复杂度为 $O(n + m)$，不保证字典序最小。

点双采用连通且无割点的定义：桥两端也成块，孤立点单独成块，割点允许属于多个块。自环不改变顶点间连通性，不额外产生点双；原边号仍保留，且不标桥。每条逻辑无向边只调用一次 add，不能把两个方向分别当作两条边加入。共享 lowlink 只跳过父边编号，不能跳过所有指向父顶点的重边，否则会误判桥。附件点双单独跳过父顶点虽不改变点双阈值的真假，但它的 low 不可直接复用来判桥。

BiconnectedCore 的点号为 $1 \dots n$、边号从零开始。run 后 blocks 是点双顶点集，bridge 是原边桥标记，bel 是 $1 \dots cnt$ 的边双号；边双编号不承诺拓扑方向。不连通输入的圆方图和桥缩点结果均为森林。st 仅保存未归块顶点，不是模拟 DFS 的执行栈；每次 run 重新计算所有结果。

独立子集、删点、删边、真值表与最大规模驱动检查均有普通及 ASan/UBSan 记录。LC 点双 22 组、边双 21 组官方数据双模式本地 checker 接受，不等于线上 AC。百万变量矛盾链在 ASan 下需更大栈，本地以 1 GiB 线程栈运行原递归算法；提交仍用普通 main。细节见仓库 docs/GRAPH-ATTACHMENT.md。当前共享核心的边双输出已由 CCF_NOI 提交 QOJ999 获 AC（记录 3061643，87ms，42352kb）；本次在线结果不核验 blocks、cut、森林或补边方案。

SCC 的完整拓扑序输出见第 162 页，注意其编号方向与 TarjanSCC 相反。固定 LC scc 的 12 组官方数据由本地 checker 接受；枚举三点有向图、独立传递闭包以及 50 万点长链、环、分离二元环核对分组和跨组顺序。同一 Kosaraju 驱动已由 CCF_NOI 提交 QOJ906 获 AC（记录 3061625，226ms，76512kb）；dag 接口及其他未调用的捆绑算法不在本次覆盖范围，速度排名尚未核验。

已有点双分解时，removal_components 返回原图块数 before 和独立删点后的总块数 after。删除孤立点会使总数减一；非单点点双每包含该顶点一次，给删除后的所在连通分量多贡献一块。AOJ 割点用法见第 177 页，只判 after[u]>before；UVA10765 完整数量及排名见第 178 页，以负块数和原点号排序，别把输入的输出名额 m 当作边数，保留每组输出后的空行。独立逐点删除图并重新求连通块，检查数量及同分顺序。

连通无向图能强连通定向当且仅当无桥。必要性：桥不论朝哪边定向，都不能双向通过；充分性：DFS 树边向下，非树边向祖先，每个非根子树都有返回严格祖先的边，逐层可回到根，根又能沿树边到达所有顶点。第 176 页先判桥再定向，每条逻辑边仅 add 一次，输出保留原边编号顺序。完整驱动枚举至五点的 771 个连通简单图，以逐边删除判可行性，再检查每条输出边及正反可达性；10 万点、30 万边边界和重边、自环的额外契约测试均通过。上述测试均有普通及 ASan/UBSan 记录，算法保留递归，不是线上 AC 或速度排名。

圆方森林保留全部原点 $1 \dots n$，第 i 个点双对应方点 $n + i + 1$（i 从零开始）。简单路径三元组应用见第 189 页：原点权为 -1，方点权为块的大小。对于同一连通分量中的不同原点 s, f，圆方树路径经过的各块，恰好给出可能作为中间点的顶点并集。同一块内任意第三点可放在两个指定端点间的简单路径上；块间只通过割点连接。方点权先计块大小，路径内部割点的 -1 去重，两端的 -1 排除 s, f，故路径权和就是可选中间点数，而不是简单路径条数。

设所在树有 C 个原点，删去节点 u 后各支的原点数为 a_i。有序原点端点对经过 u 的次数为 $2\sum_{i<j} a_i a_j + 2(C-1)[u \le n]$；后项计端点等于 u 的情形。sz 仅统计原点，父侧大小为 $C - sz[u]$，遍历子树时用前缀和累计两两乘积。乘以节点权后求和，使用 long long。每棵树分别求 C，孤立点贡献零；总时间空间 $O(n + m)$。

桥森林邻接项是（边双号，原桥号），不是原点端点；原点所属边双读 bel。第 168 页直接计度一叶子，第 180 页返回原点补边方案。连通且允许新增重边时，每个叶子至少需一个新增边端点，故至少 $\lceil L/2 \rceil$ 条边。DFS 顺序收集叶子，奇数时补首叶，再前后半段配对；每条桥两侧的叶子区间都有配对跨过，故达到下界。仅一个边双时不加边。非连通森林不能直接套此公式或调用构造接口。P2860 只检查条数，方案端点及桥编号另以本地逐边删除和多重集检验；小图额外枚举全部新增边多重集验证最优性。P4630、P2860 都是比赛应用，不代替正式模板题覆盖或线上 AC。

17.9 排列约束与最大流单位边分类

NERC 2024 K（QOJ 10424）给两套排列与同一大小为 k 的集合所形成的部分子序列。已知元素切开未知段：段内候选来自原排列的对应区间，需求是未知位置数；已知元素单独成需求一的组。检查锚点严格递增与区间长度足够；首尾设虚拟锚点。保留零需求组，使每个元素在两侧都恰好属于一组。

源连左组、右组连汇，容量为需求；每个元素直接对应左右组间的一条单位边。两侧需求总和均为 k，所以流量达到 k 等价于所有组恰好选够，等价于原题存在合法集合。这替代附件的元素拆点路径，减少节点且保留逐元素对应关系。仅区间检查不足以保证两侧同时可行。

保存 add 返回的原边号，满流后调用 unit_flow_edges。返回的 forced 为真输出 Y，否则 possible 为真输出问号，否则输出 N。原始网络分层无环，流恰好对应所选集合；残量 SCC 用于判断能否沿环改变某条单位边，不是最小割边分类。网络规模 O(n)，整数流值至多 n，因此本题增广求流 $O(n^2)$，之后一次 SCC 为 O(n)。

完整驱动通过 8166 个小输入穷举、2000 个独立子集枚举参考与最大 n=2000 边界检查。另按官方资料包 117 个配方本地重建输入，经官方 validator、两份 accepted 参考实现及普通/ASan/UBSan 下的官方 checker 核对；生成器无用越界迭代的修订有单独记录。这是比赛应用与本地证据，不增加正式模板题覆盖，也不表示线上 AC。细节见仓库 docs/KNOWNS-UNKNOWNS.md。

17.10 点分治、启发式合并与虚树用法

P3806 在建完树后只调用一次 build，询问 count_exact 是否非零。计数对象是无序不同顶点对，零权路径允许产生零距离，但不能把点与自身配对。预处理 $O(n \log^2 n)$，每次查询 $O(n \log n)$；边修改后必须重建。

CF600E 输出子树所有最高频颜色编号之和，读 answer 而非 max_freq。并列颜色全部相加，十万种不同颜色的和超过 32 位；颜色压缩后仍须累加原编号。轻子树先算后清空，重子树保留；清空整份临时统计后再重置最大频率与颜色和。该程序为比赛应用，非比赛模板题仍待补。

P2495 每次显式加入根 1，再构造虚树。虚边和 vertices 都是原树编号，只清零本次 vertices 的 dp；逆序遍历虚边，孩子为关键点时取路径最小边权，否则取路径最小边权与孩子 dp 的较小值。使用 minimum，不能用路径总长 distance。只清除本次 keys 的关键标记；祖先与后代同时关键也允许。题目来自省选，本册将其作为应用用法保留；最大答案可达 24999900000，须用 long long。

三份完整驱动分别通过路径枚举、祖先直方图、删边子集枚举；最大规模长链仍用递归 DFS，本地通过扩大测试进程栈匹配竞赛环境。本地证据与历史线上 AC 快照分开保存。

17.11 严格次小生成树与换边

`SecondMST` 用 Kruskal 选定一棵 MST，再用递归 DFS 和倍增维护树路径上两个不同的最大边权及其原边编号。每条非树非自环边换入后，非严格模式删路径最大边；若要求严格增大且换入边恰等于路径最大值，应删路径第二大的不同权边。没有这样的第二值则跳过，而不是把另一棵同权 MST 当作严格次小。路径权为 0, 1、换入权为 1 就是两种定义不同的最小反例。

返回的 `tree` 是一份 MST 边号集合；从中删除 `out`、加入 `in` 得到 `next` 的方案。同端点同权重边也使用不同编号，自环永远不进入树。无次小树时 `next` 为空；断图时 `connected=false`，不能把默认总权零当成答案。所有总权和差值用 int128，边号 -1 表示缺项，避免以最小有符号权当哨兵。普通 dsu 保持路径减半和 y 根挂 x 根，DFS 不改为显式栈。

交换配对说明单边换入即可覆盖次小答案：将另一棵树与 MST 的不同边作合法交换配对，MST 最优性保证各配对差值非负。若总权严格增大，其中至少有一个正差不超过总差；枚举所有环上的最优严格换边就能达到全局最小正差。P4180 为 BJWC 应用题，保证严格次小存在；其权值范围可安全转回 long long，但一般接口不可如此假设。6368 份生成树全子集对照、原边方案检查、十万点链和三十万边完整驱动双模式证据见 `docs/SECOND-MST.md`。

17.12 闭合集、树形图与全局割的完整用法

P2762 把实验设为正权点、仪器设为负权点，实验到仪器连依赖边。依赖数量不固定，先用 getline 读一整行，再用 istringstream 分解；这样兼容题面的 CRLF，也不会把下一实验的赞助误读成仪器编号。maximum_closure 返回的点号中，实验为 $1..m$，仪器为 $m+1..m+n$；第二类输出时减去 m。空方案仍输出两个空行，再输出零收益。共享仪器只计费一次，赞助总额用 long long。输出方案可不唯一，不能只比较固定编号。

P4716 的点号和根都减一，调用 Arborescence::solve。有值时才解引用；无解输出 -1。每个非根点有入边不代表根能到达所有点，必须处理入边环。该接口只返回费用，不恢复边方案；核心用 optional 区分无解与合法的负费用 -1。P5632 先累加对称矩阵，再调用 StoerWagner::solve；side 是一侧的原顶点编号，仍为 0-based。核心要求至少两个点、非负权。单点不存在非平凡割，驱动另行约定输出零。两题的费用能转 long long 是题面范围保证，不能据此缩小核心的 128 位费用类型。

660 份实验计划、355 份最小树形图和 891 份全局割输入，分别以子集、入边组合、非平凡割枚举核验，另测最大规模及打印程序；普通与 ASan/UBSan 均通过。完整契约与证据见 `docs/CUT-APPLICATIONS.md`。P2762 为应用，P4716/P5632 为正式模板题；本地验证不等于线上 AC。

第一部分

跨类依赖

附录 A　数论

A.1　模算术简介

A.1.1　静态模整数

固定 int 模数 1≤mod≤INT_MAX，可为合数。构造接受 signed64 并正规化，v 保持 0..mod-1；不要手动破坏正规化。加减乘 O(1)，pow 非负 signed64 指数、时间 O(log(e+1))。try_inv 用欧几里得求逆：gcd(v,mod)=1 时返回 optional 值，否则 nullopt；mod=1 返回代表 0。inv、/、/= 要求分母可逆；断言开启时诊断违反前提，关闭断言不承诺拒绝非法除法。逆元 O(log(mod+1))、额外空间 O(1)，只用 long long，不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用，支持自赋值与链式运算，可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
template <int mod> struct ModInt
{
    static_assert(mod >= 1);
    int v;

    ModInt(long long x = 0) : v((x % mod + mod) % mod) {}

    ModInt operator+(ModInt b) const { return ModInt((long long)v + b.v); }

    ModInt operator-(ModInt b) const { return ModInt((long long)v - b.v); }

    ModInt operator*(ModInt b) const { return ModInt(1LL * v * b.v); }

    ModInt pow(long long e) const
    {
        assert(e >= 0);
        ModInt a = *this, r = 1;
        for (; e; e >>= 1, a = a * a)
            if (e & 1) r = r * a;
        return r;
    }
```

求逆与除法（接上页同一结构体）：

```
    optional<ModInt> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            a -= q * b;
            swap(a, b);
            x -= q * y;
            swap(x, y);
        }
        if (a != 1) return nullopt;
        return ModInt(x);
    }

    ModInt inv() const
    {
        auto r = try_inv();
        assert(r);
        return *r;
    } // requires an invertible residue

    ModInt operator/(ModInt b) const { return *this * b.inv(); }

    ModInt &operator+=(ModInt b) { return *this = *this + b; }

    ModInt &operator-=(ModInt b) { return *this = *this - b; }

    ModInt &operator*=(ModInt b) { return *this = *this * b; }

    ModInt &operator/=(ModInt b) { return *this = *this / b; }
};
```

联合使用示例见第 332 页（Luogu P1313 计算系数），其中直接调用本模板。

A.2 卢卡斯定理

A.2.1 Lucas：小素数模的大下标组合数

Lucas(p) 预处理素数模数 p，choose(n,k) 返回 C(n,k) 模 p；n、k 为完整 unsigned long long 范围，k>n 返回 0，C(0,0)=1。重新赋值 Lucas(p) 可更换模数并重新预处理；不检测素性，要求调用者保证 p 为素数且 p≥2。预处理时间 O(p+log p)，动态版空间 O(p)；单次查询 O(1+log_p(n+1))。适合小素数模数，通常 $p \le 10^6$；大素数不能因查询下标小就忽略这里整段 O(p) 的预处理。fac、ifac 下标为 0..p-1，模数和表只读；中间乘法用 long long。合数模数请使用 exLucas。

```
struct Lucas
{
    int mod;
    vector<int> fac, ifac;

    Lucas(int p)
    {
        assert(p >= 2);
        mod = p;
        fac.resize(p);
        ifac.resize(p);
        fac[0] = 1;
        for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
        long long a = fac[p - 1], r = 1;
        for (int e = p - 2; e; e >>= 1)
        {
            if (e & 1) r = r * a % p;
            a = a * a % p;
        }
        ifac[p - 1] = r;
        for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
    }
```

```
    int choose(unsigned long long n, unsigned long long k) const
    {
        if (k > n) return 0;
        long long answer = 1;
        while (n || k)
        {
            int a = n % mod, b = k % mod;
            if (b > a) return 0;
            answer = answer * fac[a] % mod;
            answer = answer * ifac[b] % mod;
            answer = answer * ifac[a - b] % mod;
            n /= mod;
            k /= mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P3807

题意：多组查询 C(n+m,n) 模 p，题目保证 p 为素数且不超过 10^5。每组构造 Lucas(p)，choose 的参数为 n+m、n；此题范围内 n+m 不会溢出。

所需模板：Lucas（第 329 页，A.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3807

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        unsigned long long n, m;
        int p;
        cin >> n >> m >> p;
        Lucas solver(p);
        cout << solver.choose(n + m, n) << '\n';
    }
    return 0;
}
```

附录 B　组合数学

B.1　排列组合

B.1.1　阶乘组合数

代码依赖：ModInt（第 327 页，A.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

Binomial<prime>(n=0) 预处理 0..n，要求 prime 为素数且 0≤n<prime。init(n) 只扩大当前上限，较小或相同的 n 不改变表；choose(n,k) 返回 C(n,k)，permute(n,k) 返回 n!/(n-k)!。查询前显式预处理到 n，查询本身不扩表。n<0、k<0 或 k>n 返回 0，C(0,0)=P(0,0)=1。fac、ifac 为可直接读取的阶乘与逆阶乘表，增长可能使旧引用失效。单次查询 O(1)；从 old 扩到 n 新增 O(n-old+log prime) 次域运算，重分配还可能复制旧表。首次预处理 O(n+log prime)，空间 O(n)。超过模数不能使用阶乘除法：小素数用 Lucas，小合数用 exLucas。

```
template <int mod> struct Binomial
{
    using Z = ModInt<mod>;
    vector<Z> fac{Z(1)}, ifac{Z(1)};

    Binomial(int n = 0) { init(n); }

    void init(int n)
    {
        assert(0 <= n && n < mod);
        int old = (int)fac.size() - 1;
        if (n <= old) return;
        fac.resize(n + 1);
        ifac.resize(n + 1);
        for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
        ifac[n] = fac[n].inv();
        for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
    }
```

```
    Z choose(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[k] * ifac[n - k];
    }

    Z permute(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[n - k];
    }
};
```

应用题补充：Luogu P1313 计算系数

题意：求 (by+ax) 的 k 次方中指定项的系数，模 10007。x、y 的指数分别为 n、m，均非负且 n+m=k。k 至多 1000，a、b 均为不超过 1000000 的非负数。组合 C(k,n) 与两个幂即可。模数为素数且 k 小于模数，满足阶乘逆元前提；零次幂按空乘积为 1。

所需模板：ModInt（第 327 页，A.1.1 节）、Binomial（第 331 页，B.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1313

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int a, b, k, n, m;
    cin >> a >> b >> k >> n >> m;
    Binomial<10007> c(k);
    auto x = ModInt<10007>(a).pow(n);
    auto y = ModInt<10007>(b).pow(m);
    cout << (c.choose(k, n) * x * y).v << '\n';
}
```

附录 C　线性代数

C.1　行列式

C.1.1　素数模行列式

代码依赖：ModInt（第 327 页，A.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

det_prime<p>(a) 输入 n×n 方阵，返回行列式模 p；p 必须是素数。允许奇异矩阵和零维矩阵，det(empty)=1。按行交换改变符号，主元求逆消元。$O(n^3)$ 时间、传值副本 $O(n^2)$ 空间，可传 move(a)。合数模数改用 determinant_mod。

```
template <int mod> ModInt<mod> det_prime(vector<vector<ModInt<mod>>> a)
{
    using Z = ModInt<mod>;
    int n = (int)a.size();
    Z ans = 1;
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n);
        int p = i;
        while (p < n && !a[p][i].v) ++p;
        if (p == n) return 0;
        if (p != i)
        {
            swap(a[p], a[i]);
            ans = Z(0) - ans;
        }
        ans = ans * a[i][i];
        Z inv = a[i][i].inv();
        for (int j = i + 1; j < n; j++)
        {
            Z f = a[j][i] * inv;
            for (int k = i; k < n; k++) a[j][k] = a[j][k] - f * a[i][k];
        }
    }
    return ans;
}
```

模板题使用：Library Checker matrix_det

题意：求方阵行列式，模 998244353。输入元素已取模；函数按值接收矩阵，示例用 move 交出矩阵避免额外复制。

所需模板：ModInt（第 327 页，A.1.1 节）、det_prime（第 333 页，C.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/matrix_det

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Z = ModInt<998244353>;
    int n;
    cin >> n;
    vector<vector<Z>> a(n, vector<Z>(n));
    for (auto &row : a)
        for (auto &x : row) cin >> x.v;
    cout << det_prime<998244353>(move(a)).v << '\n';
}
```

C.1.2 任意正模数行列式：欧几里得消元

determinant_mod(a,mod) 返回 det(a) 的最小非负余数。a 为 long long 方阵，元素支持完整 signed 64 位；1≤mod≤LLONG_MAX，可以是合数。不使用模逆元，允许不可逆主元。矩阵按值传入，调用后原矩阵不变；空矩阵返回 1%mod，mod=1 时返回 0。换行改变符号，最终乘上三角矩阵的对角元素。所有乘法先转 int128。$O(n^3 \log mod)$ 时间、$O(n^2)$ 空间；素数模通常优先用已有高斯行列式。

```
long long determinant_mod(vector<vector<long long>> a, long long mod)
{
    assert(mod >= 1);
    int n = a.size();
    for (auto &row : a)
    {
        assert((int)row.size() == n);
        for (auto &x : row)
        {
            x %= mod;
            if (x < 0) x += mod;
        }
    }
    long long answer = 1 % mod;
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
            while (a[j][i])
            {
                long long quotient = a[i][i] / a[j][i];
                for (int k = i; k < n; k++)
                {
                    __int128 value = (__int128)a[i][k] - (__int128)quotient * a[j][k];
                    value %= mod;
                    if (value < 0) value += mod;
                    a[i][k] = (long long)value;
                }
                swap(a[i], a[j]);
                answer = answer ? mod - answer : 0;
            }
        if (a[i][i] == 0) return 0;
        answer = (__int128)answer * a[i][i] % mod;
    }
    return answer;
}
```

模板题使用：Library Checker matrix_det_arbitrary_mod

题意：给方阵和模数，求行列式的最小非负余数，模数可以为合数。函数无需模逆元，按值接收矩阵，示例用 move 交出数据。0 阶方阵的行列式为 1，再对模数取余。

所需模板：determinant_mod（第 335 页，C.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/matrix_det_arbitrary_mod

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long mod;
    cin >> n >> mod;
    vector<vector<long long>> a(n, vector<long long>(n));
    for (auto &row : a)
        for (auto &x : row) cin >> x;
    cout << determinant_mod(move(a), mod) << '\n';
}
```

C.1.3 精确整数行列式（Bareiss）

determinant_exact(a) 按值接收 n 阶方阵，0≤n<INT_MAX，返回带符号行列式，空矩阵为 1，不修改调用者矩阵。Z 必须支持精确有符号整数四则运算，推荐 Boost cpp_int；不是模整数或浮点类型。固定宽度仅在所有中间乘积、差和商均不溢出时可用。每轮选择非零主元并换行，分子必须先完整计算，再除以上一主元；由 Sylvester 恒等式保证整除。当前列无主元则返回 0。$O(n^3)$ 次大整数算术操作、$O(n^2)$ 个整数对象；实际时间和位存储取决于中间数位数，不能视为机器字常数成本。

```
template<class Z>
Z determinant_exact(vector<vector<Z>> a)
{
    int n = a.size();
    for (const auto &row : a)
        assert((int)row.size() == n);
    if (!n)
        return Z(1);
    Z prev = 1;
    int sign = 1;
    for (int k = 0; k + 1 < n; k++)
    {
        int p = k;
        while (p < n && a[p][k] == 0)
            p++;
        if (p == n)
            return Z(0);
        if (p != k)
        {
            swap(a[p], a[k]);
            sign = -sign;
        }
        Z pivot = a[k][k];
        for (int i = k + 1; i < n; i++)
        {
            for (int j = k + 1; j < n; j++)
            {
                Z value = a[i][j] * pivot - a[i][k] * a[k][j];
                a[i][j] = value / prev;
            }
            a[i][k] = 0;
        }
        prev = pivot;
    }
    return sign * a[n - 1][n - 1];
}
```

接口用法演示：精确整数行列式

演示约定：T 组；每组 n 及 n^2 个有符号十进制整数，输出精确行列式。n=0 输出 1。cpp_int 为 Boost 任意精度有符号整数，输入与中间结果受可用内存和时间限制；此处为 API 用法，无线上 AC 声明。

所需模板：determinant_exact（第 337 页，C.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

额外依赖（不属于标准头文件）：`boost/multiprecision/cpp_int.hpp`，须由编译环境提供并显式包含。

来源：参考文档

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Z = boost::multiprecision::cpp_int;
    int t;
    cin >> t;
    while (t--)
    {
        int n;
        cin >> n;
        vector<vector<Z>> a(n, vector<Z>(n));
        for (auto &row : a)
            for (auto &x : row)
                cin >> x;
        cout << determinant_exact(move(a)) << '\n';
    }
    return 0;
}
```

附录 D　并查集

D.1　并查集

D.1.1　并查集

dsu(n)，点号 0..n-1，n≥0。find(x) 路径减半；same(x,y) 判断同集合；merge(x,y) 把 y 的根接到 x 的根，已同集合返回 false。size(x) 返回所在集合大小。groups() 返回所有非空分组，组内点号递增、组间按当前根编号排列；可能压缩路径，但不改变分组。fa、sz 可读，sz 仅根处有效；不要外部改写。本版只做路径减半，不按大小或秩合并；单次 find 最坏 O(n)。空间 O(n)，groups 另需 O(n) 输出与工作空间，并执行 n 次 find。空结构可调用 groups，不能查询顶点。

```
struct dsu
{
    vector<int> fa, sz;

    dsu(int n) : fa(n), sz(n, 1) { iota(fa.begin(), fa.end(), 0); }

    int find(int x)
    {
        while (x != fa[x]) x = fa[x] = fa[fa[x]];
        return x;
    }

    bool same(int x, int y) { return find(x) == find(y); }

    bool merge(int x, int y)
    {
        x = find(x), y = find(y);
        if (x == y) return false;
        sz[x] += sz[y], fa[y] = x;
        return true;
    }

    int size(int x) { return sz[find(x)]; }

    vector<vector<int>> groups()
    {
        vector<vector<int>> res(fa.size());
        for (int i = 0; i < int(fa.size()); i++) res[find(i)].push_back(i);
        res.erase(
            remove_if(res.begin(), res.end(), [&](const auto &v) { return v.empty(); }),
            res.end());
        return res;
    }
};
```

模板题使用：Luogu P3367

题意：维护集合合并与两点是否同集合。题目点号 1..n，传给 dsu 前减 1；查询输出 Y 或 N。

所需模板：dsu（第 339 页，D.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3367

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    dsu d(n);
    while (m--)
    {
        int op, x, y;
        scanf("%d%d%d", &op, &x, &y);
        --x, --y;
        if (op == 1)
            d.merge(x, y);
        else
            puts(d.same(x, y) ? "Y" : "N");
    }
}
```

附录 E　树状数组

E.1　树状数组第 k 小与组合依赖

保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
template <class T = long long> struct Fenwick
{
    int n;
    vector<T> a;

    Fenwick(int n = 0) : n(n), a(n + 1) {}

    void add(int x, T v)
    {
        assert(x >= 1);
        for (; x <= n; x += x & -x) a[x] += v;
    }

    T sum(int x) const
    {
        T ans = 0;
        for (; x; x -= x & -x) ans += a[x];
        return ans;
    }

    T query(int l, int r) const { return sum(r) - sum(l - 1); }

    // Nonnegative frequencies; returns n+1 if k exceeds total.
    int kth(T k) const
    {
        assert(k > 0);
        int x = 0, b = 1;
        while (b <= n / 2) b *= 2;
        for (; b; b >>= 1)
            if (x + b <= n && a[x + b] < k)
            {
                k -= a[x + b];
                x += b;
            }
        return x + 1;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based，kth 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。$n \le 100000$，$|x| \le 10000000$；总复杂度 $O(n \log n)$，空间 $O(n)$。

所需模板：Fenwick（第 341 页，E.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<pair<int, int>> q(n);
    vector<int> a;
    for (auto &[op, x] : q)
    {
        cin >> op >> x;
        if (op != 4) a.push_back(x);
    }
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    Fenwick<int> f(a.size());
    for (auto [op, x] : q)
    {
        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
        if (op == 1)
            f.add(p + 1, 1);
        else if (op == 2)
        {
            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
        }
        else if (op == 3)
            cout << f.sum(p) + 1 << '\n';
        else if (op == 4)
            cout << a[f.kth(x) - 1] << '\n';
        else if (op == 5)
            cout << a[f.kth(f.sum(p)) - 1] << '\n';
        else
        {
            p = upper_bound(a.begin(), a.end(), x) - a.begin();
            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
        }
    }
}
```

附录 F　线段树

F.1　线段树基础

F.1.1　通用懒标记线段树与边界搜索

模板参数依次为 S、F、op、e、mapping、composition、id；以数组 a 构造，0-based 半开区间；n 构造将 n 个元素置为 e。0≤n≤2^28-1，受内存限制。op 满足结合律、e 为双侧单位元；单位标记不改变节点，mapping 必须保持 op 的合并结果；composition(f,g) 表示先 g 后 f，须满足作用复合一致性，id 为标记单位元。无需比较标记是否等于 id，每次下传都应用标记。set/get 单点，apply(l,r,f) 区间作用，prod(l,r) 保序合并，all() 读取根；空区间不更新且查询返回 e。区间长度若参与修改，必须存入 S，不能从 e 推断长度：区间和节点为 {sum,len}，真实叶 len=1，e={0,0}。max_right/min_left 的谓词须满足 ok(e())、无副作用、扩展时真变假后不再变真；两者查询会下传标记，非 const。递归深度 O(log n)，构造 O(n)、操作/搜索 O(log n)、all O(1)，空间 O(n)，假设运算和类型复制 O(1)。d/tag 及其他递归辅助方法仅供内部使用。

```
template <class S, class F, auto op, auto e, auto mapping, auto composition, auto id>
struct lazy_segtree
{
    int n;
    vector<S> d;
    vector<F> tag;

    lazy_segtree(int n = 0) : lazy_segtree(vector<S>(n, e())) {}

    lazy_segtree(const vector<S> &a)
        : n(a.size()), d(4 * n + 4, e()), tag(4 * n + 4, id())
    {
        if (n) build(1, 0, n, a);
    }

    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }

    void build(int p, int l, int r, const vector<S> &a)
    {
        if (r - l == 1)
        {
            d[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
```

```
    void act(int p, F f)
    {
        d[p] = mapping(f, d[p]);
        tag[p] = composition(f, tag[p]);
    }

    void push(int p)
    {
        act(p * 2, tag[p]);
        act(p * 2 + 1, tag[p]);
        tag[p] = id();
    }

    void set(int x, S v)
    {
        assert(0 <= x && x < n);
        put(1, 0, n, x, v);
    }

    void put(int p, int l, int r, int x, S v)
    {
        if (r - l == 1)
        {
            d[p] = v;
            tag[p] = id();
            return;
        }
        push(p);
        int m = (l + r) / 2;
        if (x < m)
            put(p * 2, l, m, x, v);
        else
            put(p * 2 + 1, m, r, x, v);
        pull(p);
    }

    void apply(int l, int r, F f)
    {
        assert(0 <= l && l <= r && r <= n);
        if (l < r) update(1, 0, n, l, r, f);
    }

    void update(int p, int l, int r, int a, int b, F f)
    {
        if (a <= l && r <= b)
        {
            act(p, f);
            return;
        }
        push(p);
        int m = (l + r) / 2;
        if (a < m) update(p * 2, l, m, a, b, f);
        if (m < b) update(p * 2 + 1, m, r, a, b, f);
        pull(p);
    }
```

```
87      S all() const { return d[1]; }
88
89      S get(int x)
90      {
91          assert(0 <= x && x < n);
92          return prod(x, x + 1);
93      }
94
95      S prod(int l, int r)
96      {
97          assert(0 <= l && l <= r && r <= n);
98          return l == r ? e() : query(1, 0, n, l, r);
99      }
100
101     S query(int p, int l, int r, int a, int b)
102     {
103         if (a <= l && r <= b) return d[p];
104         push(p);
105         int m = (l + r) / 2;
106         if (b <= m) return query(p * 2, l, m, a, b);
107         if (m <= a) return query(p * 2 + 1, m, r, a, b);
108         return op(query(p * 2, l, m, a, b), query(p * 2 + 1, m, r, a, b));
109     }
110
111     template <class G> int max_right(int l, G ok)
112     {
113         assert(0 <= l && l <= n && ok(e()));
114         S s = e();
115         return l == n ? n : right(1, 0, n, l, s, ok);
116     }
117
118     template <class G> int right(int p, int l, int r, int a, S &s, G &ok)
119     {
120         if (r <= a) return n;
121         if (a <= l && ok(op(s, d[p])))
122         {
123             s = op(s, d[p]);
124             return n;
125         }
126         if (r - l == 1) return l;
127         push(p);
128         int m = (l + r) / 2;
129         int x = right(p * 2, l, m, a, s, ok);
130         return x != n ? x : right(p * 2 + 1, m, r, a, s, ok);
131     }
```

```
    template <class G> int min_left(int r, G ok)
    {
        assert(0 <= r && r <= n && ok(e()));
        S s = e();
        return r == 0 ? 0 : left(1, 0, n, r, s, ok);
    }

    template <class G> int left(int p, int l, int r, int b, S &s, G &ok)
    {
        if (b <= l) return 0;
        if (r <= b && ok(op(d[p], s)))
        {
            s = op(d[p], s);
            return 0;
        }
        if (r - l == 1) return r;
        push(p);
        int m = (l + r) / 2;
        int x = left(p * 2 + 1, m, r, b, s, ok);
        return x != 0 ? x : left(p * 2, l, m, b, s, ok);
    }
};
```

模板题使用：Library Checker range_affine_range_sum

题意：对数组区间施加 x→b*x+c，并查询区间和。S 保存和与长度；空单位元长度 0，单个元素长度 1。composition(f,g) 表示先 g 后 f，mapping 按长度累加常数项。区间为 0-based 半开，模 998244353。

所需模板：lazy_segtree（第 343 页，F.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_affine_range_sum

```
using S = pair<long long, int>;
using F = pair<long long, long long>;
const int mod = 998244353;

S op(S a, S b)
{
    return {(a.first + b.first) % mod, a.second + b.second};
}

S e()
{
    return {0, 0};
}

S mapping(F f, S a)
{
    return {(f.first * a.first + f.second * a.second) % mod, a.second};
}

F composition(F f, F g)
{
    return {f.first * g.first % mod, (f.first * g.second + f.second) % mod};
}

F id()
{
    return {1, 0};
}
```

使用入口（接上页配置函数）：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n);
    for (auto &[x, len] : a)
    {
        cin >> x;
        len = 1;
    }
    lazy_segtree<S, F, op, e, mapping, composition, id> s(a);
    while (q--)
    {
        int t, l, r;
        cin >> t >> l >> r;
        if (t == 0)
        {
            long long b, c;
            cin >> b >> c;
            s.apply(l, r, {b, c});
        }
        else
            cout << s.prod(l, r).first << '\n';
    }
}
```

F.1.2 通用线段树与左右边界搜索

segtree<S,op,e>(a) 从 0-based vector 构建，或 segtree(n) 初始化为单位元。0≤n≤2^29，实际受内存限制。op(S,S) 须满足结合律，e() 是双侧单位元；无需交换律。set(p,x)、get(p) 操作单点，prod(l,r) 返回半开区间 [l,r) 从左到右的合并，空区间返回 e；all() 返回全区间结果。max_right(l,ok) 返回最大的 r 使 ok(prod(l,r)) 为真；min_left(r,ok) 返回最小的 l。要求 ok(e()) 为真，ok 无副作用，并随对应方向扩展从真变假后不再变真；例如非负和不超过阈值，不能对任意负数和这样搜索。l=n、r=0 支持空端点。构建 O(n)，单点/区间/边界搜索 O(log n)，all O(1)，空间 O(n)，假设 op、e、ok 及 S 复制为 O(1)。d、sz、pull 为内部实现，不直接修改。函数复合时 op(f,g) 表示 g(f(x))，对应先执行左区间再执行右区间；min_left 不能照抄 max_right 的合并顺序。

```
template <class S, auto op, auto e> struct segtree
{
    int n, sz = 1;
    vector<S> d;

    segtree(int n = 0) : segtree(vector<S>(n, e())) {}

    segtree(const vector<S> &a) : n(a.size())
    {
        while (sz < n) sz <<= 1;
        d.assign(2 * sz, e());
        copy(a.begin(), a.end(), d.begin() + sz);
        for (int i = sz - 1; i; i--) pull(i);
    }

```

```
    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }

    void set(int p, S x)
    {
        assert(0 <= p && p < n);
        d[p += sz] = x;
        while (p >>= 1) pull(p);
    }

    S get(int p) const
    {
        assert(0 <= p && p < n);
        return d[p + sz];
    }

    S all() const { return d[1]; }
```

```
    S prod(int l, int r) const
    {
        assert(0 <= l && l <= r && r <= n);
        S a = e(), b = e();
        for (l += sz, r += sz; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1) a = op(a, d[l++]);
            if (r & 1) b = op(d[--r], b);
        }
        return op(a, b);
    }

    template <class F> int max_right(int l, F ok) const
    {
        assert(0 <= l && l <= n && ok(e()));
        if (l == n) return n;
        S a = e();
        l += sz;
        do
        {
            while (!(l & 1)) l >>= 1;
            if (!ok(op(a, d[l])))
            {
                while (l < sz)
                {
                    l *= 2;
                    if (ok(op(a, d[l]))) a = op(a, d[l++]);
                }
                return l - sz;
            }
            a = op(a, d[l++]);
        } while ((l & -l) != l);
        return n;
    }
```

```
    template <class F> int min_left(int r, F ok) const
    {
        assert(0 <= r && r <= n && ok(e()));
        if (r == 0) return 0;
        S a = e();
        r += sz;
        do
        {
            --r;
            while (r > 1 && (r & 1)) r >>= 1;
            if (!ok(op(d[r], a)))
            {
                while (r < sz)
                {
                    r = r * 2 + 1;
                    if (ok(op(d[r], a))) a = op(d[r--], a);
                }
                return r + 1 - sz;
            }
            a = op(d[r], a);
        } while ((r & -r) != r);
        return 0;
    }
};
```

模板题使用：Library Checker point_set_range_composite

题意：单点修改一次函数，查询区间内从左到右依次作用于 x 的结果。op(f,g) 表示先 f 后 g，单位元为 x；prod(l,r) 使用 0-based 半开区间，空区间返回单位元。配置函数须和 main 一起抄写。

所需模板：segtree（第 348 页，F.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/point_set_range_composite

```
using S = pair<long long, long long>;
const int mod = 998244353;

S op(S f, S g)
{
    return {f.first * g.first % mod, (f.second * g.first + g.second) % mod};
}

S e()
{
    return {1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n);
    for (auto &[x, y] : a) cin >> x >> y;
    segtree<S, op, e> s(a);
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(p, {c, d});
        }
        else
        {
            int l, r;
            long long x;
            cin >> l >> r >> x;
            auto [c, d] = s.prod(l, r);
            cout << (c * x + d) % mod << '\n';
        }
    }
}
```

F.1.3 线段树：区间乘、加、赋值与模意义求和

AffineSegTree(a,mod) 传入非空 0-based 初值数组；查询与修改采用 1-based 非空闭区间。update(l,r,mul,add) 将区间元素 x 改为 mul*x+add；乘法用 (mul,0)，加法用 (1,add)，赋值用 (0,value)。query(l,r) 返回区间和。1≤mod≤INT_MAX，模数无需为素数，初值和修改参数允许 long long 全范围，入口统一归一化。旧标记 (m,b) 后应用新标记 (a,c)，复合为 (m*a,b*a+c)，乘法取模后再相加。建树 O(n)，单次 O(log n)，空间 O(n)。内部 build、apply 等函数不作为公共接口；动态版重建新对象。

```
struct AffineSegTree
{
    using ll = long long;

    struct Node
    {
        ll sum = 0, mul = 1, add = 0;
    };

    int n, mod;
    vector<Node> t;

    AffineSegTree(const vector<ll> &a, int mod) : n(a.size()), mod(mod), t(4 * n + 4)
    {
        assert(n > 0 && mod > 0);
        build(1, 1, n, a);
    }

    ll norm(ll x) const
    {
        x %= mod;
        if (x < 0) x += mod;
        return x;
    }

    void pull(int p) { t[p].sum = (t[p * 2].sum + t[p * 2 + 1].sum) % mod; }

    void build(int p, int l, int r, const vector<ll> &a)
    {
        t[p] = Node{};
        if (l == r)
        {
            t[p].sum = norm(a[l - 1]);
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m + 1, r, a);
        pull(p);
    }

    // New transform follows the old pending transform: x -> x*a+b.
    void apply(int p, int len, ll a, ll b)
    {
        t[p].sum = (t[p].sum * a % mod + len * b % mod) % mod;
        t[p].mul = t[p].mul * a % mod;
        t[p].add = (t[p].add * a % mod + b) % mod;
    }
```

```
49
50      void push(int p, int l, int r)
51      {
52          if (t[p].mul == 1 && t[p].add == 0) return;
53          int m = (l + r) / 2;
54          apply(p * 2, m - l + 1, t[p].mul, t[p].add);
55          apply(p * 2 + 1, r - m, t[p].mul, t[p].add);
56          t[p].mul = 1;
57          t[p].add = 0;
58      }
59
60      void update(int l, int r, ll a, ll b)
61      {
62          assert(1 <= l && l <= r && r <= n);
63          update(1, 1, n, l, r, norm(a), norm(b));
64      }
65
66      void update(int p, int l, int r, int ql, int qr, ll a, ll b)
67      {
68          if (ql <= l && r <= qr)
69          {
70              apply(p, r - l + 1, a, b);
71              return;
72          }
73          push(p, l, r);
74          int m = (l + r) / 2;
75          if (ql <= m) update(p * 2, l, m, ql, qr, a, b);
76          if (qr > m) update(p * 2 + 1, m + 1, r, ql, qr, a, b);
77          pull(p);
78      }
79
80      ll query(int l, int r)
81      {
82          assert(1 <= l && l <= r && r <= n);
83          return query(1, 1, n, l, r);
84      }
85
86      ll query(int p, int l, int r, int ql, int qr)
87      {
88          if (ql <= l && r <= qr) return t[p].sum;
89          push(p, l, r);
90          int m = (l + r) / 2;
91          ll answer = 0;
92          if (ql <= m) answer += query(p * 2, l, m, ql, qr);
93          if (qr > m) answer += query(p * 2 + 1, m + 1, r, ql, qr);
94          return answer % mod;
95      }
96  };
```

模板题使用：Luogu P3373

题意：对数组执行区间乘、区间加和区间求和，结果对给定模数取模。区间为 1-based 闭区间；update 的两个参数表示先乘再加。

所需模板：AffineSegTree（第 353 页，F.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3373

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, q, mod;
    std::cin >> n >> q >> mod;
    std::vector<long long> a(n);
    for (auto &x : a) std::cin >> x;
    AffineSegTree tree(a, mod);
    while (q--)
    {
        int op, l, r;
        std::cin >> op >> l >> r;
        if (op == 3)
            std::cout << tree.query(l, r) << '\n';
        else
        {
            long long k;
            std::cin >> k;
            if (op == 1)
                tree.update(l, r, k, 0);
            else
                tree.update(l, r, 1, k);
        }
    }
}
```

算法索引

AffineSegTree, 353
Arborescence, 147
AssignmentSpectrum, 286

BellmanFord, 112
BiconnectedCore, 163
Binomial, 331
BipartiteMatching, 274
block_cut_forest, 188
Blossom, 292
BoundedCirculation, 265
BoundedMaxFlow, 267
bridge_augmentation, 179
bridge_component_forest, 167

Cactus, 183
Cayley formula, 295
CentroidDiameter, 66
CentroidNearest, 62
CentroidPairs, 49
CentroidSum, 57
count_four_cycles, 190
cut_tree_values, 249

dag_path_determinant, 307
DagDominator, 313
DagLongest, 75
DenseDijkstra, 101
det_prime, 333
determinant_exact, 337
determinant_mod, 335
Dijkstra, 84
Dinic, 227
DirectedEuler, 209
DominatorTree, 309
dsu, 339
DynamicDiameter, 4

EdgeCompression, 172
enumerate_triangles, 192
EulerLCA, 15

Fenwick, 341
flow_unique, 241
Floyd, 88

FunctionalGraph, 155

gomory_hu, 247

HLD, 30
HLPP, 221
Hungarian, 281

independent_set, 279
Isap, 237

Johnson, 81

Kruskal, 142
KruskalTree, 125
KShortestWalks, 122

lazy_segtree, 343
LexTwoSAT, 206
LiftingLCA, 18
LongChain, 25
Lowlink, 180
Lucas, 329

Manhattan MST, 146
ManhattanMST, 130
matching_edges, 278
MatrixTree, 297
MatrixTreeExact, 302
MatrixTreeMod, 299
maximum_closure, 245
MinCostFlow, 250
mincut_edges, 243
MinCycle, 198
minimum spanning tree, 146
minimum_mean_cycle, 194
mixed_euler_orientation, 217
mixed_euler_trail, 220
ModInt, 327

NegativeCostFlow, 258

odd_cycle_vertices, 169
odd_induced_partition, 1
OfflineLCA, 12
orient_edges, 175

path_intersection, 23
Prim, 138
Prüfer sequence, 295

release_bfs, 86

removal_components, 176

SCC, 160
SecondMST, 133
segtree, 348
ShortestPathTree, 95
Spfa, 105
SpfaFlow, 254
StoerWagner, 271
StrictSecondShortest, 119
SubtreeColors, 42

TarjanSCC, 150
TreeDiameter, 9
TreeIsomorphism, 72
TreeMarket, 53
TwoSAT, 203

UndirectedEuler, 214
unit_flow_edges, 233

VirtualTree, 45

WeightedMatching, 283
word_chain, 212

ZkwFlow, 261

开放时间最短路, 322

树上路径交, 318

贪吃蛇, 322